

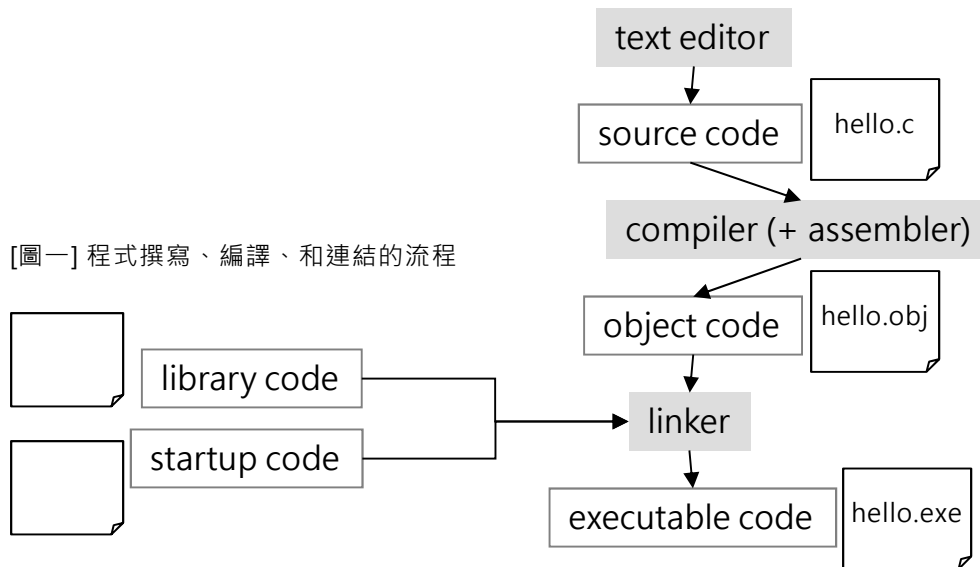
NTHU 100010CS 135502

Introduction to Programming

2011

第一章：C 程式設計簡介 課程簡介

程式編寫流程



[圖一] 程式撰寫、編譯、和連結的流程

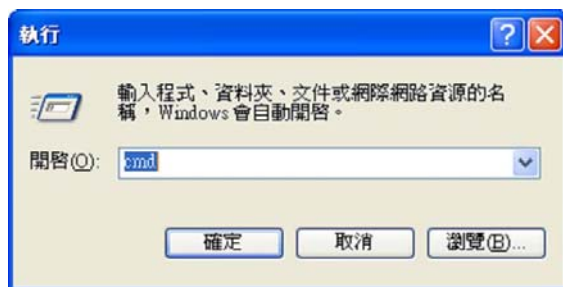
text editor 是用來寫程式的純文字編輯器，通常還會有一些額外的輔助功能，譬如縮排或改變關鍵字顏色，讓你的程式碼看起來更一目了然。

compiler 是程式語言的核心工具，用來將你寫的 C source code 變成一堆要讓電腦執行的指令。由於我們要使用的程式開發工具 Visual C++、Dev C++ 或 Code::Blocks 所包含的 compiler 其實可以編譯 C 或 C++ source code，不過這兩種語言還是有一定程度的差異，所以必須強迫 VC++ 或 Dev C++ compiler 使用 C 語言的語法來編譯你的 source code。只要你的 source code 檔案存檔時選擇用的副檔名是 ".c"，VC++ 或 Dev C++ 就會用 C compiler 來編譯你的程式。VC++ 使用的是 Microsoft 自行開發的 compiler，而 Dev C++ 和 Code:Blocks 則通常是用公開原始碼的 MinGW GCC。另外，在 Dev C++ 裡可以從主選單裡選 [Tools]→[Compiler Options]→[Settings] 然後在 C compiler 的項目裡把 "Support all ANSI standard C programs" 項目選成 "Yes"。

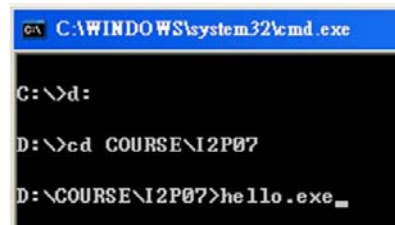
linker 是用來將編譯好的 object code 連結成最後的執行檔。因為除了我們自己寫的程式碼之外，無可避免一定會用到許多現成的 library (函式庫)。在現實的情況下，不可能每一東西都要靠自己創造，所以絕大多數的時候我們都會使用內建的或別人寫好的函式庫，來達到自己想要的功能。linker 就是用來把我們的程式碼和需要的函式庫裡的程式碼連結在一起，形成單一的可執行檔。在 Windows 裡，執行的副檔名為 ".exe"。

由於這門課並不會介紹到視窗程式設計，所以全部的程式最後都是在 console mode 命令列狀態下執行。從 [附屬應用程式]→[命令提示字元] 可以直接開啟 command prompt。另外也可以從桌面左下選 [開始]→[執行]，在 [開啟] 的欄位裡輸入 **cmd** 按下 [Enter] 鍵或按 [確定] (見圖二)。

開出了 command prompt 之後再用指令 "**cd** 目錄名稱" 切換到你的執行檔的目錄，然後再打你的執行檔檔名 (譬如 hello.exe) 就可以執行你寫的程式 (如圖三)。



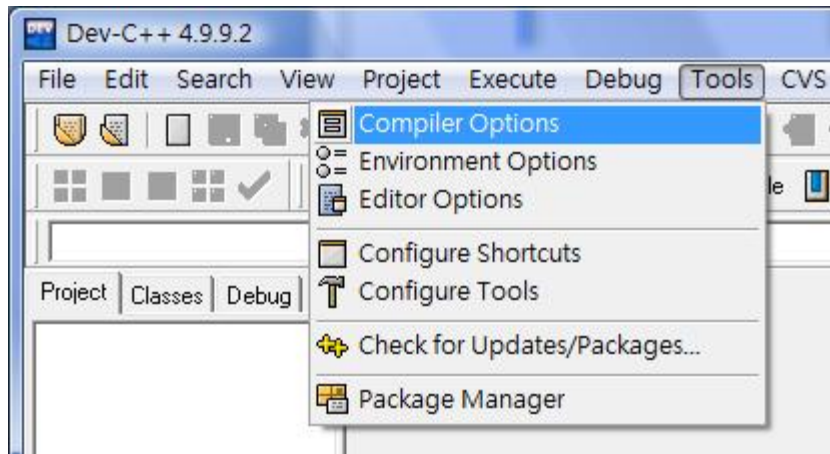
[圖二] 開啟 DOS 視窗



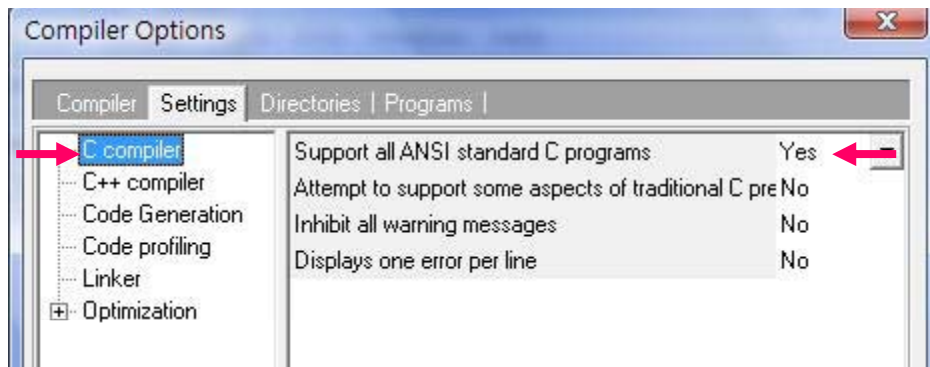
[圖三] 假設執行檔 hello.exe 放在 D:\COURSE\I2P07 目錄裡

第一章：C 程式設計簡介 課程簡介

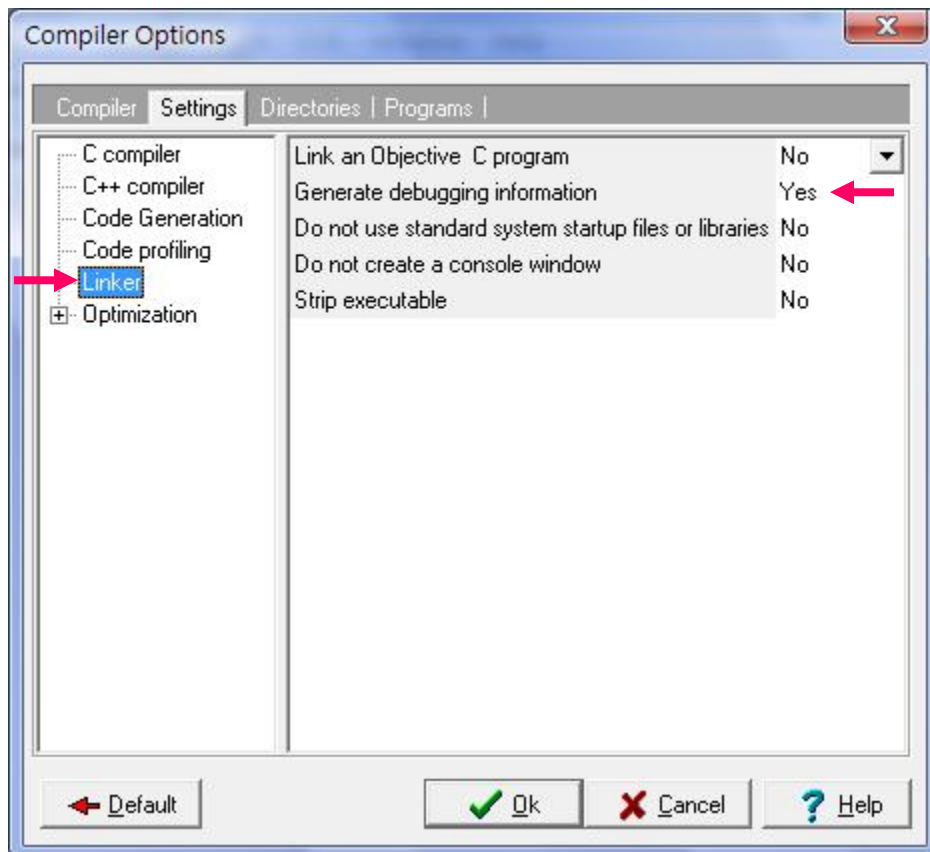
Dev C++ 設定方式



[圖四] 從選單的 [Tools] 選 [Compiler Options]

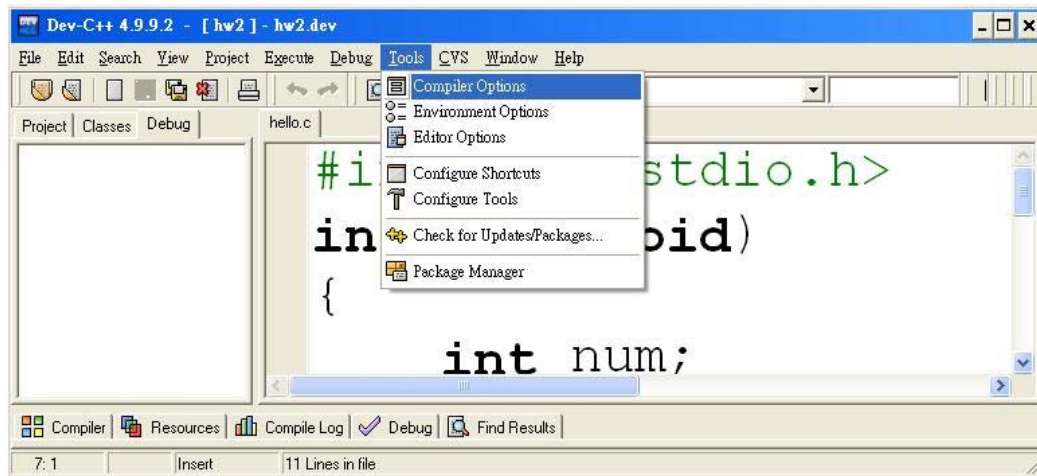


[圖五] 開出對話框之後，切換到 [Settings] 分頁，然後把 C compile 裡的 "Support all ANSI standard C programs" 改成 [Yes]



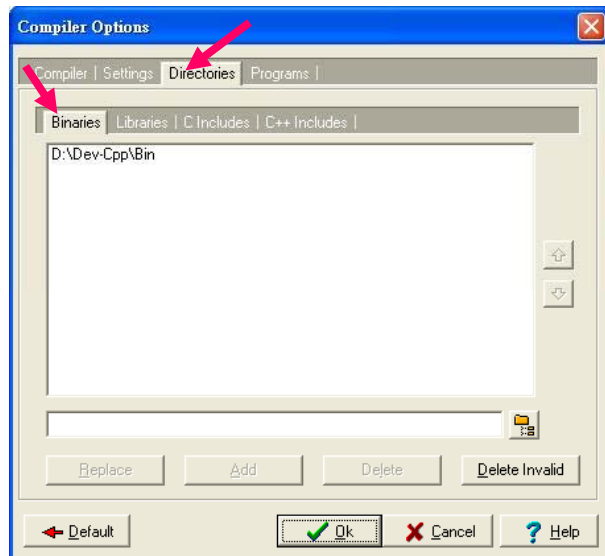
[圖六] 然後再選到 Linker 把 "Generate debugging information" 改成 [Yes]。最後按 [Ok] 就完成了。

第一章：C 程式設計簡介 課程簡介

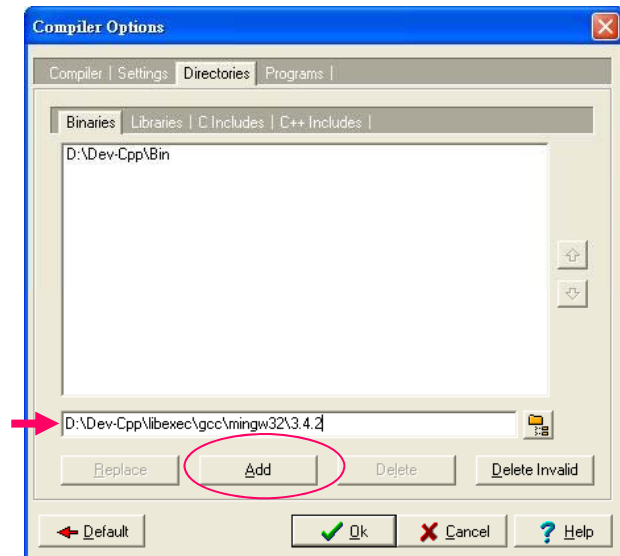


[圖七] 從選單的 [Tools] 選 [Compiler Options]

[圖八]

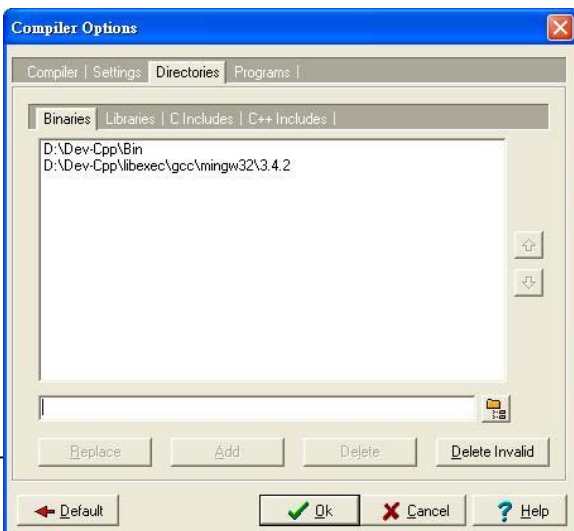


[圖九]

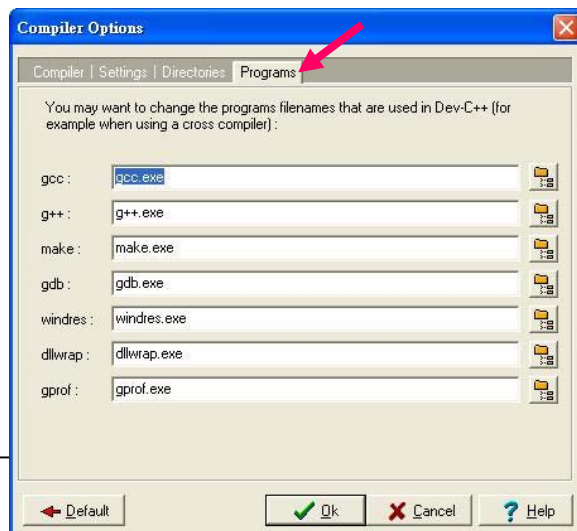


出現對話框之後切換到 [Directories] 分頁裡的 [Binaries 分頁]，然後用瀏覽的方式找到路徑 "C:\Dev-Cpp\libexec\gcc\mingw32\3.4.2" 或是 "D:\Dev-Cpp\libexec\gcc\mingw32\3.4.2" 要看你是把 Dev-C++ 灌在哪裡，我的是灌在 D: 硬碟。按下 [Add] 加進去之後變成底下這樣。接著就切換到 [Program] 分頁

[圖十]



[圖十一]



本來裡面長得像是



[圖十二]

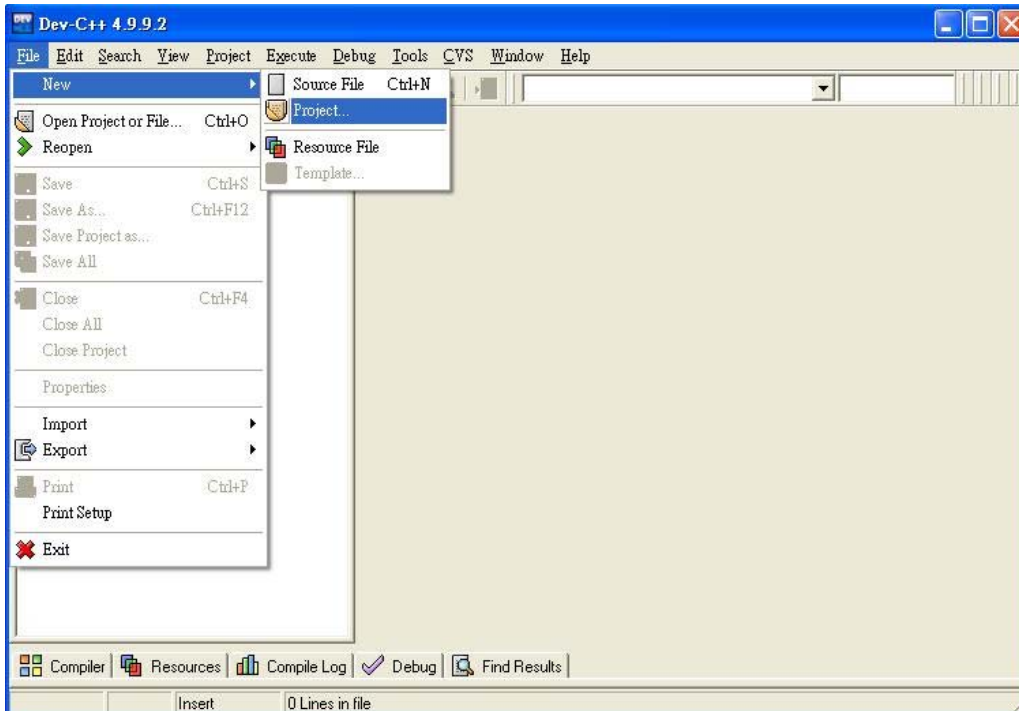
把它全部改成底下那樣。也就是在前面多加入路徑 "C:\Dev-Cpp\Bin\" 或 "D:\Dev-Cpp\Bin\" 看你是把 Dev-C++ 灌在哪个硬碟，因為我是灌在D:碟 所以用這個路徑。按下 [Ok] 之後，應該就可以 Compile 你的程式。



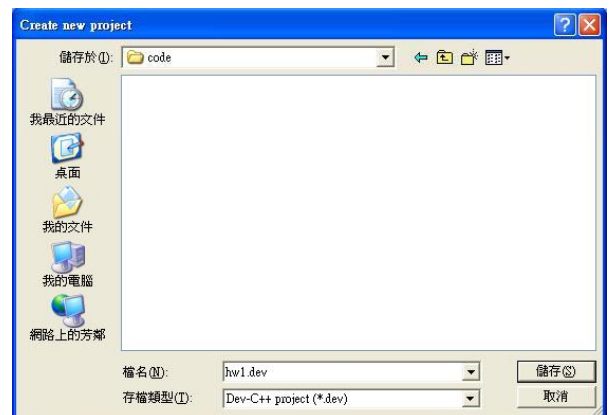
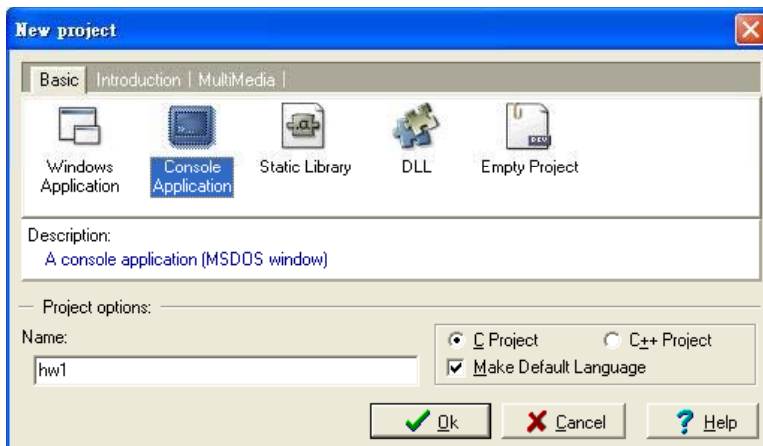
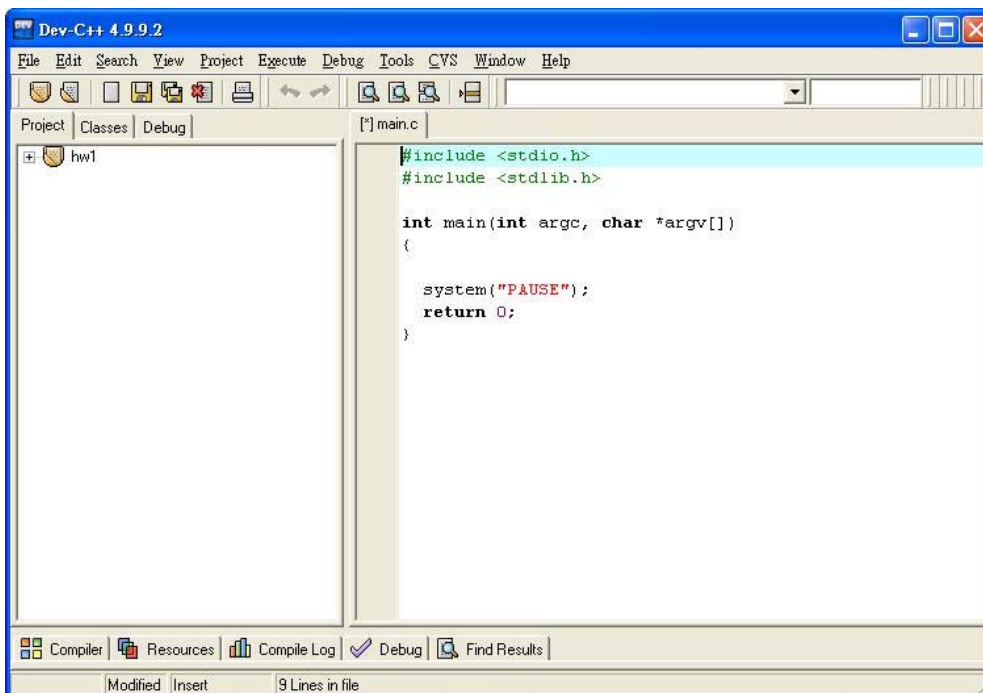
[圖十三]

第一章：C 程式設計簡介 課程簡介

Dev C++ 程式編寫範例

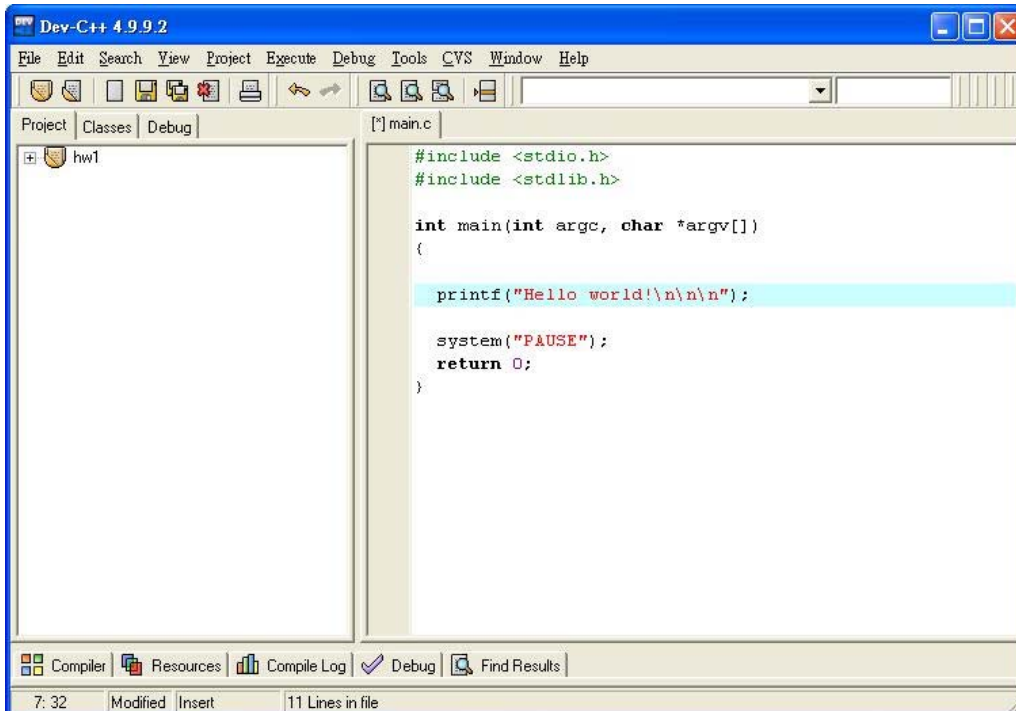


[圖十四] 產生新的 Project

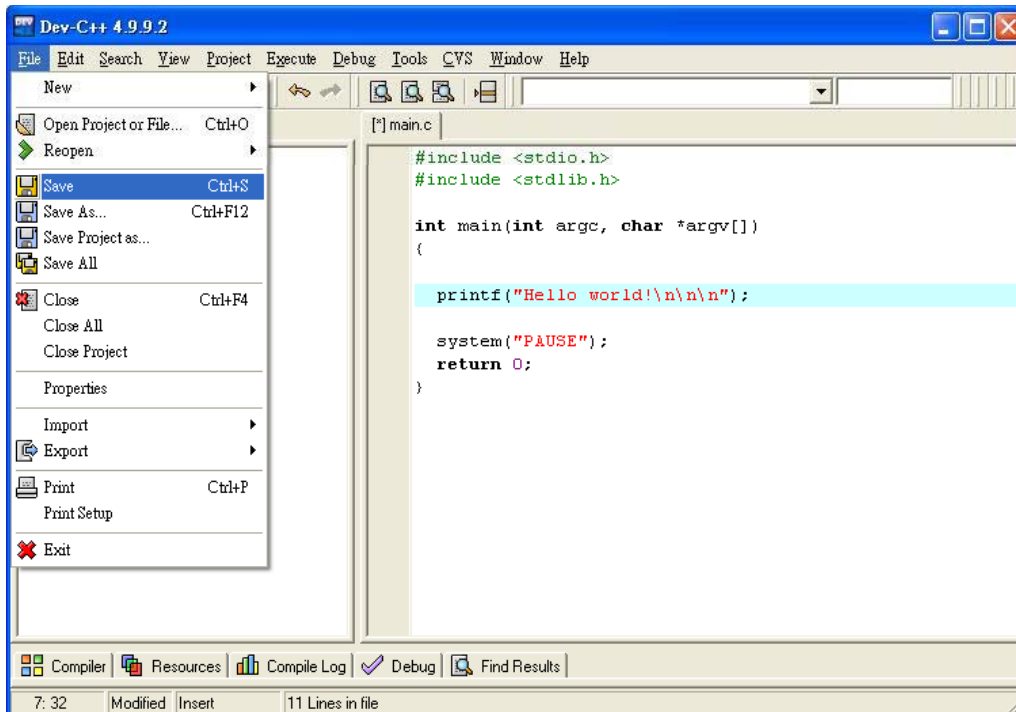
[圖十五& 圖十六]
選擇 Console Application ·
把 Project 取名為 hw1 ·
並且設定成 C Project · 在
對話框裡選擇要存放的目錄。[圖十七] 自動產生的樣本程式。
只是一個空殼子，必須再把
我們的程式碼填進去。

第一章：C 程式設計簡介 課程簡介

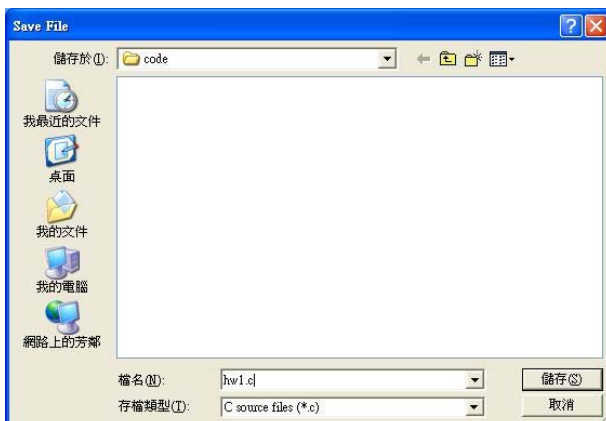
Dev C++ 程式編寫範例



[圖十八] 加入一行程式碼。



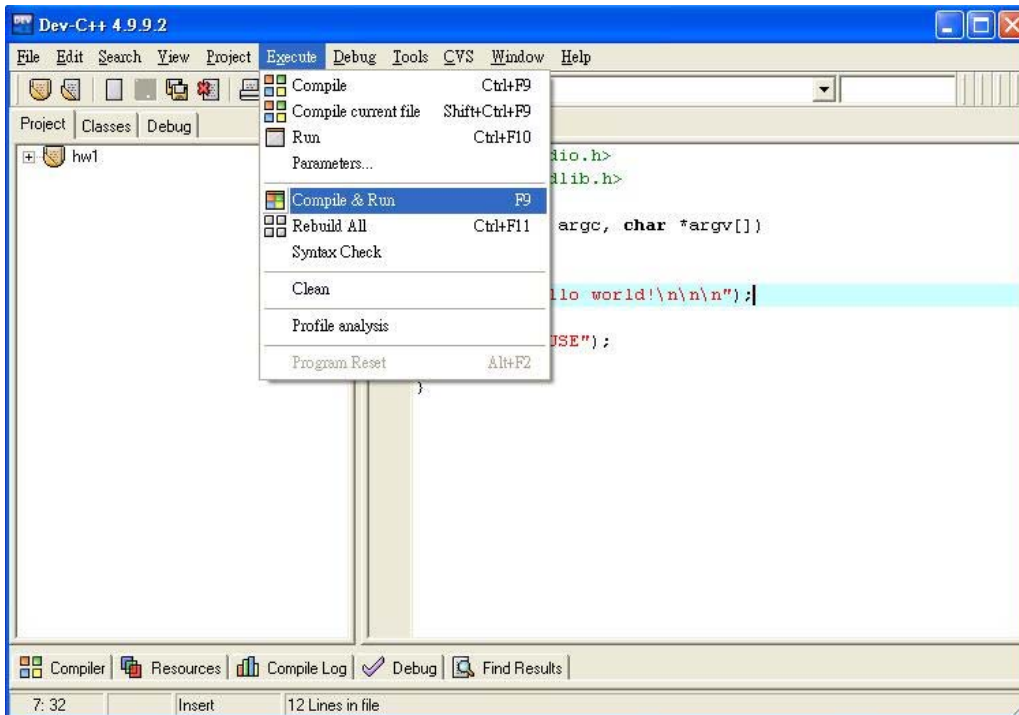
[圖十九] 存檔。



[圖二十] 輸入檔名 (hw1.c) · 並且選擇存放的目錄。

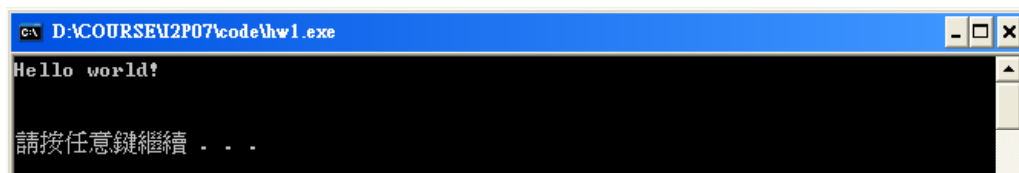
第一章：C 程式設計簡介 課程簡介

Dev C++ 程式編寫範例



[圖二十一]

選擇 "Compile & Run"，編譯並執行程式。也可以分成兩個步驟，先選 "Compile"，如果沒有出錯，再選 "Run" 執行程式。



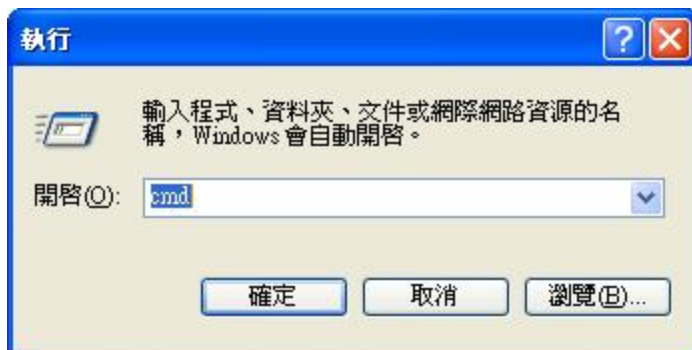
[圖二十二]

程式執行的結果會顯示在 DOS 視窗中。

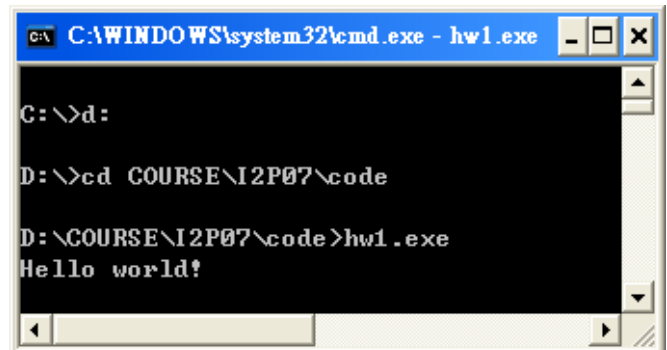
注意：`system("PAUSE");` 是 Dev C++ 替你產生的，它的做用是在程式執行完後，讓 DOS 視窗繼續開著，不要被自動關掉 (如圖二十二)。平常我們寫程式並不會有這一行。因此建議大家用前面提到的方式來執行程式。

在 Windows 裡面，從桌面左下選 [開始]→[執行]，在 [開啟] 欄位裡輸入 `cmd` 按下 [Enter] 鍵 (見圖二十三)，就會跑出一個 DOS 視窗。再用指令 "`cd 目錄名稱`" 切換到你的執行檔的目錄，然後再打你的執行檔檔名 (在這個例子是 `hw1.exe`) 就可以執行你寫的程式 (如圖二十四)，看到程式執行的結果。

[圖二十三]

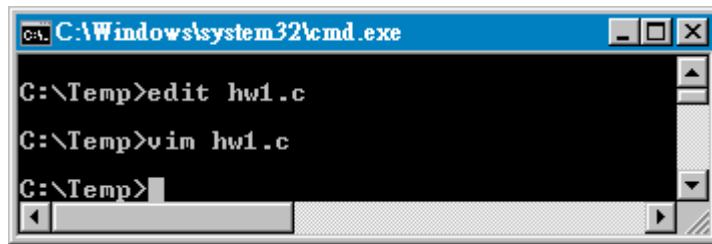


[圖二十四]



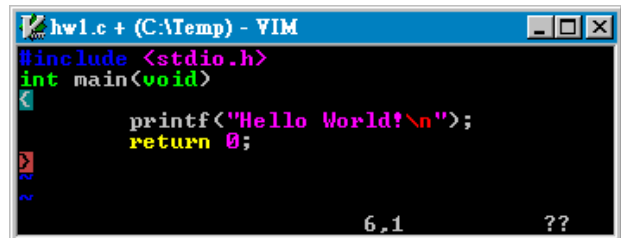
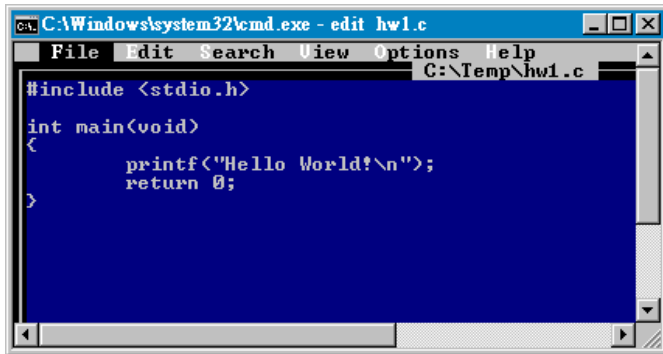
第一章：C 程式設計簡介 課程簡介

直接在命令列編譯程式而不使用 Dev C++



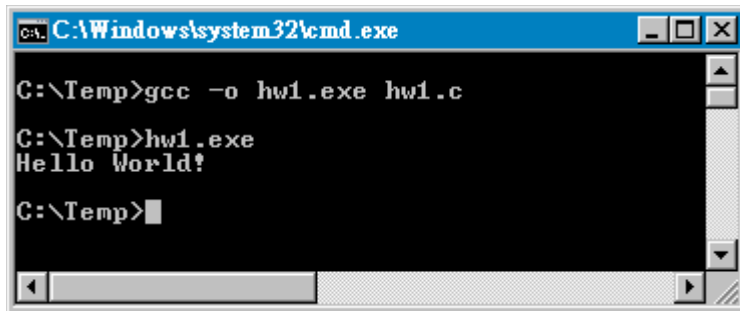
[圖二十五]

使用你電腦上有安裝的文字編輯器，例如 edit 或是 vim，編輯 hw1.c，程式寫完之後存檔。



[圖二十六]

在命令列中輸入 `gcc -o hw1.exe hw1.c` 就能將 hw1.c 編譯成 hw1.exe
-o 參數後面接的是想要產生的執行檔名
執行 hw1.exe 之後就可以看到結果



除了使用 Dev C++ 或 Code::Blocks 這類整合開發環境之外，也可以直接使用 gcc 指令，在命令列中編譯程式。gcc 是 GNU 的 compiler，可以編譯 C 程式。在 Windows 系統上安裝的版本是 MinGW，可以單獨下載安裝，或是之前安裝 Dev C++ 的時候其實裡面就包含了 MinGW 版本的 gcc compiler。先用任意的文字編輯器將程式寫好並存檔 (如圖二十五)，然後參考在命令列用 gcc 就可以編譯程式碼 (如圖二十六)。

另外還有一種方式是撰寫 makefile。當你的程式是由好幾個檔案組成時，就可以用 makefile 的方式一次編譯所有相關檔案。例如將底下文字 (注意要保持 Tab 鍵縮排格式) 儲存成一個叫做 makefile 的檔案：

```

all: tank_simple.exe

tank_simple.exe: kb_input2.o console_draw2.o tank_simple.o
    gcc -o tank_simple.exe kb_input2.o console_draw2.o tank_simple.o

tank_simple.o: tank_simple.c
    gcc -c tank_simple.c

console_draw2.o :console_draw2.c
    gcc -c console_draw2.c

kb_input2.o :kb_input2.c
    gcc -c kb_input2.c

clean:
    del *.o

```

假如已經安裝了 MinGW，就可以在命令列中執行
`mingw32-make -f makefile`

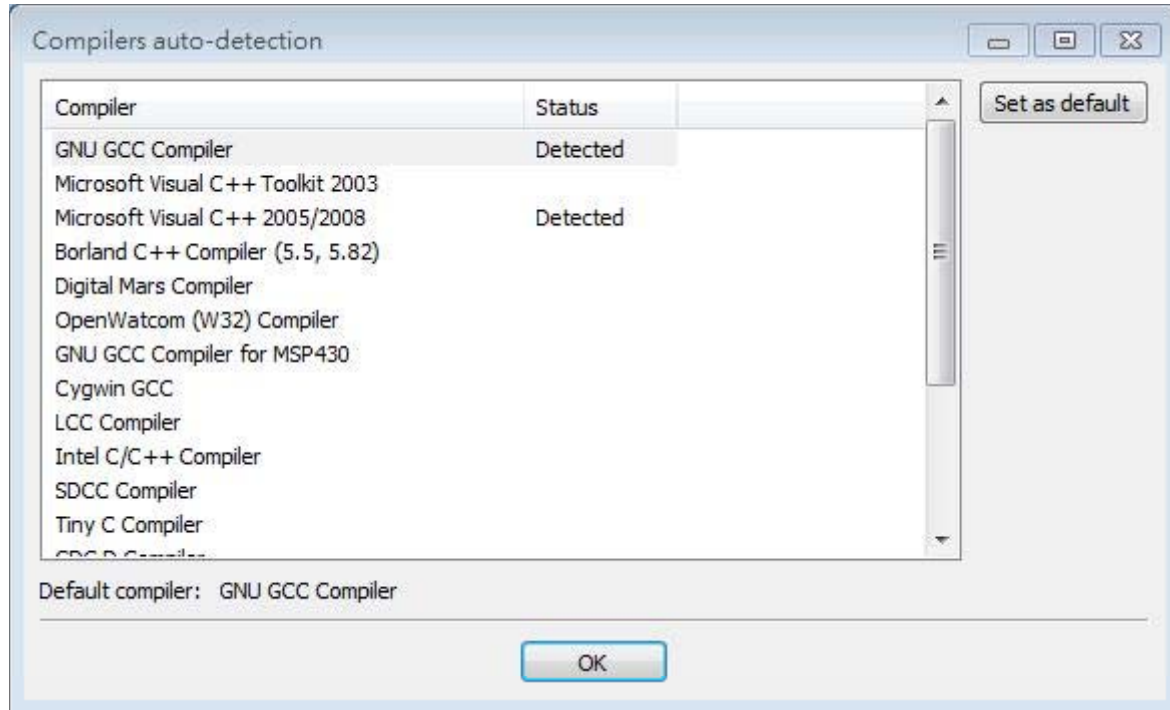
將相關檔案 kb_input2.c、console_draw2.c 以及 tank_simple.c 全部編譯並產生 tank_simple.exe 執行檔。
如果是安裝 Cygwin 或是在 GNU/Linux 系統下，則是直接執行 make 就可以得到同樣效果。

第一章：C 程式設計簡介 課程簡介

Code block 設定方式

安裝前，先到 <http://sourceforge.net/projects/mingw/files/> 下載GNU C++ Compiler安裝
或是直接下載Code Block 含有MinGW的版本安裝

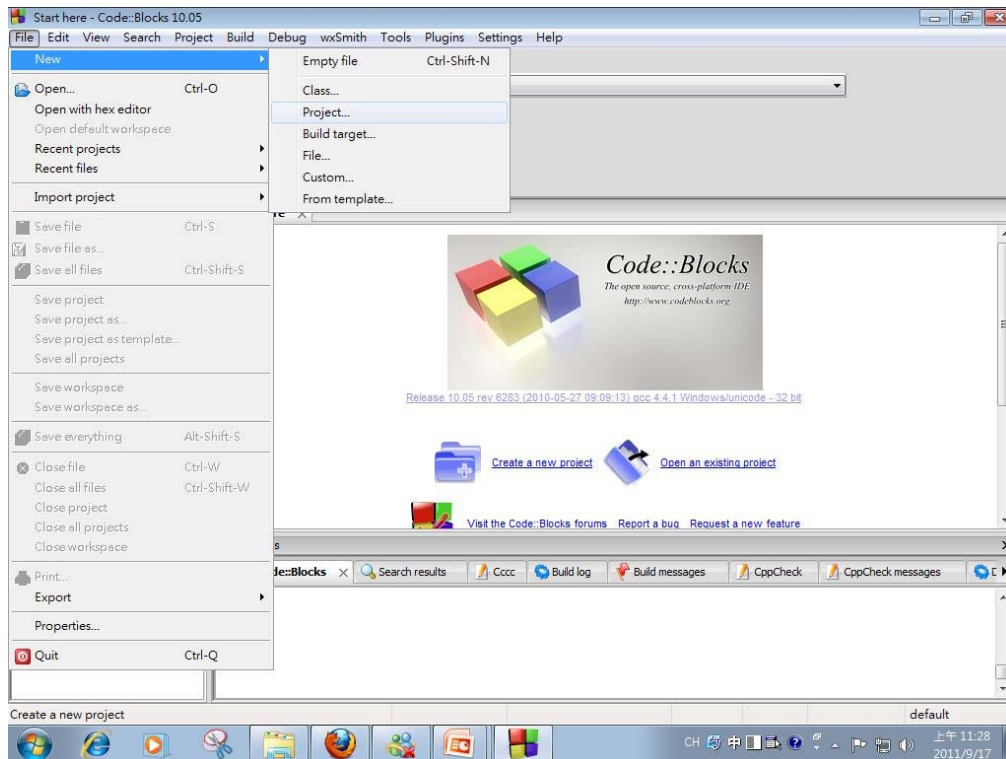
安裝完畢後，第一次開啟時就會詢問預設的compiler



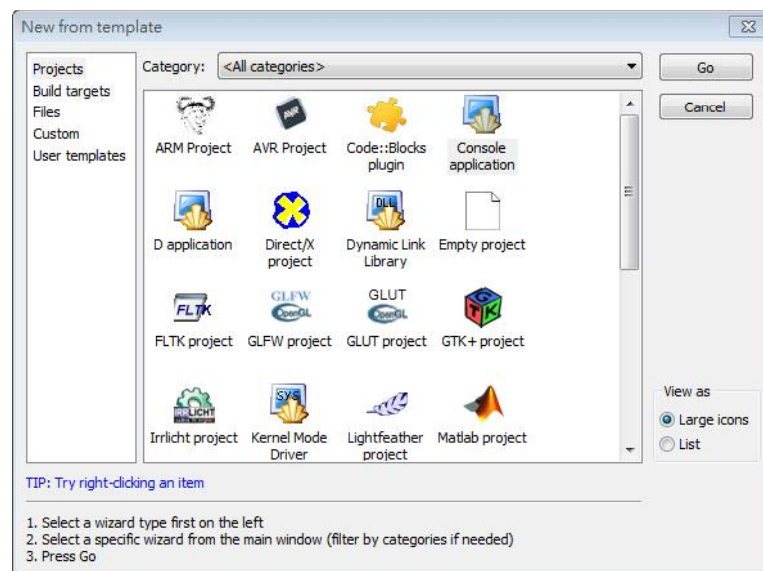
若前面已有安裝了MinGW的Compiler
則在List中就會偵測到GNU GCC Compiler，選擇GNU GCC Compiler

第一章：C 程式設計簡介 課程簡介

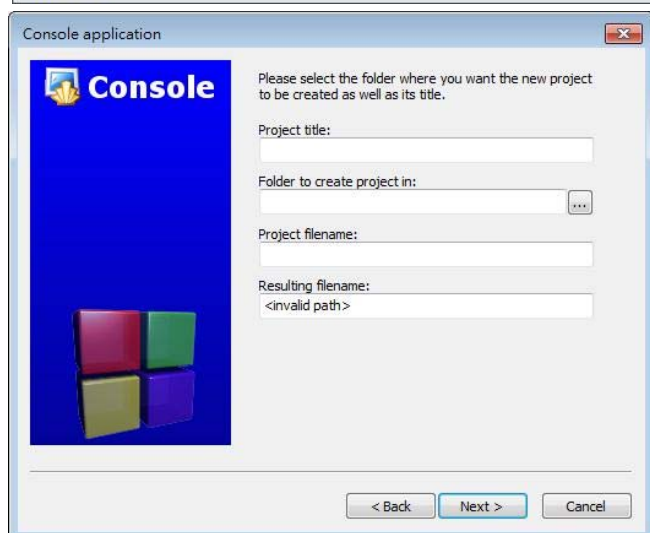
Code block 編寫程式範例



開啟新專案



選擇Console Application

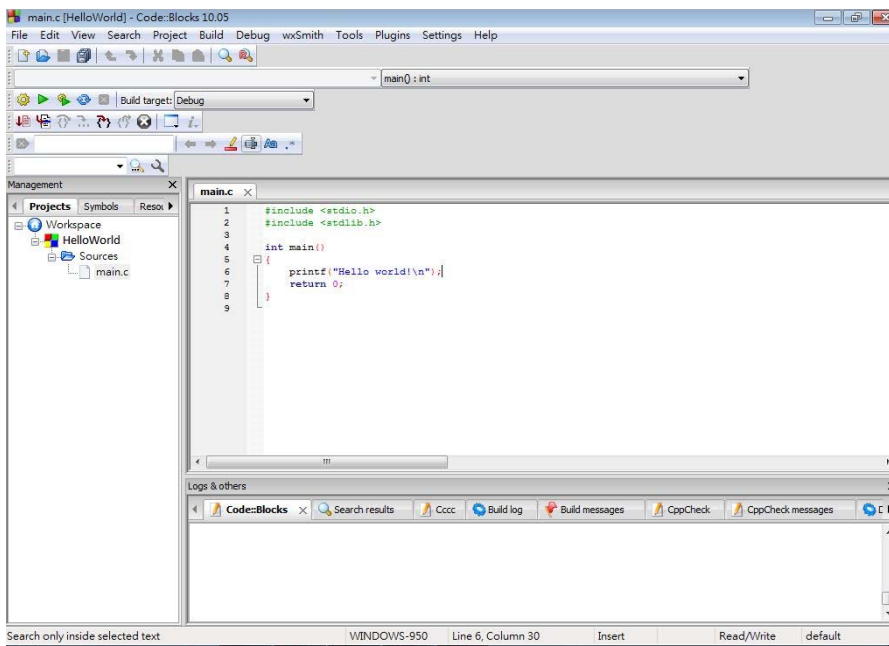
進入專案精靈後
輸入專案名稱以及存取路徑

第一章：C 程式設計簡介 課程簡介

Code block 編寫程式範例

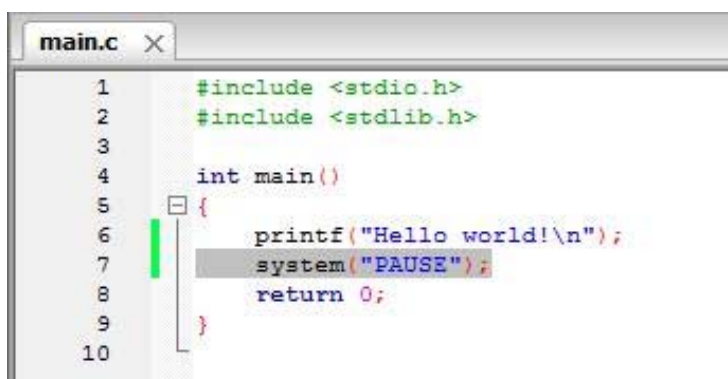


最後設定compiler的部分維持預設即可



開啟旁邊source裡面的main.c編寫

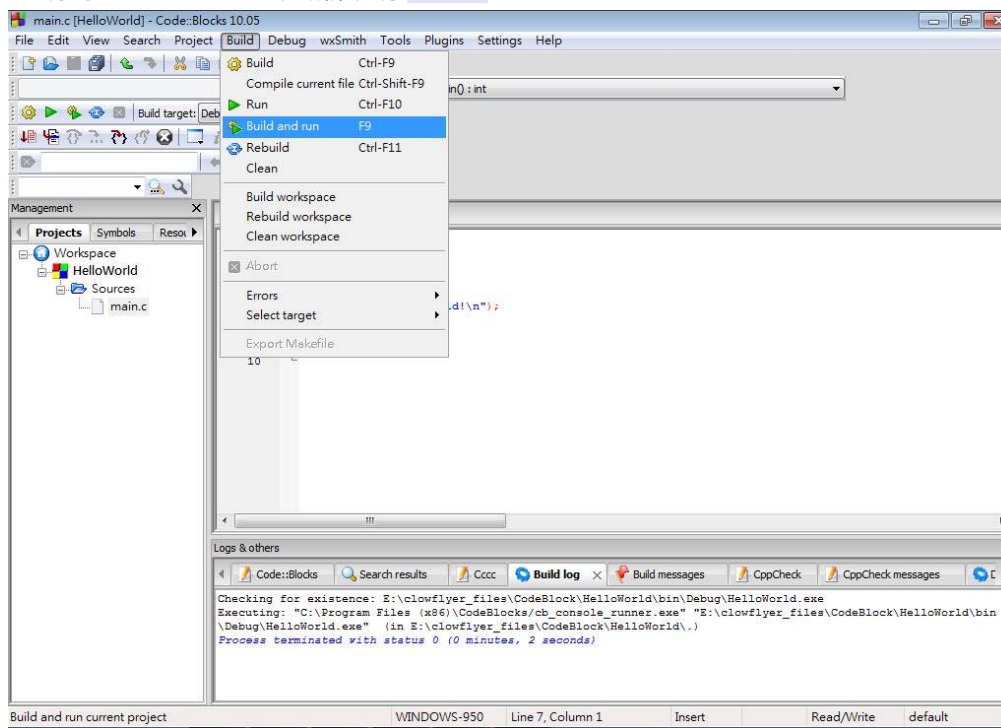
[圖] 加上一行程式碼



第一章：C 程式設計簡介 課程簡介

Code block 編寫程式範例

編寫完畢後，利用Build and Run來編譯執行



得到的執行結果如下



安裝 Pelles C

1. 進入下列網頁 <https://sites.google.com/site/nthui2p2011/programming-environment>
2. 依據下列步驟下載 Pelles C(在此示範 6.5 版本)

i. 進入下載頁面

NTHU 10010CS 135502

Search this site

I2P 2011

Home
Homework Assignments
Lecture Notes and Sample Code
Term Project
Contact Me

More Stuff

Class Announcements
Problem Sets
Reading List
Useful Links
FAQ

Programming Environment

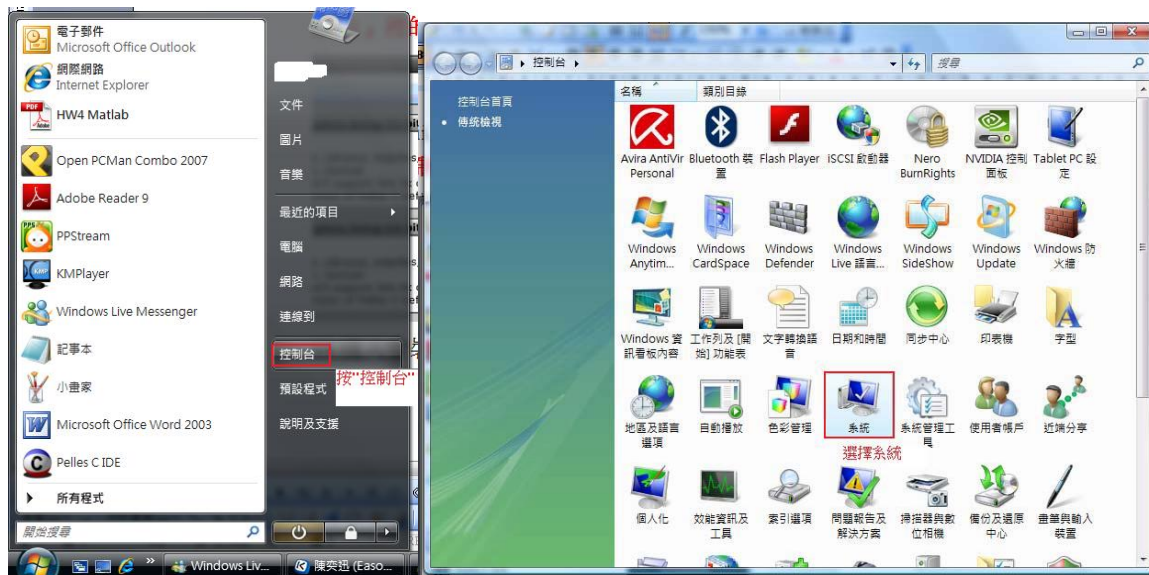
依照個人偏好，選擇其中一種安裝即可
Several options:

1. **Pelles C for Windows** (Compiler: LCC)
 - 下載 PellesC Complete Setup 32-bit 或 PellesC Complete Setup 64-bit 安裝程式 (6.5 或 6.0 版)
 - 使用方法簡介與範例程式編譯
2. **Visual C++ 2010 Express**
或是透過校務資訊系統連到計算機中心的軟體下載網頁
下載校園授權的 Visual Studio
 - 使用方法簡介與範例程式編譯 (by 湯道言)

按此連接

ii. 下載 setup.exe 並執行安裝

請依據自己的作業系統選擇 32bit or 64bit。可從『控制台』裡的『系統』得知作業系統屬於哪種

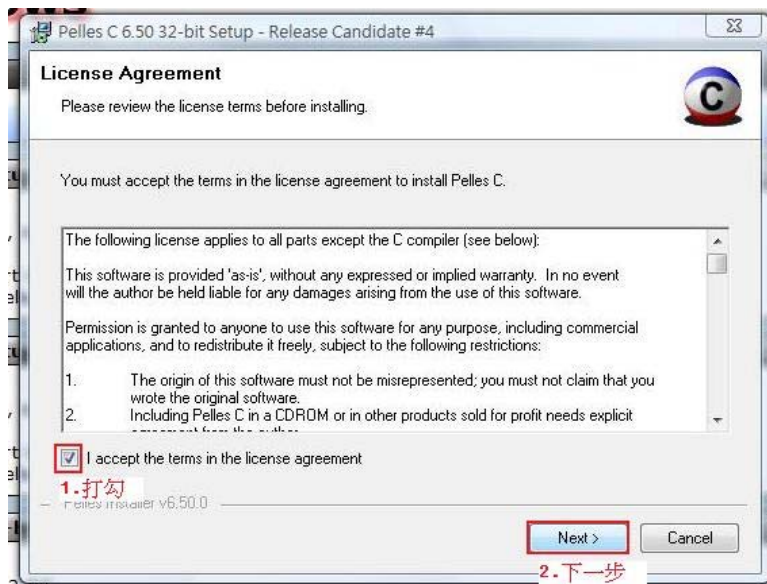


因為此作業系統為 32-bit，所以選擇下列連結。

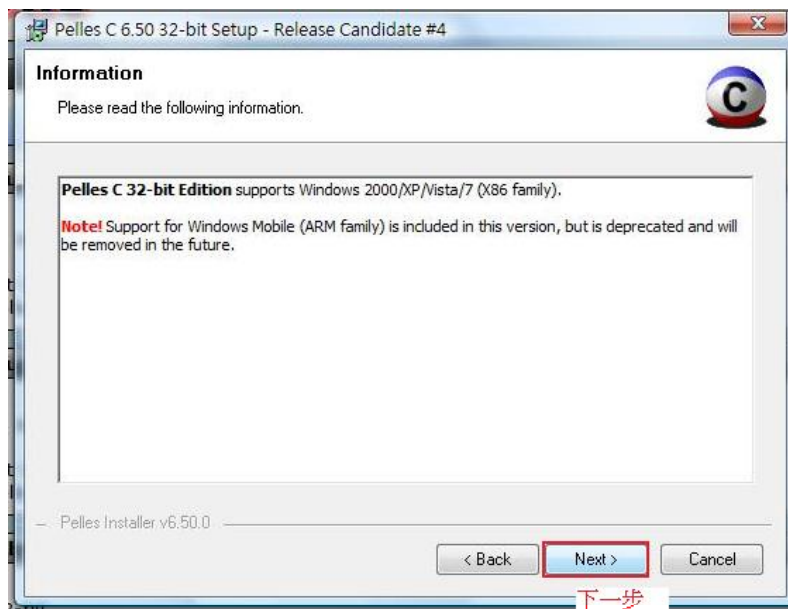
Select: Version 6.50 (BETA) ▼		
Complete Setup 32-bit RC4:	Version:	File:
ir, Librarys, Helpfiles, Examples, Registryeditor for 32-bit Windows n, German sn't support Win 9x or ME anymore, Win 9x or ME users can use version 4.50 of Pelles C. ersion of Pelles C before installing version 6.00.	6.50 (2011-04-17) Multilanguage	DOWNLOAD 點選此連結
Complete Setup 64-bit RC4:	Version:	File:
ir, Librarys, Helpfiles, Examples, Registryeditor for 64-bit Windows n, German sn't support Win 9x or ME anymore, Win 9x or ME users can use version 4.50 of Pelles C. ersion of Pelles C before installing version 6.00.	6.50 (2011-04-17) Multilanguage	DOWNLOAD (11,36 MB) 1509 downloads

iii. 進入安裝程序

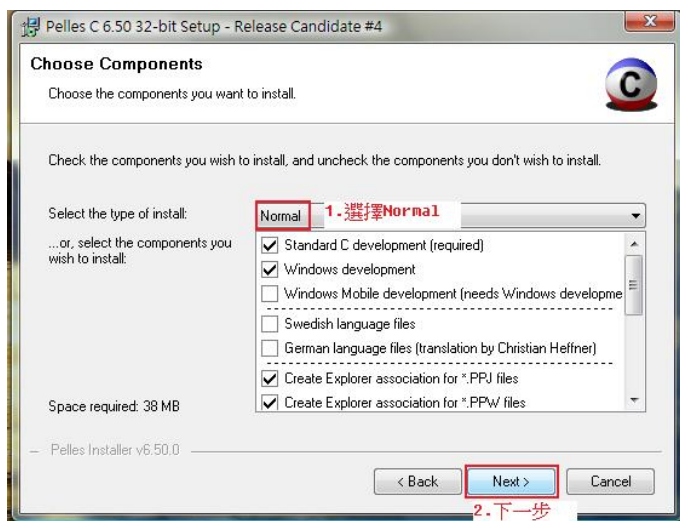
點選打勾，並按 Next



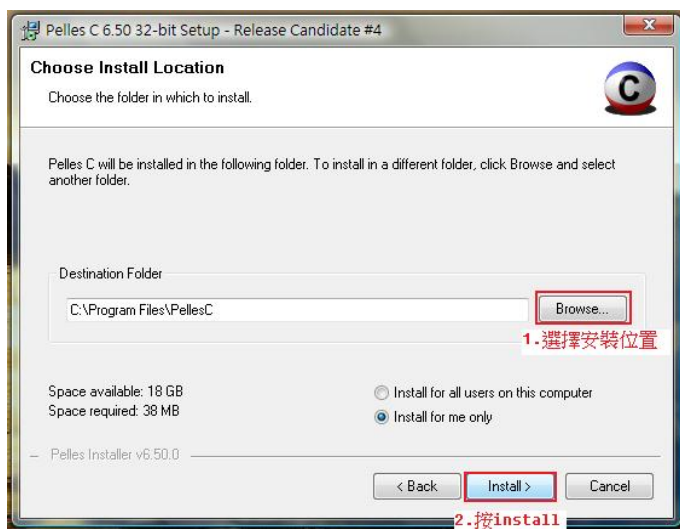
按 Next



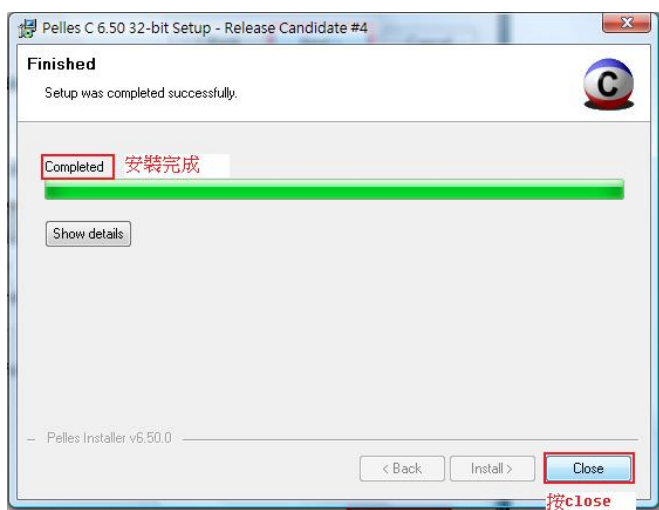
選擇組建 Normal，並按 Next



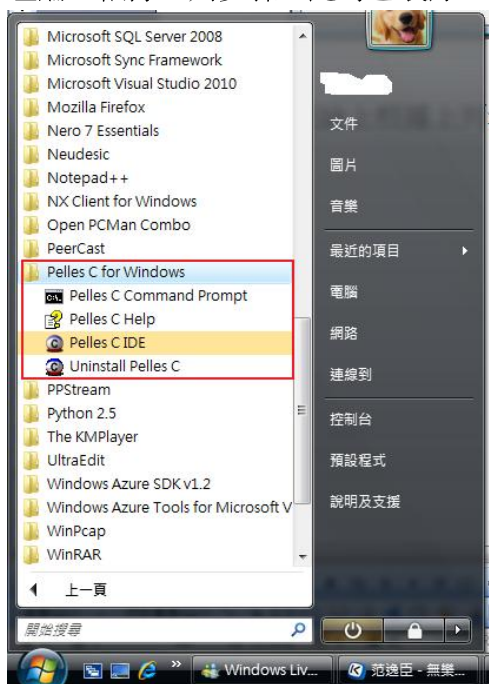
選擇安裝位置，並按 install



出現 completed 表示安裝完成。此時再按下 close

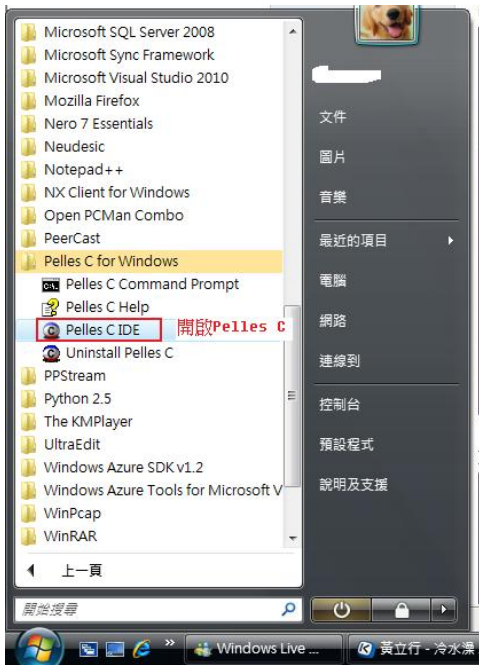


理論上根據上列步驟，此時已裝好 Pelles C。可至『開始』→『所有程式』確認是否安裝好

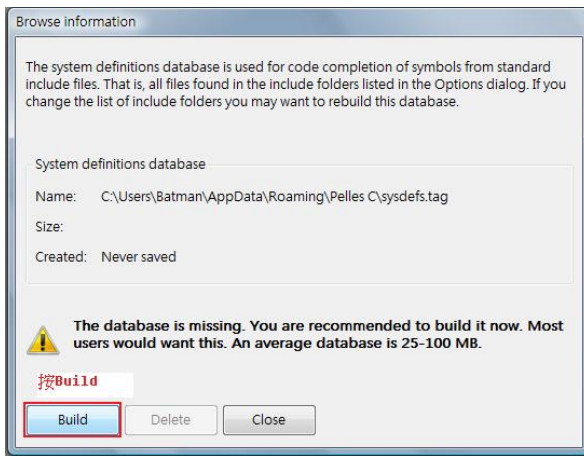


撰寫一個簡單的程式並編譯

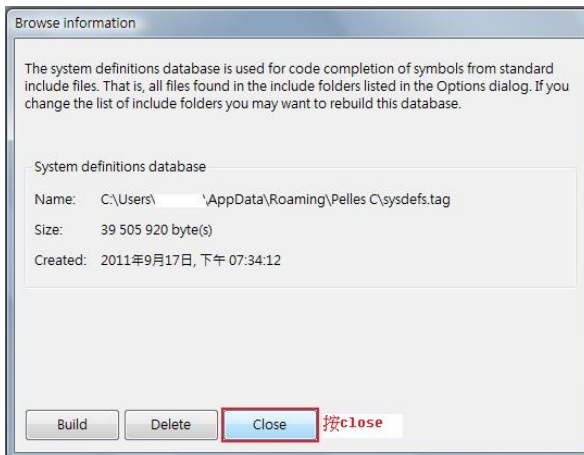
1. 從『開始』→『所有程式』→『Pelles C for Windows』→執行『Pelles C IDE』



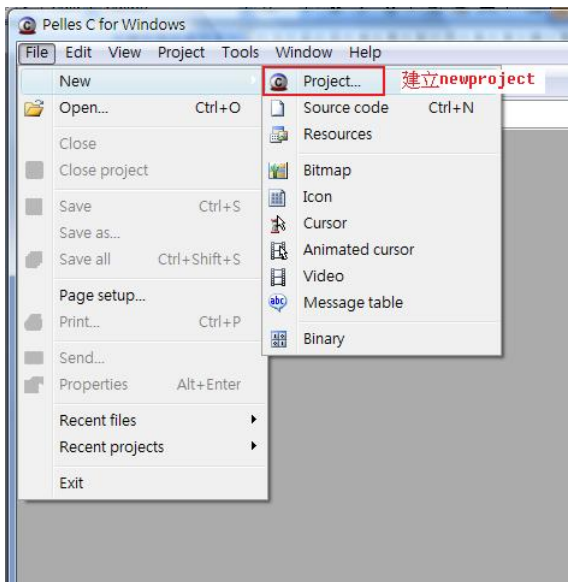
2. 此時會出現下列視窗，按下 Build 建立 database



3. database 建立完之後，按下 Close

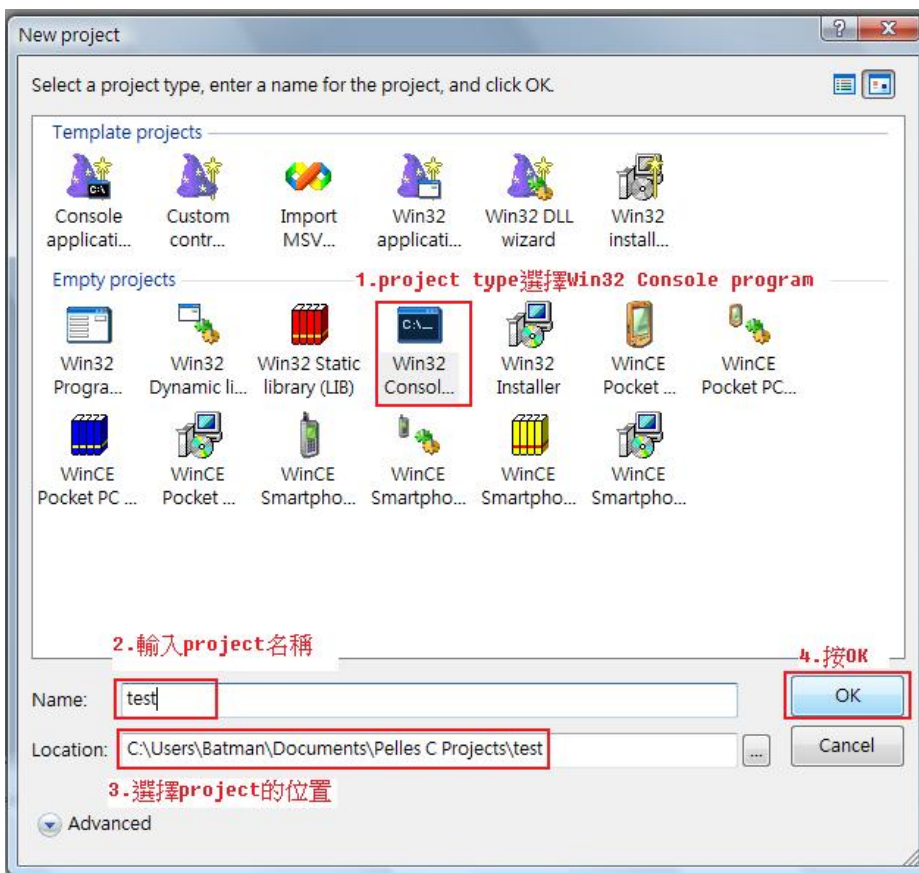


4. 建立 new project 。點選『File』→『New』→『Project...』

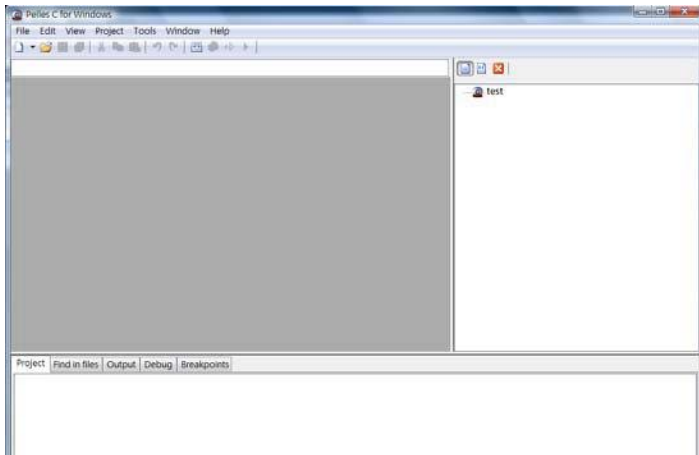


5. 出現下列視窗後

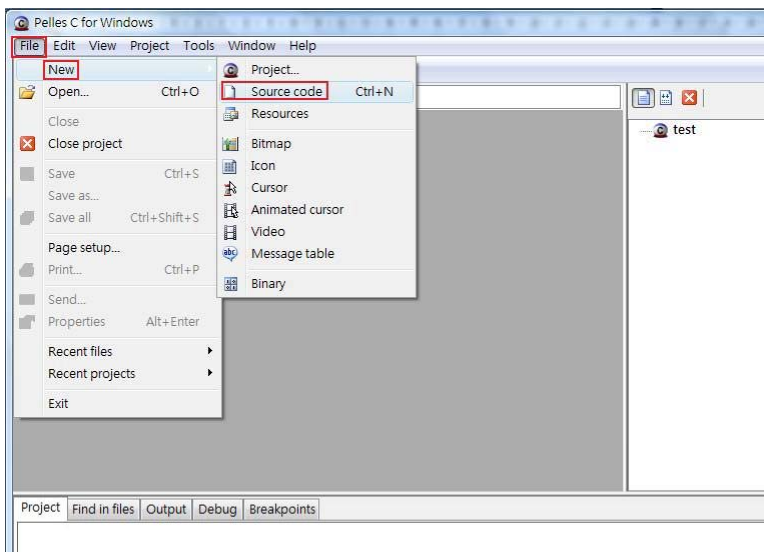
(1) 選擇 project type 『Win32 Console program』 (2) 輸入 project 名稱 (3) 選擇 project 要存在哪個位置 (4) 按下 OK



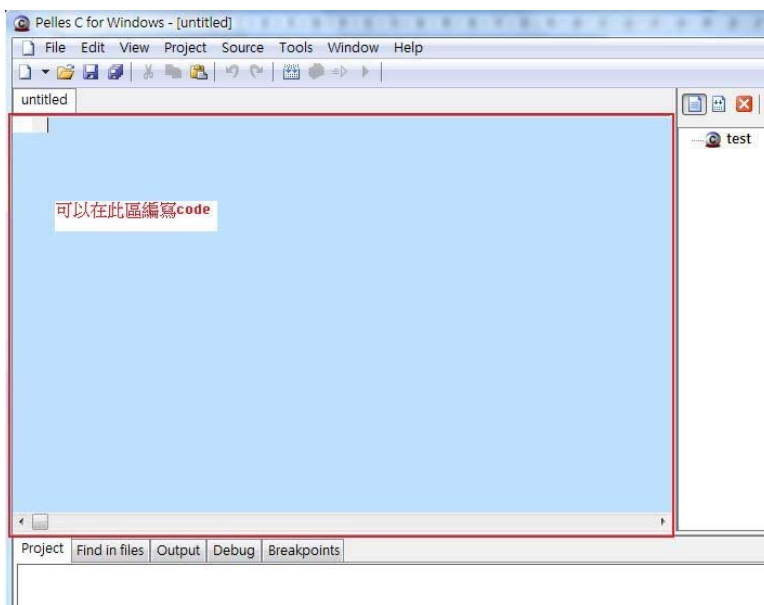
6.此時我們成功建立一個 Project。



7.接下來產生一個檔案來編寫 code。點選『File』→『New』→『Source code』

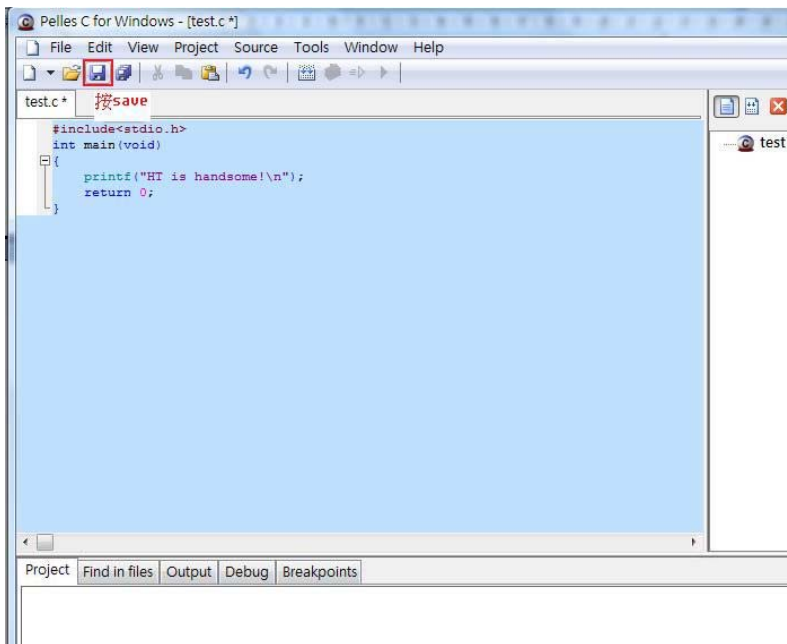


8.出現下列視窗後即可以開始編寫 code

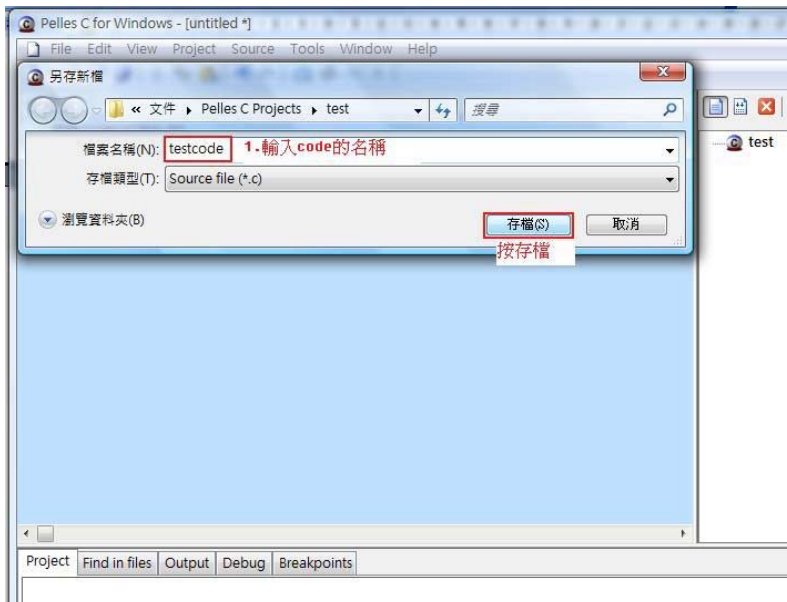


9.先不理會 code 的內容。當撰寫好 code 之後按下『儲存』
code 內容

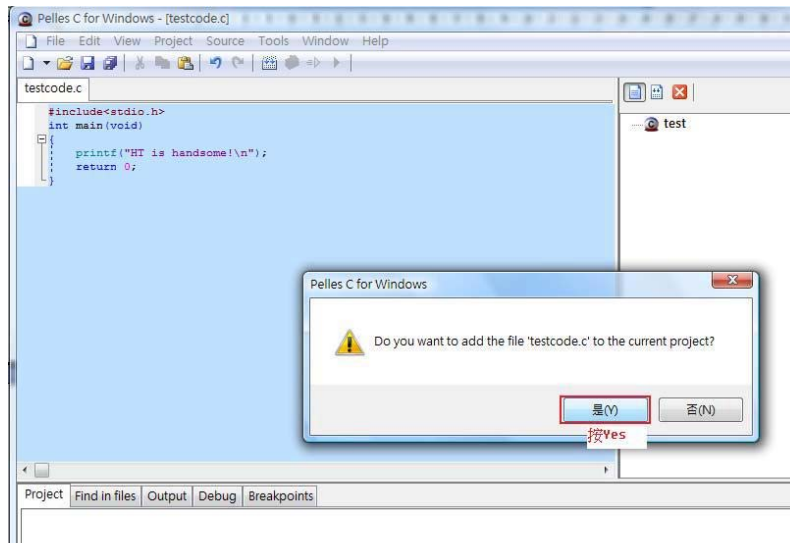
```
#include<stdio.h>
int main(void)
{
    printf("HT is handsome!\n");
    return 0;
}
```



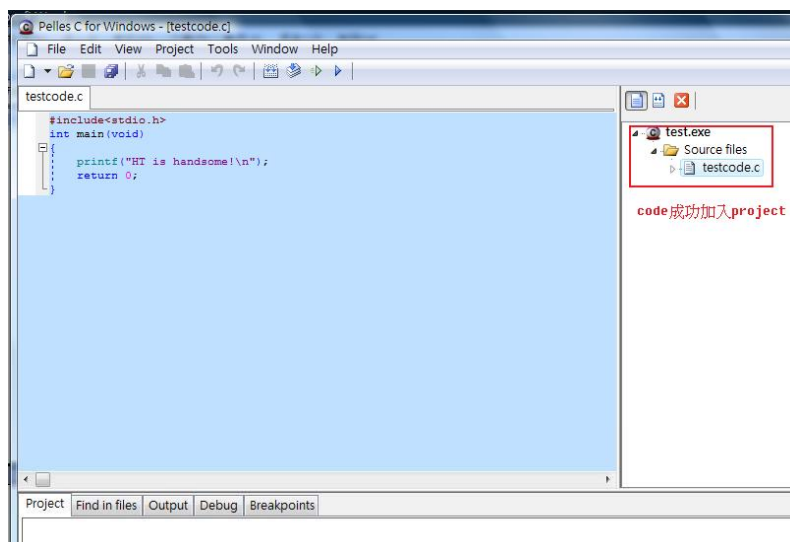
10.輸入 code 檔案的名稱，並按下存檔



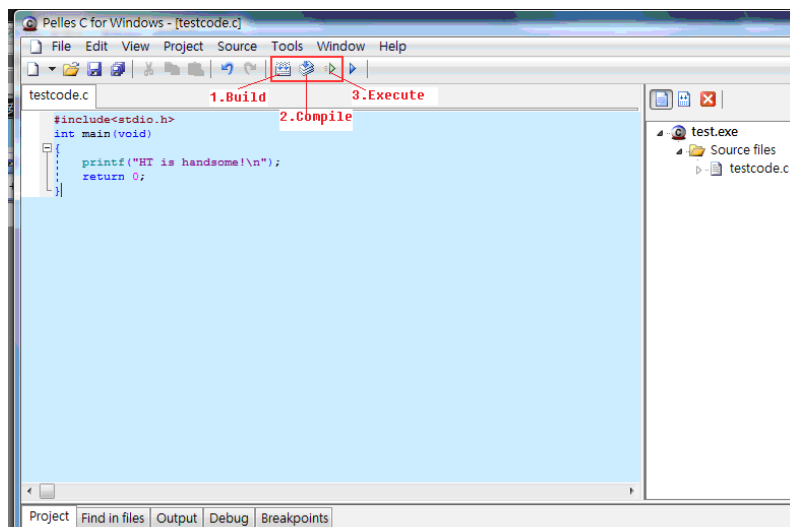
11.此時會跳出一個視窗問你是否加入 project。請選『是』



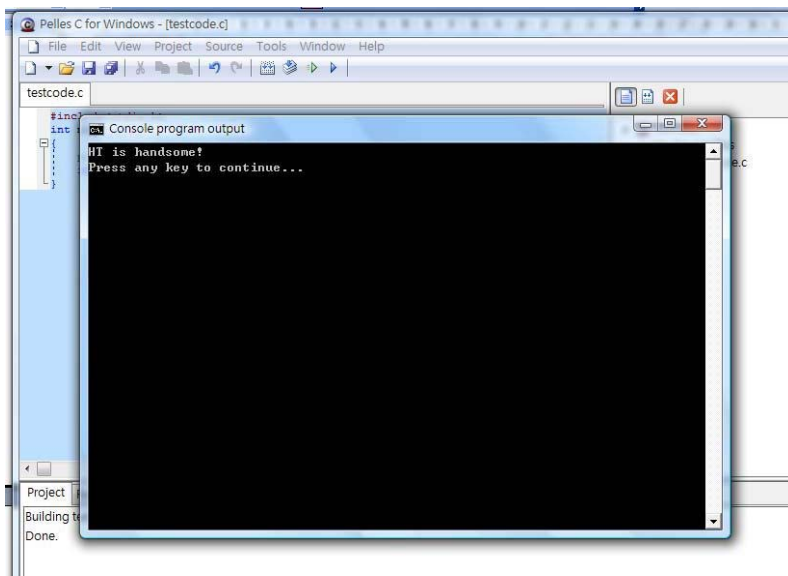
12.出現下列視窗表示 code 已經成功加入 project



13.接下來開始編譯 code。分別依照順序按下『Build』、『Compile』、『Execute』

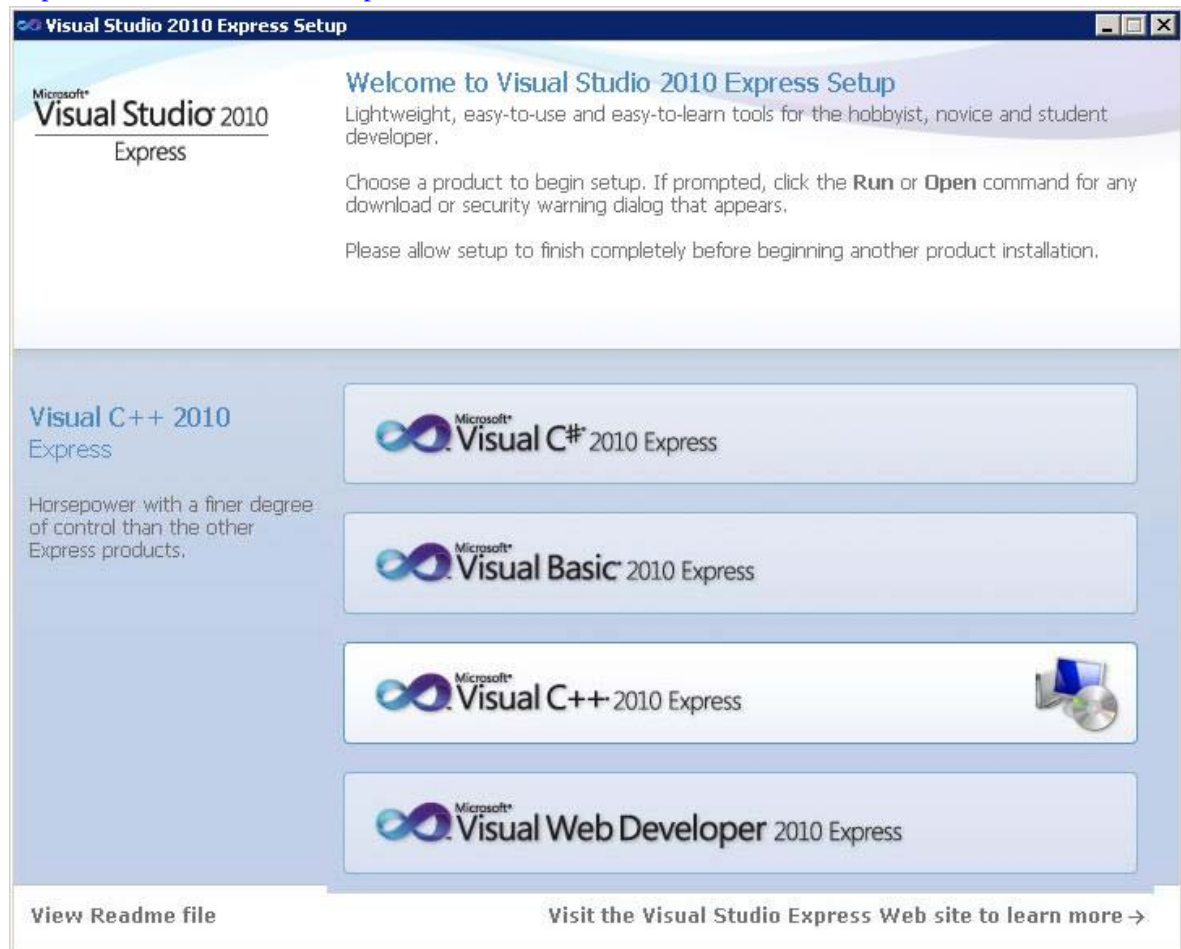


14.此時如果出現下列視窗表示你已經成功寫出一個簡單的程式!

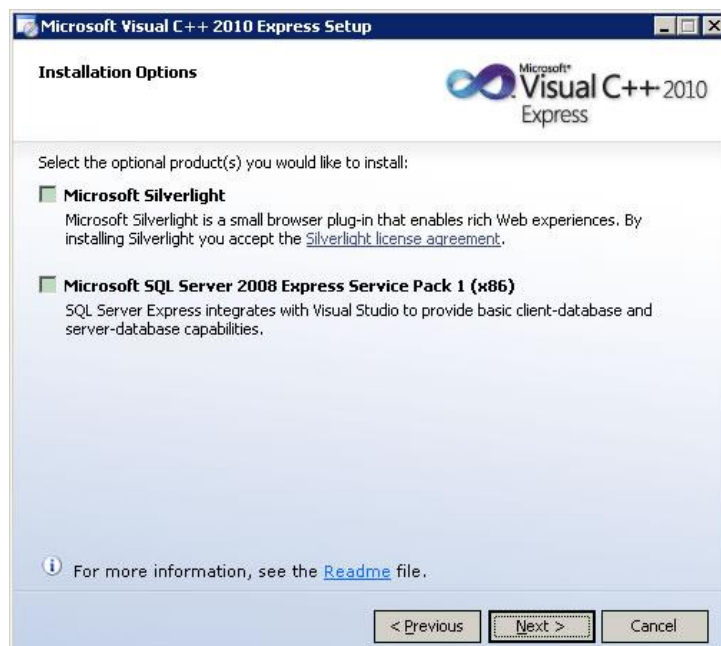


首先下載 Visual C++ 2010 Express，到官網下載離線安裝的 ISO：

<http://www.microsoft.com/express/Downloads/#2010-All>



選擇 Visual C++ 2010 安裝



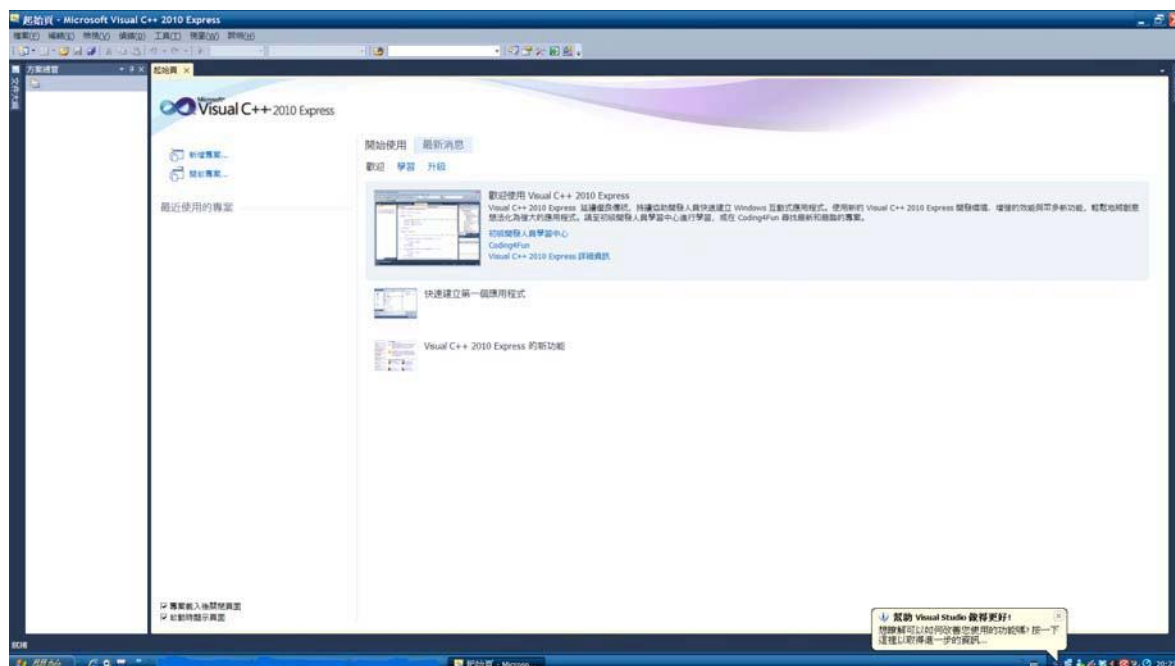
這兩個安裝可選可不選，要是覺得自己以後會用到的話就安裝了吧。接下來就照著提示一步步安裝下去，可能要花一點時間且需要重新開機。

要注意，這版的 visual studio express 並不是一開始就是免費的，還要再到官網上註冊才行，不然只是 30 天免費試用版本。



點選說明->註冊產品，需要登入 msdn 的帳號，填一些問券的東西，就可以得到產品序號。

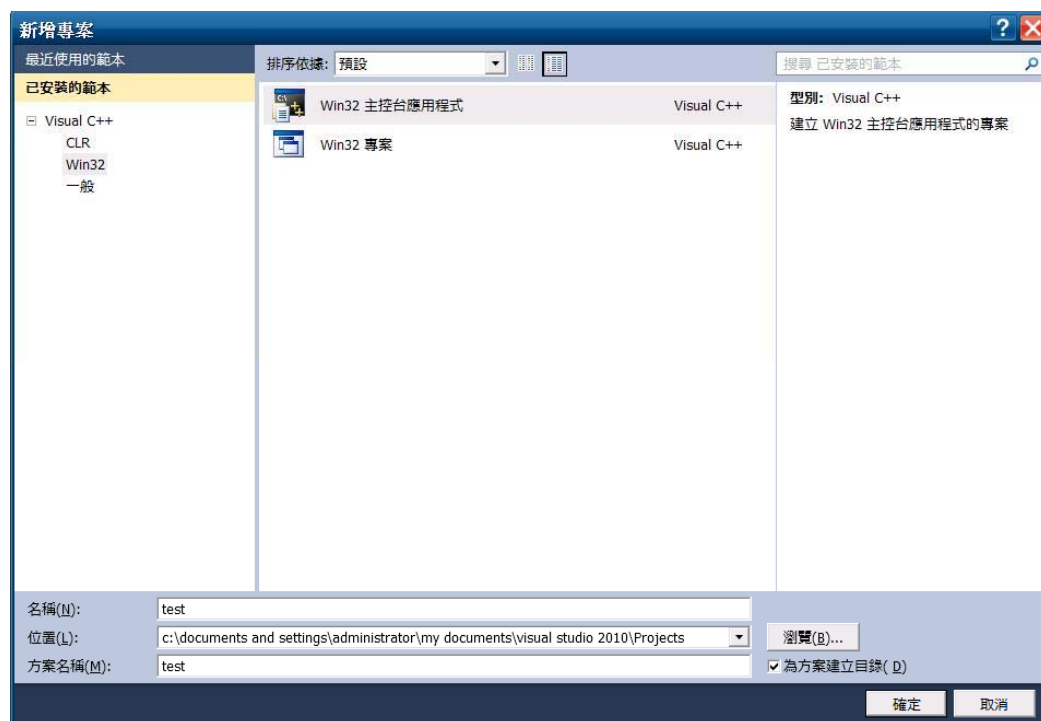
安裝完成後，讓我們試著編寫一份簡單的程式吧。首先，執行 visual c++ 2010，如下圖



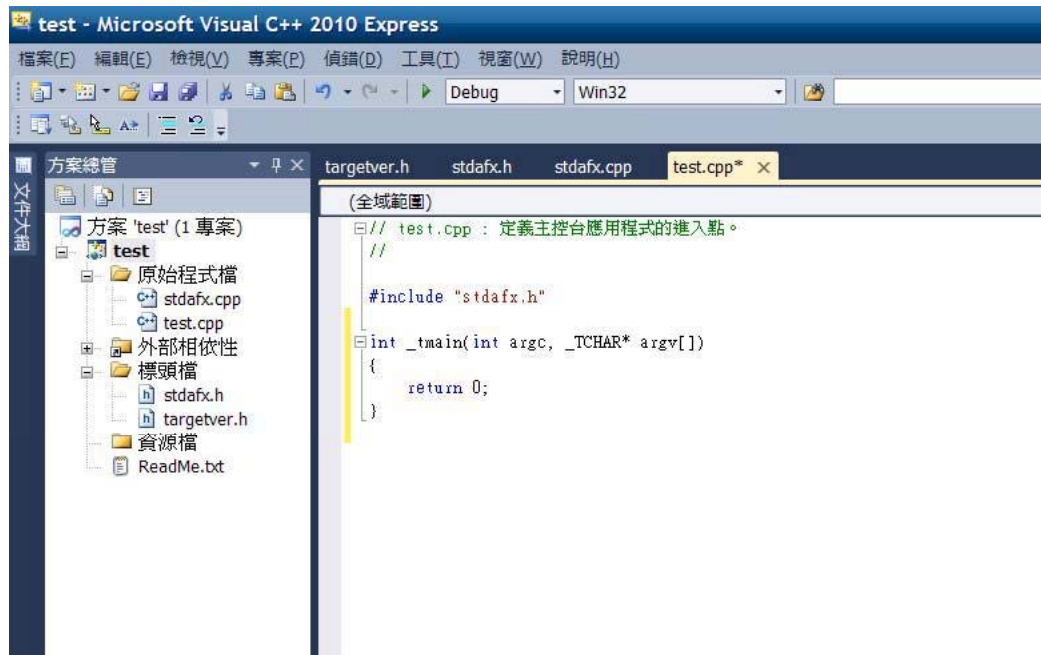
接著，點選檔案->新增->專案



這時候應該會出現下圖視窗



選擇 Win32 主控台應用程式，並記得為你的專案取一個閃亮亮的名字。然後，按下確定。



完成!!!!!!!

我們可以看到左方方案總管裡有一些自動幫我們生成的原始碼及標頭檔等，在原始程式檔底下會有一個附檔名為.cpp 的檔案，名稱會跟剛剛建專案時候的名稱一樣，範例裡就叫 test.cpp，這是主程式碼所在的檔案，可以看到它已經幫我們先寫了一些東西，我們暫時不要管它寫什麼，將檔案的內容改成下列 code

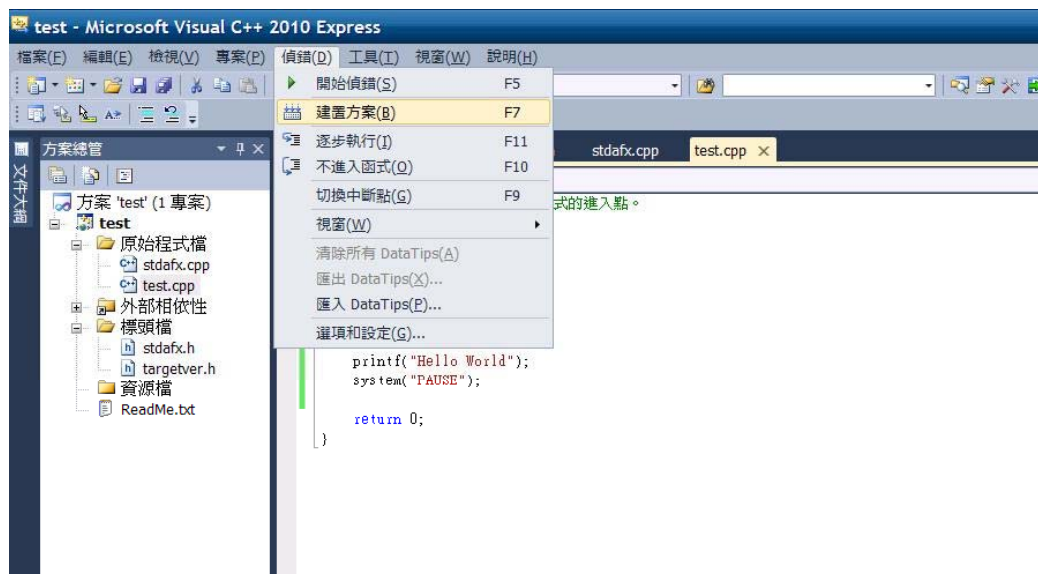
```
// test.cpp : 定義主控台應用程式的進入點。
//

#include "stdafx.h"
#include <stdio.h>
#include <stdlib.h>

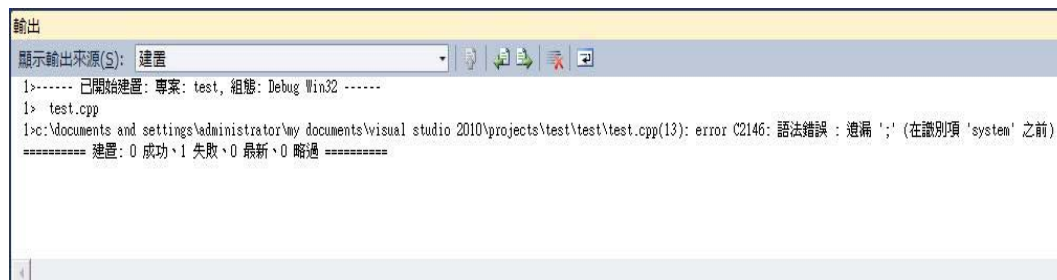
int main(void)
{
    printf("Hello World\n");
    system("PAUSE");

    return 0;
}
```

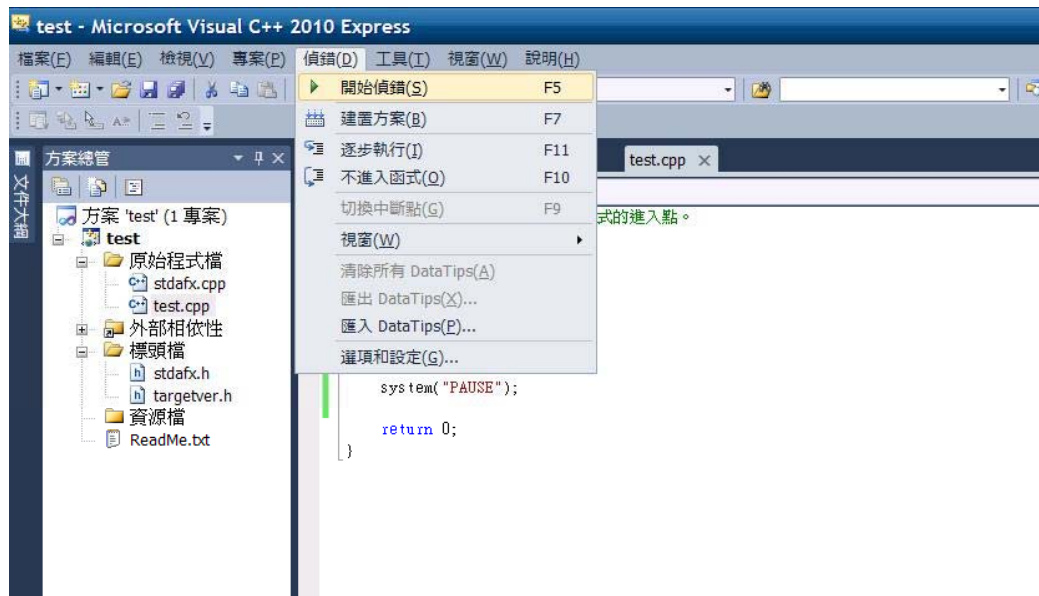
Visual C++ 當中的建置包括了編譯(compile)和連結(link)兩個動作，按下偵錯->建置方案或 F7，如下圖所示



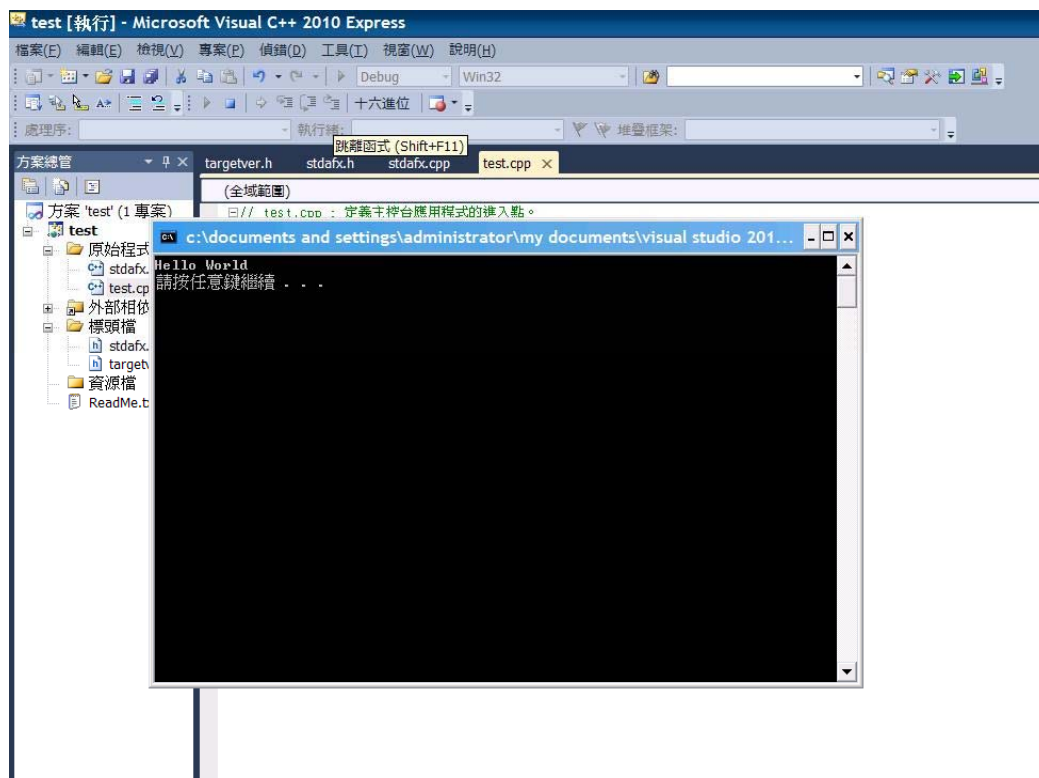
建置過程有出錯的話在下面的輸出欄位可以看到有哪些錯誤，下圖的錯誤就是少了一個分號。



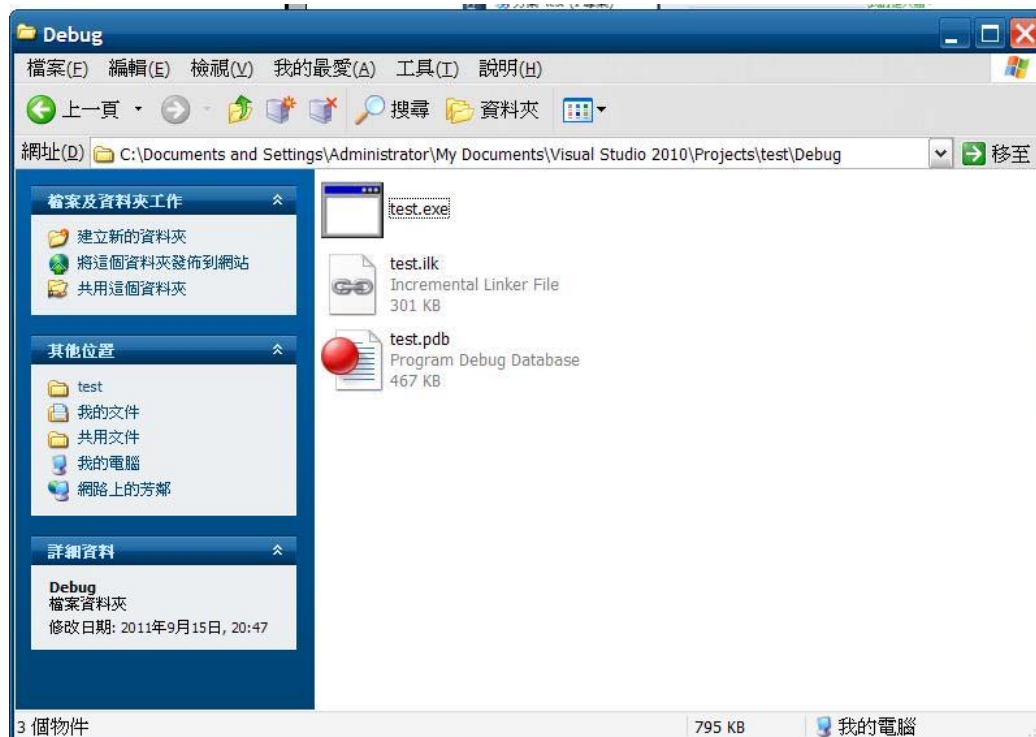
編譯完成後，我們就可以執行剛剛寫的程式了，按下偵錯->開始偵錯或 F5，visual c++就會執行我們的程式。這部分它其實會檢查你程式碼有沒有更新過，有的話它會告訴你組態已過期是否要重新建置，因此如果想要直接執行可以跳過建置步驟，直接開始偵錯。



執行成功的話就可以看到以下畫面囉



我們也可以到 visual studio 的資料夾下找一找建置出來的.exe 檔



到這邊我們已經可以使用 VC++來寫一些簡單的程式了，當然 VC++的功能絕對不只如此，更多更強大的功能就有待同學們自己去發掘了。

此外，目前我們所寫的是主控台應用程式(console application)，這是一種以終端機文字為輸出入的程式，很類似從前的 DOS 模式下的程式，以商業應用而言，這樣的程式不是主流，但是因為主控台程式架構非常的簡單，所以在從頭學習 C/C++的時候，以此為基礎程式架構可以让你專心在 C/C++語言本身的學習上，或是你日後急需一些小程式，不想要寫一大篇視窗程式碼的時候，也可以回頭來採用一下。

第二章：C 語言簡介

C 程式範例

[範例 E02_01.c]

```
#include <stdio.h>
int main(void) /* display an integer on screen */
{
    int num;
    num = 40;

    printf("The number is %d.\n", num);

    return 0;
}
```

程式要在 console mode 命令列狀態下執行。在 Windows 裡面，從桌面左下選 [開始]→[執行]，在 [開啟] 欄位裡輸入 **cmd** 按 [Enter] 鍵，就會跑出一個 DOS 視窗。再用指令 "**cd** 目錄名稱" 切換到你的執行檔的目錄，然後再打你的執行檔檔名 (譬如 **E02_01.exe**)，就可以執行你寫的程式。

這個簡單的程式會在螢幕上顯示一串文字 **The number is 40.**。這個程式包含了每個程式幾乎都會有的幾個主要結構。我們一行一行來解釋它們的作用。程式開頭第一行

```
#include <stdio.h>
```

使用 **#include** 來將 **stdio.h** 檔案的內容加到你的程式的最前面。因為 **stdio.h** 檔案裡包含了許多和輸入輸出 (I/O) 有關的東西，當我們需要用到標準的輸入輸出功能時，只要用 **#include** 的方式把 **stdio.h** 引入，就不用重打 **stdio.h** 的內容。**stdio.h** 這個檔案包含在你安裝的 C compiler 中，可以試著搜尋 Dev C++ 或 VC++ 路徑裡的 **stdio.h** 檔案，然後用 Dev C++ 或其他純文字編輯器 (例如 notepad 或 UltraEdit) 打開來看看裡面有多少內容。接下來看下一行

```
int main(void)
```

基本上 C 程式可以看成是許多 functions 的集合，每個 function 會負責某項工作，依照輸入的參數對資料作處理，然後將結果傳回。其中最重要的而且一定要包含的 function 就是 **main()**，它是你的程式的進入點，當你的程式開始執行時，首先就會執行 **main()**，然後在 **main()** 裡面再去呼叫其他 functions。我們在接下來的課程會在詳細介紹 C functions，目前只要先記得你的程式裡一定要有一個名稱是 **main()** 的 function。在 **main()** 前面的 **int** 表示 **main()** 結束後回傳值 (return value) 的型別是 **int**，也就是整數 (integer)。這個回傳值會傳回給 function 的呼叫者。因為我們是在作業系統下執行你的程式，所以在這個例子裡 **main()** 會把值回傳給作業系統。另外，在括號中的 **void** 表示這個程式的 **main()** 在呼叫時不需要傳入任何參數，我們稍後也會再介紹如何讓 **main()** 從作業系統中取得參數。

這個範例還包含了註解 **/* display an integer on screen */** 所有被包含在 **/*** 和 ***/** 之間的文字，都會被 C compiler 忽略。註解的主要作用讓我們能在程式碼片段中夾雜文字說明，幫助撰寫或修改程式的人看懂程式碼。當程式碼越寫越多，程式變得很複雜的時候，註解可以讓我們更快速了解每段程式碼的意義。

接下來會看到波浪括號 **{ ... }**，中間夾著 **main()** 的內容。波浪括號的作用相當於標示出 **main()** 的勢力範圍。當你的程式還有其他 functions 時，也是用 **{ }** 來決定每個 function 涵蓋的範圍。

再來看到的是 **int num;** 這行程式，這是所謂的變數宣告，我們要先讓 compiler 知道在 **{ }** 範圍內，將會用到一個整數 (integer) 變數 **num**。前面提過 **int** 代表 integer，**num** 則是我們自己取的變數名稱。**int** 屬於的 C keywords 之一，而每個 keyword 都有特殊的意義，所以在替變數取名時要注意不要和這些 keywords 重複。(如果想先知道有哪些 keywords，可以在網路上搜尋 "C keywords" 就會找到全部的 keywords 列表。) 在 C 裡面每個被用到的變數都必須先宣告過，如果忘了先宣告變數就直接使用，在 compile 的時候就會出錯。除了不能和已有的 keywords 重複之外，變數的取名還有一些限制。變數的名稱只能包含連續的大小寫英文字母、**_** (底線)、以及數字等三類字元，而且只能用英文字母或底線當作第一個字元，變數開頭不能用數字，例如，例如：**num_dogs**、**B612**、**iPod**，都是合法的變數名稱。變數名稱的長度最長不能超過 63 個字元，但是其實只有前 31 個字元是有效的，超過的部份會被忽略。C 語言會區分字母的大小寫，所以 **u2** 和 **U2** 代表兩個不同變數。寫程式時最好能使用固定命名規則。可以上網搜尋 "Naming Conventions (programming)"，參考慣用的規則來替變數和 function 取名。最後要特別注意的是，整個敘述句後面要用分號 **;** 來結尾，不能省略。剛開始寫程式時很容易因為忘記加分號而在 compile 時得到錯誤訊息。

num = 40; 這個敘述是所謂的 assignment statement，用來把變數 **num** 的值設定成 **40**。因為已經宣告過 **num** 是個整數變數，在記憶體會有個位置存放它的值。等號的意義就是把右邊的值放到左邊的變數裡。結尾同樣要有分號。

printf("The number is %d.\n", num); 這個敘述句呼叫了 **printf()** 這個 function。在剛開始學 C 的階段，**printf()** 會是我們最常用到的 function 之一，它的作用是把文字顯示在螢幕上。我們必須把想要顯示的字串當作參數傳給 **printf()**，在這個例子裡，我們傳了兩個參數給 **printf()**，兩個參數用逗號隔開。第一個參數是 **"The number is %d.\n"**，第二個參數則是 **num**。然後 **printf()** 會把這兩個參數做適當的結合，把第一個參數裡的 **%d** 用 **num** 變數目前的值 (**40**) 取代，再把結果顯示在螢幕上。

第二章：C 語言簡介

第一個字串參數裡還有一個奇怪的東西 `\n`，它代表換行字元，你可以試著把它去掉看看程式執行結果會有什麼不同。另外可以試試看如果把那一行打成下面這種形式 (在程式碼中直接按 Enter 鍵換行) 會發生什麼結果：

```
printf("The number is %d.\n", num);
```

在 compile 的時候應該會出現錯誤訊息。由此可見，`\n` 是為了描述一些難以輸入的字元而設計的變通表示法 (稱作 escape sequence)，類似的字元還有 `\t` 表示 tab 空格，`\b` 表示 backspace，`\"` 表示雙引號，`\\` 表示反斜線。我們再進一步解釋 `%d` 這個用法，它的作用是告訴 `printf()` 這個地方將會顯示一個整數，而且要用十進位 (decimal) 方式顯示。這樣的用法稱作格式化輸出，`printf()` 還有更多格式化輸出的選擇，我們會在稍後的例子裡一一介紹。

最後來到了 `return 0`；這個敘述句。我們前面說過 `main()` 需要回傳一個整數給作業系統，這也是 `main()` 前面的 `int` 所代表的意義。既然已經指定要回傳整數，所以在程式結束時要用 `return` 這個 keyword 來指出我們要回傳的值是多少。在這個例子回傳值是 `0`，至於它所代表的意義我們後續會再做介紹，目前暫時先忍耐一下，容許你的腦中有一些不明確的東西存在。

到目前為止，我們已經學會了最基本的 C 程式結構：

```
#include <stdio.h>
int main(void)
{
    <statements>
    return 0;
}
```

在我們前幾週的課程中會不斷地用到類似的結構。剛開始學寫程式，都有一些固定的模式可以套用，只要照著把固定的程式碼輸入就可以，真正需要變動 (設計) 的部份是 `<statements>` 那一段，這部份就要依照我們想達到的功能，填入適當的程式碼。

[範例 E02_02.c]

```
#include <stdio.h>
int main(void) /* display an integer on screen */
{
    int feet;
    int inches;
    int centimeters;

    feet = 6;
    inches = 3;
    centimeters = (feet*12 + inches) * 2.54;

    printf("%d feet %d inches = %d centimeters.\n", feet, inches, centimeters);

    return 0;
}
```

變數的數目增加成三個，一樣都是整數變數。這個範例還多了乘法和加法運算，把英制呎吋換算成公分。變數宣告的部份，由於三個變數都是整數，也可以用另一種寫法，把它們合在一起宣告，看起來稍微簡潔一點：

```
int feet, inches, centimeters;
```

要注意變數之間是用逗號隔開，最後再用分號結束。

主程式 `<statements>` 的部份，首先是設定 `feet` 和 `inches` 的值，然後再依照換算公式 (星號 `*` 代表乘法運算)，一呎等於 12 吋，一吋等於 2.54 公分，把運算的結果設定給 `centimeters`，所以 `centimeters` 就會記錄著換算出來的公分是多少，最後再呼叫 `printf()` 把結果顯示在螢幕上。這次為了顯示三個變數值，總共需要傳四個參數給 `printf()`，第一個是描述顯示格式的字串，裡面包含了三個 `%d`，分別對應到後面三個參數 `feet`，`inches`，`centimeters`，到時候這三個變數所儲存的值就會顯示在對應的位置，而且都是以十進位方式顯示。接下來我們可以更改 `feet` 和 `inches` 的值，但是要重新 compile 產生新的執行檔，再執行才會得到新的換算結果。

第二章：C 語言簡介

[練習 W02_01] 在螢幕上顯示出下面的圖案：

```

**_**
-_*_-
**_**

```

這個題目只要用 **printf()** 就可以做到，不需要宣告變數。

[練習 W02_02] 宣告兩個整數變數分別代表兩個正方形的邊長，計算第一個正方形和第二個正方形的面積差，然後把答案顯示在螢幕上。這個練習只要修改範例 E02_02.c 就可以做到。

我們再來替程式加一些變化，看看如何在 **main()** 之外再增加一個自己寫的 function。

[範例 E02_03.c]

```

#include <stdio.h>
void strike(void); /* function prototyping */
void ball(void);
int main(void)
{
    strike();
    strike();
    ball();
    strike();

    return 0;
}

void strike(void) /* start of function definition */
{
    printf("Strike!\n");
}

void ball(void) /* start of function definition */
{
    printf("Ball!\n");
}

```

這個程式多了兩個 functions，分別是 **strike()** 和 **ball()**，如果對照 **main()**，應該會發現這兩個 functions 不僅不需要傳入參數（因為參數的位置寫的是 **void**），也不需要回傳任何東西（回傳的資料型別也是 **void**）。程式最前面出現的 **void strike(void);** 稱作 prototype，它的作用是讓 compiler 先知道 **strike()** 這個 function 要如何使用。它真正被用到的地方是在 **main()** 裡面，其中 **strike()** 被呼叫了三次，而 **ball()** 被呼叫了一次。呼叫的方式是 **strike();** 因為沒有參數，所以括號裡面什麼都不用寫（但是不能把括號省略，因為這樣才知道它是個 function）。每次呼叫都會去執行 function 裡面所定義的動作。程式下半部就是 **strike()** 和 **ball()** 這兩個 functions 的定義，它們都只是做簡單的 **printf()** 顯示一些文字到螢幕上。這兩個 functions 都沒有像 **main()** 最後有一行 **return**，因為它們的回傳值都定成 **void**，表示不需要回傳任何資訊。把一些會反覆使用的動作寫成 function，可以讓你的程式更有結構。就像組合積木一樣，把一塊塊具有常用的基本功能的元件組合在一起，就可以做到更複雜的功能。除了我們自己寫的 function 之外，程式還呼叫了 **printf()**。而 **printf()** 的 prototype 其實是寫在 **stdio.h** 裡面，再靠 **#include** 貼到程式的最前面。至於 **printf()** 的定義（也就是 function 的主體）則是寫在另外的程式碼裡，而且被預先 compile 過，放進 library 中，當我們 compile 這個範例程式的時候，就會去和 library 連結起來，這樣一來，當程式執行到了 **printf("Strike!\n");** 的時候，電腦就知道該做什麼事以及該如何做了。

[練習 W02_03] 利用練習 W02_01 寫出來的程式，把它改寫成另一個 function，取名為 **draw_x()**，然後在 **main()** 裡面呼叫兩次，產生下面的輸出結果：

```

**_**
-_*_-
**_**

**_**
-_*_-
**_**

```

這個練習主要靠修改範例 E02_03.c，然後配合練習 W02_01 就可以做到題目要求的效果。

第二章：C 語言簡介



程式除錯 (Debug)

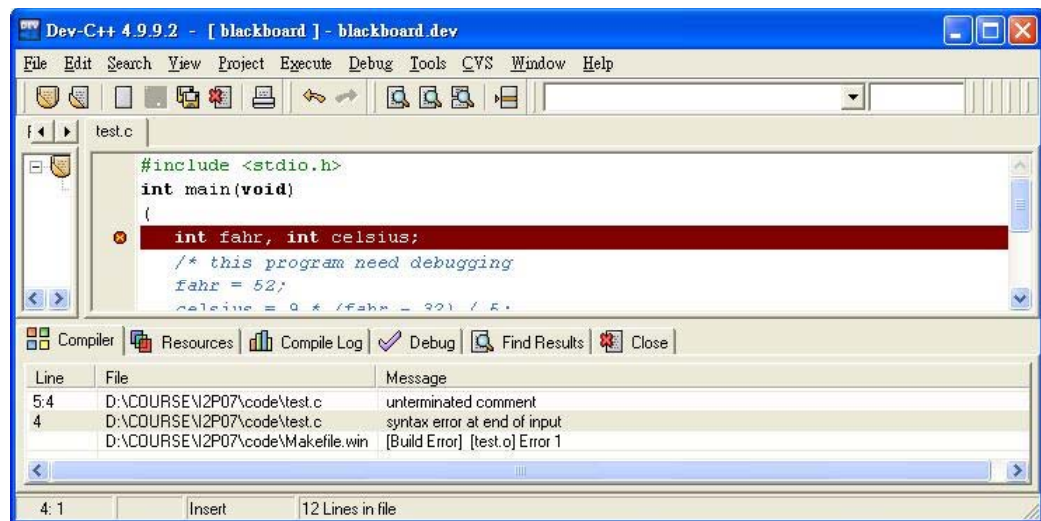
通常大家都會把程式裡的錯誤叫做 bug，而找出錯誤的過程就稱為 debugging。程式除錯是一項很重要的技巧，再厲害的程式設計者也不可能每次都能一次就寫出完全正確的程式，免不了會有些小錯誤或盲點。如果對於除錯很熟練，就可以很快找出錯誤而把程式寫完，而且可以避免程式在未來的執行過程中當掉。我們用一個錯誤的程式來解釋如何做最基本的除錯。底下這個程式做的事情是把華氏溫度換算成攝氏溫度，其中斜線符號 / 代表除法運算。

```
#include <stdio.h>
int main(void)
(
    int fahr, int celsius;
    /* this program need debugging
    fahr = 52;
    celsius = 9 * (fahr - 32) / 5;
    printf(" %d degrees Fahrenheit = %d degrees Celsius.\n", Fahr)

    return 0;
)
```

程式 compile 之後會出現像下圖的錯誤訊息：

[圖一] compile 失敗，得到錯誤訊息。



基本上 compiler 會幫你找出 "語法錯誤" (syntax errors)，以上面的例子來說，`main()` 的定義應該用波浪括號，變數宣告 `int fahr, int celsius;` 應該寫成

```
int fahr, celsius;
```

或是

```
int fahr;
int celsius;
```

註解 `/*` 少了對應的另一邊 `*/`，因此變成沒有結束。再來是 `printf()` 最後忘了加分號。還有在 `printf()` 裡面，變數 `fahr` 打成了 `Fahr` (大小寫有別)。這些錯誤改了之後，compile 就不會再產生錯誤訊息，能夠順利產生執行檔。

```
#include <stdio.h>
int main(void)
{
    int fahr, celsius;
    /* this program need debugging */
    fahr = 52;
    celsius = 9 * (fahr - 32) / 5;
    printf(" %d degrees Fahrenheit = %d degrees Celsius.\n", fahr);

    return 0;
}
```

第二章：C 語言簡介



程式除錯 (Debug)

還別高興得太早。在執行了修改過的程式之後，會發現顯示出來的結果怪怪的，出現類似下面的結果

52 degrees Fahrenheit = 4009752 degrees Celsius.

再看一下程式碼 `printf()` 的地方，發現原來問題出在 `%d` 出現了兩次，但是對應的參數只有 `fahr`。應該還要傳另一個參數 `celsius` 才符合程式想要顯示的結果。再改成

```
printf(" %d degrees Fahrenheit = %d degrees Celsius.\n", fahr, celsius);
```

結果變成

52 degrees Fahrenheit = 36 degrees Celsius.

還是有點問題。再仔細檢查一下，原來是公式代錯了，應該是

```
celsius = 5 * (fahr - 32) / 9;
```

修改過後再 compile 並執行，終於可以顯示出想要的結果：

52 degrees Fahrenheit = 11 degrees Celsius.

整個除錯過程大致就像這樣，逐一把錯誤找出來，重新 compile 看看還有沒有錯誤訊息，如果可以順利 compile，就執行程式檢查輸出結果是否正確。前面提過，compiler 可以幫你檢查語法錯誤，但是當你順利 compile 而得到執行檔，接下來的執行結果的錯誤 compiler 就幫不上忙，因為程式的語法完全正確，只是得到的不是我們要的結果，而 compiler 還沒聰明到能夠理解我們設計這個程式的意圖。像這樣的錯誤稱作 "語意錯誤" (semantic errors)，平常說話也可能犯類似的錯誤 (譬如："整件事有了三百六十度的轉變")。程式中的語意錯誤通常不是很明顯，這也是除錯最棘手的地方，有時候甚至連執行結果看起來都沒問題，但是卻在執行到了某些特殊狀況才發現程式還是有錯。

我們安裝的程式開發工具除了有 compiler 之外，還會包含除錯器 (debugger)，我們可以用它來找出程式的問題出在哪裡，所以希望大家盡快熟悉如何使用 debugger。我們用底下的例子來示範如何用 debugger 來 trace 程式的進行和變數的狀態。

[範例 E02_04.c]

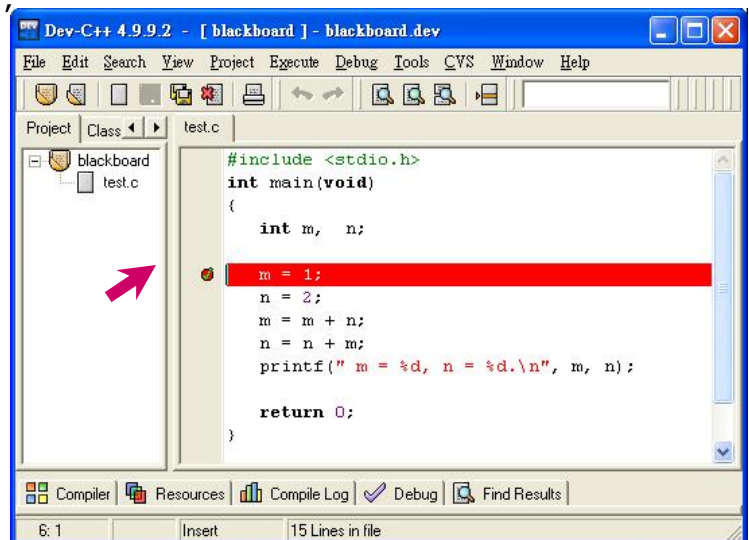
```
#include <stdio.h>
int main(void)
{
    int m, n;

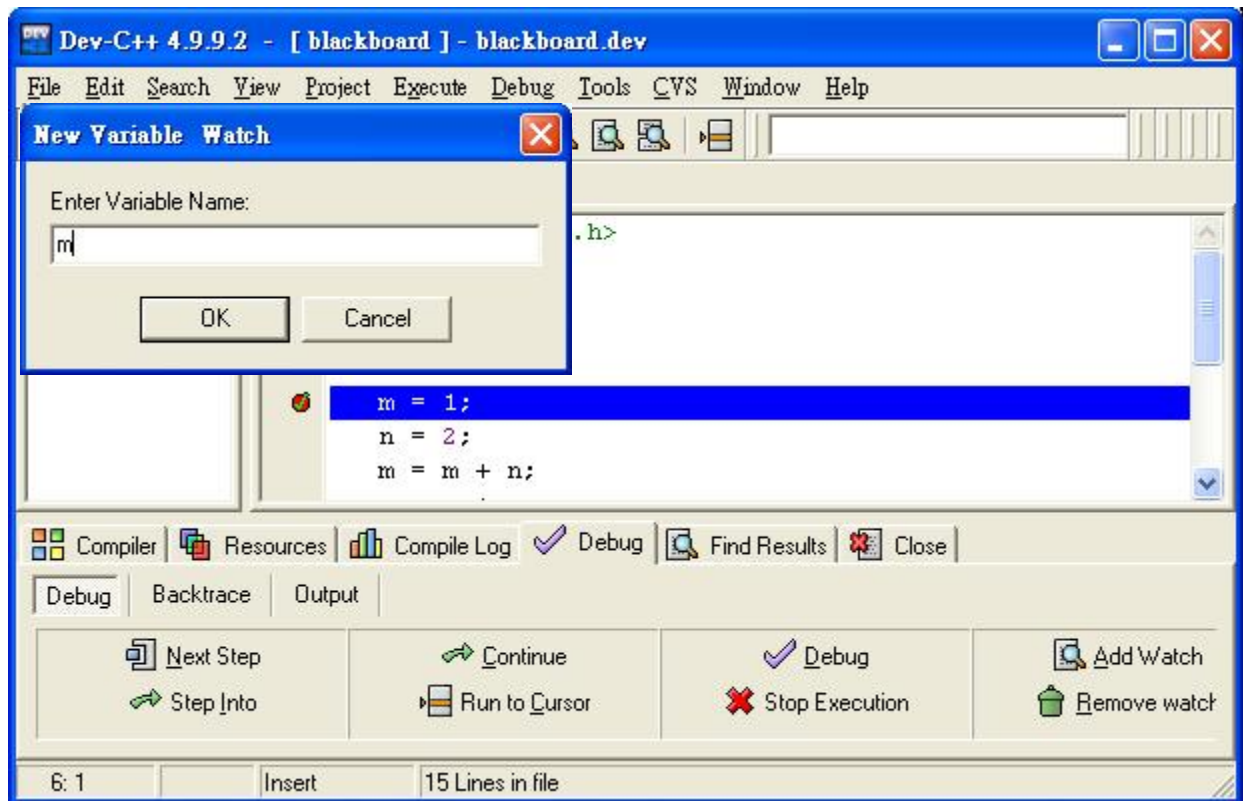
    m = 1;
    n = 2;
    m = m + n;
    n = n + m;
    printf(" m = %d, n = %d.\n", m, n);

    return 0;
}
```

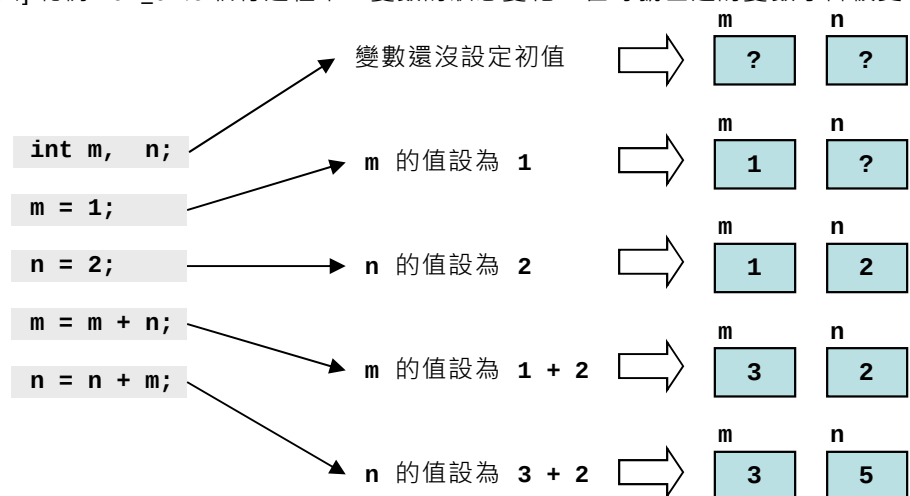
[圖二] 放置一個中斷點。

先用滑鼠在箭頭指的地方按一下，這樣就會做 toggle breakpoint 的動作，在那一行程式設定一個中斷點，接著從 Menu 裡選 [Debug]→[Debug]，然後按 Add Watch 把 `m` 加入觀察名單中，同樣也把 `n` 加入觀察 (如圖三)。接下來按 [Next Step] 就可以單步執行程式，然後觀察變數的值有什麼變化。按 [Next Step] 執行完 `m = 1;` 這一行之後，可以看到 Watch 視窗裡 `m` 的值變成 1。執行完 `n = 2;` 之後 `n` 的值就變成 2。再來執行了 `m = m + n;` 之後，就會計算出 `m + n` 是多少，然後把這個值存回 `m`，所以 `m` 的值被改成 3。同理當執行完 `n = n + m;` 之後，`n` 的值變成 5。整個執行的過程，變數的狀態變化可以用圖四來表示。



[圖三] 把變數 *m* 加入觀察名單。

[圖四] 範例 E02_04.c 執行過程中，變數的狀態變化。在等號左邊的變數才會被更改。



[練習 W02_04] 下面的程式會在螢幕上顯示出什麼？試著用 debugger 追蹤變數的狀態。

```

#include <stdio.h>
int main(void)
{
    int m;

    m = 1;
    printf("m = %d\n", m + 1);
    m = m + 1;
    printf("m = %d\n", m + 1);

    return 0;
}

```

第三章：讀取和輸出各種型別的資料

輸入數值

[範例 E03_01.c]

```
#include <stdio.h>
int main(void) /* calculate earned run average */
{
    float er, ip, era;

    printf("How many earned runs did you give up?\n");
    scanf("%f", &er);
    printf("How many innings did you pitch?\n");
    scanf("%f", &ip);

    era = 9.0 * er / ip;
    printf("ERA = %.2f\n", era);

    return 0;
}
```

這個簡單的程式會讓使用者輸入自責分和投球局數，然後算出防禦率並顯示在螢幕上。在解釋程式的細節之前，先來看看這個程式裡有哪些新東西。首先是變數宣告的地方，出現了 **float** 這個沒看過的型別 (之前是用 **int**)。再來是除了 **printf()** 之外還出現了 **scanf()** 這個新的 function。另外 **printf()** 的字串參數裡還出現 **%.2f** 的格式。

我們就來一一解釋這些新東西。首先是 **float** 型別，為了計算的精確度，可以把變數宣告成 **float** (浮點數)。在這個例子中，由於我們要計算的數值包含小數點後的位數，所以不適合再用 **int** 整數型別來儲存數值，而要用能夠表示更精細的數值的浮點數型別變數。整數和浮點數儲存數值的方式不一樣，我們會再做更詳細的介紹，目前暫時先記得，處理整數的資料，就用 **int** 變數；處理有小數點的資料，就用 **float**。

程式接下來用 **printf()** 顯示訊息，指示使用者輸入自責分是多少，然後用 **scanf()** 來讀取使用者鍵盤輸入的數值。我們先前看過的例子都沒讓使用者自己輸入變數的值，變數值都在程式碼裡面設定，所以每次更改數值，都要重新 compile 程式再執行，才能得到新的結果，這是很不方便的操作方式。所以我們要讓程式能在執行中讀取使用者想要設定的變數值，多了這樣的互動功能，才比較符合一般的使用需求。我們已經學過，要讓 C 程式輸出資訊可以用 **printf()**，而要讓 C 程式讀取輸入的資料則可以用 **scanf()**，它的使用方法和 **printf()** 類似，都要先用第一個參數指定格式，然後剩下的參數就是對應的變數。不一樣的地方是變數名稱前面多了 **&** 符號。所以這一行程式碼

```
scanf("%f", &er);
```

就會把使用者輸入的數值存入變數 **er** 中。讀取的格式為 **float**，因此用 **%f** 告訴 **scanf()** 要讀取的資料的格式是浮點數。至於為什麼 **scanf()** 傳參數的時候要多一個 **&**？底下用一個簡單的比喻來解釋：

如果把變數想成一個盒子，則變數名稱 (譬如 **er**) 就是寫在盒子上的標籤，用來和其他盒子區別，而盒子裡的東西就是那個變數所儲存的數值。當我們要將變數當作參數傳給某個 function (譬如 **printf()**) 的時候，C 語言的作法就是把盒子裡的東西拿出來交給那個 function。但是對於像是 **scanf()** 這樣功能的 function 來說，它要的並不是盒子裡的東西，而是要整個盒子，因為它要把使用者輸入的數值放進盒子裡。要達到把整個盒子當參數 (而不是把盒子裡的東西當參數) 的這個目的，作法就是在參數變數前加上 **&**，告訴 compiler 要把整個盒子傳給 **scanf()**。初學者最常犯的錯誤就是在使用 **scanf()** 讀取整數或浮點數變數的時候，忘了在變數前面加上 **&**。

接下來的數值運算 **era = 9.0 * er / ip;** 就是依照我們要的計算公式把答案算出來存放在變數 **era** 裡。其中 ***** 代表乘法運算，而 **/** 代表除法運算。最後再把結果用 **printf()** 顯示在螢幕上。

[練習 W03_01] 讓使用者輸入星期一到星期五每天花多少錢，計算出總和以及平均值，然後把結果顯示在螢幕上。所有變數都是用 **float** 型別。

整數 **int** 型別和浮點數 **float** 型別

C 語言裡面當你的數值有小數點的時候，就不適合再用整數來儲存，而要用浮點數。當你寫 **3** 的時候，這個數會被當成是整數，當你寫 **3.0** 就變會被認為是浮點數。除了直接寫出整個數值，浮點數還可以用以十為底的科學記號的方式來表示，譬如：**2.5e-4** 就相當於 **2.5** 乘上 **10** 的 **-4** 次方，也就是 **0.00025**。浮點數和整數在電腦裡的儲存格式不一樣，整數會完全用二進位編碼來儲存整個數值，浮點數則會拆成幾個部份，包括正負號、底數、指數三個組成元素。用這樣的方式來儲存，讓我們可以用浮點數來表達更大範圍的數值 (譬如 **-2** 後面接 **50** 個零，這麼大的數只要寫成 **-2e50**，所以只要記錄 i) 是個負數，ii) 底數是 **2**，iii) 指數是 **50** 就夠了)。然而浮點數的精確度仍舊是有限的，所以有時候浮點運算結果可能和你預期的結果之間會有點誤差，譬如你覺得算出來的答案應該是 **1.0**，但是實際上儲存的浮點數可能是 **0.999999**。以後我們會再用一些例子實際說明使用浮點數必須特別注意哪些情況。

第三章：讀取和輸出各種型別的資料

整數型別 `int`

大多數的電腦用的是 32-bit 整數，但是隨著轉換到 64 位元處理器，整數長度也會跟著更改成 64-bit。位元長度決定整數可以表示的數值的大小範圍。如果是用長度 32-bit 的空間來儲存一個整數，那麼可以表達的整數值範圍就是 $-2^{31} \sim 2^{31}-1$ (包含了負數和 0)。宣告整數變數的方法我們已經看過，就是 `int` 後面接變數名稱：

```
int birds;
int cats, dogs;
```

宣告變數之後變數還要設定變數值，可以直接在程式碼中設定，譬如 `cats = 3;` 或是使用 `scanf()` 或其他 function 來取得並設定變數的值。還有一種方式是在宣告的時候就順便把值設定好，也就是所謂的初始化 (initialization)：

```
int birds = 10;
int cats = 3, dogs = 2;
```

注意，第二行如果寫成 `int cats, dogs = 2;` 則只有 `dogs` 會做初始化。整個宣告加初始化做的就是右圖所表示的動作。



找到一塊儲存空間，標示成 `birds`，把 10 存放在裡面。

在 C 程式裡面，`5`，`8`，`-32` 這樣的直接寫出來的整數稱作常數 (不需用變數來儲存的數值)。前面提過，如果寫成 `5.0` 就會被當成浮點數常數。

我們已經學過如何用 `printf()` 來顯示 `int` 變數：`printf()` 的第一個參數可以搭配 `"%d"` 來指定輸出格式為十進位整數。要注意的是格式的部份要由程式設計者負責，如果對應錯誤，出來的結果就會是錯的。譬如下面的例子：

[範例 E03_02.c]

```
#include <stdio.h>
int main(void)
{
    int ten = 10;
    int two = 2;

    printf("%d minus %d is %d\n", ten, two, ten - two);
    printf("%d minus %d is %d\n", ten); /* 少了兩個參數 */

    return 0;
}
```

程式輸出的結果可能是

```
10 minus 2 is 8
10 minus 2 is 8
```

少了兩個參數竟然還會跑出結果！這只是湊巧，因為當 `%d` 找不到對應的參數時，`printf()` 取得的的就是當時記憶體中剛好在本來預期的位置裡存放的數值，至於裡面會有什麼值，就要看你的運氣，在這個例子缺少的兩個參數所對應到的記憶體位置，裡面湊巧存放的是 `2` 和 `8`。類似的情況會讓你以為程式沒有錯誤，要特別小心這種 bugs。

除了可以用 `"%d"` 十進位表示法來顯示整數，`printf()` 還可以用 `"%x"` 十六進位表示法來顯示整數，譬如 `0x1f` 表示十進位的 `31` ($=1*16+15$)，這樣的表示法常用在 binary 資料。

[範例 E03_03.c]

```
#include <stdio.h>
int main(void)
{
    int x = 100;

    printf("decimal: %d, hexadecimal: %x\n", x, x);
    printf("decimal: %d, hexadecimal: %#x\n", x, x);

    return 0;
}
```

輸出：

```
decimal: 100, hexadecimal: 64
decimal: 100, hexadecimal: 0x64
```

第二個 `printf()` 用 `"%#x"` 只是會在顯示的數值前面多加 `0x`。

第三章：讀取和輸出各種型別的資料

其他整數型別

除了 `int` 之外，C 程式裡面還可以把變數宣告成其他種類的整數，包括不同長度的型別：`short`, `long`, `long long`。關於 `short`, `long`, `long long` 的位元數的定義，大家可以到課程網頁的參考資料連結，去查一查 *The GNU C Programming Tutorial* (ctut.pdf) Chapter 5, pp. 19-20。另外我們還可以在整數型別前面加上 `unsigned`，用來表示這個整數變數不包含負數的部份：

```
unsigned int x = 20;
```

如果你很確定變數 `x` 的值不會是負的，就可以把它宣告成 `unsigned`，這樣原本用來表達負數所需的空間就可以拿來表達更大的正數。譬如同樣使用 32 bits，原本 `int` 如果範圍是 `-2,147,483,648 ~ 2,147,483,647`，宣告成 `unsigned int` 範圍就變成 `0 ~ 4,294,967,295`。

[範例 E03_04.c]

```
#include <stdio.h>
int main(void)
{
    unsigned int un = 3000000000;
    short small = 200;
    long big = 65537;

    printf("un = %u and not %d\n", un, un);
    printf("small = %hd and %d\n", small, small);
    printf("big = %ld and not %hd\n", big, big);

    return 0;
}
```

輸出：

```
un = 3000000000 and not -1294967296
small = 200 and 200
big = 65537 and not 1
```

得到的輸出結果如右。用 `printf()` 輸出時要配合型別設定顯示格式，才能得到正確的輸出結果。

接下來我們要看一種情況：當我們想要儲存的數值超過了整數型別可以表達的範圍時，會發生所謂的 Integer Overflow 的問題，譬如下面的例子。

[範例 E03_05.c]

```
#include <stdio.h>
int main(void)
{
    int i = 2147483647;
    unsigned int j = 4294967295U;

    printf("%d %d %d\n", i, i+1, i+2);
    printf("%u %u %u\n", j, j+1, j+2);

    return 0;
}
```

輸出：

```
2147483647 -2147483648 -2147483647
4294967295 0 1
```

大家可以想想看如何解釋這樣的現象。

使用字元 `char` 型別

[範例 E03_06.c]

```
#include <stdio.h>
int main(void)
{
    char ch;

    printf("Please enter a character: ");
    scanf("%c", &ch);
    printf("ASCII code for '%c' is %d.\n", ch, ch);

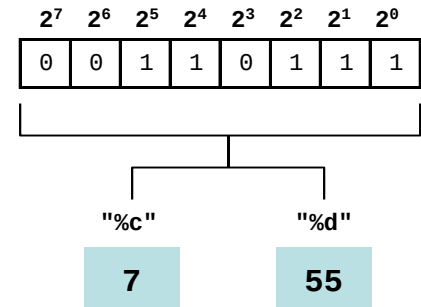
    return 0;
}
```

第三章：讀取和輸出各種型別的資料

前面的範例用到了幾個新的東西。首先是 **char** 型別，它會用到 8-bit 的記憶體空間，靠 8 bits 所儲存的數值來表達字元。譬如 'A', 'e', '7' 等等單一的英文字母或數字。要將數值對應到字元，我們需要特定的編碼方式，通常會有一個編碼的對照表來對應數值和字元。最常用的是 ASCII code 編碼，可以把 0 到 127 的數值對應到不同的字元。前面的範例就是要使用者輸入一個字元，然後程式會把對應的 ASCII 編碼秀出來。範例裡的 **scanf()** 和 **printf()** 還用到了 **%c**。對於 **scanf()** 來說，使用 **%c** 的格式，表示輸入的資料會被當成字元，譬如輸入的是 7，就會把 7 當成字元而不是整數 7。同樣地，**printf()** 裡用 **%c** 的格式，也是要把傳入的參數的數值所對應的字元顯示出來。我們用下面的範例和練習來看看 **%c** 和 **%d** 的差別。

[範例 E03_07.c]

```
#include <stdio.h>
int main(void)
{
    printf("character: %c  ASCII code: %d\n", 55, '7');
    return 0;
}
```



在數字或字母兩邊加上單引號 ' '，就表示要把它當成字元，譬如 '7'，如右圖。

(另外還有一種八進位表示法，譬如十進位的 55 寫成八進位就是 067，也可以寫成 '\067' 來表示 '7'。)

[練習 W03_02] 讓使用者輸入一個 ASCII code 值 (整數值)，程式要顯示對應的字元。

我們平常使用的中文編碼是 BIG5，中文字數目比較多，所以需要兩個 bytes 才足夠編碼常用的中文字，譬如

```
printf("%c%c\n", 0xa7, 0x70);
```

在支援中文的 DOS console 下會顯示出一個中文字。(有時候網頁或郵件出現亂碼就是因為編碼錯誤或是用錯了對照表。)

使用浮點數型別 float, double

[範例 E03_08.c]

```
#include <stdio.h>
int main(void)
{
    float x = 12345.0;
    double y = 2.34e12;

    printf("%f or %e\n", x, x);
    printf("%f or %e\n", y, y);
    return 0;
}
```

輸出：

```
12345.000000 or 1.234500e+004
2340000000000.000000 or 2.340000e+012
```

可以用 **%f** 格式或是 **%e** 科學記號格式來表示浮點數。

浮點數運算也有 overflow 的問題，譬如

```
float s = 3.4e38 * 10.0;
printf("%e\n", s);
```

由於 3.4e38 已經接近 float 可容許的範圍的邊緣，再乘上 10.0 之後就爆掉，所以會得到輸出結果是 1.#INF00e+000，表示無限大。使用浮點數另外有一個要注意的是 round-off error 的問題，譬如

```
float a, b;
b = 1.0e12 + 1.0;
a = b - 1.0;
printf("%f\n", a);
```

照理答案應該是 1 後面接 12 個 0，但是實際上會得到奇怪的數字。這是因為要對兩個大小差很多的數目做運算時，只用 float 並不足夠同時精確表達兩個數目。

第三章：讀取和輸出各種型別的資料

使用 `sizeof` 查詢型別的佔用的 byte 數

[範例 E03_09.c]

```
#include <stdio.h>
int main(void)
{
    printf("int: %u bytes, char: %u bytes,\n", sizeof(int), sizeof(char));
    printf("long: %u bytes, double: %u bytes.\n", sizeof(long), sizeof(double));
    return 0;
}
```

這個範例使用 `sizeof` 這個內建 operator 來顯示 `int`, `char`, `long`, `double` 這些型別所佔用的 byte 數目。

[練習 W03_03] 用 `sizeof` 查 `unsigned int`, `float` 的大小。

當我們把數值設定給變數時，要注意變數的型別是否符合我們的需求，譬如

```
int a = 9.99;
float pi = 3.1415926536;
```

這兩個情況都無法把我們想要的數值存到變數裡。對變數 `a` 來說，小數點後面的數目會被無條件捨去，所以 `a` 的值會是 `9`。至於 `float` 的精確度只有保證六個有效位數，因此超過的位數就不一定正確。如果用 `double` 的話可以保證到十個有效位數。

Flushing the Buffer

當你使用 `printf()` 把資料顯示到螢幕上時，其實並不會立即顯示在螢幕上，而是先送到所謂的 buffer 裡。要等到下列幾種情況才會做 flushing the buffer 的動作，把 buffer 裡的資料沖到螢幕上：i) 當 buffer 滿的時候；ii) 當 `'\n'` 字元出現的時候；iii) 當接下來是做輸入的動作的時候（譬如遇到 `scanf()`）。

有時候為了讓資料能立刻顯示到螢幕上，可以用 `fflush()` 強迫把 buffer 裡的東西送出。當你發現有時候輸入或輸出的顯示順序會亂掉，可以試著在 `printf()` 之後用 `fflush()` 來確保資料不會被卡在 buffer 裡。

[練習 W03_04] 讓使用者輸入一星期的零用錢有多少，假設假日可以花的錢是平常的 1.5 倍，在螢幕上顯示每天可以花的數目。（譬如，`Mon: 200, Tues: 200, ..., Sat: 300, Sun: 300`。）

[練習 W03_05] 讓使用者輸入時速 (公里/小時)，用程式換算成 (公尺/秒) 把結果輸出到螢幕上。

第四章：字串與格式化輸入輸出

字串

[範例 E04_01.c]

```

#include <stdio.h>
#include <string.h>
#define SPEED 0.083
int main(void)
{
    float minutes, distance;
    int size, letters;
    char name[10];

    printf("Hi! What's your first name? ");
    scanf("%s", name);
    printf("\n%s, how many minutes does it take to walk from\n", name);
    printf("your dormitory to the CS building? ");
    scanf("%f", &minutes);
    size = sizeof(name);
    letters = strlen(name);

    distance = minutes * SPEED;
    printf("\nThe distance from your dormitory to the CS building\n");
    printf("is about %.3f kilometers.\n\n", distance);

    printf("By the way, your first name has %d letters,\n", letters);
    printf("and we have %d bytes to store it in.\n", size);

    return 0;
}

```

我們來看看這個程式做了哪些事情以及用到了哪些新的東西。執行這個程式會得到類似下面的結果：

Hi! What's your first name? Stevie

Stevie, how many minutes does it take to walk from
your dormitory to the CS building? 8

The distance from your dormitory to the CS building
is about 0.664 kilometers.

By the way, your first name has 6 letters,
and we have 10 bytes to store it in.

程式先用 `printf()` 顯示訊息，詢問使用者的名字，然後用 `scanf()` 來讀取使用者輸入的 "字串"，把字串儲存在變數 `name` 裡面。再來就可以用儲存起來的字串稱呼使用者，並且問一個簡單的問題，再把計算結果顯示在螢幕上。程式裡面用到的新東西首先是 `#include <string.h>`。回想當初程式需要 `#include <stdio.h>` 是因為要用到 `printf()`，同樣地，引入 `string.h` 是為了要用到其中相關的 function，在這個例子用到的是 `strlen()` 這個 function，它可以計算字串變數裡面所儲存的字串的長度。傳入的參數是字串，傳回的整數值就是字串長度，以 **Stevie** 為例，長度是 6。程式第三行的 `#define SPEED 0.083` 利用 C Preprocessor 來做前處理，在 compile 之前，會先把程式裡所有出現的 `SPEED` 字眼通通取代成 `0.083`。字串其實是靠所謂的 character array 來儲存，

```
char name[10];
```

表示 `name` 會佔用到十個 bytes 的記憶體，等於是十個空位，每個空位的大小是一個 byte，每個 byte 的空間可以存放一個字元。接下來會看到 `printf()` 和 `scanf()` 裡都用到 `%s`，表示要處理的輸入輸出資料是字串 (string)。有一個比較奇怪的地方是在 `scanf()` 裡 `weight` 前面有加 `&`，但是 `name` 前面卻沒有。(由此可以推測 `&weight` 和 `name` 應該代表著對等的性質。稍後我們就會解釋，其實它們都對應到記憶體位址。)

字元陣列 character arrays 與字串

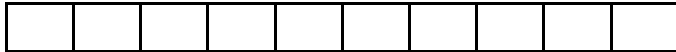
S	t	e	v	i	e	\0			
---	---	---	---	---	---	----	--	--	--

字串裡的字元必須連續地存放在記憶體中，所以剛好可以用陣列來儲存，因為陣列就是一連串的記憶體空間。字元陣列的每一格空間可以存放一個字元 (`char`)，當我們宣告 `char name[10];` 表示要保留十格空間存放十個字元，每一格可以容納一個 `char` 型別的資料。當然除了 `char` 之外，也可用其他型別如 `int` 來宣告陣列，目前我們先專心探討字元陣列。為了標記整個字串究竟在哪裡算是結尾，C 語言使用一個特殊的字元 `'\0'` 來表示字串結尾。字元 `'\0'` 對應到的 ASCII 值是 0。我們也可以用整數 0 來代替字元 `'\0'`，但為了有所區別，當字元使用時最好寫成 `'\0'`。

第四章：字串與格式化輸入輸出

宣告一個字元變數和宣告一個陣列的差別可以用下圖來表示。

`char ch;`  佔用一個 byte，型別為 `char`

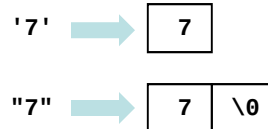
`char name[10];`  佔用十個 bytes，型別為 `char`

宣告陣列產生一個可以容納十個字元的 array，準備用來記錄使用者輸入的字串 (使用者的名字)。因為要保留一格給 `'\0'` 字元來標示字串結尾，所以其實真正能用來記錄的人名長度，最多只能包含九個字元。如何把字串存入陣列中呢？在前面的例子裡用的方法是 `scanf("%s", name);` 讀取使用者輸入的字串。所以 `%s` 就表示要把使用者輸入的東西當作 "字串" 讀進來，然後參數 `name` 就是要存放字串的陣列名稱，這個名稱所代表的意義是整個字串的開頭位址。因此 `scanf()` 就能由 `name` 找到陣列開頭位址，一格一格把字元填進去，而且會自動在最後加上 `'\0'` 當作結束。

現在暫時來想想下面這個沒什麼用的程式，其中的 `printf()` 有什麼差別。

[範例 E04_02.c]

```
#include <stdio.h>
int main(void)
{
    printf("%d\n", '7');
    printf("%c\n", '7');
    printf("%s\n", "7");
    return 0;
}
```



最需要注意的是 `'7'` 代表的是字元 (ASCII 碼是 55)，但是 `"7"` 是字串，它除了包含 `'7'` 這個字元之外，還多藏了一個 `'\0'` 字元。

再回到範例 E04_01.c，裡面又用到 `sizeof`，它可以計算型別 (譬如 `sizeof(float)`) 或資料 (譬如 `sizeof(name)` 或 `sizeof("Stevie")`) 的 byte 數。若把它用在宣告過的陣列上，它會告訴我們陣列有多少個 bytes，如果是 `char name[10];` 就是 10 bytes。另外 `#include <string.h>` 是為了用 `strlen()` 這個 function，它是針對字串而設計的 function，可以算出字串長度。字串長度是靠 `'\0'` 來界定，所以如果把某個字串當參數傳給 `strlen()`，它就會一個字元一個字元地數，直到遇到 `'\0'` 就停下來，然後回報字串長，看看底下例子中的差異：

[範例 E04_03.c]

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    printf("The length of string is %d, but %d bytes are occupied.\n",
        strlen("Stevie"), sizeof("Stevie"));
    return 0;
}
```

Constants and the C Preprocessor

接下來解釋如何在程式裡設定常數。在範例 E04_01.c 裡用到 `#define SPEED 0.083` 這一行的作用是告訴 C preprocessor 要在 compile 之前，先把程式碼裡面出現的 `SPEED` 字眼都取代成 `0.083`。它的主要功能是為了寫程式方便，因為你的程式可能會在許多地方用到 `0.083` 這個數據來做計算，為了避免打錯或是忘記，可以把 `0.083` 取名作 `SPEED`，程式裡只要直接使用 `SPEED` 就可以，等到要 compile 的時候，會有前處理的動作先幫你將 `SPEED` 都換成實際上需要的數值 `0.083`。另外還有一個方便的地方是當你想要更改數據，譬如 `0.083` 改成 `0.082`，你只需要改 `#define SPEED 0.082` 這個地方，其餘的程式碼都不需要更改。假如你不是用 `#define` 而是直接把數字寫死，你就要自己找出每個出現 `0.083` 的地方，然後把它改成 `0.082`，這樣做不但麻煩而且容易出錯。所以當你的程式用到某些常數時，最好不要直接使用常數的數值，而應該替它取個代名詞。

(還有一種設定 "整數常數" 的方法是使用 `enum`，有興趣的話可以查詢課程網頁上的參考資料連結 *The GNU C Programming Tutorial* (ctut.pdf) Chapter 19, pp. 189-191。)

[練習 W04_01] 用 `#define` 替 `3.14159` 取個名字叫做 `PI`，用 `scanf()` 讓使用者輸入圓半徑並存入變數 `radius` 中，用 `PI` 和 `radius` 計算圓面積並用 `printf()` 顯示出結果。

第四章：字串與格式化輸入輸出

在送到 compiler 之前已經先做了取代的動作。

```

#define SPEED 0.083
int main (void)
{
    .
    .
    distance = minutes * SPEED;
    .
    .
}

                                preprocessing
                                →
int main (void)
{
    .
    .
    distance = minutes * 0.083;
    .
    .
}

                                compiling
                                →

```

關於 limits.h 與 float.h

如果進入你的 compiler 軟體所在的目錄裡的 include 資料夾，譬如 D:\Dev-Cpp\include\ 裡面會有一個叫做 limits.h 的檔。用文字編輯器打開檔案你會發現裡面用 **#define** 設定了許多常數值，譬如

```
#define LONG_MAX 2147483647L
```

這些常數值和型別的範圍有關。假如你想知道整數最大可以到多大，可以使用 **INT_MAX** 來表示。所以相較於把數字都寫死在程式碼裡面，使用這些常數帶來的額外好處是，假如你的程式拿到另外一台不同的電腦上要執行，只需要重新 compile 就可以取得相對應的 **INT_MAX**，程式碼完全不必更改，因為那台電腦也會有對應的 **INT_MAX**，縱使這個常數值和原本電腦上的有所差異也不會產生問題。另外 float.h 檔也設定了類似的常數（檔案可能在 D:\Dev-Cpp\lib\gcc\mingw32\3.4.2\include 路徑下），底下的範例會把這些值顯示出來。

[範例 E04_04.c]

```

#include <stdio.h>
#include <float.h>
int main(void)
{
    printf("Number of bits in the mantissa of a float: %d\n", FLT_MANT_DIG);
    printf("Minimum number of significant decimal digits for a float: %d\n", FLT_DIG);
    printf("Minimum value for a positive float retaining full precision: %e\n", FLT_MIN);
    printf("Maximum value for a positive float: %e\n", FLT_MAX);
    printf("Difference between 1.00 and the least float value greater than 1.00: %e\n",
        FLT_EPSILON);
    return 0;
}

```

更進一步了解 printf() 和 scanf()

接下來我們要再多介紹一些 **printf()** 和 **scanf()** 的功能，也藉此多熟悉一下寫程式的技巧。首先再來做個練習複習一下：

[練習 W04_02] 讓使用者輸入某個 NBA 球員的名字，接著要求輸入該球員的身高幾公分 (**int**)，然後用程式換算成英制單位，並在螢幕上顯示球員名字和他的身高是幾呎幾吋。(除法運算的符號是 /，譬如 **x = 5 / 2;**，另外如果還有些印象，整數的除法運算結果，小數點會被無條件捨去。)

大家目前應該和 **printf()** 混得蠻熟了，**printf()** 除了常用的 **%d, %f, %s** 這些格式之外，還有很多其他格式可用，詳細用法請參考課程網頁上所附的連結 *The GNU C Library Reference Manual* (glibc-manual-2.2.5.pdf) 12.12 Formatted Output, p. 264。

雖然都沒正式提過，但是大家應該有都觀察的出來，**printf()** 的格式其實就是

```
printf(control-string, item1, item2, ...);
```

其中 *control-string* 是用 **"..."** 指定的那一連串輸出格式設定，而 *item1, item2, ...* 就是你要輸出的資料。再次提醒要特別小心 *control-string* 裡的 **%d** 之類的輸出格式設定要和後面傳入的參數有一對一的對應，譬如

```
printf(": %d ft %d in\n", "LeBron James", foot, inches);
```

是錯的，因為少了一個 **%s**，對應到 **"LeBron James"**。

第四章：字串與格式化輸入輸出

用實際的例子來看看常用的格式設定的用法。

[範例 E04_05.c]

```
#include <stdio.h>
#define ENGINE 1499.99
int main(void)
{
    printf("%f~\n", ENGINE);
    printf("%e~\n", ENGINE);
    printf("%4.2f~\n", ENGINE);
    printf("%3.1f~\n", ENGINE);
    printf("%10.3f~\n", ENGINE);
    printf("%-10.3f~\n", ENGINE);
    printf("%12.3e~\n", ENGINE);
    printf("%+4.2f~\n", ENGINE);
    printf("%010.2f~\n", ENGINE);
    return 0;
}
```

輸出：

```
~1499.990000~
~1.499990e+003~
~1499.99~
~1500.0~
~ 1499.990~
~1499.990 ~
~ 1.500e+003~
~+1499.99~
~0001499.99~
```

從輸出結果可以猜出每種格式的功用。第三個 `printf()`，`%4.2f` 表示最少要顯示出四個字元寬度，而小數點後要顯示二位小數。第四個 `printf()` 用的是 `%3.1f`，所以最少要顯示出三個字元寬，而小數點後只顯示一位小數，這麼做會使得原本的數字被自動四捨五入。第五個 `printf()` 因為用了 `%10.3f` 所以最少要顯示十個字元寬 (不足的會補空白)，然後小數點後顯示三位小數；可以數數看在兩個 ~ 符號中間是否正好包含十個字元 (包含空白字元和小數點)。接下來 `%-10.3f` 意思和 `%10.3f` 相同，差別只是變成要向左靠齊 (這是加了 - 號的效果)。第七個 `printf()` 用的是 `%12.3e` 所以總寬度是 12 個字元，一樣可以數數看是否有 12 個字元，不足的也會用空白來填補。而由於小數點後只顯示三位小數，所以無法完整表示成 `1.49999e+003`，會被自動四捨五入成 `1.500e+003`。第八個 `printf()` 用 `%+4.2f` 多了 + 號只是表示要在數字前面顯示正負號，因為 `1499.99` 是個正數，所以就多顯示了 + 號。最後一個 `printf()` 用到 `%010.2f` 意思是要顯示最少十個字元寬，不足個地方不再是用空白來補，而是改成補零。以上這些就是可能會用到的格式設定。

[練習 W04_03] 用 `#define` 設定一個叫做 `COMPOSER` 的字串，字串內容是 "J. S. Bach"。測試看看用 `printf()` 顯示 `COMPOSER` 字串時，格式如果用 `%10.5s` 和 `%-10.5s` 有什麼差別。

電腦是用二進位表示法來儲存資料，它記錄的只是一堆 0 與 1 的 bits。譬如 76 這個十進位數字用二進位表示是 `01001100`。假設有某個變數 `x` 它對應到的記憶體空間裡儲存著 `01001100`，這樣的資料究竟代表什麼意義要看我們怎麼解讀這些 bits。假設我們想要把它當作十進位整數，它的值就等於 76。而當我們想把 76 這個數字顯示到螢幕上時，如果使用 `printf()` 的 `%d` 格式，`printf()` 做的事情就變成要先顯示 '7' 這個字元，然後再顯示 '6' 這個字元 ('7' 的 ASCII code 是 55，'6' 是 54)。

再更進一步，如果我們把 `01001100` 當作 ASCII code 來解讀，使用 `printf()` 的 `%c` 把它顯示出來，它就變成對應到 'L' 這個字元。這些差異其實不難理解，雖然乍看之下很容易讓人混淆，但只要掌握一個原則就是二進位表示是資料真正的本質，至於如何解讀這些二進位 bits，就要看我們的需求和意圖來決定。

簡單地說，`printf("%d", x);` 的動作就是把 `x` 的值當成十進位整數，然後把每個位數取出來，一個個用字元顯示到螢幕上。至於 `printf("%c", x);` 的動作則是把 `x` 當成 ASCII code，把這個 ASCII code 對應的字元顯示到螢幕上。

關於 `scanf()`

用鍵盤打字是我們輸入資料到電腦裡的主要方式之一。譬如我們要輸入 2008 這個數目到電腦裡，我們會依次按下鍵盤上的數字鍵 2 0 0 8，每個鍵送出的資訊相當於一個字元，所以我們有四個字元 '2' '0' '0' '8'。如果我們想要的是把這四個數字合在一起看，當作是一個數值 (兩千零八)，這時候就需要用到 `scanf()` 幫我們把這一連串字元轉成一個整數，同時再把轉換的結果存進某個變數裡面，譬如

```
scanf("%d", &x);
```

要再次強調的重點是使用 `scanf()` 時，如果輸入的資料要存入一般的變數中，變數前面要加 `&`；如果輸入的是字串，要存入字元陣列中，陣列名稱前不需要加 `&`。所以如果是

```
char name[10];
```

就只要

```
scanf("%s", name);
```

第四章：字串與格式化輸入輸出

我們再更深入地了解 `scanf()` 的運作方式。試試看底下的程式用各種不同格式輸入資料會產生什麼樣的結果。

[範例 E04_06.c]

```
#include <stdio.h>
int main(void)
{
    char name[10];
    int x = -1;
    scanf("%d", &x);
    scanf("%s", name);
    printf("x = %d\n", x);
    printf("name: '%s'\n", name);
    return 0;
}
```

1) 如果先輸入一堆空白字元 (例如按下空白鍵或 Enter 鍵)，然後再輸入一個數字，接著再打一堆空白字元，然後輸入一串字母，得到的結果會像下面這樣 (加底線的是使用者輸入的資料)：

```
      89      James
x = 89
name: 'James'
```

2) 假如沒有輸入數字，而是直接先輸入一串字母，則會得到類似下面的結果：

```
Andy
x = -1
name: 'Andy'
```

3) 假如輸入數字但是緊接著又輸入字母，則會得到

```
789Alex13 R
x = 789
name: 'Alex13'
```

出四個字元寬度，而小數點後要顯示兩位小數。

整個運作的原則其實很單純。對於 `%d` 來說，`scanf()` 會忽略一開始輸入的空白字元，期待第一個數字出現，然後就開始讀取數字，一直到出現非數字的字元就停止，然後把數字所表示的十進位數值存到變數裡。對於 `%s` 來說，則是從第一個非空白字元開始讀，直到出現空白字元為止，把這中間所有字元當作一個字串存入字元陣列裡，同時還會在最後多加一個 `'\0'` 放入字元陣列中，表示字串的結尾。

最後我們來看看 `%*` 在 `printf()` 和 `scanf()` 各有什麼作用。

[範例 E04_07.c]

```
#include <stdio.h>
int main(void)
{
    int width, precision;
    double rate = 123.45;
    printf("Enter a width and a precision: ");
    scanf("%d %d", &width, &precision);
    printf("rate: '%*.*f'\n", width, precision, rate);
    return 0;
}
```

先讓使用者輸入想要顯示的浮點數的寬度和精確度 (小數點後的位數)，把這兩個數據存在 `width` 和 `precision` 中，例如使用者輸入的分別是 `8` 和 `3`。然後在 `printf()` 裡用 `%*.f` 的方式，按照 `width` 和 `precision` 所指定的格式來顯示浮點數。所以 `%*.f` 裡的兩個 `*` 號，會被取代成 `%8.3f`，所以 `rate` 這個變數就會顯示成 `' 123.450'`。這樣的功能提供了更大的彈性，讓我們的程式可以動態地決定如何顯示資料。

接著再來看 `scanf()` 中使用 `%*` 有什麼不同的作用。

第四章：字串與格式化輸入輸出

[範例 E04_08.c]

```
#include <stdio.h>
int main(void)
{
    int x;
    printf("Please enter three integers:\n");
    scanf("%*d %*d %d", &x);
    printf("Only the last one is stored: %d\n", x);
    return 0 ;
}
```

程式要求使用者輸入三個整數，但是在 `scanf()` 裡面，前兩個位置是用 `%*d` 格式，意思是忽略這兩個被輸入的整數。只有第三個格式是標準的 `%d`，所以它才是真正對應到 `&x`，整個程式也只有記錄第三個被輸入的整數。

[練習 W04_04] 先猜猜底下的程式執行後，如果使用者輸入 Mississippi，程式會輸出什麼？

```
#include <stdio.h>
int main(void)
{
    char word[46];
    scanf("%45s", word);
    printf("%.4s\n", word);
    return 0 ;
}
```

因為 `printf()` 裡面用了 `%.4s`，如果對照浮點數的用法 `%.4f` 表示顯示時精確度是 4，套用到字串上，意思變成只要顯示前四個字元，所以螢幕上會顯示 Miss 四個字元，而不是整個 `word` 字串 Mississippi。至於 `scanf()` 裡面用到 `%45s` 則是表示輸入的字串最長不超過 45 個字元 (第 46 個要留給 `'\0'`)。

試著修改上面的程式碼，讓使用者輸入一個英文字，然後在 `printf()` 裡利用 `%.*s` 格式設定要輸出的字串長度，讓程式只輸出原本輸入的英文字的前半段，譬如 Mississippi 只輸出 Missi，或 university 只輸出 unive。

提示：用 `strlen()` 計算字串長度。

[補充] 用 `printf()` 輸出字串時的格式設定

各種用法的整理：

1. 如果寫

```
printf("%.3s\n", "university");
```

會輸出 `uni` 也就是只會輸出前三個字元

2. 如果寫

```
printf("%15s\n", "university");
```

會輸出 `university` 前面會多五個空格

3. 如果寫

```
printf("%15.3s\n", "university");
```

則會輸出 `uni` 也就輸出 `university` 的前三個字元 然後前面會多加入 12 個空格 總共長度是 15 個字元

4. 如果寫

```
printf("%.*s\n", 15, 3, "university");
```

則是用動態的方式來設定格式 在這個例子得到的輸出結果也是 `uni` 也就是把兩個 `*` 號分別用 `15` 和 `3` 取代 相當於 `"%15.3s\n"` 的效果

第五章：Operators, Expressions, and Statements

我們已經大致知道如何輸入資料給程式，以及如何透過程式輸出資料到螢幕上。具備了與使用者互動的基本能力之後，再來就可以開始寫更複雜的程式。我們接下來要介紹一些運算符號，並介紹如何在程式裡加入迴圈，讓程式反覆做類似的運算，畢竟這才是電腦的強項。我們就來看看如何利用電腦重複地做大量的計算。

[範例 E05_01.c]

```
#include <stdio.h>
#define STEP 10
#define LOWER 0
#define UPPER 100
int main(void)
{
    int fahr, celsius;

    printf("Celsius    Fahrenheit\n");
    printf("-----\n");
    celsius = LOWER;
    while (celsius <= UPPER) {
        fahr = 1.8*celsius + 32;
        printf("%4d %12d\n", celsius, fahr);
        celsius = celsius + STEP;
    }
    printf("-----\n");
    return 0;
}
```

輸出：

Celsius	Fahrenheit
0	32
10	50
20	68
30	86
40	104
50	122
60	140
70	158
80	176
90	194
100	212

這個程式會列出攝氏和華氏溫度的對照表，範圍從攝氏零度到一百度。程式用到了 **while** 迴圈來達到反覆呼叫 **printf()** 的效果。當程式執行到 **while** 時，會先檢查括號裡的式子：

```
(celsius <= UPPER)
```

這個式子是在判斷 **celsius** 是否小於 **UPPER**。由於在執行到 **while** 之前的一行，**celsius** 的值被設為 **LOWER** (值為 0)，所以目前 **celsius** 的確小於 **UPPER**。由於 **celsius** 與 **UPPER** 的實際大小關係符合這個式子所描述，所以這個式子會得到 **true**，表示這個式子是成立的，因此程式就可繼續執行 **while** 迴圈接下來的敘述。整個 **while** 迴圈的內容被包含在 **{}** 所定義的有效範圍內。接下來要執行的程式碼就是

```
fahr = 1.8*celsius + 32;
printf("%4d %12d\n", celsius, fahr);
```

代入換算公式，再把對照值有固定格式印出來。接下來 **celsius** 的值會被累加上 **STEP**

```
celsius = celsius + STEP;
```

這一行的意思就是把 **celsius** 加上 **STEP** 之後，得到的值再存回 **celsius**。所以 **celsius** 的值從 0 變成 10，然後程式的進行會回到 **while** 的判斷式，再次判斷 **celsius <= UPPER** 是否成立。這樣一直反覆進行下去，不斷地在螢幕上印出攝氏華氏對照值，而且 **celsius** 的值也會不斷增加。一直到 **celsius** 的值變成 100，然後再經過 **celsius = celsius + STEP;** 之後變成 110，這時候回到判斷式 **celsius <= UPPER** 就會發現條件不成立，因為 110 已經比 100 大，所以 **while** 迴圈就此結束，程式接下來直接跳到 **while { ... }** 之外，執行下一行程式碼 **printf("-----\n");**。

運算符號 Operators

基本的運算符號大家其實都已經看過，包括加減乘除 **+** **-** ***** **/**，以及把值設定給變數時要用 **=**。其中 **=** 是最需要再特別解釋的符號。標準用法像是

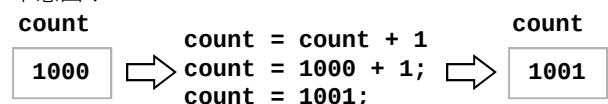
```
count = 1000;
```

把 1000 設定給變數 **count**。另外一種常用的情況是

```
count = count + 1;
```

表示要把變數 **count** 的值加 1 之後再存回相同的變數 **count**。

示意圖：



第五章：Operators, Expressions, and Statements

[練習 W05_01] 在螢幕上顯示一個簡單的對照表，列出數字 **1** 到 **30** 以及它們對應的平方值。

[範例 E05_02.c]

```
#include <stdio.h>
#define INTEREST_RATE 0.05
#define FUND 10000
#define YEAR 10
int main(void)
{
    double cumulative;
    int count;

    printf("YEAR      PROFIT\n");
    cumulative = FUND;
    count = 1;
    while (count <= YEAR) {
        cumulative = cumulative * (1 + INTEREST_RATE);
        printf("%2d %12.2f\n", count, cumulative - FUND);
        count = count + 1;
    }
    return 0;
}
```

輸出：

YEAR	PROFIT
1	500.00
2	1025.00
3	1576.25
4	2155.06
5	2762.82
6	3400.96
7	4071.00
8	4774.55
9	5513.28
10	6288.95

這個範例依照年度，以複利計算獲利。假設本金 10,000 元，年利率 5%，總共投資十年，則每年能夠賺到的利息依照年度列表。初始值的設定很重要。變數 **cumulative** 是用來計算累計的本利，一開始設成 **FUND**，在迴圈中會不斷累乘 **1+0.05**，顯示結果的時候，把本金扣除就可以算出獲利。變數 **count** 則是用來累加年度，當 **count** 超過 **YEAR** 時，迴圈就結束。

[練習 W05_02] 延續前面範例，但是假設每過一年又會再存入一次 **FUND**。把新的獲利表列出來。(提示:只需修改兩個地方。)

運算符號 **sizeof** 以及型別 **size_t**

之前已經學過使用 **sizeof** 來看某個型別或某個資料所佔用的 byte 數，使用 **sizeof** 所得到的值 (byte 數) 的型別為 **size_t** (其實相當於 **unsigned int**)。

[範例 E05_03.c]

```
#include <stdio.h>
int main(void)
{
    int n = 0;
    size_t intsize;

    intsize = sizeof (int);
    printf("n = %d, n has %u bytes; all ints have %u bytes.\n", n, sizeof(n), intsize);

    return 0;
}
```

其中 **intsize** 是一個宣告成 **size_t** 型別的變數，用來儲存 **sizeof (int)** 所計算出來的值。此外，**%u** 代表顯示的格式是 unsigned decimal integer。其實 **size_t** 這個型別只是透過 **typedef** 的方式，來將它變成 **unsigned int** 的同義辭。當 compiler 遇到 **size_t** 的時候，它就會想起 **size_t** 其實就是 **unsigned int**。所以像底下這樣

```
typedef double real;
```

就可以替 **double** 定一個新的同義辭叫做 **real**。接下來程式裡面可以用

```
real x;
```

宣告變數 **x**。乍看之下 **x** 屬於一個新的型別 **real**，但是其實 compiler 知道 **x** 的型別就是標準的 **double**。

第五章：Operators, Expressions, and Statements

求餘數要用 % 符號 (Modulus Operator)

輸出：

[範例 E05_04.c]

```
#include <stdio.h>
#define SEC_PER_MIN 60

int main(void)
{
    int sec, min, left;

    printf("Convert seconds to minutes and seconds!\n");
    printf("Enter the number of seconds (<=0 to quit):\n");
    scanf("%d", &sec);
    while (sec > 0) {
        min = sec / SEC_PER_MIN;
        left = sec % SEC_PER_MIN;
        printf("%d seconds is %d minutes, %d seconds.\n", sec, min, left);
        printf("Enter next value (<=0 to quit):\n");
        scanf("%d", &sec);
    }
    printf("Done!\n");
    return 0;
}
```

```
Convert seconds to minutes and seconds!
Enter the number of seconds (<=0 to quit):
123
123 seconds is 2 minutes, 3 seconds.
Enter next value (<=0 to quit):
340
340 seconds is 5 minutes, 40 seconds.
Enter next value (<=0 to quit):
0
Done!
```

這個程式使用 % 符號來把秒數換算成幾分幾秒。另外還用了一個新的程式技巧，透過 **while** 迴圈讓使用者不斷輸入資料，當使用者輸入的資料小於或等於零，迴圈就停止。

累加符號 ++ 與 --

[範例 E05_05.c]

```
#include <stdio.h>
int main(void)
{
    int lower = 0, upper = 10;

    while (lower < upper) {
        lower++;
        upper--;
        printf("lower = %d, upper = %d \n", lower, upper);
    }
    return 0;
}
```

輸出：

```
lower = 1, upper = 9
lower = 2, upper = 8
lower = 3, upper = 7
lower = 4, upper = 6
lower = 5, upper = 5
```

程式用到 ++ 和 -- 來改變變數的值。其中 **lower++** 就是把 **lower** 的值增加一，相當於 **lower = lower + 1**；同理 **upper--**；相當於 **upper = upper - 1**。當 **lower** 的值比 **upper** 還小，迴圈就會繼續進行；而當 **lower** 的值增加到等於或超過 **upper** 時，迴圈就結束。

[範例 E05_06.c]

```
#include <stdio.h>
int main(void)
{
    int a = 1, b = 1;
    int aplus, plusb;
    aplus = a++;
    plusb = ++b;
    printf("a   aplus   b   plusb \n");
    printf("%1d%6d%6d%6d\n", a, aplus, b, plusb);
    return 0;
}
```

輸出：

a	aplus	b	plusb
2	1	2	2

兩種寫法都達到了把 **a** 和 **b** 的值增加一的效果。但是從 **aplus** 和 **plusb** 的值可以看出來，**a++** 和 **++b** 的時間點稍有不同。如果 ++ 寫在後面，相當於先做 **aplus = a**；然後再做 **a = a + 1**；而如果把 ++ 寫在前面，相當於先做 **b = b + 1**；再做 **plusb = b**；的動作。

第五章：Operators, Expressions, and Statements

[範例 E05_07.c]

```
#include <stdio.h>
int main(void)
{
    int i = 'A';

    while (i <= 'Z') {
        printf(" %c: %3d", i, i);
        i++;
    }
    return 0;
}
```

程式輸出的結果如下：

```
A: 65 B: 66 C: 67 D: 68 E: 69 F: 70 G: 71 H: 72 I: 73 J: 74
K: 75 L: 76 M: 77 N: 78 O: 79 P: 80 Q: 81 R: 82 S: 83 T: 84
U: 85 V: 86 W: 87 X: 88 Y: 89 Z: 90
```

這個程式非常簡短，作用是把 'A' 到 'Z' 的 ASCII code 列出來。所以我們可以用 'A' 或 'Z' 這種字元表示方式來設定整數的值，例如 `i = 'A'`；同時還可以拿來做比較，譬如 `i < 'Z'`。整個迴圈會不斷增加 `i` 的值，直到 `i` 超過 'Z' 的 ASCII code 值。

[練習 W05_03] 讓使用者輸入兩個大寫英文字母，然後程式必須輸出字母順序介於兩者之間的其他字母。

[範例 E05_08.c]

```
#include <stdio.h>
#include <string.h>
#define STAR "*****"
int main(void)
{
    int i = 1, j = 0;
    while (i <= strlen(STAR)) {
        printf("%.5s\n", i++, STAR);
        j = 0;
        while (j < 50000000) j++;
    }
    return 0;
}
```

輸出:

```
*
**
***
****
*****
*****
*****
*****
*****
*****
*****
```

這個程式也很簡短，但可以輸出有趣的結果。與 `j` 有關的 `while` 迴圈沒有實質運算上的意義，純粹只是為了拖延程式執行的速度。(這個寫法雖然能達到讓程式“暫停”的效果，但並不是一個好方法，我們以後會介紹更好的做法。)

[練習 W05_04] 修改上面的範例，不使用 `#define`，而是讓使用者輸入一串字串，譬如 "ABCDE"，然後輸出像下面這樣的結果：

```
A
AB
ABC
ABCD
ABCDE
```

(提示：可以宣告一個陣列 `char word[40]`；然後用 `scanf()` 讀入字串並存入 `word` 陣列中。另外再宣告整數 `word_len` 記錄 `strlen(word)` 計算出來的長度。顯示時 `printf()` 裡面用 `%.5s` 來設定格式。)

Expressions 與 Statements

稍微介紹一下 C 語言語法裡兩個常用的名詞：expression 與 statement。底下的運算式都是 expression

```
a + 5
x = 3 * 4
k > 3
```

最需要記得的觀念是每個 expression 都代表某個值，譬如 `b = 1 + 2` 的值是 3，甚至條件判斷式也有值，譬如 `5 > 3` 的值就是 1 (在 C 裡面 1 代表 true，而 0 代表 false)。至於 statement 簡單的說就是以分號結尾的句子，譬如 `x = 3 * 4;`。稍微知道這兩個名詞的意思，往後我們提到的時候就會有概念，解釋起來也會比較方便。

第五章：Operators, Expressions, and Statements

型別轉換

當 expression 或 statement 中所出現的變數或常數之間型別不同時，C 會做型別轉換將變數或常數轉成相同型別。底下是型別轉換的基本規則

- 1) 在 expression 中的 **char** 會轉換成 **int**，譬如我們前面看到的 **i <= 'Z'**。
 - 2) 當兩種型別混用時，位階較低的型別會轉成位階較高的；位階高低順序如下：**double, float, unsigned long, long, unsigned int, int**。
 - 3) 在等式的 statement 中，型別會被轉換成等號左邊的變數的型別。
 - 4) 在傳參數到 function 的時候，**char** 會被轉成 **int**，而 **float** 會被轉成 **double**。
- 通常，第三個規則所造成的型別轉換有可能會讓我們不小心寫出 bug。

[範例 E05_09.c]

```
#include <stdio.h>
int main(void)
{
    char ch;
    int i;
    float fl;
    ch = 'C'; /* line 7 */
    i = ch; /* line 8 */
    fl = i; /* line 9 */
    printf("ch = %c, i = %d, fl = %2.2f\n", ch, i, fl); /* line 10 */
    ch = ch + 1; /* line 11 */
    i = fl + 2 * ch; /* line 12 */
    fl = 2.0 * ch + i; /* line 13 */
    printf("ch = %c, i = %d, fl = %2.2f\n", ch, i, fl); /* line 14 */
    return 0;
}
```

程式輸出的結果如下：

```
ch = C, i = 67, fl = 67.00
ch = D, i = 203, fl = 339.00
```

所以第七行把字元 'C' 存在變數 **ch** 中 (只佔 1 byte)，接下來轉成 **int** 存在 **i** 中，再轉成 **float** 存在 **fl** 中。第 12 行的運算，由於 **ch** 的值是 'C' (ASCII code 67) 會被轉成 **int** 來運算，然後得到的結果會被轉成 **float** 再和 **fl** 相加，最後再降成 **int** 存到 **i** 之中。第十三行道理相同，只是最終都轉成 **float**。

強制型別轉換 cast

除了自動做型別轉換之外，還可以透過下面的方式來強制做型別轉換，做法是在數值或變數前面加上 (型別)，也就是用括號將型別括起來的形式，其中 型別 可以替換為我們想要強制轉換的型別：

```
x = (int) 2.3 + (int) 2.8;
```

假設已經宣告 **int x**；則整數變數 **x** 得到的結果會是 4，因為 2.3 被轉成 **int** 變成 2，而 2.8 被轉成 **int** 也變成 2，所以 2+2 就得到 4。(如果是 **x = 2.3 + 2.8**；則 **x** 的值會是 5，因為會先計算出 5.1 然後無條件捨去轉換成 **int**。)

如何寫出能夠傳入參數的 function

之前看過如何把一連串動作合在一起寫成一個 function，讓主程式可以重複呼叫。接下來我們進一步來看如何讓自己寫的 function 也可以傳入參數 (譬如像 **strlen()** 可以傳入字串)。

[範例 E05_10.c]

```
#include <stdio.h>
void star(int n);
int main(void)
{
    int i = 5;
    float fl = 7.0;
    star(i);
    star((int) fl);
    return 0;
}
```

```
/* function definition */
void star(int n)
{
    while (n-- > 0) {
        printf("*");
    }
    printf("\n");
}
```

第五章：Operators, Expressions, and Statements

在使用 function 之前要先宣告 function 的 prototype，讓 compiler 知道要怎麼使用它。程式最前面的 **void star(int n);** 意思就是 **star()** 是一個 function，需要傳入一個型別為 **int** 的參數；不過 **star()** 並不傳回任何值，所以在 function 名稱的左邊寫 **void**。接下來在 function 主體的地方 (function definition)，要符合宣告時的 prototype。在 **star()** 的主體做的事情是傳入一個整數，取名叫做 **n**，然後持續把 **n** 的值遞減 (用 **n--** 的方式)，迴圈每次會在螢幕上顯示一個 *****，直到 **n** 不再大於 0 迴圈就停止。主程式 **main()** 裡面使用 **star()** 的方式就跟使用 **strlen()** 或 **printf()** 類似，把要傳進去的參數寫在 function 的括號裡，像是 **star(i);**。呼叫 **star(i);** 的動作會把 **i** 傳入 **star()** 裡，而 **i** 的值就會被 **star()** 接收，當作變數 **n**。由於這個程式裡當 **i** 傳入 **star()** 時，它的值是 5，所以 **n** 的初值就是 5，因此 **while** 迴圈會做五次印出五個星號。後來 **fl** 傳入時有用 **(int) fl** 把它強制從 **float** 轉成 **int**，所以到了 **star()** 裡面時 **n** 的值得到七，所以迴圈會做七次。如果我們把強制型別轉換拿掉，則 compiler 也會自動幫我們做轉換成 **int** 的動作，因為在 prototype 中我們已經宣告了 **star()** 傳入的參數是 **int**，所以 compiler 知道要把參數轉成 **int**。

另外值得一提的是 **star()** function 主體裡面所用到的變數，他們的有效範圍只限於 **star()** 那兩個波浪括號 **{ }** 所包含的範圍內。這個意思就是說，如果我們在 **star()** 裡面把某個變數名稱取成和 **main()** 裡另一個變數名稱完全相同，compiler 也不會因此混淆。譬如，若把 **star()** 的 function definition 的內容改成

```
void star(int i)
{
    while (i-- > 0) {
        printf("*");
    }
    printf("\n");
}
```

程式並不會出錯，還是會得到相同的結果。

輸入輸出：

[範例 E05_11.c]

```
#include <stdio.h>
#define S_PER_M 60
#define S_PER_H 3600
int main(void)
{
    double length, dist, speed;
    int laps;
    int hour, min, sec;
    int time, laptime, avmin, avsec;

    printf("Please enter the length of the track (km): ");
    scanf("%lf", &length); /* lf for type double */
    printf("Enter the number of laps for this GP: ");
    scanf("%d", &laps);
    printf("Enter the time in hours, minutes, and seconds.\n");
    printf("hours: ");
    scanf("%d", &hour);
    printf("minutes: ");
    scanf("%d", &min);
    printf("seconds: ");
    scanf("%d", &sec);
    time = S_PER_H * hour + S_PER_M * min + sec;
    dist = length * laps;
    speed = dist / time;
    laptime = (int) (length / speed);
    avmin = laptime / S_PER_M;
    avsec = laptime % S_PER_M;
    printf("The average speed is %5.2f km/h\n", speed * S_PER_H);
    printf("The average time for one lap is %d min %d sec\n", avmin, avsec);

    return 0;
}
```

```
Please enter the length of the track (km): 5.338
Enter the number of laps for this GP: 58
Enter the time in hours, minutes, and seconds.
hours: 1
minutes: 26
seconds: 42
The average speed is 214.26 km/h
The average time for one lap is 1 min 29 sec
```



程式要使用者輸入 F1 賽道單圈長度，然後輸入比賽圈數，接著輸入跑完整場比賽花了幾小時幾分幾秒。程式就會依照這些數據算出平均速度，以及單圈平均時間。程式裡面包含了一些自動型別轉換以及強制型別轉換，也用到了 % (modulus) 運算。其餘的算式意義應該都很明確，大家可以用 debugger 逐行檢查每個變數算出來的值是否正確。

[**練習 W05_05] 讓使用者輸入一個正整數，譬如 13579，利用 **while** 迴圈以及 % 求餘數，輸出如下的結果
The number 13579 can be written as 9 + 7*10 + 5*100 + 3*1000 + 1*10000.

第六章：C Control Statements: Looping

已經學了如何用 **while** 來做迴圈，再來將會介紹其他的迴圈形式，包括 **for** 和 **do while**，並且練習一些稍微複雜一點的範例。在這之前，先來複習 **while**，讓我們把迴圈用得更熟練。

[範例 E06_01.c]

```
#include <stdio.h>
int main(void)
{
    long num;
    long sum = 0L;
    int status;

    printf("Please enter an integer to be summed ");
    printf("(q to quit): ");
    status = scanf("%ld", &num); /* %ld for long, status is the return value */
    while (status == 1) {        /* == means "is equal to" */
        sum = sum + num;
        printf("Please enter next integer (q to quit): ");
        status = scanf("%ld", &num);
    }
    printf("Those integers sum to %ld.\n", sum);
    return 0;
}
```

這個程式會不斷要求使用者輸入整數，然後當使用者輸入的資料不是數字時，迴圈就會結束，並且把所有數目的總和算出來，譬如：

```
Please enter an integer to be summed (q to quit): 12
Please enter next integer (q to quit): 34
Please enter next integer (q to quit): -56
Please enter next integer (q to quit): 789
Please enter next integer (q to quit): q
Those integers sum to 779.
```

型別 **long** 我們之前比較少用，宣告為 **long** 的變數可以儲存更大範圍的整數值。程式一開始顯示一些資訊請使用者輸入整數，然後就用 **scanf()** 讀取輸入的數值並存到 **num** 中。因為要讀的是 **long**，所以用 **%ld**。另外有個新東西是我們用到了 **scanf()** 的回傳值。其實每次呼叫 **scanf()**，它除了把輸入的資料存入參數之外，還會回傳一個整數值給呼叫者。不過因為我們之前用不到，所以都忽略掉而沒有拿變數去接收這個回傳值。由於 **scanf()** 回傳的是它成功讀取的資料數目，以這個例子來說，如果 **scanf()** 能成功讀到 1 個長整數 (對應到 **%ld**)，它就會回傳 **1**，如果使用者輸入的不是整數 (譬如輸入英文字母 'q')，則 **scanf()** 會回傳 **0** 表示沒有讀到任何整數，所以 **status** 的值就會是 **0**。當進入 **while** 迴圈的時候，會先判斷 **status** 是否等於 **1**，等於 **1** 才表示 **scanf()** 有讀到整數資料。在 C 語言裡判斷兩個數是否相等要用 **==** 符號，也就是兩個等號連在一起。注意，千萬不要錯用成一個等號，因為 **=** 的意義是 *assignment*，也就是設定變數值，剛開始學 C 語言最常犯的錯誤之一就是 **把 == 寫成 =**，大多數時候如果有這樣的誤用，程式 compile 還是會通過，而且也可以執行，但是意義完全不同，所以得到的結果會是錯的。

C Style Reading Loop

前面範例的迴圈，在寫法上通常會被簡化。譬如，原來是

```
status = scanf("%ld", &num);
while (status == 1) {
    sum = sum + num;
    printf("Please enter next integer (q to quit): ");
    status = scanf("%ld", &num);
}
```

會習慣寫成

```
while (scanf("%ld", &num) == 1) {
    sum = sum + num;
    printf("Please enter next integer (q to quit): ");
}
```

這樣的寫法可以省去變數 **status**。這樣的用法等於一次做兩個動作，先利用 **scanf()** 讀入資料，同時判斷回傳值是否等於 **1**。熟練之後，這樣的寫法會蠻簡潔方便。如果還不熟練，寫的時候就要稍微小心不要造成迴圈停不下來。

第六章：C Control Statements: Looping

使用 **while** 迴圈要注意停止條件，看看底下這個錯誤示範

```
int i = 1;
while (i < 5) {
    printf("Hello!\n");
}
```

執行這個程式會發生什麼事？迴圈裡面完全沒有更改 **i** 的值，所以 **i < 5** 的條件會一直成立，迴圈無法停下來，只能用暴力手段把程式停掉。而如果程式改成

```
int i = 1;
while (--i < 5) {
    printf("Hello!\n");
}
```

程式還是錯的，但是有可能會自己停止。當 **i** 不斷遞減變成絕對值越來越大的負數，到了極限之後反而變成絕對值最大的正數，等於繞了一圈循環回來，所以 **i** 的值就比 **5** 大，迴圈就停止了。底下的程式碼可以看出循環的現象。

[範例 E06_02.c]

```
#include <stdio.h>
int main(void)
{
    char i = 1;
    while (--i < 1) {
        printf("%d\n", i);
    }
    printf("%d\n", i);
    return 0;
}
```

```
#include <stdio.h>
int main(void)
{
    char i = 1;
    while (++i > 0) {
        printf("%d\n", i);
    }
    printf("%d\n", i);
    return 0;
}
```

```
#include <stdio.h>
int main(void)
{
    unsigned char i = 1;
    while (++i > 0) {
        printf("%d\n", i);
    }
    printf("%d\n", i);
    return 0;
}
```

由於 **char** 型別只佔一個 byte 所以可以比較快停下來。如果換成 **int** 就要跑比較久。

迴圈何時停止

[範例 E06_03.c]

```
#include <stdio.h>
int main(void)
{
    int n = 1;
    while (n < 3) {
        printf("n = %d\n", n);
        n++;
        printf("Now n = %d\n", n);
    }
    printf("The loop has finished.\n");
    return 0;
}
```

輸出：

```
n = 1
Now n = 2
n = 2
Now n = 3
The loop has finished.
```

當程式第二次執行到 **n++**；之後，**n** 的值變成 **3**，雖然已經不再滿足迴圈預期的條件，但是迴圈並不會在這個時候立刻停止，要等到執行完接下來的 **printf("Now n = %d\n", n);** 然後回到 **while** 的開頭，再次判斷 (**n < 3**) 時，條件不成立，才會整個跳出被 { } 所包含的 **while** 區域。

用來做比較的符號

總共有六種：小於 **<** 小於等於 **<=** 相等 **==** 大於等於 **>=** 大於 **>** 不相等 **!=**

比較新奇的應該只有 **==** 和 **!=**，其他跟我們在算數學的時候用的寫法一樣。在比較兩個數值的大小關係時，如果比較的是浮點數，要特別注意，只能使用 **>** 和 **<**，因為經過一些數學運算，兩個浮點數是否還能完全相等往往會出乎我們意料之外。譬如

```
double a = 0;
```

而如果接下來判斷

```
while (a != 1.0) { a = a + 1.0/6.0; }
```

基本是迴圈是停不下來的，雖然照理說只要執行六次迴圈 **a** 的值就會累加到 **1.0**，但是由於是以數值來儲存 **1.0/6.0**，所以會有誤差，最後使得 **a** 不能精確地等於 **1.0**。

第六章：C Control Statements: Looping

既然使用浮點數無法精確得到相等的數值，我們就只能用近似的方式，當兩個數值的差異小於某個可容忍範圍，就應該接受，讓迴圈停止。在 `math.h` 檔裡有宣告一個叫做 `fabs()` 的 function，可以用來計算浮點數的絕對值。我們用它來算兩個浮點數相減的絕對值。

[範例 E06_04.c]

```
#include <math.h>
#include <stdio.h>
#define ANSWER 3.14159
int main(void)
{
    double response;

    printf("What is the value of pi?\n");
    scanf("%lf", &response);
    while (fabs(response - ANSWER) > 0.0001) {
        printf("Try again!\n");
        scanf("%lf", &response);
    }
    printf("Close enough!\n");
    return 0;
}
```

當使用者輸入的值和 `3.14159` 的差異超過 `0.0001`，迴圈就會繼續要求使用者輸入，直到誤差小於 `0.0001` 才能結束迴圈。

[範例 E06_05.c]

```
#include <stdio.h>
int main(void)
{
    int true_val, false_val;

    true_val = (10 > 2);
    false_val = (10 == 2);
    printf("true = %d; false = %d \n", true_val, false_val);

    return 0;
}
```

輸出：

```
true = 1; false = 0
```

這個程式的目的是要觀察 `true` 和 `false` 對應的整數值到底是多少。描述兩個數值關係的 expression 如果成立 (`true`)，則可以用整數值 `1` 來表示，如果關係不成立 (`false`) 值就是 `0`。所以在 C 程式裡我們可以用 `1` 來代表 `true` 而用 `0` 來代表 `false`。有時候程式為了製造無窮迴圈，會用下面的寫法

```
while (1) {
    ...
}
```

其實在 C 程式裡不只 `1` 可以代表 `true`，任何非零的數都代表 `true`，但是只有 `0` 代表 `false`。看看下面的範例：

[範例 E06_06.c]

```
#include <stdio.h>
int main(void)
{
    int n = 3;

    while (n) {
        printf("%2d is true\n", n--);
    }
    printf("%2d is false\n", n);

    return 0;
}
```

輸出：

```
3 is true
2 is true
1 is true
0 is false
```

只要 `while` 迴圈繼續，就表示 `n` 的值相當於 `true`。

第六章：C Control Statements: Looping

我們來看看如果是底下的範例 (把範例 E06_01.c 更改了一個小地方) · 執行結果會如何。

[範例 E06_07.c]

```
#include <stdio.h>
int main(void)
{
    long num;
    long sum = 0L;
    int status;

    printf("Please enter an integer to be summed ");
    printf("(q to quit): ");
    status = scanf("%ld", &num); /* %ld for long, status is the return value */
    while (status = 1) {          /* == means "is equal to" */
        sum = sum + num;
        printf("Please enter next integer (q to quit): ");
        status = scanf("%ld", &num);
    }
    printf("Those integers sum to %ld.\n", sum);

    return 0;
}
```

當我們輸入整數時，程式看起來還正常，但是當我們想要結束，輸入 'q'，則程式就掉進了無止境的迴圈，自己不停的印出 `printf()` 裡的資訊。這一切都是因為 `status = 1` 這個 expression 寫錯。本來應該用 `status == 1` 判斷 `scanf()` 是否有讀到一個整數。但是寫成了 `status = 1` (變成在做 *assignment* 的動作) 所以 `status` 的值永遠都是 1，所以這個 `while` 迴圈跟 `while(1)` 沒兩樣，因此會不斷要求輸入整數。當你在 `scanf()` 的地方輸入 'q' 的時候，雖然 `scanf()` 會回傳 0 給 `status`，但是到了 `while (status = 1)` 的地方 `status` 又被設成 1。這是迴圈停不下來的主因。至於為什麼程式會開始自動顯示 `printf()` 裡的資訊，而不再等待使用者輸入，這就和 `scanf()` 運作機制有關。當 `scanf()` 讀不到要讀的東西時 (在這個例子裡是長整數 `long`)，它會把讀到但格式不符的資料又丟回 buffer，暫時保留在那裡，留待下次再處理。所以下次再呼叫 `scanf()` 的時候，'q' 依舊在那裡，`scanf()` 讀不到長整數因此依舊回傳 0，依舊把 'q' 留在那裡，就這樣沒完沒了。總之，在寫停止條件時要特別小心，千萬不要讓 expression 變成出乎我們意料的數值，尤其是 `==` 和 `=` 不要錯用。

使用 for 迴圈

假如我們一開始就知道迴圈將會執行多少次，可以使用 `for` 迴圈來達到反覆計算的效果，使用上會比較方便。最標準的狀況就是使用一個 counter 來計算迴圈執行次數，當 counter 達到預定的次數迴圈就停止。

[範例 E06_08.c]

```
#include <stdio.h>
int main(void)
{
    int num;
    int i;

    printf("Enter an integer: ");
    scanf("%d", &num);
    for (i = 0; i < num; i++) {
        printf("$");
    }
    printf("\n");
    return 0;
}
```

輸出：

```
Enter an integer: 30
$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
```

由於已經用 `scanf()` 讓使用者輸入迴圈執行的次數，所以我們知道迴圈將會執行 `num` 所代表的次數。假設我們用變數 `i` 當作 counter，變數 `num` 是要反覆執行的次數，則 `for` 迴圈的語法就是

```
for (i = 0; i < num; i++) {
    ...
}
```

這個語法包含三個部份，用兩個分號隔開成三個區域，第一個部份用來設定 counter 的初值，只會被執行一次。第二個部份判斷是否應該繼續執行。第三個部分可用來更改 counter 的值，每跑回一次迴圈就會再被執行一次。

第六章：C Control Statements: Looping

學會了 **for** 迴圈之後很多事做起來就會變得蠻方便。底下這些範例做的事情都很容易理解，主要是讓大家熟悉 **for** 的語法。

[範例 E06_09.c]

```
#include <stdio.h>
int main(void)
{
    int n;

    for (n = 2; n < 60; n = n + 13) {
        printf("%d \n", n);
    }
    return 0;
}
```

輸出：

```
2
15
28
41
54
```

[練習 W06_01] 用 **for** 迴圈計算 0 到 100 之間，奇數數字的平方和。

[練習 W06_02] 用 **for** 迴圈——列出 1 到 100 每個數字的立方。

[範例 E06_10.c]

```
#include <stdio.h>
int main(void)
{
    char ch;

    for (ch = 'a'; ch <= 'z'; ch++) {
        printf("The ASCII value for %c is %d.\n", ch, ch);
    }
    return 0;
}
```

所以 **for** 迴圈的語法就是

```
for ( 初始化 initialize ; 是否繼續執行的條件判斷 test ; 更新變數值 update ) {
    ...
}
```

其中三個區域的每個區域所做的事情還可稍加變化。譬如如果什麼都不做，寫成

```
for ( ; ; ) {
    printf("Do something.\n");
}
```

作用則相當於無窮迴圈。還可以像底下的範例這樣寫。

[範例 E06_11.c]

```
#include <stdio.h>
#define CODE 7
int main(void)
{
    int num = 0;

    for (printf("Keep entering numbers!\n"); num != CODE; ) {
        scanf("%d", &num);
    }
    printf("Bingo!\n");
    return 0;
}
```

輸出：

```
Keep entering numbers!
1
6
3
7
Bingo!
```

這個程式會不斷讓使用者輸入數字，直到使用者猜到密碼是 7 為止。比較奇怪的地方是在初始化的地方偷渡了一個 **printf()** 的動作，所以剛進入迴圈時會顯示一次 "Keep entering numbers!"。第二部份條件判斷檢查是否輸入的數字等於預設密碼。第三部份則沒有東西，因為 **num** 靠迴圈裡的 **scanf()** 來做更新的動作。

第六章：C Control Statements: Looping

其他 Assignment 符號：+=, -=, *=, /=, %=

這些可看成是簡寫的 assignment 符號， $x += 5$ ；相當於 $x = x + 5$ ；而 $y %= 7$ ；相當於 $y = y \% 7$ ；以此類推。在 **for** 迴圈的第三部份 (更新變數值) 用這樣的寫法會比較簡潔，但你不一定要用這樣的寫法，只要順著自己的習慣就可以。知道有這樣的寫法，看到別人的程式用到這種寫法才能知道它的意思。

在 **for** 迴圈的第一部份 (初始化) 和第三部份 (更新變數值)，其實允許我們做兩個以上的 statements。

[範例 E06_12.c]

```
#include <stdio.h>
#define FIRST_PACK 7
#define NEXT_PACK 5
int main(void)
{
    int n, cost;
    printf(" packs   costs\n");
    for (n=1, cost=FIRST_PACK; n<=10; n++, cost+=NEXT_PACK) {
        printf("%5d   $%-5d\n", n, cost);
    }
    return 0;
}
```

輸出：

packs	costs
1	\$7
2	\$12
3	\$17
4	\$22
5	\$27
6	\$32
7	\$37
8	\$42
9	\$47
10	\$52

在初始化的部份做了兩個動作，兩個動作用逗號分開，分別是設定 $n=1$ 以及設定 $cost=FIRST_PACK$ 。更新變數值的部份也做了兩個動作，先做 $n++$ 接著做 $cost+=NEXT_PACK$ ，這樣就能讓變數 n 的值被加一 (負責累計迴圈反覆次數)，而且也累計 $cost$ 值。在這裡用逗號隔開的兩個動作是以循序方式執行，也就是做完第一個動作才做第二個。

使用 do ... while 迴圈

某些情況下，迴圈會保證至少被執行一次，這時候就適合用 **do ... while** 迴圈，它的效果是先做迴圈內容，最後在判斷條件是否要繼續，譬如下面的例子，要求使用者要輸入密碼，直到輸入正確為止。由於這個動作至少要做一次，所以用 **do ... while** 程式會比較簡潔，它的語法如下：

[範例 E06_13.c]

```
#include <stdio.h>
#define CODE 13
int main(void)
{
    int code_entered;

    do {
        printf("Please enter the secret code number: ");
        scanf("%d", &code_entered);
    } while (code_entered != CODE);
    printf("Bingo!\n");
    return 0;
}
```

要注意在 **do ... while** 的用法裡面，**while** 的條件判斷之後有個分號，不能漏掉，這是 **do ... while** 和標準 **while** 的語法不同之處。可以比較看看，如果用 **while** 而不是 **do ... while**，寫起來會有點累贅：

```
#include <stdio.h>
#define CODE 13
int main(void)
{
    int code_entered;

    printf("Please enter the secret code number: ");
    scanf("%d", &code_entered);
    while (code_entered != CODE) {
        printf("Please enter the secret code number: ");
        scanf("%d", &code_entered);
    }
    printf("Bingo!\n");
    return 0;
}
```

第六章：C Control Statements: Looping

Nested Loops 多重迴圈

更複雜的迴圈使用方法是有一個迴圈包住另一層迴圈。使用雙重迴圈可以讓我們很簡單就做到許多複雜功能。雙重迴圈最常用來處理以二維霍表格方式呈現的資料。譬如我們想要輸出一個用 * 號填滿的長方形，寬和高分別是 25 和 7。我們已經學過用一層迴圈反覆輸出，所以要印出一排 25 個星號可以用下面的寫法

```
for (j = 0; j < 25; j++) {
    printf("*");
}
printf("\n");
```

顯示了一串 25 個 * 之後，再用 `printf("\n");` 換行。接下來如果我們再用一個迴圈把上面的程式碼包起來，變成

```
for(i = 0; i < 7; i++) {
    for (j = 0; j < 25; j++) {
        printf("*");
    }
    printf("\n");
}
```

就會重複做 7 次，而每次都會顯示一排 25 個 * 號。

[範例 E06_14.c]

```
#include <stdio.h>
int main(void)
{
    int i, j;
    for(i = 0; i < 7; i++) {
        for (j = 0; j < 25; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

輸出：

```
*****
*****
*****
*****
*****
*****
*****
```

[練習 W06_03] 修改上面範例，讓使用者輸入要顯示的長方形的長和寬。

[練習 W06_04] 輸出九九乘法表的內容。(只需要把上面範例裡的條件判斷改調，讓內外層迴圈各做九次，另外再把 `printf("*");` 改成顯示 `i*j`。)

[範例 E06_15.c]

```
#include <stdio.h>
int main(void)
{
    int row;
    char ch;

    for(row = 0; row < 6; row++) {
        for (ch = ('A' + row); ch < ('A' + 10); ch++) {
            printf("%c", ch);
        }
        printf("\n");
    }

    return 0;
}
```

輸出：

```
ABCDEFHIJ
BCDEFHIJ
CDEFHIJ
DEFGHIJ
EFGHIJ
FGHIJ
```

外層迴圈負責反覆六次印出六行字串，內層迴圈則負責在每一行顯示連續的英文字母，每一行顯示的字母範圍都不一樣。

[練習 W06_05] 輸出九九乘法表的下半部，也就是只顯示 `1*1, 2*1, 2*2, 3*1, 3*2, 3*3, 4*1, ...` 的相乘結果。

第六章：C Control Statements: Looping

陣列 (Arrays) 使用方法簡介

我們在後面的課程會詳細介紹陣列。但是其實陣列的基本概念也不複雜，而且我們也用過字元陣列，所以我們就先簡介陣列的使用方法，這樣接下來的程式也有更多寫法可以運用。陣列就是一連串相同型別的資料存放在連續的空間中，譬如 15 個 **int** 型別的資料，或是十個 **char** 字元。整個陣列的內容只需要用一個名字來統稱，而其中每個元素可以藉由索引 **index** 方式取出。宣告陣列的方法如下

```
float nums[20];
```

這樣就表示 **nums** 是一個陣列，包含 20 個元素，每個元素可用來記錄一個型別為 **float** 的數值。陣列的第一個元素叫做 **nums[0]**，第二個元素是 **nums[1]**，以此類推，最後一個元素是 **nums[19]**。所以 C 的陣列編號方式是從 **0** 開始編號而不是從 **1** 開始。這一點要特別注意，因為這也是初學者常忘記的規則，用錯了會產生 **bug**。每個元素可以當作一個變數來使用，譬如

```
nums[3] = 3.4;
nums[9] = 18.5;
```

或是

```
scanf("%f", &nums[4]); /* 讀入一個小數存放在第五個元素裡 */
```

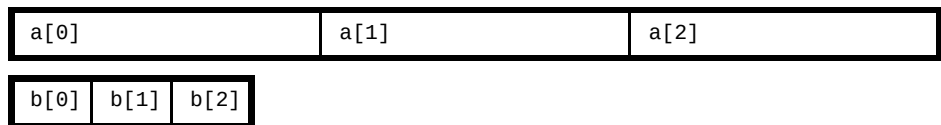
由於 C 並不會去檢查我們給的 **index** 是否超出當初宣告的陣列大小範圍，所以可能會寫出下面有 **bug** 的程式而沒有察覺，**compiler** 也不會跟我們說程式有 **error**

```
nums[20] = 2.5; /* A BUG */
nums[30] = 3.6; /* A BUG */
```

程式 **compile** 之後，執行到像是上面那兩行的時候，程式非常可能就會當掉，因為它試圖去讀取錯誤的記憶體位置中的東西。

我們還可以宣告其他類型的陣列，譬如

```
int a[3];
long c[1000];
char b[3];
```



這裡順便複習一下，字元陣列和字串的差別只在最後是否有 **'\0'** 結束符號，如果要當作字串來使用，字元陣列要加入 **'\0'** 當作結束字元。不同型別的陣列佔用的記憶體空間也不同，例如 **int** 陣列每個元素佔用四個 **bytes**，而 **char** 陣列每個元素只佔一個 **byte**。

搭配陣列來使用迴圈

[範例 E06_16.c]

```
#include <stdio.h>
int main(void)
{
    int i;
    float hours[7], average;
    printf("Enter the hours of sleep per night last week\n");
    for(i = 0; i < 7; i++) {
        scanf("%f", &hours[i]);
    }
    printf("The numbers you entered are as follows:\n");
    for(i = 0; i < 7; i++) {
        printf("  %4.1f", hours[i]);
    }
    printf("\n");
    for(i = 0, average = 0; i < 7; i++) {
        average += hours[i];
    }
    average /= 7;
    printf("Your average hours of sleep per night were %.1f hours.\n", average);
    return 0;
}
```

輸出：

```
Enter the hours of sleep per night last week
9 10 4 12 13 14 3.5
The numbers you entered are as follows:
  9.0  10.0   4.0  12.0  13.0  14.0   3.5
Your average hours of sleep per night were 9.4 hours.
```

第六章：C Control Statements: Looping

前面的範例先用迴圈讓使用者輸入七個數字，分別存在陣列裡，然後再用迴圈——把陣列裡的元素再顯示出來，然後再用迴圈把元素的值累加起來，最後計算出平均值。有幾個要注意的地方，第一是迴圈裡的用來當陣列 index 的變數要從 0 開始；第二是用來累加總和並計算平均的變數 **average** 要先設定初值等於 0；第三是 **scanf()** 裡陣列元素的寫法，當我們要把值存入某個陣列元素時，別忘了加 **&** 符號，例如 **&hours[1]**，只要記得每個陣列元素其實就相當於我們平常在使用的變數，所以要遵守的原則都相同。

[範例 E06_17.c]

```
#include <stdio.h>
double power(double n, int p);

int main(void)
{
    double x, xpow;
    int exp;

    printf("Enter a number and the positive integer power\n");
    printf("to which the number will be raised. Enter q to quit.\n");
    while (scanf("%lf%d", &x, &exp) == 2) {
        xpow = power(x, exp);
        printf("%.3f to the power %d is %.5f\n", x, exp, xpow);
        printf("Enter next pair of numbers or q to quit.\n");
    }
    printf("\nBye!\n");
    return 0;
}

double power(double n, int p)
{
    double pow = 1;
    int i;

    for (i = 1; i <= p; i++) {
        pow *= n;
    }

    return pow;
}
```

這個範例主要是要介紹如何自己寫一個 function 來計算某個數的整數次方。這個自定的 function 取名為 **power()**，在主程式的前面要先宣告 **power()**，把它需要的參數型別，以及回傳值的型別都指定好。所以由宣告可以看出 **power()** 要傳入兩個參數，分別是型別為 **double** 的底數 **n** 以及型別為 **int** 的指數 **p**。至於 function 主體 (definition) 的部份則在 **main()** 後面，我們用了迴圈來達到計算次方的效果。首先 **pow** 的值被設成 1，然後迴圈要做 **p** 次，每次都會把 **pow** 乘上 **n** 再存回 **pow** 自己。所以做了 **p** 次之後 **pow** 的值就等於 **n** 的 **p** 次方。最後把 **pow** 的值傳回去，用的語法是 **return pow;**。有了自己訂的 **power()**，就可以在主程式裡重複呼叫。這樣的模組化程式設計方式是我們學習 C 語言要多加練習的重點之一。我們把較複雜而會重複使用的程式碼寫成 function，不僅可以讓程式看起來更簡潔，而且程式會比較有結構、比較容易理解、也比較容易修改。當程式越寫越龐大的時候，這項優勢就會更加突顯。

[練習 W06_06] 寫一個 function 叫做 **rectangle()**，它需要傳入三個參數，分別是兩個 **int**，各代表寬和高，以及一個 **char**，function 要依照寬和高畫出長方形，長方形用第三個參數所指定的字元來填滿。最後 function 還會把長方形面積當作回傳值傳回去。另外，主程式的部份則是在 **main()** 裡面呼叫兩次 **rectangle()**，分別畫出一個 7 乘 5 用 '#' 填滿的長方形，以及一個 12 乘 9 用 '@' 填滿的長方形，並且把面積印出來。

[**練習 W06_07] 讓使用者輸入一個大寫英文字母，譬如 'E'，然後輸出像右圖一樣排列的形狀。提示：總共需要兩層迴圈。外層用一個迴圈負責跑每一行，然後裡面包了三個迴圈，第一個內層迴圈負責印出適當數量的空白字元，第二個內層迴圈負責以遞增順序印出字母，然後第三個內層迴圈以遞減順序印出字母。

```
A
ABA
ABCA
ABDCBA
ABCDEDCBA
```

[*練習 W06_08] 讓使用者輸入一個字串 (譬如一個英文單字)，把這個字存入 **char** 陣列中，然後把整個字以相反順序印出來。譬如輸入 **dog**，則輸出的結果會是 **god**。

提示：用 **strlen()** 取得字串長度 (要記得 **#include <string.h>**)。另外在取出陣列元素時，index 編號要小心。

第六章：C Control Statements: Looping

[補充] 用多重迴圈輸出九九乘法表上半部

```
#include <stdio.h>
int main(void)
{
    int i, j;

    for (i = 1; i <= 9; i++) {
        for (j = 1; j <= i; j++) {
            printf(" ");
        }
        for ( ; j <= 9; j++) {
            printf("%3d", i*j);
        }
        printf("\n");
    }

    return 0;
}
```

輸出：

1	2	3	4	5	6	7	8	9
4	6	8	10	12	14	16	18	
9	12	15	18	21	24	27		
	16	20	24	28	32	36		
		25	30	35	40	45		
			36	42	48	54		
				49	56	63		
					64	72		
						81		

左下角有一個隱藏的空白三角形，是用內層 `j` 迴圈的的第一部份畫出來。

[補充] 用多重迴圈畫出空心的長方形

```
#include <stdio.h>
int main(void)
{
    int w, h, d;
    int i, j;

    scanf("%d%d%d", &w, &h, &d);

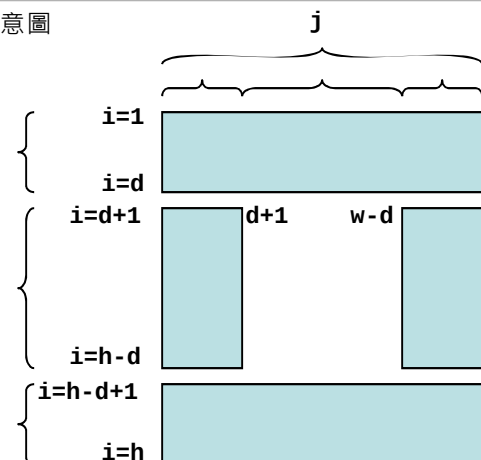
    for (i = 1; i <= d; i++) {
        for (j = 1; j <= w; j++) {
            printf("");
        }
        printf("\n");
    }
    for ( ; i <= h-d; i++) {
        for (j = 1; j <= d; j++) {
            printf("");
        }
        for ( ; j <= w-d; j++) {
            printf(" ");
        }
        for ( ; j <= w; j++) {
            printf("");
        }
        printf("\n");
    }
    for ( ; i <= h; i++) {
        for (j = 1; j <= w; j++) {
            printf("");
        }
        printf("\n");
    }

    return 0;
}
```

輸出：

```
15 8 2
*****
*****
**          **
**          **
**          **
**          **
*****
*****
```

示意圖



關鍵是要能正確設定"負責控制迴圈次數的變數"的初始值和結束值。

第六章：C Control Statements: Looping

[補充] 字元陣列的讀寫

```

#include <stdio.h>
#include <string.h>
int main(void)
{
    int i;
    char word[50];           /* word 字元陣列用來記錄使用者輸入的字串 */
    char out[100];          /* out 是用來暫存輸出的結果 */
    int len;

    scanf("%s", word);      /* 讓使用者輸入一個字串 */
    len = strlen(word);     /* 計算使用者輸入的字串長度 */

    for (i = 0; i < len; i++) { /* 用迴圈把整個 word 從頭到尾看一遍 */
        out[i] = word[i];      /* 把 word 逐字複製到 out 對應的位置 */
        out[2*len-i-1] = word[i]; /* out 另外一邊對稱的位置也要填上同樣的字元 */
    }
    out[2*len] = '\0';      /* 在 out 的後面填上 '\0'，讓 out 變成字串 */
    printf("%s\n", out);    /* 這樣就可以用 printf() 配合 %s 格式直接輸出整個字串 */

    for (i = 0; i < 2*len; i++) { /* 先把 out 用空白字元填滿 (注意: '\0' 還在，不會被蓋掉) */
        out[i] = ' ';
    }

    for (i = 1; i < len; i++) { /* 接下來要顯示垂直的部份 */
        out[0] = word[i];      /* 把 out 的第一個字元填上 word[i] */
        out[2*len-1] = word[i]; /* 把 out 最後面的字元填上 word[i] */
        printf("%s\n", out);  /* 把 out 輸出 */
    }

    for (i = len-1; i > 0; i--) { /* 接下來要顯示垂直的另一半對稱的部份，迴圈要倒著跑 */
        out[0] = word[i];      /* 把 out 的第一個字元填上 word[i] */
        out[2*len-1] = word[i]; /* 把 out 最後面的字元填上 word[i] */
        printf("%s\n", out);  /* 把 out 輸出 */
    }

    for (i = 0; i < len; i++) { /* 重複做一次第一部份的迴圈 */
        out[i] = word[i];
        out[2*len-i-1] = word[i];
    }
    printf("%s\n", out);      /* 因為 out 的結尾 '\0' 並沒有被更動，所以不用再填 */

    return 0;
}

```

底下是程式執行的情況：

```

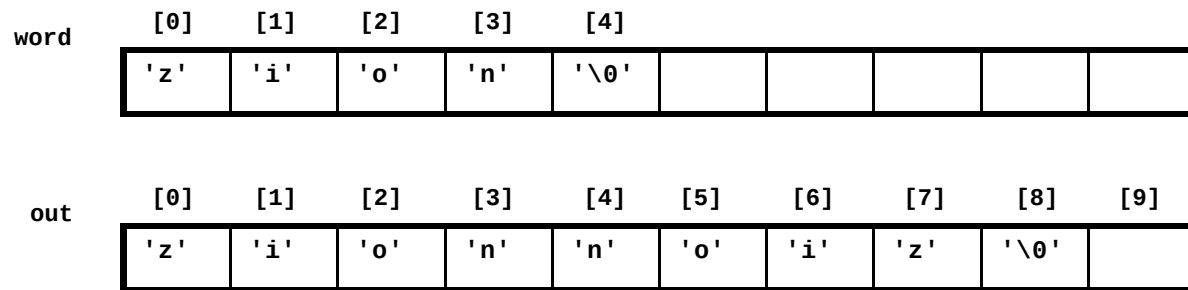
zion
zionnoiz
i      i
o      o
n      n
n      n
o      o
i      i
zionnoiz

```

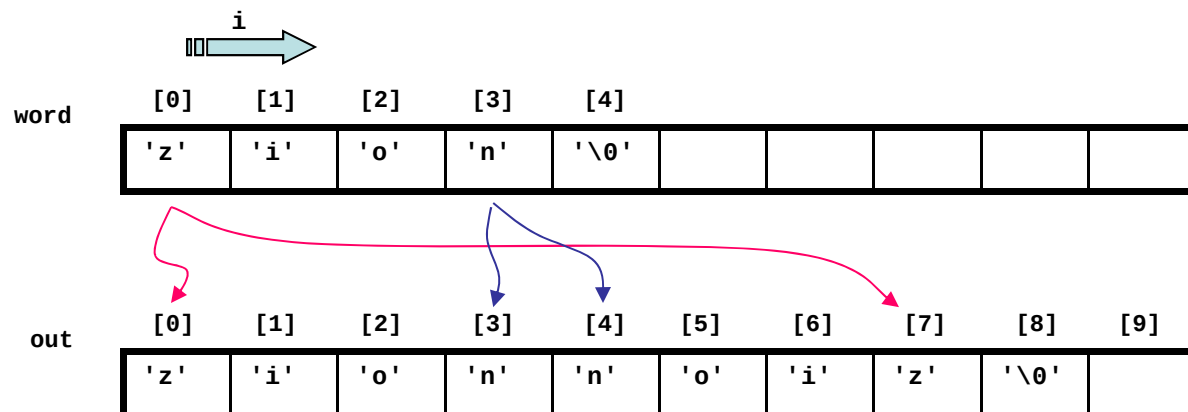
請看下一頁的圖文說明

第六章：C Control Statements: Looping

首先要想辦法產生下面的東西 (注意陣列的索引編號)：

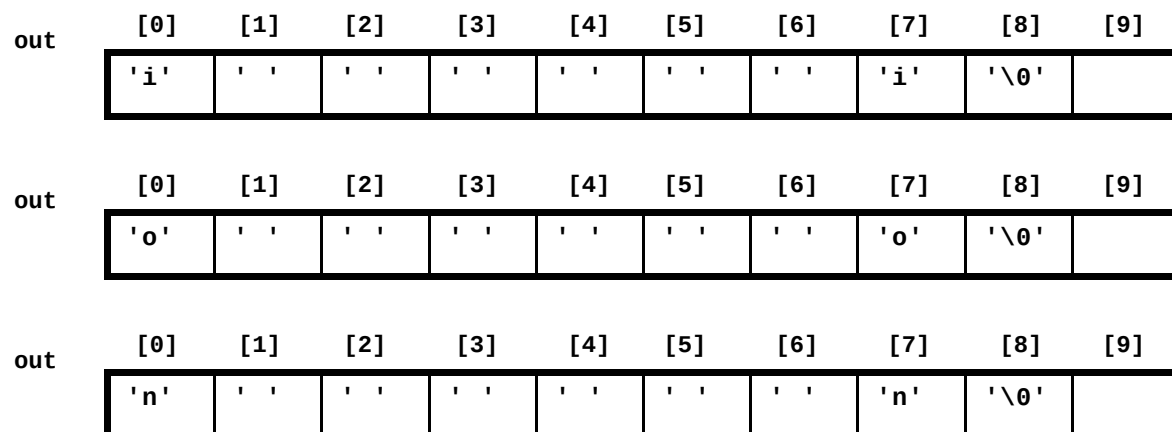


做法是用迴圈把對應位置該放的字元填好：



填好 **out** 之後，就可以用 `printf("%s\n", out);` 把 **out** 的內容整個顯示出來。

接下來垂直的部份，只需要依序產生下面的陣列。先把 **out** 清空，然後在頭尾填上該放的字元。(在這個例子該放的字元分別是要從 'i'，'o'，'n'，也就是 `word[1]`，`word[2]`，`word[3]`。)



完成前面兩個部份之後，剩下的部分只要把前兩個動作反過來再做一次就可以。

整個程式的關鍵在於如何設定正確的陣列索引 (index)，在陣列中適當的位置放入該放的字元。要特別注意陣列的索引要從 0 開始。字串的最後一個字元要加上 '\0'。

第七章：C Control Statements: Branching

使用 if Statement

[範例 E07_01.c]

```

#include <stdio.h>
#define PAR 5
int main(void)
{
    float hours;
    int tired_days = 0;
    int all_days = 0;

    printf("Please enter the hours of sleep per night during the past few days.\n");
    printf("(q to quit): ");
    while (scanf("%f", &hours) == 1) {
        all_days++;
        if (hours < PAR) {
            tired_days++;
        }
    }
    if (all_days != 0)
        printf("%.1f%% of the time you didn't sleep enough.\n",
            100.0 * (float) tired_days / all_days);
    if (all_days == 0)
        printf("No data entered!\n");

    return 0;
}

```

這個程式執行起來像底下這樣：

```

Please enter the hours of sleep per night during the past few days.
(q to quit): 4 4 5 3 4.5 q
80.0% of the time you didn't sleep enough.

```

迴圈 **while** 用來讓使用者反覆輸入睡眠時數，直到輸入 'q' 才結束。在 **while** 迴圈裡用一個 **if** 來判斷輸入的睡眠時數是否低於標準值，如果低於標準就把 **tired_days** 的值加一。同時在迴圈中會累計 **all_days**，來計算總共輸入了幾筆資料。在迴圈結束後也有兩個 **if** 判斷式，它們用來判斷使用者是否輸入了任何數字，如果有就把低於標準值的比率印出來，如果使用者沒有輸入任何資料而只是直接打 'q'，則會因為完全沒進入迴圈，所以 **all_days** 的值還是零，這時就要印出 "No data entered!\n"。

從上面的例子應該可以觀察到，**if** 的語法就是

```

if (expression)
    statement

```

其中 *expression* 是用來做判斷的條件，如果 *expression* 得到的值不為零 (代表 true)，則就會執行 *statement* 所包含的動作，如果 *expression* 的值是零 (代表 false)，則略過 *statement*，進行 **if** 之後的程式。注意，*expression* 外面一定要用括號 () 包住。在 **if** 的用法裡，最標準的 *expression* 是關係判斷式，譬如：**a > b** 或 **c == 3**。從上面的例子還可以看出，我們用了兩種 *statement* 寫法，第一種是

```

if (hours < PAR) {
    tired_days++;
}

```

這樣的 *statement* 是由一個 block 構成，在括號 { } 中間可以包含多個 statements，如果 **if** 的條件成立，整個 block 中的 statements 都會被執行。當我們想在 **if** 條件下執行多個動作時，一定要用這種形式。另外一種簡便的形式是當 **if** 只要做單一動作時，可以用

```

if (all_days == 0)
    printf("No data entered!\n");

```

直接接一個 statement。這種情況可以省略波浪括號 { }，但僅限單一 statement 時才能這樣用。

使用 if ... else ...

當 **if** 條件只有兩種可能的情況時，譬如 **a == 0** 或 **a != 0**，一旦其中一個條件成立，另一個一定就不會成立。這時候可以用 **if ... else ..** 的寫法，會比較省事。

第七章：C Control Statements: Branching

譬如範例 E07_01.c 裡

```
if (all_days != 0)
    printf("%.1f%% of the time you didn't sleep enough.\n",
           100.0 * (float) tired_days / all_days);
if (all_days == 0)
    printf("No data entered!\n");
```

可以寫成

```
if (all_days != 0)
    printf("%.1f%% of the time you didn't sleep enough.\n",
           100.0 * (float) tired_days / all_days);
else
    printf("No data entered!\n");
```

因為只要 `all_days != 0` 成立，`all_days == 0` 其實可以完全不用再測試，一定會成立。

從上面的例子應該可以觀察到，`if ... else ...` 的語法就是

```
if (expression)
    statement1
else
    statement2
```

其中 `statement1` 和 `statement2` 都可以是由波浪括號 `{ }` 包含的多個 statements。

使用 `getchar()` 以及 `putchar()`

到目前為止我們使用的輸入輸出 function 只有 `scanf()` 和 `printf()`。我們接下來要看一組針對 `char` 的輸入輸出 function：`getchar()` 和 `putchar()`。這兩個 function 也是在 `stdio.h` 裡宣告，所以只要有 `#include <stdio.h>` 就可以在程式裡使用它們。

[範例 E07_02.c]

```
#include <stdio.h>
#define SPACE ' '
int main(void)
{
    char ch;

    ch = getchar();          /* read a character */
    while (ch != '\n') {     /* while not end of line */
        if (ch == SPACE)    /* leave the space */
            putchar(ch);    /* character unchanged */
        else
            putchar(ch + 1); /* change other characters */
        ch = getchar();      /* get next character */
    }
    putchar(ch);             /* print the newline */

    return 0;
}
```

輸出：

```
CALL ME HAL.
DBMM NF IBM/
```

這個程式做的事情有點像是做編碼加密的動作（非常容易破解的加密方法）。程式用 `getchar()` 讀取使用者輸入的字元，然後判斷字元是否等於 `'\n'`（按下 Enter 鍵）。如果不是就會判斷使用者剛才輸入的字元是否是空白，如果是的話就不做處理，把空白直接用 `putchar()` 送出，如果不是空白，就會把字元的 ASCII code 加 1 當作新的編碼，然後用 `putchar()` 輸出，接著再使用者輸入的字元並繼續 `while` 迴圈。

所以 `gechar()` 的用法就是

```
ch = getchar();
```

它會傳回讀取的字元，所以我們用字元變數 `ch` 來接收。而 `putchar()` 的用法則是

```
putchar('A');
```

把字元當參數傳入 `putchar()` 中。

我們可以再修改上面的範例，讓它變得更簡潔。我們可以用下面這樣的寫法：

```
while ((ch = getchar()) != '\n') {
    ...
}
```

第七章：C Control Statements: Branching

這樣的寫法把 assignment 的動作和比較的動作合在一起：先做了 `(ch=getchar())` 取得輸入的字元，再比較 `ch` 得到的值是否不等於 `'\n'`，不等於才會做 `while` 迴圈。此外，從 `putchar(ch + 1);` 再次看出，當字元要拿來做計算的時候，譬如 `ch + 1`，會把 `ch` 自動從 `char` 型別轉成 `int` 型別。(其實 `putchar()` 要傳入的參數型別也是 `int`。)

使用 ctype.h 中的用來處理字元的 functions

我們來看看如何利用 ctype.h 所提供的字元處理 functions 來改良前一個範例。

[範例 E07_03.c]

```
#include <stdio.h>
#include <ctype.h>
int main(void)
{
    char ch;

    while ((ch = getchar()) != '\n') {
        if (isalpha(ch)) /* if a letter, */
            putchar(ch + 1); /* change */
        else /* otherwise, */
            putchar(ch); /* print as is */
    }
    putchar(ch); /* print the newline */

    return 0;
}
```

範例裡用了 `isalpha()` 來判斷字元是否是英文字母。為了用 `isalpha()` 要先在程式前面加上 `#include <ctype.h>`。如果 `ch` 字元是英文字母，`isalpha(ch)` 就會得到非零的值 (代表 true)，所以就會執行 `putchar(ch + 1);`。如果 `ch` 不屬於英文字母則 `isalpha(ch)` 會傳回 0 (代表 false)，所以就執行 `putchar(ch)`，直接把字元印出來。執行程式會得到類似下面的效果：

```
Look! It's a programmer!
Mppl! Ju't b qsphsbnnfs!
```

只有英文字母被加密。

還有許多其他有用的字元處理 functions，大家可以查看課程網頁上的參考資料連結，*The GNU C Library Reference Manual* (檔案是 `glibc-manual-2.2.5.pdf`)，其中 *Chapter 4: Character Handling* 有詳細的介紹。

我們在這裡只再多介紹一組 functions，叫做 `tolower()` 和 `toupper()`，它們可以用來把大寫字母轉成小寫，或是把小寫轉大寫。它們的用法是 `ch = tolower(ch);` 以及 `ch = toupper(ch);`。要把字元傳入，然後用變數接收它們的回傳值。

[練習 W07_01] 讓使用者輸入一串字元 (一樣是讀到 `'\n'` 就結束)，把這串字元裡所有的英文字母都轉成大寫再輸出。

使用 else if ...

[範例 E07_04.c]

```
#include <stdio.h>
#include <ctype.h>
int main(void)
{
    char ch;

    while ((ch = getchar()) != '\n') {
        if (isupper(ch)) /* line 8 */
            ch = tolower(ch); /* line 9 */
        else if (islower(ch)) /* line 10 */
            ch = toupper(ch); /* line 11 */
        putchar(ch);
    }
    putchar(ch);

    return 0;
}
```

輸出：

```
C Control Statements: Branching!
c cONTROL sTATEMENTS: bRANCHING!
```

第七章：C Control Statements: Branching

首先還是用 `while ((ch = getchar()) != '\n')` 的方式把 assignment 和 條件判斷寫在一起，先取得 `ch`，再比較 `ch` 是否等於 `'\n'`。進入 `while` 迴圈之後，有三種可能的狀況，1) 如果 `ch` 是大寫字母，`isupper(ch)` 得到非零值 (代表 true)，就把 `ch` 轉成小寫。2) 如果 `ch` 不是大寫，就會進入 `else if (islower(ch))` 的判斷，檢查看看 `ch` 是否為小寫字母，如果是就把 `ch` 轉成大寫。3) 最後如果不是大寫也不是小寫，就什麼轉換都不做，所以我們就直接把原本該有的 `else` 的部份省略。迴圈裡的最後一行再用 `putchar(ch)` 把字元印出來。注意：第 8 行和第 10 行這兩個條件判斷，一但第一個 `if` 條件成立，就不會再去判斷 `else if` 裡的條件，所以在 `if` 裡面可以對 `ch` 做修改。只有當第一個 `if` 不成立時，才會做 `else if`。我們可以來看看，假如不使用 `else if`，而是用下面這樣的寫法，是否行得通

```
if (isupper(ch))
    ch = tolower(ch);
if (islower(ch))
    ch = toupper(ch);
```

用這樣的寫法，每個 `if` 就變成各自獨立，所以判斷完第一個 `if` 之後，不管條件是否成立，都還是會繼續判斷第二個 `if`。這樣可能會發生一種錯誤情況，假如 `ch` 原本是大寫字母，由於 `(isupper(ch))` 得到 true，所以會做 `ch = tolower(ch);`，做完之後 `ch` 就變成小寫字母。接著又會判斷 `if (islower(ch))`，這時候由於 `ch` 已經被轉成小寫，所以條件又成立，就會做 `ch = toupper(ch)`，所以 `ch` 又被轉成大寫，這樣整個程式就等於白忙了一場。

來看看底下這個有點複雜的範例。它的功能是讓使用者輸入一個正整數，然後程式會列出除了 1 和該正整數本身以外的所有因數。如果無法因數分解，則程式就會顯示這個正整數是質數。

[範例 E07_05.c]

```
#include <stdio.h>
int main(void)
{
    unsigned long num, div;
    int isPrime;

    printf("Please enter an integer (q to quit): \n");
    while (scanf("%lu", &num) == 1) {
        for (div = 2, isPrime = 1; (div * div) <= num; div++) {
            if (num % div == 0) {
                if ((div * div) != num)
                    printf("%lu, %lu, ", div, num / div);
                else
                    printf("%lu, ", div);
                isPrime = 0;
            }
        }
        if (isPrime)
            printf("%lu is prime.\n", num);
        else
            printf("\b\b are divisors of %lu\n", num);
        printf("Please enter another integer (q to quit): \n");
    }
    printf("Bye.\n");

    return 0;
}
```

這個範例用到不少有用的程式技巧，我們來一一解釋。在主程式一開始的地方我們宣告了 `num` 和 `div` 兩個型別為 `unsigned long` 的變數，以及一個整數 `isPrime`。我們會把 `isPrime` 當作 flag 來使用，用來記錄正整數是否是質數。迴圈 `while` 用 `scanf()` 讀取使用者輸入的正整數並存放在 `num`，如果確實有讀到一個正整數，就可以進入迴圈，接著就要檢查 `num` 是否可以因數分解。檢查 `num` 可否分解要靠 `for` 迴圈，變數 `div` 代表除數，一開始設為 2，我們用迴圈不斷增加 `div` 的值 (`div++`)，直到 `div*div` 超過 `num` 為止。(不需要檢查從 2 到 `num` 之間所有的數目，因為因數分解是對稱的，如果所有小於 `num` 開根號的數都無法整除 `num` 的話，接下來的數字也就不用再檢查了，這樣可以省去非常多沒必要的檢查時間，對程式執行速度有很大的影響。) 我們用 `isPrime` 來記錄 `num` 是否不可以被整除，一開始設為 1，也就是先假定 `num` 是個質數，等到 `for` 迴圈進行中發現某個 `div` 可以整除 `num`，則我們原先的假定就破滅，那時候就把 `isPrime` 設為 0。如果 `for` 迴圈進行完後 `isPrime` 都還能維持值是 1，表示我們完全無法找到可以整除 `num` 的 `div`，也就表示 `num` 是個質數。檢查是否可以整除，用的方法是 `num % div == 0`。另外，每組因數包含 `div` 和 `num/div`，為了避免完全平方的情況出現時，同一個因數會被列出兩次，所以我們又判斷是否 `div*div != num`，如果不是，則表示 `num` 是 `div` 的平方，這時候只要印出 `div` 就夠了，不用印 `num/div`。

第七章：C Control Statements: Branching

[**練習 W07_02] 修改範例 E07_05.c (原本是以交替的次序輸出因數)，讓程式能夠由小到大輸出因數。

總結 **if** statement 的語法有下面三種形式

```

if (expression)
    statement

if (expression)
    statement1
else
    statement2

if (expression1)
    statement1
else if (expression2)
    statement2
else
    statement3

```

我們可以把它們任意組合或一層層包在一起。寫程式的時候要注意條件判斷一定要加括號。

邏輯運算

[範例 E07_06.c]

```

#include <stdio.h>
#include <ctype.h>
int main(void)
{
    int ch;
    int digitcount = 0;

    while ((ch = getchar()) != 'q' && ch != 'Q') {
        if (isdigit(ch))
            digitcount++;
    }
    printf("There are %d decimal digits.\n", digitcount);
    return 0;
}

```

這個程式讓使用者不斷輸入字元，直到輸入 'q' 或 'Q' 才結束。迴圈中會用 **isdigit()** (也是在 **ctype.h** 裡宣告的 function) 檢查 **ch** 是否是數字，如果是數字就累加 **digitcount**。我們用到了 **&&** 符號，它的意思是邏輯上的 AND，譬如 **(a > 2) && (b != 0)**，就是要判斷 **(a > 2)** 且 **(b != 0)** 是否同時成立，只有兩者皆為 true 時，**(a > 2) && (b != 0)** 整體的值才會是 true。除此之外，**&&** 的運算是從左到右，所以我們在範例裡會先判斷

```
(ch = getchar()) != 'q'
```

這時候 **ch** 就取得了輸入的字元，接下來在 **&&** 右邊就可以直接判斷

```
ch != 'Q'
```

所以整個判斷條件的用意就是當輸入的字元不等於 'q' 而且也不等於 'Q'，迴圈才會繼續。

除了 **&&** 之外，還有 **||** 和 **!**。符號 **||** 代表邏輯的 OR 運算，譬如 **(a != 0) || (b != 0)** 表示只要 **a** 或 **b** 其中之一不等於零，整個 expression 的值就是 true。所以假如 **a** 的值是 1 而 **b** 的值是 0，則 **(a != 0) || (b != 0)** 還是成立。至於 **!** 符號代表邏輯的 NOT 運算，譬如 **!(a > 0)** 相當於判斷 **a <= 0**，而 **!(a < 1 || a > 100)** 則相當於 **a >= 1 && a <= 100**。又譬如像之前的範例宣告了 **isPrime** 當作 flag，所以程式裡如果寫 **if (isPrime)** 就表示要檢查 **isPrime** 是否為 true。相反地，如果寫 **if (!isPrime)** 就變成要檢查 NOT **isPrime** 是否為 true，也就是檢查 **isPrime** 是否為 false。

[練習 W07_03] 令 **x = 5**；而 **y = 3**；判斷下列 expressions 是 true 還是 false。

```

x > 2 && y == 3
!(x < 2 || y == 3)
x != 0 && y
x == 0 || !y
x != y && (20/x) < y
!(y > 5 || y < 2) && !(x%5)
!x && x

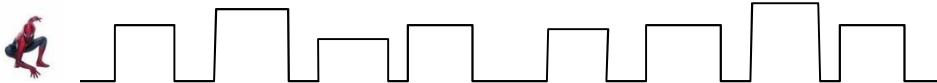
```

第七章：C Control Statements: Branching

計算英文字數

再來我們要嘗試挑戰一個更複雜的程式。我們要讓使用者不斷輸入，直到輸入 '|' 字元才結束，然後程式會統計出使用者究竟輸入了幾個字元，以及輸入了幾個單字 (用空白隔開就算一個，不管是否真的是字典查得到的單字)，並且算出總共輸入了幾行。由於這個程式要做的事情比較多，所以我們先來規劃一下，想想看程式要怎麼寫。

首先我們要讓使用者輸入字元，這個用 `getchar()` 可以辦到。為了讓使用者可以不斷輸入，我們需要用 `while` 迴圈反覆呼叫 `getchar()`，同時判斷讀入的字元是否是 '|'，不是就繼續，是的話就結束迴圈。接下來就想想迴圈裏面要怎麼做。因為我們需要計算字元數，所以要有一個變數來累計字元數，每做一次迴圈就把這個變數加一。假設我們宣告一個變數 `n_chars` 來負責計算字元數，`long n_chars = 0L;`，要先把它的初值設成零，然後在迴圈裡只要用 `n_chars++`；就可以累加字元數。接下來由於我們想計算總共輸入的行數，這可以藉由計算 '\n' 總共出現了幾次來達成，每出現一次 '\n' 就多一行。所以我們可以宣告一個變數 `n_lines` 來計算 '\n' 出現的次數，在迴圈裡每次發現輸入的字元等於 '\n'，就做 `n_lines++`。接下來由於我們想知道使用者輸入了幾個單字 (用空白隔開)，所以我們必須判斷目前輸入的字元是否是空白字元 (包括 space, tab, 和換行)，這可以用 `ctype.h` 裡的 `isspace()` 辦到。我們可以把計算單字數量的動作想成底下的問題：



如果我們用望遠鏡追蹤觀察 Spiderman 從左到右沿著大樓牆壁爬上爬下，但是望遠鏡只鎖定 Spiderman 的臉，其他的景物都觀察不到，請問 Spiderman 要怎麼讓我們知道他爬上了多少棟大樓？

我們先來想想爬大樓的過程：每次遇到一棟大樓，就會開始往上爬，爬到頂樓後繼續前進，然後往下爬直到回到平地。我們需要能夠知道他什麼時候是在平地上，什麼時候是在大樓。假設在平地的時候 Spiderman 的眼睛閉著，一但碰壁開始往上爬，爬到樓頂的時候就把眼睛睜開來，然後當他往下爬碰到平地就又把眼睛閉起來。所以每次我們只要觀察到他的眼睛本來是閉著但突然睜開，就表示他又爬上了一棟大樓。我們就是用這樣的觀念完成底下的範例。(題外話：要如何分辨 Spiderman 的眼睛是睜開還是閉著？)

[範例 E07_07.c]

```
#include <stdio.h>
#include <ctype.h>
#define STOP '|'
int main(void)
{
    char c;
    char prev;
    long n_chars = 0L;
    int n_lines = 0;
    int n_words = 0;
    int p_lines = 0;
    int inword = 0;

    printf("Enter text to be analyzed (| to terminate): \n");
    prev = '\n';
    while ((c = getchar()) != STOP) {
        n_chars++;
        if (c == '\n')
            n_lines++;
        if (!isspace(c) && !inword) {
            inword = 1;
            n_words++;
        }
        if (isspace(c) && inword)
            inword = 0;
        prev = c;
    }
    if (prev != '\n')
        p_lines = 1;

    printf("characters = %ld, words = %d, lines = %d, ", n_chars, n_words, n_lines);
    printf("partial lines = %d\n", p_lines);

    return 0;
}
```

第七章：C Control Statements: Branching

這個程式執行起來像底下這樣：

```
Enter text to be analyzed (| to terminate):
We shall not fail or falter;
we shall not weaken or tire...
Give us the tools and we will finish the job.
|
characters = 106, words = 22, lines = 3, partial lines = 0
```

總共輸入了 106 個字元，包含了 22 個字，總共有 3 行。需要稍作解釋的部分是

```
if (!isspace(c) && !inword) {
    inword = 1;
    n_words++;
}
```

以及

```
if (isspace(c) && inword)
    inword = 0;
```

變數 **inword** 就是用來記錄目前是在空白還是單字裡 (平地還是大樓)。如果 **inword** 的值是 **1** 就表示在單字裡，如果 **inword** 的值是 **0** 就表示回到空白。所以如果目前的狀態是在空白區域 (不在單字裡 **inword** 是 false) 則相反的 **!inword** 的值就是 true，這時候如果讀進來的字元不是空白字元，就表示開始進入單字裡面，這時候要做兩件事，首先要把 **inword** 的狀態改成 **1**，另外還要把 **n_words** 的個數加 **1**。如果本來是在單字裡，當讀進來的字元是空白字元時，就要把 **inword** 的狀態再切換回 **0**。

使用？：語法

你可能會看到別人寫的 C 程式裡有下面這樣的語法

```
x = (y < 0) ? -y : y;
```

它的意義相當於

```
if (y < 0)
    x = -y;
else
    x = y;
```

也就是說，如果 **y < 0** 則把 **x** 的值設成問號？後面的值，如果 **y < 0** 不成立，就把 **x** 的值設成冒號：後面的值。這樣的寫法確實比較簡單扼要，但是依照自己的習慣就好，不見得一定要用這樣的語法。我們在這裡介紹的主要用意是讓大家以後看到別人這樣寫的時候不會被嚇到，而且也知道它的意思是什麼。

[練習 W07_04] 把底下的程式 (作用是取得 **a** 和 **b** 兩數中較大的數)，改用？：語法來寫。

```
if (a > b) { max = a; }
else { max = b; }
```

--

使用 continue 和 break

在迴圈裡面我們可以用 **continue** 和 **break** 來改變迴圈原有的進行步驟。比較底下範例左右兩邊的輸出。

[範例 E07_08.c]

```
#include <stdio.h>
int main()
{
    int i = 'A';
    do {
        if (i == 'C')
            continue;
        putchar(i);
    } while (++i < 'F');
    return 0;
}
```

輸出：

ABDE

```
#include <stdio.h>
int main()
{
    int i = 'A';
    do {
        if (i == 'C')
            break;
        putchar(i);
    } while (++i < 'F');
    return 0;
}
```

輸出：

AB

第七章：C Control Statements: Branching

前面的範例，本來 **while** 迴圈應該要顯示從 'A' 到 'E' 的字元，但是因為用了 **continue**，當 **i** 等於 'C' 的時候，會跳過接下來的步驟 (也就是 **putchar(i);**) 直接進行下一次迴圈，所以會跳去做 **++i**，然後比較 **i** 是否小於 'F'。至於右邊的情況則是當 **i** 等於 'C' 的時候，因為 **break**，所以程式就直接跳到 **while** 之外，也就是 **return 0;** 的地方。

[練習 W07_05] 讓使用者輸入一個字串，判斷這個字串是否對稱 (所謂的 palindrome，正著讀或倒著讀都一樣)。提示：用 **scanf()** 讀入字串，存入陣列中，用 **strlen()** 找到字串的最後一個字元 ('\0' 的前一個字元)，用迴圈依序檢查第一個和最後一個字元是否相等，然後檢查第二和倒數第二個字元是否相等，這樣反覆做下去，直到每個字元都通過檢查過就表示這個字串是個 palindrome。如果檢查的過程中發現頭尾字元不相等，就用 **break** 跳出迴圈。程式最後要輸出一段訊息指出這個字串是否為 palindrome。底下是未完成的程式碼

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[1000];
    int i, j;

    scanf("%s", str);
    i = 0;
    j = ???
    while (i < j) {
        ???
    }
    if ( ??? )
        printf("Not a palindrome.\n");
    else
        printf("Palindrome!\n");

    return 0;
}
```

使用 **switch ... case ... break**

[範例 E07_09.c]

```
#include <stdio.h>
#include <ctype.h>
int main(void)
{
    char ch;
    printf("Give me a letter of the alphabet, and I will give ");
    printf("an animal name \nbeginning with that letter.\n");
    printf("Please type in a letter; type # to quit.\n");
    while((ch = getchar()) != '#') {
        if ('\n' == ch)
            continue;
        if (islower(ch)) {
            switch (ch) {
                case 'a':
                    printf("argali, a wild sheep of Asia\n");
                    break;
                case 'b':
                    printf("babirusa, a wild pig of Malay\n");
                    break;
                case 'c':
                    printf("coati, racoonlike mammal\n");
                    break;
                case 'd':
                    printf("desman, aquatic, molelike critter\n");
                    break;
                case 'e':
                    printf("echidna, the spiny anteater\n");
                    break;
                case 'f':
                    printf("fisher, brownish marten\n");
                    break;
            }
        }
    }
}
```

未完待續

第七章：C Control Statements: Branching

[範例 E07_09.c · 接上頁]

```

        default :
            printf("That's a stumper!\n");
    }
}
else {
    printf("I recognize only lowercase letters.\n");
}
while (getchar() != '\n')
    continue; /* skip rest of input line */
printf("Please type another letter or a #.\n");
}
printf("Bye!\n");

return 0;
}

```

這個範例很長，但是其實並沒有做太複雜的事情。它執行起來的結果像底下這樣

```

Give me a letter of the alphabet, and I will give an animal name
beginning with that letter.
Please type in a letter; type # to quit.
e
echidna, the spiny anteater
Please type another letter or a #.
b
babirusa, a wild pig of Malay
Please type another letter or a #.
ab
argali, a wild sheep of Asia
Please type another letter or a #.
#
Bye!

```

程式主要的部分被包在一個大的 **while** 迴圈裡，每次迴圈會用 **getchar()** 讀一個字元存到變數 **ch**，如果 **ch** 不等於 '#' 就繼續跑迴圈，否則迴圈就結束。程式用 **switch** 的語法來檢查 **ch** 的值。**switch** 的語法如下

```

switch (integer expression) {
    case constant1:
        statements [optional]
    case constant2:
        statements [optional]
    default:
        statements [optional]
}

```

其中 [optional] 的意思是可以沒有 *statements*。這個用法的意思是以 **switch** 後面接的 *integer expression* 為參考標準，然後逐一檢查後續的 **case**，如果 *integer expression* 的值等於 *constant1*，就做底下的 *statements*，如果 *integer expression* 的值不等於 *constant1*，就繼續看下一個 **case**，如果每個 **case** 都不成立，就做 **default** 底下的 *statements*。(由於 **default** 可有可無，如果沒有指定 **default** 要做什麼，當所有 **case** 都不成立時，就什麼 *statements* 都不會做。)由於 **case** 會一個接一個被檢驗，所以通常當某個 **case** 符合，做完該做的 *statement* 之後，我們會用 **break**；結束這組 *statement*，這樣就可以直接跳到 **switch** 外面，不用再繼續檢查後面的 **case**。除了使用 **switch ... case ... break** 之外，這個範例還有一個特別的地方。如果使用者在同一行輸入兩個以上的字元，也只有第一個字元會發生作用，後面的字元會被忽略。這項功能是靠外層迴圈所包含的程式碼的最後面那個內層 **while** 迴圈達成，也就是

```

while (getchar() != '\n')
    continue;

```

這個 **while** 迴圈會不斷消耗剩餘的字元，直到遇到 '\n' 為止。這時候回到外層迴圈，**ch = getchar()** 讀到的就會是新的第一行的第一個字元。

此外，如果使用者在新的一行的一開始什麼英文字母都不打，就直接按 Enter，這時候應該什麼事都不要做，所以在一進入外層 **while** 迴圈的地方我們要加上

```

if ('\n' == ch)
    continue;

```

跳過 '\n' 換行字元。

第八章：Character Input/Output and Input Validation

我們已經學會用 `printf()`、`scanf()`、`putchar()`、`getchar()` 來做輸入輸出。這些用來處理 I/O 的 functions 並不是 C 語言內建的，而是由 C library 提供，我們接著就來解釋輸入輸出的運作機制。另外我們也會介紹如何過濾輸入的資料，讓讀進來的資料格式能符合程式需求，這樣才能按照我們設計的方式順利處理資料。

使用 `getchar()` 與 `putchar()`

這兩個 functions 每次只能讀取或輸出一個字元，功能看似簡單，但是足以讓我們拿來處理各式各樣的輸入輸出狀況。用底下的範例來複習一下 `getchar()` 和 `putchar()`。

[範例 E08_01.c]

```
#include <stdio.h>
int main(void)
{
    char ch;

    while ((ch = getchar()) != '@')
        putchar(ch);

    return 0;
}
```

這個程式執行起來像底下這樣：

```
Hi, there. My email address
Hi, there. My email address
is mac@apple.com.
is mac
```

迴圈 `while` 處理的過程是用 `getchar()` 讀取一個字元，然後用 `putchar()` 輸出一個字元。這個範例照理說應該是輸入一個字元就會回應一個字元例如

```
HHii,, tthheerree..
```

但是程式執行起來卻是輸入了一整行之後（按下 Enter 鍵），才會把輸出的字元顯示出來。我們必須先了解 C 程式如何處理鍵盤輸入，以及所謂的 buffering 機制。由於 `getchar()` 會透過 buffer 來緩衝輸入的字元，所以要等到換行字元出現，才會把 buffer 裡的東西送到程式做處理。所以我們會看到鍵盤輸入的字元先被直接回應到螢幕上（系統為了要讓我們知道究竟打了哪些字），同時這些輸入的字元會被存在 buffer 中，等到使用者打了 Enter 才把 buffer 裡的字元送到程式，然後程式的迴圈就把字元一一處理。

另一方面，使用特殊符號（譬如 '@'）當作結束輸入的字元，也會讓那個字元原本的用途變得無法被正常使用（有時候我們還是會需要用到它，譬如要打 email address）。所以最好是能用某個絕對不會出現在輸入裡的字元，當作輸入終止的判斷條件。C 確實也提供了一個專門用來表示輸入終止的符號讓我們使用。

我們先暫時跳開，先簡介一個概念：在 C 裡面，輸入和輸出的裝置被當成檔案來看待，輸入就像在讀檔，輸出就像是在寫入檔案，這麼做的好處是可以讓程式可以有統一的介面來處理輸入和輸出，不會因為換到不同系統和不同裝置就變得不能執行。在 `stdio.h` 裡定了兩個標準的檔案串流叫做 `stdin` 和 `stdout`，分別對應到標準輸入裝置（通常假設是鍵盤）和標準輸出裝置（通常假設為螢幕）。所以如果我們從 `stdin` 讀取資料，就等於是從鍵盤讀取使用者輸入的內容，如果把資料寫入 `stdout` 就等於把資料送到螢幕上。我們之前學過的 `scanf()` 和 `getchar()` 都是預設從 `stdin` 讀資料，而 `printf()` 和 `putchar()` 都預設為寫入 `stdout`。在 `stdio.h` 裡面定了一個常數 `EOF` (End of File)，用來代表檔案結束。既然輸入被當作檔案來處理，我們在讀取 `stdin` 的資料的時候，就可以判斷讀入的字元是否等於 `EOF`，用這個檔案結束符號來判斷使用者是否已經結束輸入的動作。所以前面的範例程式我們可以改寫成

```
#include <stdio.h>
int main(void)
{
    int ch;

    while ((ch = getchar()) != EOF)
        putchar(ch);

    return 0;
}
```

我們剛才說過，`EOF` 在 `stdio.h` 裡面有定義，通常它的值被定成 `-1`，但是可能因作業系統而有不同。另外，配合 `EOF` 我們把 `ch` 的型別改成 `int`（其實 `getchar()` 和 `putchar()` 的回傳值和參數本來就都是 `int` 型別）。最後我們要知道的是如何用鍵盤輸入 `EOF`（不是輸入 `E O F` 三個字母，也不是直接打 `-1`）。這在不同作業系統下會有不同的方式，在 DOS 下的作法是在新的一行的開頭按 `Ctrl+Z`（在 Unix 則是 `Ctrl+D`），這樣就會相當於是 End of File。

第八章：Character Input/Output and Input Validation

假設我們把範例 E08_01.c 改成 EOF 的版本，取名為 reflect.c，然後 compile 並產生執行檔，假設執行檔的名稱叫做 reflect.exe。在 command line 模式下試試看下面的指令

```
D:\code\> reflect.exe < reflect.c
```

你會發現 reflect.c 的內容會被顯示到螢幕上。不論是在 DOS 或是 Unix, Linux 都提供了所謂 redirection 的功能。這項功能可以把作業系統裡一般的檔案 (譬如純文字檔)，導向成為標準輸入裝置 (對我們的程式來說就是 **stdin**)。用法很簡單，就是像上面的例子用 < 符號，看起來就像把 reflect.c 餵給 reflect.exe。所以 reflect.exe 不再是從鍵盤讀取資料，而是從檔案 reflect.c 讀取資料。對我們的程式來說完全沒差，因為我們的程式本來就是把 **stdin** 當作檔案來處理。由於 reflect.c 本身是純文字檔，它的檔案結束正是靠 **EOF** 來標記，所以 reflect.exe 讀了整個檔案直到碰到 **EOF** 就結束。

除了輸入可以導向，輸出也同樣可以，用法是

```
D:\code\> reflect.exe > ten_words.txt
```

也就是用 > 符號接上我們要輸出的檔案名稱。執行程式之後我們可以輸入一堆字元，輸入完畢後，在新的一行的開頭按 Ctrl+Z 結束程式。這時候如果去檢查目錄會發現多了一個檔案叫做 ten_words.txt。我們可以看看它的內容是什麼。我們就直接用剛學會的方式 (reflect.exe < ten_words.txt) 來看檔案內容，會發現它的內容就是我們剛才用鍵盤輸入的那些字元。所以 reflect.exe > ten_words.txt 這個指令的效果相當於把資料寫入檔案 ten_words.txt 裡面。因為我們用 > 符號把 **stdout** 導向到檔案 ten_words.txt，所以資料不再送到螢幕上，而是送到 ten_words.txt。最後還可以把兩種導向合併，變成

```
D:\code\> reflect.exe < reflect.c > reflect_copy.c
```

這個動作相當於把檔案 reflect.c 複製到檔案 reflect_copy.c。

學會了 redirection 這項小技巧之後，處理檔案資料就變得很方便。我們可以先把資料編輯好，再用 < 導向送入程式作處理，不需要每次都在線上打字輸入。而程式輸出的資料也可以用 > 導向，送到檔案裡儲存起來。

處理 Buffered Input

[範例 E08_02.c]

```
#include <stdio.h>
int main(void)
{
    int guess = 1;

    printf("Pick an integer from 1 to 100. I will try to guess it.\n");
    printf("Respond with a y if my guess is right and with an n if it is wrong.\n");
    printf("Uh...is your number %d?\n", guess);
    while (getchar() != 'y')
        printf("Well, then, is it %d?\n", ++guess);
    printf("I knew I could do it!\n");

    return 0;
}
```

這個程式執行起來像底下這樣：

```
Pick an integer from 1 to 100. I will try to guess it.
Respond with a y if my guess is right and with an n if it is wrong.
Uh...is your number 1?
n
Well, then, is it 2?
Well, then, is it 3?
n
Well, then, is it 4?
Well, then, is it 5?
y
I knew I could do it!
```

這程式有 bug，每次都多回應了一行。問題出在哪裡？從輸出的結果可以猜到，程式應該是連做了兩次用 **getchar()** 讀取字元的動作。雖然看起來我們只打了 **n** 這個字母，但實際上我們還按了 Enter 鍵，所以多藏了一個 '\n' 字元。

第八章：Character Input/Output and Input Validation

解決這個問題的方法是再用一個迴圈把剩餘的字元吃掉。

```
while (getchar() != 'y') {
    printf("Well, then, is it %d?\n", ++guess);
    while (getchar() != '\n')
        continue;
}
```

改了之後程式執行起來就比較正常，不會再一次印兩行

```
Pick an integer from 1 to 100. I will try to guess it.
Respond with a y if my guess is right and with an n if it is wrong.
Uh...is your number 1?
n
Well, then, is it 2?
no
Well, then, is it 3?
forget it
Well, then, is it 4?
give me a break
Well, then, is it 5?
y
```

程式還可以再改進，因為只要輸入的不是 **y**，都會被當成 **n**。

[練習 W08_01] 把範例修改成只能接受 **y** 或 **n**，如果是其他的輸入（譬如 **no** 或 **forget it**），就顯示 "Sorry, I understand only y or n."。

提示：宣告一個變數 **char response**；用它來記錄讀到的字元，例如 **while ((response = getchar()) != 'y')**，然後再用一個變數 **isn** 當作 flag 來記錄讀到的是否是單一字元 **n**，如果是的話 **isn** 的值要設成 **1**。迴圈的最後面就依照 **isn** 的狀態來顯示訊息，例如

```
if (isn)
    printf("Well, then, is it %d?\n", ++guess);
else
    printf("Sorry, I understand only y or n.\n");
--
```

[範例 E08_03.c]

```
#include <stdio.h>
void display(char cr, int lines, int width);
int main (void)
{
    int ch;
    int rows, cols;
    printf("Enter a character and two integers:\n");
    while ((ch = getchar()) != '\n') {
        scanf("%d %d", &rows, &cols);
        display(ch, rows, cols);
        printf("Enter another character and two integers;\n");
        printf("Enter a newline to quit.\n");
    }
    printf("Bye.\n");
    return 0;
}

void display(char cr, int lines, int width)
{
    int row, col;
    for(row = 1; row <= lines; row++) {
        for (col = 1; col <= width; col++)
            putchar(cr);
        putchar('\n');
    }
}
```

輸出：

```
Enter a character and two integers:
u 4 7
uuuuuuuu
uuuuuuuu
uuuuuuuu
uuuuuuuu
Enter another character and two integers;
Enter a newline to quit.
Bye.
```

這個例子定義了一個叫做 **display()** 的 function，傳入一個字元，和兩個整數代表行數和寬度，就會用雙重迴圈把字元按照指定的行數和寬度重複輸出。定義和呼叫 function 的部分應該沒什麼問題，我們來看看輸入的處理。

第八章：Character Input/Output and Input Validation

前面的範例程式裡的迴圈應該要做很多次才對，但是執行了一次就停止了，原因是因為 `scanf()`。我們輸入了一個字元和兩個數字，然後按下 Enter 鍵，`scanf()` 就會讀我們輸入的東西。但是 `scanf()` 只會取它要的東西，剩下的會被留在 buffer 裡，所以換行字元 `'\n'` 會被留在那裡，然後下次迴圈 `getchar()` 讀到的就是 `'\n'`，因此 `while` 迴圈結束。我們可以再用同一招，加入一個 `while` 迴圈把 `scanf()` 之後的字元都吃掉：

```
while (getchar() != '\n')
    continue;
```

改良版如下：

[範例 E08_04.c]

```
#include <stdio.h>
void display(char cr, int lines, int width);
int main (void)
{
    int ch;
    int rows, cols;
    printf("Enter a character and two integers:\n");
    while ((ch = getchar()) != '\n') {
        if (scanf("%d %d", &rows, &cols) != 2)
            break;
        display(ch, rows, cols);
        while (getchar() != '\n')
            continue;
        printf("Enter another character and two integers;\n");
        printf("Enter a newline to quit.\n");
    }
    printf("Bye.\n");
    return 0;
}

void display(char cr, int lines, int width)
{
    int row, col;
    for(row = 1; row <= lines; row++) {
        for (col = 1; col <= width; col++)
            putchar(cr);
        putchar('\n');
    }
}
```

輸出：

```
Enter a character and two integers:
u 1 5
uuuuu
Enter another character and two integers;
Enter a newline to quit.
r 2 7
rrrrrrr
rrrrrrr
Enter another character and two integers;
Enter a newline to quit.

Bye.
```

另外我們也替 `scanf()` 檢查是否確實讀入了兩個整數，如果沒有成功讀入兩個數，就用 `break` 把迴圈中斷。改過之後就可達到預期的效果。

檢查輸入是否符合規定

雖然我們會在程式裡加入一些指示，要求使用者依照我們規定的格式輸入資料，但是使用者還是可能輸入格式不符的資料，這種情況如果沒有妥善處理，有可能會讓我們程式出錯。例如為了預防使用者輸入負數，我們可能加入類似底下的迴圈。

```
#include <stdio.h>
int main(void)
{
    int n;
    scanf("%d", &n);
    while (n >= 0) {
        /* something to do with non-negative n */
        scanf("%d", &n);
    }
    return 0;
}
```

如果使用者輸入負數程式就會停止。另外為了更進一步預防使用者輸入的資料型別不對，我們可以再改成

```
while (scanf("%d", &n) == 1 && n >= 0) {
    /* something to do with non-negative n */
}
```

第八章：Character Input/Output and Input Validation

所以只有當使用者輸入的是整數而且大於等於零迴圈才會繼續。如果使用者輸入的是浮點數，程式就會停止。我們可以自己寫一個 function 專門用來讀取整數，例如

```
int get_int(void)
{
    int input;

    char ch;
    while (scanf("%d", &input) != 1) {
        while ((ch = getchar()) != '\n')
            putchar(ch);
        printf(" is not an integer.\nPlease enter an ");
        printf("integer value, such as 25, -178, or 3: ");
    }

    return input;
}
```

這個 function 不需要傳入參數，但它會傳回一個整數。它所做的事情就是不斷讓使用者輸入，直到使用者乖乖地輸入一個整數為止。如果使用者輸入的不是整數，由於 `scanf()` 會把資料留在 buffer 裡，所以我們要做把 buffer 清掉的動作，這就是第二個 `while` 在做的事情。我們先把這個 function 留著，等一下會派上用場。

```
int bad_limits(int begin, int end, int low, int high)
{
    int not_good = 0;

    if (begin > end) {
        printf("%d isn't smaller than %d.\n", begin, end);
        not_good = 1;
    }
    if (begin < low || end < low) {
        printf("Values must be %d or greater.\n", low);
        not_good = 1;
    }
    if (begin > high || end > high) {
        printf("Values must be %d or less.\n", high);
        not_good = 1;
    }
    return not_good;
}
```

這個 function 會檢查 `begin` 和 `end` 是否介於我們設定的範圍 `low` 和 `high` 之間，同時也會判斷 `begin` 的值沒有超過 `end`。程式本身並沒有太複雜的運算，只是用 `if` 來逐一檢查範圍。我們用 `not_good` 當作 flag 來記錄是否有不符合規定的情況發生，剛開始 `not_good` 的值是 0，一旦發生不符合規則的情況則 `not_good` 的值就會被設為 1。最後 function 會把 `not_good` 傳回去，讓主程式知道 `begin` 和 `end` 是否有超出範圍。

接下來我們就用上面寫好的 `get_int()` 和 `bad_limits()` 來處理輸入資料的格式，我們要來寫一個大一點的程式，這個程式要做的事情是輸入兩個整數，然後計算出介於兩個整數之間 (包含自己) 所有整數的平方和。我們也先寫好 function 來計算介於兩整數之間所有整數的平方和。

```
unsigned long sum_squares(int a, int b)
{
    unsigned long total = 0;
    int i;
    for (i = a; i <= b; i++)
        total += i * i;

    return total;
}
```

[練習 W08_02] 做個簡單的複習，把 `sum_squares()` 改成為 `while` 來寫。

[練習 W08_03] 做個簡單的複習，寫一個 function 叫做 `sum_squares_odd(int a, int b)`，它的功用是計算 `a` 和 `b` 之間所有奇數的平方和。

第八章：Character Input/Output and Input Validation

[範例 E08_05.c]

```

#include <stdio.h>
#define MIN -1000
#define MAX 1000
int get_int(void);
int bad_limits(int begin, int end, int low, int high);
unsigned long sum_squares(int a, int b);
int main(void)
{
    int start;
    int stop;
    unsigned long answer;

    printf("This program computes the sum of the squares of ");
    printf("integers in a range \nwithin [-1000, 1000].\nEnter the ");
    printf("limits (enter 0 for both limits to quit):\nlower limit: ");
    start = get_int();
    printf("upper limit: ");
    stop = get_int();
    while (start != 0 || stop != 0) {
        if (bad_limits(start, stop, MIN, MAX))
            printf("Please try again.\n");
        else {
            answer = sum_squares(start, stop);
            printf("the sum of the squares of the integers ");
            printf("from %d to %d is %lu\n", start, stop, answer);
        }
        printf("Enter the limits (enter 0 for both ");
        printf("limits to quit):\n");
        printf("lower limit: ");
        start = get_int();
        printf("upper limit: ");
        stop = get_int();
    }
    printf("Done.\n");

    return 0;
}

```

我們可以試試看各種可能的輸入錯誤是否都被正確地處理，例如

```

This program computes the sum of the squares of integers in a range
within [-1000, 1000].
Enter the limits (enter 0 for both limits to quit):
lower limit: low
low is not an integer.
Please enter an integer value, such as 25, -178, or 3: -10
upper limit: a big number
a big number is not an integer.
Please enter an integer value, such as 25, -178, or 3: 900
the sum of the squares of the integers from -10 to 900 is 243405535
Enter the limits (enter 0 for both limits to quit):
lower limit: 10
upper limit: -5
10 isn't smaller than -5.
Please try again.
Enter the limits (enter 0 for both limits to quit):
lower limit: 0
upper limit: 0
Done.

```

整個程式的撰寫過程也是我們要學會的一項有用的技巧。我們要把要達成的工作分成一些獨立的 functions，再組合起來就可以做到更複雜的功能。不要把所有步驟都混雜在一起，這樣程式很容易寫錯，也很難再改寫或擴充。主程式主要是一個 `while` 迴圈反覆讓使用者輸入 lower limit 和 upper limit，當兩個 limits 都是 0 就停止。其餘部分都只要呼叫寫好的 functions：`get_int()`、`bad_limits()` 和 `sum_squares()`，這樣就能讓程式變得很簡潔。

第八章：Character Input/Output and Input Validation

scanf() 做了什麼事

假設我們輸入了 **42** 然後按下 Enter。如果程式是用 **scanf()** 加上 **%c** 來讀取，它會讀到 **'4'** 這個字元，然後存入變數裡。如果我們用 **%s**，則會讀入兩個字元 **'4'** 和 **'2'**，然後把他們存成字串 (結尾多加 **'\0'**)。如果用的是 **%d**，則 **scanf()** 也是讀取兩個字元 **'4'** 和 **'2'**，並且計算他們所代表的十進位整數值 (計算 $4 \times 10 + 2$ 得到 **42**)，然後把這個數值用二進位表示法存入型別為 **int** 的變數中。如果用 **%f**，則一樣是把字元轉成數值 **42**，再用浮點數的儲存方式存入 **float** 變數中。

[範例 08_06.c]

```
#include <stdio.h>
int get_choice(void);
int get_first(void);
float get_float(void);
void add(void);
void subtract(void);
void multiply(void);
void divide(void);
int main(void)
{
    int choice;
    while ( (choice = get_choice()) != 'q') {
        switch (choice) {
            case 'a' : add(); break;
            case 's' : subtract(); break;
            case 'm' : multiply(); break;
            case 'd' : divide(); break;
            default : printf("Program error!\n"); break;
        }
    }
    printf("Bye.\n");
    return 0;
}
int get_choice(void)
{
    int ch;
    printf("Enter the operation of your choice:\n");
    printf("a. add          s. subtract\n");
    printf("m. multipl       d. divide\n");
    ch = get_first();
    while ( ch != 'a' && ch != 's' && ch != 'm' && ch != 'd' && ch != 'q') {
        printf("Please respond with a, s, m, d, or q.\n");
        ch = get_first();
    }
    return ch;
}
int get_first(void)
{
    int ch;
    while ((ch = getchar()) == '\n');
    while (getchar() != '\n')
        continue;
    return ch;
}
float get_float(void)
{
    float input;
    char ch;
    while (scanf("%f", &input) != 1) {
        while ((ch = getchar()) != '\n')
            putchar(ch);
        printf(" is not a number.\nPlease enter a ");
        printf("number, such as 2.5, -1.78E6, or 3: ");
    }
    return input;
}
```

[範例 08_06.c] 接上一頁

```

void add(void)
{
    float a, b;
    printf("Enter first number: ");
    a = get_float();
    printf("Enter second number: ");
    b = get_float();
    while (getchar() != '\n')
        continue;
    printf("%f + %f = %f\n", a, b, a+b);
}
void subtract(void)
{
    float a, b;
    printf("Enter first number: ");
    a = get_float();
    printf("Enter second number: ");
    b = get_float();
    while (getchar() != '\n')
        continue;
    printf("%f - %f = %f\n", a, b, a-b);
}
void multiply(void)
{
    float a, b;
    printf("Enter first number: ");
    a = get_float();
    printf("Enter second number: ");
    b = get_float();
    while (getchar() != '\n')
        continue;
    printf("%f * %f = %f\n", a, b, a*b);
}
void divide(void)
{
}

```

程式執行起來的效果如下：

```

Enter the operation of your choice:
a. add          s. subtract
m. multipl     d. divide
a
Enter first number: 18
Enter second number: b
b is not a number.
Please enter a number, such as 2.5, -1.78E6, or 3: 2398
18.000000 + 2398.000000 = 2416.000000
Enter the operation of your choice:
a. add          s. subtract
m. multipl     d. divide
m
Enter first number: 1.8
Enter second number: 9.3
1.800000 * 9.300000 = 16.740000
Enter the operation of your choice:
a. add          s. subtract
m. multipl     d. divide
q
Bye.

```

[練習 W08_04] 把 `divide()` 的 definition 的部份寫出來。要記得檢查是否出現分母為零的情況，有的話就顯示錯誤訊息，並且讓使用者重新輸入。

第八章：Character Input/Output and Input Validation

主程式 `main()` 的部份最簡單，只是利用 `while` 迴圈以及 `switch` 來讓使用者選擇要做哪一種運算，總共有加減乘除四種運算可以選擇。在 `while` 迴圈的判斷式裡我們用自己寫的 function `get_choice()` 來取得使用者輸入的字元。取得使用者輸入的選項之後，就靠 `switch` 來找到該執行哪種運算，每種運算都有對應的 function 負責處理，包括 `add()`、`subtract()`、`multiply()`、`divide()`，我們也要自己寫這些 functions。

先來看 `get_choice()`，它會先列出選單，然後讀取使用者輸入的字元，如果使用者輸入的不是 'a', 's', 'm', 'd', 'q' 之中的某個字元，會靠迴圈繼續讓使用者輸入直到符合為止。在 `get_choice()` 裡用 `get_first()` 來讀取字元，`get_first()` 會讀取到第一個不等於 '\n' 的字元，而且只讀每一行輸入的第一個字元，第一個之後的字元會被忽略(靠 `while` 來將 buffer 裡剩的字元吃掉)。

另外還要自己寫一個 `get_float()`，用來讀取並回傳一個浮點數，一樣是靠迴圈不斷重複直到輸入正確為止。最後 `add()`、`subtract()`、`multiply()`、`divide()` 這幾個 function 的內容都差不多，只有 `divide()` 需要再額外處理除以零的情況。

[補充] 用字元陣列儲存數字

```
int main(void)
{
    char str1[8];
    int str2[8]; /* char str2[8]; */
    int i;

    scanf("%7s", str1);

    for (i=0; i<strlen(str1); i++)
        str2[i] = str1[i] - '0';
    ...
    return 0;
}
```

str1[0]				str1[4]			
'4'	'5'	'0'	'3'	'\0'			

str1[0]				str1[4]			
52	53	48	51	0			

```
str2[0] = str1[0] - '0'
str2[1] = str1[1] - '0'
str2[2] = str1[2] - '0'
str2[3] = str1[3] - '0'
```

str2[0]				str2[4]			
4	5	0	3	???			

用 `scanf("%7s", str1)` 方式讀進來的資料是字串，存入字元陣列中每個陣列的元素都是一個字元。由於在 C 裡面字元是用 ASCII 值來表示，所以 `str1` 的內容就像上圖第二列顯示的一樣。我們為了要把這些阿拉伯數字字元當做 0 到 9 的整數值來運算，必須先把他們從數字字元換算成整數，方法是減去 '0' 這個字元，或是說減去 48。

[註] 雖然 `int str2[8];` 宣告成 `int` 型別的陣列，但是其實也可以宣告成字元陣列，也就是 `char str2[8];`，這樣對於後續把 `str2` 拿來做整數運算完全沒有影響，因為 `char` 型別其實也是整數，只是長度只能記錄到一個 byte (8 bits) 而已，可以表示的整數範圍只在 -128 到 127 之間。

請試試看底下幾種顯示方式各自會得到什麼結果

顯示 `str1` 的第四個元素

```
printf("%d\n", str1[3]);
printf("%c\n", str1[3]);
```



顯示 `str2` 的第四個元素

```
printf("%d\n", str2[3]);
printf("%c\n", str2[3]);
```

把 `str1` 當作字串顯示

```
printf("%s\n", str1);
```

把 `str2` 當作字串顯示

```
printf("%s\n", str2);
```



第九章：Functions

看過了最近幾個範例，我們已經稍微體會到使用 function 來建構我們的程式的好處。除了現成的 functions 例如 `printf()`、`scanf()`、`putchar()`、`getchar()`、`strlen()`，我們還可以把某些會重複使用的功能寫成 function，然後在其他 function 裡面呼叫。對 C 程式來說，用 functions 當作基本架構來組成複雜的程式碼，才是比較適當的程式設計方法。雖然程式執行過程其實是循序一行接著一行地執行，但是如果程式碼真的用這樣的方式來撰寫的話，不僅沒有效率而且很容易出錯。程式設計的過程可以用 top-down 的方式，先想想看我們要寫的程式需要做哪些事情，把這些任務加以區分，用不同的 function 來達成，然後先宣告我們需要用的 functions，先不要管 function 內部要怎麼做，只要先定出 prototype 就可以 (要傳什麼參數，會傳回什麼值)。有了這些 function prototypes 就可以開始寫主程式，需要做某項任務就呼叫負責的 function。整個架構都完成了之後，再把 function 的主體 definition 補好，讓 function 能實際做到我們想要的功能。

想像我們要寫一個程式來做底下幾件事：1) 讀取一組數據；2) 把數據排列；3) 計算這組數據的平均值；4) 把數據的直方圖畫出來。主程式可能長得像這樣

```
#include <stdio.h>
#define SIZE 50
int main(void)
{
    float list[SIZE];
    readlist(list, SIZE);
    sort(list, SIZE);
    average(list, SIZE);
    histogram(list, SIZE);
    return 0;
}
```

程式看起來一目了然，每個步驟要做什麼工作都很清楚。接下來如果能正確的把每個 function 的程式碼一一寫好，讓它們能完成該做的事情，那麼程式也就寫完了。我們把大問題拆成很多小任務來完成，就不至於陷入不知從何下手的窘境，每次只要專心把某個 function 的功能寫出來就可以。我們也可以把 function 想像成黑盒子，只要知道它的作用以及使用方法 (傳什麼東西進去它會傳回什麼)，而不需要知道內部如何運作，除非我們自己要來撰寫 function。用這種方式來設計程式，讓我們不會在一開始的時候就被細節淹沒。我們可以先思考在整個程式的架構，先想想需要哪些 functions，以及它們和主程式之間的關連。如果有現成的 function 可用就直接拿現成的來用，最後再來擔心其餘非現成的 functions 該怎麼寫。(善用 Libraries 也是學寫程式的過程需要漸漸培養的一項能力，當開發大型的軟體的時候，如果知道某項工作有現成可用的 functions，就不必再自己花時間重寫，才可以加快程式開發的速度。)

[範例 E09_01.c]

```
#include <stdio.h>
#define NAME "NTHU CS"
#define ADDRESS "101, Sec. 2, Kuang Fu Rd."
#define PLACE "Hsinchu, 300 Taiwan"
#define WIDTH 40
void starbar(void); /* function prototype */
int main(void)
{
    starbar();
    printf("%s\n", NAME);
    printf("%s\n", ADDRESS);
    printf("%s\n", PLACE);
    starbar();
    return 0;
}
void starbar(void) /* function definition */
{
    int count;
    for (count = 1; count <= WIDTH; count++)
        putchar('*');
    putchar('\n');
}
```

這個程式執行起來像底下這樣：

```
*****
NTHU CS
101, Sec. 2, Kuang Fu Rd.
Hsinchu, 300 Taiwan
*****
```

第九章：Functions

在 `main()` 前面的 `void starbar(void);` 是所謂的 function prototype，目的是讓 compiler 先知道有個 `starbar()`，這樣在呼叫 `starbar()` 的時候才會認得這個 function。和變數一樣，function 也牽涉到型別，我們必須指定 function 的回傳值以及傳入參數的型別，在這裡 `starbar()` 的回傳值和參數的型別都是 `void`，也就是什麼都沒有，不需要傳入參數，也不需要傳回任何值。要注意 function prototype 宣告的結尾要加上分號。在程式裡如果要呼叫 `starbar()` 只需要寫 `starbar();`，因為不需要傳參數，也不需要接收任何回傳值。呼叫 function 的動作會讓程式的流程轉而進入 function 裡面，等做完 function 該做的工作之後，就會回到原來呼叫 `starbar()` 的主程式的下一行繼續執行。

在 `void starbar(void) { ... }` 兩個波浪括號中間的程式碼，負責定義 `starbar()` 究竟要做哪些事情。在這個例子裡，`starbar()` 用來畫出寬度為 `WIDTH`，以 '*' 所組成的分隔線。在 `starbar()` 裡面有一個 `int` 變數叫做 `count`，它是所謂的 local variable。之所以稱為 local，是因為它的有效範圍只有在 `starbar()` 這個 function 裡。所以，假設在 `main()` 裡面也有個變數叫做 `count`，但因為它的有效範圍也只有在 `main()` 裡面，所以就算兩個變數（一個在 `starbar()`，另一個在 `main()` 裡面）都叫做 `count` 也完全不會造成衝突。

在這個範例裡 `starbar()` 和 `main()` 是在同一個檔案裡，實際上我們也可以把它們分別存成不同的檔案，這樣的好處是可以讓其他的程式如果有需要也可以呼叫 `starbar()`，增加 function 重覆使用的便利性。我們稍後就會介紹怎麼把程式碼分成不同檔案來儲存以及 compile。

接下來我們要來改良前面的範例，讓輸出的字串能夠置中，順便介紹怎麼寫出需要傳入參數的 function。

[範例 E09_02.c]

```
#include <stdio.h>
#include <string.h>
#define NAME "NTHU CS"
#define ADDRESS "101, Sec. 2, Kuang Fu Rd."
#define PLACE "Hsinchu, 300 Taiwan"
#define WIDTH 40
#define SPACE ' '
void show_n_char(char ch, int num);
int main(void)
{
    int spaces;
    show_n_char('*', WIDTH);
    putchar('\n');
    show_n_char(SPACE, 17);
    printf("%s\n", NAME);
    spaces = (WIDTH - strlen(ADDRESS)) / 2;
    show_n_char(SPACE, spaces);
    printf("%s\n", ADDRESS);
    show_n_char(SPACE, (WIDTH - strlen(PLACE)) / 2);
    printf("%s\n", PLACE);
    show_n_char('*', WIDTH);
    putchar('\n');
    return 0;
}
void show_n_char(char ch, int num)
{
    int count;
    for (count = 1; count <= num; count++)
        putchar(ch);
}
```

這個程式執行起來像底下這樣：

```
*****
                NTHU CS
          101, Sec. 2, Kuang Fu Rd.
          Hsinchu, 300 Taiwan
*****
```

我們用自訂的 `show_n_char()` 來輸出一長條以相同字元組成的字串，我們希望 `show_n_char()` 要傳入兩個參數，分別用來指定輸出的字元以及長度，它做的事情很簡單，只要一個 `for` 迴圈就可以辦到。宣告 (prototype) 的地方要寫 `void show_n_char(char ch, int num);`

表示第一個參數的型別是 `char`，第二個參數的型別是 `int`。傳回值的型別則還是寫 `void`，表示不需要回傳任何值。

第九章：Functions

其實在宣告的時候，變數名稱並不重要，重要的是型別要寫對，所以 function 宣告也可以寫成

```
void show_n_char(char , int );
```

因為宣告的時候，還不需要真正產生變數。要等到定義的時候，才需要產生變數，那時候就一定要寫變數名稱。使用 `show_n_char()` 只要把我們想要顯示的字元和數量當參數傳進去，`show_n_char()` 就可以依照它們來顯示，例如程式碼裡面用到

```
show_n_char('*', WIDTH);
```

就是想要畫出一條寬度為 `WIDTH` 以 `'*'` 構成的分隔線。另外為了要讓字串置中，我們也用計算的方式在文字之前先顯示一些空白，譬如

```
show_n_char(SPACE, (WIDTH - strlen(PLACE)) / 2);
```

在 `main()` 前面也用了其他方式來呼叫 `show_n_char()`，例如直接傳常數 `show_n_char(SPACE, 17);` 或是傳變數，例如 `show_n_char(SPACE, spaces);`。這些不同的寫法，最後做的動作其實都一樣，因為最後其實傳的是 `int` 值。像 `show_n_char(SPACE, (WIDTH - strlen(PLACE)) / 2);`；其實會先把 `(WIDTH - strlen(PLACE)) / 2` 算出來得到 `10`，然後把 `10` 這個值當參數傳給 `show_n_char()`，而 `show_n_char()` 並不需要知道外面究竟做了什麼事，只要知道傳進來的第二個參數的值是 `10` 就夠了。若以 `show_n_char(SPACE, spaces);` 這個例子來說，傳進去的也是當時變數 `spaces` 的值 (值為 `7`)。

[練習 W09_01] 寫一個程式，其中包含一個 function 叫做 `pyramid()`，它的作用是畫出類似下面圖中的三角形。我們需要傳入兩個參數，型別各是 `char` 和 `int`，第一個參數代表要畫出的字元，第二個參數是要畫的三角形的高度，譬如呼叫 `pyramid('#', 5);`；則必須畫出

```
#
###
#####
#####
#####
```

可以利用前一個範例裡的 `show_n_char()`，直接在 `pyramid()` 裡再呼叫 `show_n_char()`。在主程式裡測試我們寫的 `pyramid()` 是否正確，分別呼叫 `pyramid('#', 5);` 以及 `pyramid('*', 8);` 看看是否達到我們預期的效果。

前面提過可以把 function 想成一個黑盒子，使用 function 的時候可以不管它裡面到底怎麼做的，只要確定它可以辦到我們想要做的事情，然後知道要傳什麼樣的參數就夠了。由於這種黑盒子的特性，我們不必擔心定義在 function 裡面的變數是否會和外部的變數衝突。假設 `main()` 裡面有個變數叫做 `x`，然後某個 function `f()` 裡也有個變數 `x`，則更改 `f()` 裡的 `x` 並不會改變 `main()` 裡的 `x`。另外再重複強調一次，當參數傳入 function 時，只有值會被複製傳入，所以在 function 裡對參數做修改，同樣也是不會影響外部的變數值，例如

[範例 E09_03.c]

```
#include <stdio.h>
void f(int x);
int main(void)
{
    int x = 1;
    printf("main: %d\n", x);
    f(x);
    printf("main: %d\n", x);
    return 0;
}
void f(int x)
{
    x++;
    printf("f: %d\n", x);
}
```

輸出：

```
main: 1
f: 2
main: 1
```

我們可想像程式執行的流程。一開始在 `main()` 裡面 `x` 的值被設為 `1`，然後把 `x` 當參數傳入 `f()`，在 function `f()` 裡的 local variable 也叫做 `x`，這個 `x` 因為接收傳入的參數，所以值也是 `1`，然後在 `f()` 裡做了 `x++`；所以 `x` 的值變成 `2`，然後 `f()` 結束回到 `main()`，這時 `main()` 裡的 `x` 和呼叫 `f()` 前沒有兩樣，所以值還是 `1`。

第九章：Functions

接下來要來看看如何從 function 傳值回來。

[範例 E09_04.c]

```
#include <stdio.h>
int imin(int, int);
int main(void)
{
    int evil1, evil2;
    printf("Enter a pair of integers (q to quit):\n");
    while (scanf("%d %d", &evil1, &evil2) == 2) {
        printf("The lesser of %d and %d is %d.\n",
            evil1, evil2, imin(evil1, evil2));
        printf("Enter a pair of integers (q to quit):\n");
    }
    printf("Bye.\n");
    return 0;
}

int imin(int n, int m)
{
    int min;
    if (n < m) { min = n; }
    else { min = m; }
    return min;
}
```

輸出：

```
Enter a pair of integers (q to quit):
300 200
The lesser of 300 and 200 is 200.
Enter a pair of integers (q to quit):
-8 9
The lesser of -8 and 9 is -8.
Enter a pair of integers (q to quit):
q
Bye.
```

在 `main()` 裡面用迴圈反覆要求使用者輸入兩個整數 (現在應該很熟悉這種寫法了)，每次輸入兩個整數之後就會用 `imin()` 來判斷出兩者中較小的數。所以 `imin()` 的 prototype 宣告成 `int imin(int, int);` 代表會傳入兩個 `int` 參數，然後要回傳一個 `int`。我們希望回傳值等於傳入參數值中較小的那一個值，這只要用 `if else` 就可以辦到。在 `imin()` 的 definition 的最後，必須用 `return` 把結果傳回去，在這個範例的寫法就是 `return min;`，表示要把變數 `min` 的當時的值傳回去。在 `main()` 裡面呼叫 `imin(evil1, evil2)` 這個 expression 的值會等於 `imin()` 做完之後 `return` 傳回來的值。譬如 `imin(5, 3)` 的值就等於 3。

關於 `return` 還有一些額外的東西可以稍微了解一下。如果我們寫了底下的 function

```
int what_if(int n)
{
    double z = 100.0/ (double) n;
    return z;    /* what happens? */
}
```

由於 `what_if()` 宣告的回傳值型別是 `int`，所以雖然 `z` 是 `double`，但是在 `return` 的時候會被強制轉成 `int`。此外，`return` 有一個效果是讓目前的 function 結束，直接跳回到原來呼叫者的程式碼片段。所以其實 `return` 一定要在程式的最後面，譬如

```
int imin(int n, int m)
{
    if (n < m)
        return n;
    else
        return m;
    printf("You can't see me.\n");
}
```

程式最後一行的 `printf()` 永遠不會被執行。如果 function 的不需要回傳值 (也就是宣告成 `void`)，那麼也可以用 `return;` 讓 function 在中途跳回去，寫法就是 `return` 直接加分號；，不傳回任何值 (因為是 `void`)，只讓 function 就此結束執行。

```
void show_postive(int x)
{
    if (x < 0)
        return;
    else
        printf("%d\n", x);
}
```

[練習 W09_02] 寫一個 function 叫做 `max()`，傳兩個 `double` 參數值進去，傳回其中較大的值。

[練習 W09_03] 寫一個 function 叫做 `hmean()`，傳入兩個 `double` 值，計算出它們的 harmonic mean，然後把值傳回去。(harmonic mean 是兩數的倒數的平均值的倒數。)

第九章：Functions

Recursion

之前看的 function 都是透過別人來呼叫，那麼如果一個 function 自己呼叫自己會發生什麼事呢？在 C 程式裡面確實允許 function 呼叫自己，這樣的動作稱作遞迴 (recursion)。使用 recursion 時最需要注意的是停止條件，因為如果停止條件不正確，有可能不斷呼叫而讓程式停不下來，最後甚至會因為使用過多資源而讓程式當掉。程式中使用到迴圈的地方通常都可以改成用 recursion 方式來做。有時候某些工作用迴圈方式來思考會比較容易，但有时候用 recursion 則會比較簡潔。通常使用 recursion 的解法會比較漂亮，但是可能會比較沒有效率，使用的資源也會比較多。我們就先來看一個使用 recursion 的例子。

[範例 E09_05.c]

```
#include <stdio.h>
void up_and_down(int);
int main(void)
{
    up_and_down(1);
    return 0;
}
void up_and_down(int n)
{
    printf("Level %d: n location %p\n", n, &n);
    if (n < 4)
        up_and_down(n+1);
    printf("LEVEL %d: n location %p\n", n, &n);
}
```

輸出：

```
Level 1: n location 0022FF60
Level 2: n location 0022FF40
Level 3: n location 0022FF20
Level 4: n location 0022FF00
LEVEL 4: n location 0022FF00
LEVEL 3: n location 0022FF20
LEVEL 2: n location 0022FF40
LEVEL 1: n location 0022FF60
```

首先 `main()` 先呼叫 `up_and_down()`，傳入的參數值是 1。所以進入 `up_and_down()` 之後顯示出 Level 1，表示 `n` 目前的值等於 1。接著由於 `n` 比 4 還小，所以會繼續呼叫 `up_and_down()`，這時候傳的參數是 `n+1` 所以會傳 2 進去，然後程式又跑到 `up_and_down()` 裡，由於 `n` 的值等於 2 所以會顯示是在 Level 2，就這樣繼續進入 Level 3，然後到了 Level 4。因為在 Level 4，這時 `n` 的值等於 4，會讓 `if` 條件不成立，所以不再繼續呼叫 `up_and_down()`，而進行到下一個 `printf()`，顯示出 LEVEL 4，再接下來 function 就結束，回到當初呼叫它的地方，也就是在 Level 3 裡面，呼叫 `up_and_down(n+1)` 的地方，接著執行下一行的 `printf()` 顯示 LEVEL 3，就這樣一路往回，回到 LEVEL 1，然後回到 `main()`，最後就是 `main()` 裡的 `return 0`；整個程式就結束。在每次呼叫進入某個 Level 的 `up_and_down()`，使用到的變數 `n` 名字雖然一樣，但是其實是不同的變數 (local variable)，這點可以從變數的記憶體位址看出來，我們用 `&n` 就表示要取得的是 `n` 的位址。在不同的 Level 下，`&n` 的值都不一樣，表示 `n` 佔用的是不同的記憶體位址。另外如果是在同一個 Level 則 `n` 的記憶體位址會一樣。(`printf()` 裡的 `%p` 格式表示要顯示的是位址。)

現在應該可以感受到 recursion 進行的方式比迴圈更容易讓人混淆。底下是幾項關於 recursion 使用上的重點：

1) 每一層遞迴都有屬於自己的變數，雖然變數名稱看起來一樣，但是其實是存放在不同位置的變數 (變數名稱只是代號)，也都會擁有不同的值。我們可以用下圖來分析上面的範例的執行過程：

variables:	n	n	n	n
after level 1 call	1			
after level 2 call	1	2		
after level 3 call	1	2	3	
after level 4 call	1	2	3	4
after return from level 4	1	2	3	
after return from level 3	1	2		
after return from level 2	1			
after return from level 1	(all gone)			

2) 程式以對稱方式進行。會先一層一層往下走，然後再一層層倒退回來。在每一次遞迴呼叫 (recursive call) 之前的程式碼，它們的執行順序和所在位置的 function 被呼叫的順序是相同的，也就是順向進行，從 Level 1，Level 2，Level 3，到 Level 4。但是如果是在 recursive call 之後的程式碼，它們則是以 LEVEL 4，LEVEL 3，LEVEL 2，LEVEL 1 相反順序倒退回來。

3) 雖然每一層 recursion 都有自己的變數，但是程式碼卻不需要被複製很多份。每個 function 只是一堆指令的集合，而程式是用循序方式執行，每次遞迴呼叫都只是再進入 function 指令集合的最前面，重新開始執行。

4) 遞迴呼叫最重要的是要能讓整個流程停下來。通常要用 `if` 來判斷傳入的參數是否已達到原先設定的停止條件，所以 function 裡除了要包含正確的程式碼做條件判斷之外，每次遞迴呼叫時所傳的參數值也必須有所改變，否則也是無法達到停止條件。例如範例裡面，遞迴呼叫的地方傳的參數是 `n+1`，所以相當於 `up_and_down(n)` 會呼叫 `up_and_down(n+1)`，這樣每次進入新的 recursion 的時候 `n` 的值就會增加，也才有可能讓 `if (n < 4)` 不成立而不再繼續遞迴下去。

第九章：Functions

[練習 W09_04] 寫 `void count_down(int n)`，用 recursion 方式顯示出由 `n` 倒數到 `0` 然後再正數到 `n` 的數列，譬如呼叫 `count_down(6)` 就會顯示 **6543210123456**。在主程式裡測試一下作用是否正確。(注意只有顯示一個 `0`。)

Tail Recursion

這是最簡單的一種 recursion 型式，因為它運作起來就像迴圈一樣。它的特色是在 `return` 之前才做 recursive call。我們來看看如何用迴圈和遞迴兩種不同方式來計算整數的階乘。

[範例 E09_06.c]

```
#include <stdio.h>
long fact(int n);
long rfact(int n);
int main(void)
{
    int num;
    printf("Enter a value in the range 0-12 (q to quit):\n");
    while (scanf("%d", &num) == 1) {
        if (num < 0)
            printf("No negative numbers, please.\n");
        else if (num > 12)
            printf("Keep input under 13.\n");
        else {
            printf("loop: %d factorial = %ld\n", num, fact(num));
            printf("recursion: %d factorial = %ld\n", num, rfact(num));
        }
        printf("Enter a value in the range 0-12 (q to quit):\n");
    }
    printf("Bye.\n");
    return 0;
}

long fact(int n)
{
    long ans;
    for (ans = 1; n > 1; n--)
        ans *= n;
    return ans;
}

long rfact(int n)
{
    long ans;
    if (n > 0)
        ans = n * rfact(n-1);
    else
        ans = 1;
    return ans;
}
```

輸出：

```
Enter a value in the range 0-12 (q to quit):
5
loop: 5 factorial = 120
recursion: 5 factorial = 120
Enter a value in the range 0-12 (q to quit):
11
loop: 11 factorial = 39916800
recursion: 11 factorial = 39916800
Enter a value in the range 0-12 (q to quit):
q
Bye.
```

先來看用迴圈的版本，`for` 從 `n` 開始，不斷減少 `n` 的值 (`n--`)，直到 `n` 的值變成 `1` 迴圈就停止。在每次的迴圈裡都把 `ans` 累乘 `n`，所以最後整個迴圈得到的效果就是 `n*(n-1)*(n-2)*...*2`。再來看使用遞迴呼叫的方式有什麼不同。由於一開始在 `main()` 裡面呼叫 `rfact()` 是傳入 `num`，所以進入 `rfact()` 之後 `if` 條件成立，因此會執行

```
ans = n * rfact(n-1);
```

這樣又會再呼叫 `rfact()`，但是傳入的參數變成是 `n-1`。就這樣繼續呼叫 `rfact()`，但是都還沒有遇到任何 `return` 的動作。直到當傳入的 `n` 的值等於 `0` 的時候，由於 `if` 不再成立，所以得到 `ans = 1`；然後就做 `return ans`；也就是把 `1` 傳回去。這時候回到上一層未完成的動作 `ans = n * rfact(n-1)`；由於 `rfact(n-1)` 得到的值是 `1`，而 `n` 目前的值是 `2`，所以就算出了 `ans` 的值是 `2` 乘上 `1` 等於 `2`，然後就可以執行到 `return ans`；把 `2` 傳回去。整個 recursion 就這樣一路返回，最後就可以得到答案 `ans`，再傳回去給 `main()`。

在這個範例使用迴圈看起來比 recursion 簡單很多，而實際上使用的資源也是迴圈比較節省。不過這是因為 tail recursion 是 recursion 的最單純的一種形式。我們接下來會看到有些問題確實必須以 recursion 方式來思考會比較簡潔，若想用迴圈來寫反而比較複雜。

[練習 W09_05] 用 recursion 方式改寫 `power()` (範例 E06_17.c)，傳入一個 `double` 變數 `n` 和一個 `unsigned int` 變數 `p`，算出 `n` 的 `p` 次方。

第九章：Functions

Recursion and Reversal

來看看什麼情況 recursion 方式比迴圈方式更好寫。我們要來寫一個程式能把十進位整數值的二進位表示法顯示出來。譬如二進位的表示法 **101** 其實就是 $1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$ ，相當於十進位的 **5**。從十進位數字轉成二進位表示法的運算要怎麼做，我們可以先用十進位來思考。譬如平常寫 **32456** 其實相當於 $3 \cdot 10000 + 2 \cdot 1000 + 4 \cdot 100 + 5 \cdot 10 + 6$ 。所以如果要得到個位數是多少，只要用 **32456 % 10**，就可以知道個位數是 **6**。那麼接下來如果要知道十位數是多少，其實只要先把十位數降成個位數，也就是 **32456/10**，然後無條件捨去得到 **3245**，接著再用同樣伎倆 **3245%10** 就可以取得十位數。就這樣一直做下去就可以把每個位數都求出來。二進位運算其實完全一樣，只要把 **%10** 改成 **%2**，然後把 **/10** 改成 **/2** 就可以。

現在拿 **13** 來試試看，先算 **13%2** 得到 **1**，所以最右邊的位數是 **1**，然後 **13/2** 得到 **6**，然後 **6%2** 得到 **0**，所以右邊第二位是 **0**，然後 **6/2** 得到 **3**，而 **3%2** 是 **1** 所以右邊數來第三位是 **1**，然後 **3/2** 得到 **1**，而 **1%2** 等於 **1** 所以右邊算起第四位是 **1**，再來 **1/2** 得到 **0**，就不需要再算下去。所以我們最後得到 **13** 的二進位表示法為 **1101**。

現在就來看看程式碼怎麼寫：

[範例 E09_07.c]

```
#include <stdio.h>
void to_binary(unsigned long n);
int main(void)
{
    unsigned long number;
    printf("Enter an integer (q to quit):\n");
    while (scanf("%ul", &number) == 1) {
        printf("Binary equivalent: ");
        to_binary(number);
        putchar('\n');
        printf("Enter an integer (q to quit):\n");
    }
    printf("Bye.\n");
    return 0;
}

void to_binary(unsigned long n)
{
    int r;
    r = n % 2;
    if (n >= 2)
        to_binary(n / 2);
    putchar('0' + r);
    return;
}
```

輸出：

```
Enter an integer (q to quit):
13
Binary equivalent: 1101
Enter an integer (q to quit):
255
Binary equivalent: 11111111
Enter an integer (q to quit):
1024
Binary equivalent: 10000000000
Enter an integer (q to quit):
q
Bye.
```

在 **to_binary()** 裡面用 **putchar('0' + r)**；是為了把數值轉成字元然後輸出，所以如果 **r** 等於 **1** 就得到字元 **'1'**。這裡當然也可以用別種寫法，例如 **printf("%d", r)**；或是 **putchar(r ? '1' : '0')**；。

這個範例充分利用了 recursion 的特性，因為整個過程是倒反過來的，所以雖然我們最早呼叫 **to_binary()** 的時候計算的是最右邊的位數，但是最早被 **putchar()** 顯示出來的卻是最左邊的位數，而這正是我們想要得到的效果。假如要用迴圈方式來做出這個範例，就必需先把答案存起來，然後再反過來顯示出來，或者是想個新的作法，先從最高位開始取，但是不管怎樣程式都會變得比較複雜。

Recursion 的優缺點

我們來看看一個有名的數列叫做 Fibonacci 數列，它的規律是 1, 1, 2, 3, 5, 8, 13, ...。最前面兩個數都是 1，然後接下來每一個數都是前兩個數的和，所以這個數列本身就是用 recursion 方式定義。我們可以用底下的 function 來表示

```
long Fibonacci(int n)
{
    if (n > 2)
        return Fibonacci(n-1) + Fibonacci(n-2);
    else
        return 1;
}
```

程式碼看起來非常簡潔。但是如果試著呼叫 **Fibonacci(50)** 會發現程式會跑不出來，因為每次呼叫 **Fibonacci()** 都會分支出兩個 **Fibonacci()**，每次也會多複製出兩個 local variable **n**，一直到 **n <= 2** 為止。然而 **2** 的 48 次方是非常大的數目，超過一般 PC 的記憶體容量，所以程式會跑不出來。

[練習 W09_06] 用迴圈方式來計算 Fibonacci 數列。

第九章：Functions

[練習 W09_07] 把下面的空格填入適當的程式碼，讓 `gcd()` 能夠計算出最大公因數。

```
#include <stdio.h>
int gcd(int x, int y);
int main(void)
{
    int x, y;
    scanf("%d%d", &x, &y);
    printf("%d\n", gcd(x, y));
    return 0;
}
int gcd(int x, int y)
{
    int ans;
    if (____) {
        ans = gcd(____, ____);
    } else {
        ans = ____;
    }
    return ____;
}
```

[練習 W09_08] 改寫 `gcd()`，用迴圈方式來達成。

如何 compile 多個原始檔

到目前為止，我們寫的程式都是把所有的 functions 放在同一個檔案裡，這樣只要 compile 那個檔案就可產生最後我們要的執行檔。但是其實 compiler 允許我們把程式碼 (以 function 為基本單位) 分散在不同檔案裡，然後再個別 compile 並且 link 產生一個執行檔。如果我們把程式碼分散在多個檔案中，由於 function 使用前要先宣告 prototype，所以必須在每個分散的檔案裡面都把要用到的 function prototype 包含進來。通常的作法都是把這些宣告放在一個 header file 裡面 (副檔名 .h)，然後用 `#include` 的方式引入原始檔中。除了 function prototype 之外，用 `#define` 方式定義的常數也可以放在 header file 裡。

我們用底下的範例來介紹多個原始檔的程式設計方法，我們把程式碼分成兩個 .c 檔和一個 .h 檔：

[範例 E09_08a.c]

```
/* room rate program -- compile with E09_08b.c */
#include <stdio.h>
#include "E09_08.h"
int main(void)
{
    int nights;
    double hotel_rate;
    int code;
    while ((code = menu()) != QUIT) {
        switch(code) {
            case 1 : hotel_rate = HOTEL1;
                    break;
            case 2 : hotel_rate = HOTEL2;
                    break;
            case 3 : hotel_rate = HOTEL3;
                    break;
            case 4 : hotel_rate = HOTEL4;
                    break;
            default: hotel_rate = 0.0;
                    printf("Oops!\n");
                    break;
        }
        nights = getnights();
        showprice(hotel_rate, nights);
    }
    printf("Thank you and goodbye.\n");

    return 0;
}
```

第九章：Functions

[範例 E09_08b.c]

```

/* hotel management functions -- this file contains no main() */
#include <stdio.h>
#include "E09_08.h"
int menu(void)
{
    int code, status;
    printf("\n%s%s\n", STARS, STARS);
    printf("Enter the number of the desired hotel:\n");
    printf("1) Ritz Carlton      2) Four Seasons\n");
    printf("3) Grand Hyatt          4) Amanusa\n");
    printf("5) quit\n");
    printf("%s%s\n", STARS, STARS);
    while ( (status = scanf("%d", &code)) != 1 || (code < 1 || code > 5) ) {
        if (status != 1)
            scanf("%*s"); /* skip a word */
        printf("Enter an integer from 1 to 5, please.\n");
    }
    return code;
}
int getnights(void)
{
    int nights;
    printf("How many nights are needed? ");
    while (scanf("%d", &nights) != 1) {
        scanf("%*s");
        printf("Please enter an integer, such as 2.\n");
    }
    return nights;
}
void showprice(double rate, int nights)
{
    int n;
    double total = 0.0;
    double factor = 1.0;
    for (n = 1; n <= nights; n++, factor *= DISCOUNT)
        total += rate * factor;
    printf("The total cost will be $%.2f.\n", total);
}

```

[範例 E09_08.h]

```

/* constants and declarations for E09_08a.c and E09_08b.c */
#define QUIT      5
#define HOTEL1    400.00
#define HOTEL2    500.00
#define HOTEL3    300.00
#define HOTEL4    350.00
#define DISCOUNT 0.95
#define STARS     "*****"
int menu(void);
int getnights(void);
void showprice(double rate, int nights);

```

總共有兩個 .c 檔，其中一個是主程式檔，包含 `main()`，另外一個包含會用到的 functions。至於 .h 檔則是 function prototypes 以及 `#define` 的常數。要 compile 這個範例程式的方法有幾種，如果是在 DOS command line 裡可以打

```
gcc E09_08a.c E09_08b.c
```

然後目錄下會產生一個我們要的執行檔，檔名是 a.exe。如果是在 Dev-C++ 的程式開發環境下面，只要先產生一個 project，然後把兩個 .c 檔都加入 project 底下，再來可以 compile，會產生和 project 名稱相同的 .exe 執行檔。

[補充說明] 在 Dev-C++ 的 menu 選 [File] → [New] → [Project...]，產生了 project 之後，再選 [Project] → [Add to Project] 選擇要加入的原始檔 E09_08a.c 以及 E09_08b.c。再來就像平常一樣 compile 就可以產生最後的執行檔。

第九章：Functions

程式執行結果如下：

```
*****
Enter the number of the desired hotel:
1) Ritz Carlton      2) Four Seasons
3) Grand Hyatt        4) Amanusa
5) quit
*****
2
How many nights are needed? 5
The total cost will be $2262.19.

*****
Enter the number of the desired hotel:
1) Ritz Carlton      2) Four Seasons
3) Grand Hyatt        4) Amanusa
5) quit
*****
3
How many nights are needed? 2
The total cost will be $585.00.

*****
Enter the number of the desired hotel:
1) Ritz Carlton      2) Four Seasons
3) Grand Hyatt        4) Amanusa
5) quit
*****
5
Thank you and goodbye.
```

主程式會呼叫 `menu()` 來顯示飯店選單，同時 `menu()` 會回傳使用者的選擇。主程式依照使用者的選擇決定費率。接著呼叫 `getnights()` 來取得使用者輸入的住宿天數。該有的資訊都搜集齊全之後，就呼叫 `showprice()` 來計算出價錢。

在 `menu()` 和 `getnights()` 裡面都用到了一個小技巧，就是當使用者輸入錯誤的時候（不是整數），用 `scanf("%*s");` 的方式來把沒有格式不符的資料跳過（當作字串讀進來，但是不儲存），同時呼叫 `printf()` 要求重新輸入。如果輸入的是整數但是不在 1 到 5 的範圍內，則 `status` 會等於 1，所以就不必呼叫 `scanf("%*s");`，只要直接呼叫 `printf()` 要求輸入 1 到 5 之間的整數。

使用 & 符號取得位址

指標 (pointer) 是 C 語言裡面的非常重要、同時也最讓初學者感到困擾的概念。如果用最簡單的方式來說明，指標就是一個專門用來儲存位址的變數。

我們在使用 `scanf()` 的時候，其實就已經用過位址來當作參數（變數前面加 `&`）。而任何 C 的 function 如果不想靠 `return` 方式來傳回值，就只能透過位址來取得 function 執行的結果。我們先來探討一下 `&` 符號，以及如何取得位址的資訊。在變數前面加上 `&`，會得到用來儲存該變數的位址。譬如變數的名稱叫做 `y`，則 `&y` 就是這個變數的位址。我們可以把位址想成記憶體中的某個位置。

[範例 E09_09.c]

```
#include <stdio.h>
int main(void)
{
    int y = 5;
    printf("%d %p\n", y, &y);
    return 0;
}
```

輸出：

5 0x22FF74

輸出的 `&y` 就是位址的十六進位值。能夠取得變數的位址是一個很強大的功能，因為這樣一來我們就可以透過位址來存取和修改變數值，而不再只是透過變數名稱來存取變數值。除此之外，我們也可以把位址當參數傳遞，這一點可以幫我們做到很多原本做不到的事情。

我們之前提過每個 function 會有自己的 local variables，不會和外部衝突，而且參數傳入 function 的時候，是把值複製過去，所以在 function 裡任意修改參數並不會影響到外部的變數。下面的範例就是在說明這個特性。

第九章：Functions

[範例 E09_10.c]

```
#include <stdio.h>
void f(int);
int main(void)
{
    int y = 2, z = 5;
    printf("In main(), y = %d and &y = %p\n", y, &y);
    printf("In main(), z = %d and &z = %p\n", z, &z);
    f(y);
    return 0;
}
void f(int z)
{
    int y = 10;
    printf("In f(), y = %d and &y = %p\n", y, &y);
    printf("In f(), z = %d and &z = %p\n", z, &z);
}
```

程式執行之後會得到類似下面的結果

```
In main(), y = 2 and &y = 0022FF74
In main(), z = 5 and &z = 0022FF70
In f(), y = 10 and &y = 0022FF44
In f(), z = 2 and &z = 0022FF50
```

觀察執行結果會發現使用什麼樣的變數名稱在 `main()` 和 `f()` 其實完全無關，而且每個變數的位址也都不一樣。所以真正能用來區別不同的資料的其實是位址。

假設我們想要寫一個 function，傳入兩個參數，然後在 function 裡面交換這兩個參數的值，這個程式該怎麼寫呢？先看看下面這種寫法：

[範例 E09_11.c]

```
#include <stdio.h>
void swap(int u, int v);
int main(void)
{
    int y = 2, z = 5;
    printf("Before calling swap(), y = %d and z = %d.\n", y, z);
    swap(y, z);
    printf("After calling swap(), y = %d and z = %d.\n", y, z);
    return 0;
}
void swap(int u, int v)
{
    int temp;
    printf("In swap(), before swapping, u = %d and v = %d.\n", u, v);
    temp = u;
    u = v;
    v = temp;
    printf("In swap(), after swapping, u = %d and v = %d.\n", u, v);
}
```

程式執行的結果是

```
Before calling swap(), y = 2 and z = 5.
In swap(), before swapping, u = 2 and v = 5.
In swap(), after swapping, u = 5 and v = 2.
After calling swap(), y = 2 and z = 5.
```

結果 `swap()` 對 `main()` 裡的 `y` 和 `z` 一點效果都沒有。雖然在 `swap()` 裡面，做完下面三行

```
temp = u;
u = v;
v = temp;
```

之後參數確實被交換了，但是出了 `swap()` 回到 `main()` 一切都沒變，因為更改的其實是存放在不同位址的變數值。

第九章：Functions

要怎麼才能做到真正把 `main()` 裡的 `y` 和 `z` 交換？我們要先學會怎麼使用**指標變數**。

指標變數

假設我們已經有一個指標變數，名字叫做 `ptr`，由於指標變數**專門用來儲存位址**，所以我們可以寫

```
ptr = &y;
```

這樣的動作等於把 `y` 的位址用 `ptr` 記下來，通常我們也會把這樣的動作叫做“把 `ptr` 指向 `y`”。雖然同樣都代表某個記憶體位址，但是 `&y` 是一個 `constant`，它的值就是 `y` 的位址，是個固定的值不能改變。但是 `ptr` 是個**變數**，所以我們可以改變 `ptr` 的值，拿它來記錄別的位址，例如，

```
ptr = &z;
```

假設我們用了上面的程式碼把 `ptr` 的值改設為 `z` 的位址，也就說用 `ptr` 記錄下 `z` 的位址，那麼我們可以對 `ptr` 做所謂 `dereferencing` 的動作，這個動作的作用是把 `ptr` 記錄的位址裡面所儲存的值取出來，例如

```
x = *ptr;
```

也就是使用 `*` 符號加在指標變數前面，可以取得 `ptr` 代表的位址裡所存放的數值。這兩個動作 `ptr = &z;` 和 `x = *ptr;` 得到的效果相當於

```
x = z;
```

如何宣告指標變數

我們知道如何宣告 `int` 變數或是其他型別的變數，但是要怎麼宣告指標變數？難道是用底下這樣：

```
pointer ptr; /* not the way to declare a pointer */
```

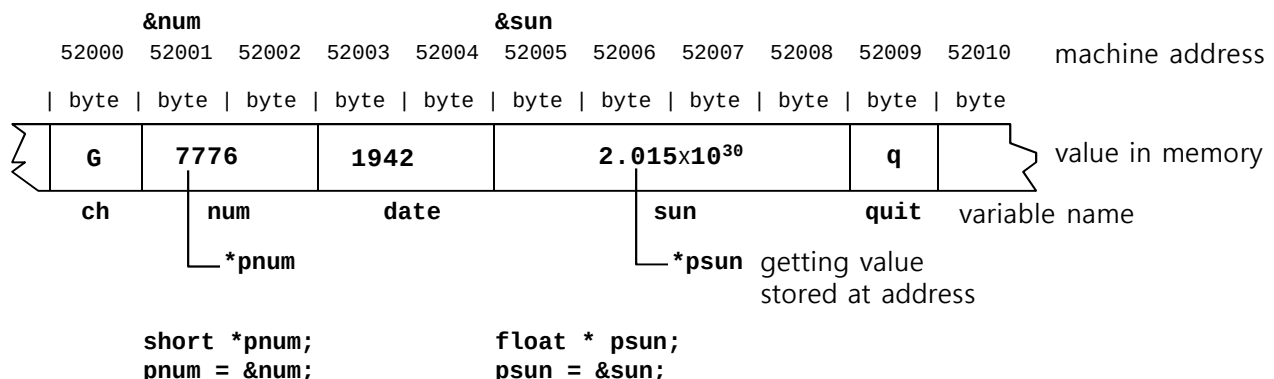
並不是這樣，並沒有叫做 `pointer` 的內建型別。而且我們除了要知道 `ptr` 是個用來記錄位址的變數之外，還要知道它所記錄的位址裡，儲存的資料型別是什麼，因為不同型別的資料在記憶體裡面需要的空間不同，而我們除了要靠 `ptr` 記錄起始位址之外，還要知道後面涵蓋了多少數量的記憶體空間。所以我們必須有各式各樣的指標類型，類似

```
int_pointer ptr; /* not the way to declare a pointer */
char_pointer ptr; /* not the way to declare a pointer */
float_pointer ptr; /* not the way to declare a pointer */
```

不過如果真的是這樣宣告就太麻煩了，因為每個原有的型別都要造出一個對應的指標型別。在 C 語言裡真正的作法是底下這樣的寫法

```
int * pi; /* pi is a pointer to an integer variable */
char * pc; /* pc is a pointer to a character variable */
float * pf, * pg; /* pf and pg are pointers to float variables */
```

這樣的宣告表示 `pi` 是指向 `int` 的指標，所以 `pi` 的值是個位址，而這個位址裡存的資料是個整數。同理，`pc` 是指向 `char` 的指標，`pf` 和 `pg` 都是指向 `float` 的指標。我們用底下的圖來模擬一下指標的宣告和使用過程中記憶體的狀態變化：



第九章：Functions

當我們在寫程式的時候，每個變數對我們來說具有兩項主要性質：**變數名稱**和**變數值**。當程式經過 `compile` 然後 `load` 到記憶體裡準備執行時，電腦認知的變數所具有的性質變成**位址**和**變數值**。也就是說電腦是以位址來區別變數。在許多語言中，位址完全交給電腦負責就好了，但是在 C 語言裡，程式設計者可以用 `&` 符號來取得位址的資訊。我們就來試試看透過記憶體位址，達到真正交換兩個變數值的效果。

[範例 E09_12.c]

```
#include <stdio.h>
void swap(int * u, int * v);
int main(void)
{
    int y = 2, z = 5;
    printf("Before calling swap(), y = %d and z = %d.\n", y, z);
    swap(&y, &z);
    printf("After calling swap(), y = %d and z = %d.\n", y, z);
    return 0;
}

void swap(int * u, int * v)
{
    int temp;
    printf("In swap(), before swapping, u = %d and v = %d.\n", *u, *v);
    temp = *u;
    *u = *v;
    *v = temp;
    printf("In swap(), after swapping, u = %d and v = %d.\n", *u, *v);
}
```

程式執行的結果是

```
Before calling swap(), y = 2 and z = 5.
In swap(), before swapping, u = 2 and v = 5.
In swap(), after swapping, u = 5 and v = 2.
After calling swap(), y = 5 and z = 2.
```

確實把 `main()` 裡的 `y` 和 `z` 的值對調了。在主程式裡呼叫 `swap()` 的方式變成

```
swap(&y, &z);
```

我們傳入的參數不再是 `y` 和 `z` 的值，而是 `y` 和 `z` 的位址。所以在宣告 `swap()` 的 `prototype` 的時候，參數的型態應該是指標，這樣才能接收位址。而且因為 `y` 和 `z` 是 `int`，所以我們需要指向 `int` 的指標，

```
void swap(int * u, int * v);
```

在 `swap()` 的程式碼主體裡，我們宣告 `int temp`；因為要用 `temp` 當作交換時的暫存區。由於 `u` 和 `v` 都是指標變數，它們的值都代表著位址，我們要把 `u` 所記錄的位址裡儲存的資料值取出來，放入 `temp` 裡

```
temp = *u;
```

(注意，絕對不要寫成 `temp = u`；我們要交換的是位址裡儲存的資料，不是要交換位址。)

同樣的道理

```
*u = *v;
```

把儲存在 `v` 所記錄的位址裡的資料取出來，把 `u` 所記錄的位址裡面儲存的值蓋掉。

如果能把範例 E09_12.c 想清楚，並且把整個寫法弄得很熟，指標的使用對我們來說就不會構成太大的困擾。最好不僅只是把原理都想通，還能熟練到可以反射式地寫出 `temp = *u`；或 `*u = *v`；或 `int * u`；這些指標的用法。建議大家可以試著先把整個範例弄懂，然後自己重寫整個範例。

[練習 W09_09] 寫一個 function 叫做 `sort2()`，它的作用是讓兩個變數 `a` 和 `b` 在經過呼叫 `sort2()` 之後，`a` 的值變成 `a` 和 `b` 之中較小的值，同時 `b` 的值被改成 `a` 和 `b` 之中較大的值。試著用指標和位址存取方式來寫。

第十章：Arrays and Pointers

終於來到了學習 C 語言的關鍵階段。我們只要再把陣列 (arrays) 和指標 (pointers) 的用法練得很熟，就能夠用 C 語言來實作各種資料結構和演算法，解決更複雜的問題。陣列和指標兩者很有關連，我們會先介紹陣列，接著再講解指標的用法。我們已經看過了一些使用陣列的範例，由其是字元陣列的操作。陣列就是一連串相同型別的元素，放置在連續的位置，因此陣列的宣告必須告訴 compiler 這個陣列的組成元素的型別以及總共包含多少元素，如此一來 compiler 才能知道如何處理我們需要的陣列，例如下面的宣告：

```
int main(void)
{
    float signal[100];
    char name[20];
    int histogram[128];
    ...
    return 0;
}
```

方括號 [] 用來表示 **signal** 是個陣列，括號裡的數字代表陣列的大小，而每個元素的型別都是 **float**。陣列的每個元素可以被單獨存取，但是要特別注意，陣列的編號從 **0** 開始，所以在上面的例子裡 **signal[0]** 是第一個元素，而 **signal[99]** 是最後一個元素。

[範例 E10_01.c]

```
#include <stdio.h>
#define NBIN 10
#define STARS "*****"
int main(void)
{
    int hist[NBIN] = {3,5,2,1,8,3,1,5,4,3};
    int index;

    for (index = 0; index < NBIN; index++) {
        printf("[%d] %2d ", index, hist[index]);
        printf(".*s\n", hist[index], STARS);
    }

    return 0;
}
```

這個程式執行起來像底下這樣：

```
[0]  3  ***
[1]  5  *****
[2]  2  **
[3]  1  *
[4]  8  *****
[5]  3  ***
[6]  1  *
[7]  5  *****
[8]  4  ****
[9]  3  ***
```

上面的範例程式一開始就先把陣列裡面的元素設定好，也就是所謂的初始化 (initialization)，對陣列做初始化的語法是在陣列宣告之後用等號來指定，然後用波浪括號包住元素的值，每個值用逗號隔開。這個程式輸出的結果是一個垂直顯示的長條圖。如果這個程式少掉了初始化的步驟，會發生什麼後果呢？底下是把宣告改成 **int hist[NBIN];** 也就是去掉初始化之後的程式執行結果：

```
[0] 2009095316 *****
[1] 2008948848 *****
[2] -1 *****
[3] 2009055971 *****
[4] 2009118740 *****
[5] 4008912 *****
[6] 4008880 *****
[7] 8 *****
[8] 2009116333 *****
[9] 28 *****
```

第十章：Arrays and Pointers

沒有經過初始化的陣列，裡面會有什麼東西我們無法控制，完全看當時記憶體中有什麼內容而決定。假如我們再把宣告改成 `int hist[NBIN] = {3, 5};`，也就是如果只設定前兩個元素的初值，則執行結果會變成

```
[0] 3 ***
[1] 5 *****
[2] 0
[3] 0
[4] 0
[5] 0
[6] 0
[7] 0
[8] 0
[9] 0
```

也就是說，如果我們只對陣列的部份元素設定初值，則其餘的元素會被自動設為 `0`。同樣的道理，如果我們想要讓 `hist` 每個元素的初值都變成 `0`，只要寫 `int hist[NBIN] = {0}` 就可以辦到。

另一方面，假如我們在設定初值時，列出的初值數量超過宣告的元素個數，譬如 `int hist[3] = {1,2,3,4};`，陣列大小是 `3`，但是設了四個初值，這樣的寫法是不被允許的，在 `compile` 時就會出錯。遇到這種情況如果改成 `int hist[] = {1,2,3,4};`，則不會有問題。使用這種寫法，我們就不必自己指定陣列大小，而讓 `compiler` 依照我們給的初值個數決定陣列要多大。

[範例 E10_02.c]

```
#include <stdio.h>
#define STARS "*****"
int main(void)
{
    int hist[] = {3,5,2,1,8,3,1,5,4,3};
    int index;

    for (index = 0; index < (sizeof hist) / (sizeof hist[0]); index++) {
        printf("[%d] %2d ", index, hist[index]);
        printf(".*s\n", hist[index], STARS);
    }

    return 0;
}
```

上面的範例執行後得到的結果和範例 E10_01.c 完全一樣，但是我們並沒有直接指定陣列的元素個數，而是靠初值個數來決定。除此之外，在迴圈裡面我們用以前看過的 `sizeof` 來算出陣列實際上總共包含幾個元素：先用 `sizeof hist` 取得整個陣列佔用的 `byte` 數目，然後除以 `sizeof hist[0]`，也就是第一個元素佔用的 `byte` 數，這樣就可以算出陣列總共有幾個元素。

除了用設定初始值的方式之外，真正比較常用的作法應該是配合迴圈，逐一把適當的值填入陣列中。像底下這樣：

[範例 E10_03.c]

```
#include <stdio.h>
#define STARS "*****"
#define MAXBIN 25
int main(void)
{
    int hist[MAXBIN];
    int i, nbin;

    for (nbin = 0; nbin < MAXBIN; nbin++) {
        if (scanf("%d", &hist[nbin]) != 1)
            break;
    }
    for (i = 0; i < nbin; i++) {
        printf("[%d] %2d ", i, hist[i]);
        printf(".*s\n", hist[i], STARS);
    }

    return 0;
}
```

第十章：Arrays and Pointers

我們可以用導向的方式來測試前面的範例。假設我們已經在 `test.txt` 這個文字檔裡輸入了資料，則執行結果如下

```
D:\COURSE\I2P07\code>cat test.txt
1 3 2 4 8 3 3 2 2
D:\COURSE\I2P07\code>E10_03.exe < test.txt
[0] 1 *
[1] 3 ***
[2] 2 **
[3] 4 ****
[4] 8 *****
[5] 3 ***
[6] 3 ***
[7] 2 **
[8] 2 **
```

範例 E10_03.c 在迴圈裡用 `scanf()` 逐次讀取整數，然後循序存入陣列的元素中，這就是實際常用的操作方式之一。要特別注意的是 C 語言本身並沒有提供把一個陣列整個設給另一個陣列的功能，所以不能用像下面這樣的寫法來複製陣列的內容：

```
int hist1[NBIN];
int hist2[NBIN];
hist1 = hist2; /* wrong */
```

我們只能透過迴圈，對每個元素以逐一存取的方式，把一個陣列的內容複製到另一個陣列，或是用 C Standard Library 提供的 function 來複製陣列（可以用 `memcpy()` 來複製陣列，我們不在此介紹，目前暫時就先用迴圈方式。）使用陣列時要注意的還有陣列的範圍，這點我們已經強調過不少次。我們要對自己用的陣列索引負責，因為 compiler 並不會替我們檢查是否超出範圍。假如不小心超出了範圍，會發生什麼狀況就不得而知，很有可能程式照樣能跑，但也可能是程式整個當掉。

[範例 E10_04.c]

```
#include <stdio.h>
#define SIZE 4
int main(void)
{
    int value1 = 55;
    int arr[SIZE];
    int value2 = 66;
    int i;

    printf("value1 = %d, value2 = %d\n", value1, value2);
    for (i = -1; i <= SIZE; i++) {
        arr[i] = 2 * i + 1;
    }
    for (i = -1; i < 7; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);
    }
    printf("value1 = %d, value2 = %d\n", value1, value2);

    return 0;
}
```

這個程式執行起來像底下這樣：

```
value1 = 55, value2 = 66
arr[-1] = -1
arr[0] = 1
arr[1] = 3
arr[2] = 5
arr[3] = 7
arr[4] = 9
arr[5] = 28
arr[6] = 0
value1 = 55, value2 = -1
```

第十章：Arrays and Pointers

由前面得到的執行結果，我們可以推測這個 compiler 似乎是把 **value2** 放在 **arr** 陣列的前面，也就是說在記憶體中，**value2** 的位址正好和 **arr[-1]** 的位址重疊，所以當我們用不正當的方式，把超出陣列合法位置的元素 **arr[-1]** 的值設定成 **-1**，就會把 **value2** 原有的值蓋掉，讓 **value2** 變成了 **-1**。不同的 compiler 對於變數會有不同的放置方式，所以在別的電腦上重新 compile 再執行，得到的結果可能會和前面的情況稍有不同。

為什麼 C 語言允許我們亂搞一通？為什麼不規定嚴格一點，或是幫我們檢查索引是否超出了陣列的範圍？這個問題的原因要回溯到 C 語言的設計理念：基本上 C 語言相信程式設計者知道自己在做什麼，因此對於程式設計者完全信賴，給予很大的自由，而這樣可以換來執行速度變快的優勢。由於陣列的索引是否超出範圍並不是在 compile 時就能完全偵測出來，大多數情況下必須到了 runtime，也就是程式執行時才能發現索引超出範圍；為了要在執行時能夠檢查出陣列索引超出範圍，compiler 必須在程式裡額外加入一些負責檢查範圍的程式碼，但這樣一來就會讓程式的執行速度變得比較慢。因此，C 語言寧願相信程式設計者永遠都是在做正確的事情，藉此換得執行速度較快的程式。話雖如此，但不見得每個程式設計者都那麼值得信賴。如果程式設計者自己亂搞而搞砸了，後果就要自行負責。

[練習 W10_01] 用導向方式 (例如 D:\COURSE\I2P07\code>W10_01.exe < test.txt) 讀入一串整數，假設整數的個數不超過 100 個數，且每個整數值都介於 0 和 1,000,000 範圍之內。先把讀入的資料存入陣列中，然後一併找出其中最大值和最小值。

[練習 W10_02] 和 W10_01 的輸入資料相同，把資料從小到大按照順序排好。

設定陣列的大小

除非我們使用的 compiler 支援最新的 C99 語法規則，否則在宣告陣列時不能用常數整數型別以外的方式來指定陣列的大小。所以底下這幾種是合法的宣告

```
float a1[5];
float a2[5*2 + 1];
float a3[(int)2.3];
```

但是下面這幾種是錯誤的

```
float a4[-4];
float a5[0];
float a6[2.3];
```

底下這幾種只有在支援 C99 的 compiler 下才可以，因為陣列大小不是常數而是變數在控制。

```
int n=5, m=6;
float a7[n]; /* C99 */
float a8[m*n]; /* C99 */
```

二維陣列

對電腦程式來說，二維陣列是非常方便的資料儲存和表示方式，例如數位影像或矩陣，本身就具有二維特性，如果要對影像或矩陣做處理，使用二維陣列的方式來操作在思考上會比較直接。在 C 程式裡要產生一個二維陣列的寫法是

```
int a[3][4];
```

我們可以把二維陣列想像成左圖的排列形式(注意陣列元素的編號)：

a[0][0]	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2]	a[2][3]

如果要設定初始值，則可以用下面的方式來設定

```
int a[3][4] = { {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} };
```

或是

```
int a[3][4] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 };
```

上面的例子傳達的一個觀念是，雖然我們把資料排列方式想像成二維，但是實際上電腦是用循序的方式來儲存資料，所以其實就只是一維，也就是說每個二維陣列裡的資料是一列接著一列放置在記憶體裡。使用二維陣列表示法只是為了方便我們在寫程式時可以用比較直觀的方式思考。

[練習 W10_03] 依照第二頁介紹設定一維陣列初始值的規則，推廣到二維陣列。底下的程式碼會得到什麼結果。

```
int a[][4] = { {1, 2, 3, 4}, {5, 6}, {9, 10, 11, 12} };
int b[][4] = { 1, 2, 3, 4, 5, 6, 9, 10, 11, 12 };
```

第十章：Arrays and Pointers

底下的程式碼用 `sizeof` 來顯示出二維陣列的大小，從輸出結果可以看出 `sizeof(a[0])` 的大小相當於 `sizeof(int)*4`，而 `sizeof(a)` 的大小則相當於 `sizeof(a[0])*3`，也就是 `sizeof(int)*4*3`。下面的程式碼同時也顯示了不同的初始化的結果。

```
#include<stdio.h>
int main(void)
{
    int a[][4] = { {1, 2, 3, 4}, {5, 6}, {9, 10, 11, 12}};
    int b[][4] = { 1, 2, 3, 4, 5, 6, 9, 10, 11, 12};

    int i, j;

    printf("a:          %d bytes\n", sizeof(a));
    printf("a[0]:       %d bytes\n", sizeof(a[0]));
    printf("a[0][0]: %d bytes\n", sizeof(a[0][0]));

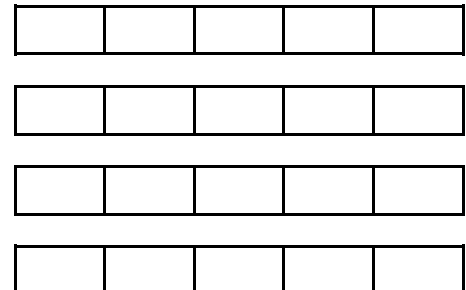
    printf("\n");
    for (i=0; i<3; i++) {
        for (j=0; j<4; j++) {
            printf("%3d ", a[i][j]);
        }
        printf("\n");
    }
    printf("\n");
    for (i=0; i<3; i++) {
        for (j=0; j<4; j++) {
            printf("%3d ", b[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

輸出：

a:	48 bytes		
a[0]:	16 bytes		
a[0][0]:	4 bytes		
1	2	3	4
5	6	0	0
9	10	11	12
1	2	3	4
5	6	9	10
11	12	0	0

row_vector



另外一種看待二維陣列的方式是把它想成 "陣列的陣列"：

```
typedef int row_vector [5];
row_vector matrix[4];
```

上面的程式碼相當於

```
int matrix[4][5];
```

也就是說，`row_vector` 是一種包含五個元素的陣列，把這樣的東西用 `typedef` 定成一種新的型別，然後 `matrix` 就是一個由四個 `row_vector` 組成的陣列。

接下來我們用一連串的範例來練習二維陣列的用法，然後再進入 C 語言最重要的主題：pointers。

首先必須聲明：接下來的幾個例子只適用於 Windows 環境。我們要介紹如何在 DOS 視窗裡的任意位置顯示文字。我們將不再侷限於循序方式一行接著一行地顯示字元，而是把 DOS 視窗當作一個二維陣列，直接把字元放到我們想要的二維座標裡。由於我們用到的 functions 並不屬於 C Standard Library，所以程式碼拿到不同平台並不能 compile 和執行。控制 Windows 環境下的 DOS 視窗的方式之一是使用 windows.h 裡面提供的 functions。為了使用上的方便，我們把它們簡化、包裝成自定的 console_draw.h。由於是架構在 windows.h 之上，所以這套程式碼只能在 Windows 環境下運作，如果想在 Linux 或 Mac OS 下做類似的事情，必須改用別的 header file 和 functions。這個 console_draw.h 檔案是參考網路上相關程式整理出來的簡化版本，只有兩個 functions，我們可以先不去管裡面究竟做了些什麼，只要在主程式裡面加入 `#include "console_draw.h"`，直接把 functions 拿來用就夠了。稍後當我們需要更多功能的時候，再去查詢 windows.h 的說明文件。包含在 console_draw.h 裡的 functions 分別是 `void putASCII(int x, int y, int ch, int color);` 以及 `void drawCmdWindow();`，它們的功能如下：
`void putASCII(int x, int y, int ch, int color)` 可以讓你在 DOS 視窗的 (x, y) 位置顯示你指定的字元 `ch`。參數 `color` 則要傳入你想要設定的字元的顏色，可以設成四種值：0, 1, 2, 3，各自代表 紫紅色, 綠色, 藍色, 黃色。把整個 DOS 視窗填好之後，再呼叫 `drawCmdWindows()` 就可以把填好的畫面秀出來。這兩個 functions 的定義是寫在 colsole_draw.c 裡面。

第十章：Arrays and Pointers

[console_draw.h]

```
#include <windows.h>
#ifndef CONSOLE_DRAW_
#define CONSOLE_DRAW_
#define SCREEN_WIDTH 80
#define SCREEN_HEIGHT 25
enum some_color { COLOR_BLUE = 0x09, COLOR_GREEN = 0x0A, COLOR_MAGENTA = 0x0D,
COLOR_YELLOW = 0x0E };
extern void putASCII( int x, int y, int ch, int color );
extern void drawCmdWindow();
#endif
```

[console_draw.c]

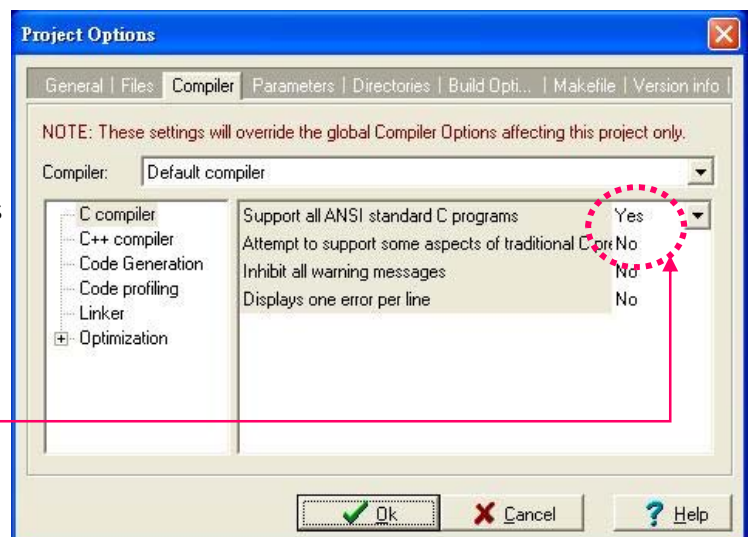
```
#include "console_draw.h"

COORD dwBufferSize = { SCREEN_WIDTH, SCREEN_HEIGHT };
COORD dwBufferCoord = { 0, 0 };
SMALL_RECT rcRegion = { 0, 0, SCREEN_WIDTH-1, SCREEN_HEIGHT-1 };
CHAR_INFO screen_buffer[SCREEN_HEIGHT][SCREEN_WIDTH];
HANDLE hOutput;
int touched = 0;

void putASCII( int x, int y, int ch, int color )
{
    if ( touched == 0 ) {
        hOutput = (HANDLE)GetStdHandle( STD_OUTPUT_HANDLE );
        ReadConsoleOutput( hOutput, (CHAR_INFO *)screen_buffer, dwBufferSize,
            dwBufferCoord, &rcRegion );
        touched = 1;
    }
    if (x<SCREEN_WIDTH && x>=0 && y<SCREEN_HEIGHT && y>=0) {
        screen_buffer[y][x].Char.AsciiChar = ch;
        if (color == 1)
            screen_buffer[y][x].Attributes = COLOR_GREEN;
        else if (color == 2)
            screen_buffer[y][x].Attributes = COLOR_BLUE;
        else if (color == 3)
            screen_buffer[y][x].Attributes = COLOR_YELLOW;
        else
            screen_buffer[y][x].Attributes = COLOR_MAGENTA;
    }
}

void drawCmdWindow()
{
    WriteConsoleOutput( hOutput, (CHAR_INFO *)screen_buffer, dwBufferSize,
        dwBufferCoord, &rcRegion );
    touched = 1;
}
```

我們不再對上面兩個檔案多做解釋，因為其中牽涉到的用法和我們目前想要探討的二維陣列較無關。有一個需要注意的小細節是，在 Dev-C++ 中要 compile 上述程式，必須要確認一下 Project Options 的設定是否如右圖。



要設成這樣

第十章：Arrays and Pointers

我們就來看看如何用自己定義的 console_draw.h 裡的 functions 來寫一些有關二維陣列的應用程式。

[範例 E10_05.c]

```
#include <stdio.h>
#include <time.h>
#include "console_draw.h"
#define WIDTH      12
#define OFFSET_X   10
#define OFFSET_Y   5

void delay(float sec);

int main(void)
{
    int a[WIDTH][WIDTH] =
    {
        '#', '#', '#', '#', '#', '#', '#', '#', '#', '#', '#', '#',
        '#', '.', '#', '.', '#', '.', '#', '.', '#', '.', '#', '.',
        '.', '#', '.', '#', '.', '#', '#', '#', '#', '.', '#',
        '#', '#', '#', '.', '#', '.', '#', '.', '#', '.', '#',
        '#', '.', '#', '.', '#', '.', '#', '.', '#', '.', '#', 'E',
        '#', '#', '#', '#', '.', '#', '.', '#', '.', '#', '.',
        '#', '.', '#', '.', '#', '.', '#', '.', '#', '.', '#',
        '#', '#', '.', '#', '.', '#', '.', '#', '.', '#', '.',
        '#', '.', '#', '.', '#', '.', '#', '.', '#', '.', '#',
        '#', '#', '#', '#', '#', '#', '#', '#', '#', '#', '#',
        '#', '.', '#', '.', '#', '.', '#', '.', '#', '.', '#',
        '#', '#', '#', '#', '#', '#', '#', '#', '#', '#', '#',
    };

    /* 二維陣列 迷宮地圖 */

    /* 方法：想像右手貼著牆壁 不斷沿著牆壁走 手不能離開牆壁 這樣一直走保證能走到出口 */

    int direction[4] = {'>', 'v', '<', '^'}; /* 面向何方 右 > 下 v 左 < 上 ^ */
    int cur_direct = 0; /* 目前的方向，對應到 direction[] 的元素 */
    int head_x[4] = {1, 0, -1, 0}; /* 下一個位置的 x 移動方向 */
    int head_y[4] = {0, 1, 0, -1}; /* 下一個位置的 y 移動方向 */
    int fx, fy; /* 下一個位置的座標 */
    int rx, ry; /* 右手邊位置的座標 */
    int px = 0, py = 2; /* 目前位置的座標 */
    int i, j;

    while (1) {
        /* 把儲存在 array 裡的地圖填到螢幕上相對的位置 */
        for (i = 0; i < WIDTH; i++) {
            for (j = 0; j < WIDTH; j++) {
                putASCII(j + OFFSET_X, i + OFFSET_Y, a[i][j], 1);
            }
        }

        /* 顯示目前所在位置 依照面對的方位 畫出對應的符號 > v < ^ */
        putASCII(px + OFFSET_X, py + OFFSET_Y, direction[cur_direct], 0);

        /* 把填好的螢幕內容顯示出來 */
        drawCmdWindow();

        if (a[py][px] == 'E') break; /* 找到出口 結束 */

        a[py][px] = ' '; /* 凡走過不留痕跡 把點點吃掉 */

        /* 計算 下一個位置 以及 右手邊位置 */
        fx = px + head_x[cur_direct];
        fy = py + head_y[cur_direct];
        rx = px + head_x[(cur_direct+1)%4]; /* (cur_direct+1)%4 相當於向右轉九十度 */
        ry = py + head_y[(cur_direct+1)%4];
    }
}
```

第十章：Arrays and Pointers

[範例 E10_05.c] 接上一頁

```

    if (a[fy][fx] != '#' && a[ry][rx] == '#') { /* 前面不是牆壁 右邊有牆壁 就向前直走 */
                                                /* (符合右手貼著牆壁的原則) */
        px = px + head_x[cur_direct];
        py = py + head_y[cur_direct];

    } else if (a[ry][rx] != '#') { /* 假如目前右邊沒有牆壁 就向右轉九十度 */
                                    /* 然後再順便向前走一格 才能繼續沿著牆壁走 */
        cur_direct = (cur_direct + 1) % 4;
        px = px + head_x[cur_direct];
        py = py + head_y[cur_direct];

    } else if (a[fy][fx] == '#' && a[ry][rx] == '#') {
                                                /* 假如前面碰壁 右邊也是牆壁 就應該向左轉 */
                                                /* (cur_direct+3)%4 左轉九十度的效果 */
        cur_direct = (cur_direct + 3) % 4;

    }
    /* 稍微停留 0.2 秒 */
    delay(0.2);

} /* while(1) */
return 0;

}

void delay(float sec)
{
    clock_t startc; /* 計算停留的時間 */
    startc = clock();
    for ( ; ; ) {
        if ((float)(clock()-startc)/CLOCKS_PER_SEC > sec) break;
    }
}

```

```

#####
#   #.....#
#   #.####.#
### #...#.#
#   ###.#.E
#### #.#.#.#
#   #.#.#.#
## #.#.#.#
#   >...#.#
#####.###.#
#.....#...#
#####

```

這個範例程式必須和 console_draw.c 一起 compiler，在 Dev-C++ 裡的做法就是把兩個 .c 檔都加入同一個 project 裡面，然後 compile 整個 project 就可以產生最後的可執行檔。

程式執行之後會跑出類似右上圖的畫面，讓箭頭符號在迷宮之中走動，直到找到迷宮出口為止。隨著箭頭移動方向不同，箭頭指的方向也會改變，這個效果是靠切換顯示的字元來達到，'>', 'v', '<', '^' 分別代表右下左上四個方向。在程式一開始我們先用二維陣列製作一個迷宮地圖。接著用一個無窮迴圈 while(1) 來把整個走迷宮的動作包起來。每一次迴圈會先把二維陣列的內容用我們自定的 putASCII() 填入螢幕的畫面中，填好之後再呼叫 drawCmdWindow() 一次把整個畫面顯示到螢幕上，這樣可以避免螢幕不斷閃爍的情況發生。

每次顯示完目前的狀態之後就要重新計算接下來的位址，要遵照的原則是讓右邊靠著牆壁，不斷沿著牆壁走，所以如果右邊是牆壁，而前面是空格，就繼續前進；如果右邊是空的就該右轉九十度然後前進一格，讓右邊能繼續貼著牆壁；另外一種狀況是如果右邊和前方都是牆壁，就要左轉九十度，左轉之後，右邊貼著牆壁的原則依然會保持。

在 while(1) 無窮迴圈的最後面我們呼叫自己寫的 delay(0.2) 來拖延計算速度，讓程式暫停 0.2 秒，避免程式跑得太快，看不清輸出的結果。我們用的是 C Standard Library 裡的 time.h 所提供的一個計時的 function，叫做 clock()。當程式呼叫 clock()，它會傳回以微秒或毫秒為單位的時間數值（實際使用的計時單位，在 time.h 裡有定義，叫做 CLOCKS_PER_SEC），時間數值是用 32 bits 來儲存累計的時間，累計到爆掉之後就會循環。我們可以靠呼叫兩次 clock() 來估計間隔的時間，因此

```

    startc = clock();
    for ( ; ; ) {
        if ((double)(clock()-startc)/CLOCKS_PER_SEC > sec) break;
    }

```

這段程式碼的效果就是會持續地去判斷目前的時間與第一次呼叫 clock() 的時間，是否已經超過了 sec 秒，如果是的話表示時間到了，就結束空轉的迴圈，讓程式能繼續進行下去。

下一個範例我們繼續利用 console_draw.h 來控制螢幕的輸出，並且結合一維陣列來記錄資料，交替使用一維和二維陣列。除此之外，我們還會簡單地運用 windows.h 的鍵盤控制功能，幫助我們模擬出即時互動的效果。

第十章：Arrays and Pointers

底下的範例除了用到 console_draw.h 之外，還要用到另一個自定的 kb_input.h 檔裡面宣告的 functions。範例程式必須與 console_draw.c 以及 kb_input.c 一起放入同一個 project 裡，然後 compile 產生可執行檔。

[範例 E10_06.c]

```
#include <stdio.h>
#include <stdlib.h> /* 要使用 srand 和 rand 來產生亂數 */
#include <time.h> /* 要使用 time 來產生 srand 所需的 seed */
#include "console_draw.h" /* 在 DOS command window 畫圖 */
#include "kb_input.h"

#define WIDTH      8
#define SIZE      (WIDTH*WIDTH) /* 地圖的大小 */
#define NUM_MINES 10
#define OFFSET_X  10
#define OFFSET_Y   5

int main(void)
{
    int a[SIZE] = {'.'};
    int i;
    int target;
    int hold;
    int ch;
    int px = 0, py = 0; /* Jack 的位置 */

    /* 把 NUM_MINES 個地雷放在 array 的最前面 但是要把 i = 0 的位置空下來放 Jack */
    for (i = 1; i <= NUM_MINES; i++) {
        a[i] = '@';
    }
    /* 剩下的位置補上地板 '.' */
    for (i = NUM_MINES+1; i < SIZE; i++) {
        a[i] = '.';
    }
    /* 用 shuffle 方式把 array 弄亂 來產生隨機的地圖 */
    srand(time(NULL));
    for (i = 1; i < SIZE; i++) {
        target = rand() % (SIZE-1) + 1; /* 隨機產生落在 1 ~ SIZE-1 的整數 */

        hold = a[i];
        a[i] = a[target];
        a[target] = hold;
    }

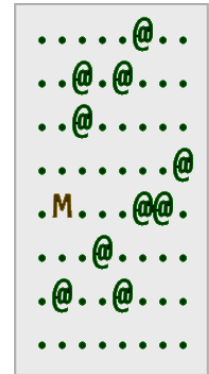
    initializeKeyInput(); /* 啟動鍵盤控制 */

    while (1) { /* 無窮迴圈 */

        /* 把儲存在 array 裡的地圖填到螢幕上相對的位置 */
        for (i = 0; i < SIZE; i++) {
            putASCII(i%WIDTH + OFFSET_X, i/WIDTH + OFFSET_Y, a[ i ], 1);
        }
        /* 畫出 Jack 目前位置 */
        putASCII(px + OFFSET_X, py + OFFSET_Y, 'M', rand()%4);

        /* 把填好的螢幕內容顯示出來 */
        drawCmdWindow();

        /* 如果踩到地雷 */
        if (a[px + py*WIDTH] == '@') { /* (px, py) 要換算成 array 的 index*/
            for (i = 0; i < SIZE; i++) {
                putASCII(i%WIDTH + OFFSET_X, i/WIDTH + OFFSET_Y, '*', rand()%4);
            }
            drawCmdWindow(); /* 把填好的螢幕內容顯示出來 */
            break; /* 跳出無窮迴圈 */
        }
    }
}
```



第十章：Arrays and Pointers

[範例 E10_06.c] 接上一頁

```

/* 停留 0.5 秒等待輸入 如果時間到了沒有輸入任何東西 就會傳回 0
   如果在 0.5 秒之內有輸入 就會立即傳回 1 */
if(waitForKeyDown(0.5)) {
    ch=getKeyEventASCII(); /* 讀取輸入的字元 */

    /* 判斷使用者按的按鍵對應到哪個移動方向 計算移動後的位置 */
    switch(ch) {
        case 'e': return 0; break;
        case 'i': py--; break;
        case 'k': py++; break;
        case 'j': px--; break;
        case 'l': px++; break;
    }
}
/* 檢查 Jack 移動過後的位置是否超出範圍 */
if (py < 0) py = 0;
if (py > WIDTH-1) py = WIDTH - 1;
if (px < 0) px = 0;
if (px > WIDTH-1) px = WIDTH - 1;

} /* while (1) 無窮迴圈 */

return 0;
}

```

[kb_input.h]

```

#include <windows.h>
#ifndef KB_INPUT_
#define KB_INPUT_
extern void initializeKeyInput(void);
extern int waitForKeyDown(float sec);
extern int getKeyEventASCII(void);
#endif

```

[kb_input.c]

```

#include "kb_input.h"

HANDLE Stdin;
INPUT_RECORD ir = {0};
DWORD Read = 0;
void initializeKeyInput(void)
{
    if ((Stdin = GetStdHandle (STD_INPUT_HANDLE)) == NULL)
        exit(1);
    SetConsoleMode (Stdin, ENABLE_LINE_INPUT);
}
int waitForKeyDown(float sec)
{
    if (WAIT_TIMEOUT == WaitForSingleObject (Stdin, (int)(sec*1000) ))
        return 0;
    else
        return 1;
}
int getKeyEventASCII(void)
{
    ReadConsoleInput(Stdin, &ir, 1, &Read);
    FlushConsoleInputBuffer(Stdin);
    if (ir.EventType == KEY_EVENT && ir.Event.KeyEvent.bKeyDown)
        return ir.Event.KeyEvent.uChar.AsciiChar;
    else
        return -1;
}

```

第十章：Arrays and Pointers

範例 E10_06.c 有幾個組成元件需要一一介紹：如何自動產生隨機放置地雷的地圖，如何用鍵盤控制，如何讓程式能模擬即時互動的效果。我們先來想想如何隨機放置地雷。

在範例中，我們用一維陣列來記錄整個地圖，把地圖的每一行接在一起，所以如果是一個八乘八的地圖，我們需要一個長度 64 的一維陣列來記錄地圖內容。假設我們想在地圖裡隨機放置十個地雷，如果隨便挑選十個位置來放置，雖然選到重複位置的機率不高，但是仍有可能選到同一個位置兩次，如此一來放置的地雷總數就會少於十。像這樣的情況，我們可以運用洗牌 (shuffle) 的概念來達到亂放的效果。首先把十個地雷依序放在一維陣列最前面的位置，假設我們用 '@' 來表示那個位置是地雷，然後在陣列的接下來位置一一補上一般的地板，假設用 '.' 來表示地板。全部放好之後，陣列裡就包含了十個地雷，接下來就把陣列的每個位置依序走過一次，每到一個位置就隨機產生一個新位置的索引編號，然後把這兩個位置裡的東西交換，達到洗牌的效果。要在程式中產生隨機亂數，可以用 C Standard Library 的 `stdlib.h` 所提供的 `rand()` 來產生，呼叫 `rand()` 會隨機傳回一個介於 0 到 `RAND_MAX` (至少是 32767) 的整數，我們可以用 % 求餘數的方式來取得一個在我們想要的範圍內大致均勻分佈的亂數。由於 `rand()` 是用所謂的 pseudorandom-number generator 方式產生，它產生的是一連串看起來很亂但是順序卻是固定的數列，所以我們還需要用 `srand()` 來產生一個不同的 "種子" (seed)，讓 `rand()` 不會每次都產生同一組數列。地圖內容完成後，就可以呼叫 `putASCII()` 把圖案填到螢幕畫面的二維陣列中，我們必須自己做座標的換算，把一維陣列的位置換成二維陣列的座標。填好畫面之後一樣是呼叫 `drawCmdWindow()` 來將畫面顯示到螢幕上。

鍵盤控制是靠 `windows.h` 提供的一些 functions 來達成，我們只取出目前需要用到的一小部份功能，把它們簡化，然後在 `kb_input.h` 裡宣告，至於 functions 的主體部份則定義在 `kb_input.c` 裡面。在 `main()` 裡面使用時，要先呼叫 `initializeKeyInput()`；把鍵盤輸入的功能啟動，然後在程式中我們會用 `ch=getKeyEventASCII()`；讀取使用者按下的按鍵。再來就是用 `switch ... case` 的方式來判斷使用者按的究竟是哪個鍵，然後就可以做對應的處理。

我們在 `kb_input.h` 裡面還宣告了 `int waitForKeyDown(float sec);` 它的作用是等待鍵盤輸入，但是只會等待一段時間，時間的長短是靠參數 `sec` 來設定，當預定等待的時間到了而使用者沒有按下任何按鍵，`waitForKeyDown()` 就自動結束等待並且傳回 0。如果預定等待的時間還沒到而使用者按下了按鍵，則會立即回傳 1。有了這個 function，我們的程式就不會為了等待使用者輸入而卡在那個地方。這個功能對於設計即時互動的程式很有用，有了這個機制我們才能模擬程式不斷在背景運作的效果，而不是和使用者一來一往地互相等待對方動作。在範例中為了顯示這樣的效果，我們會不斷隨機改變使用者操縱的 'M' 符號的顏色，就算使用者沒按任何按鍵移動 'M' 的位置，顏色還是會持續地改變，顯示程式其實正在不斷的執行中。(附註：我們會在課堂上再介紹另一種更一般化的讀取鍵盤輸入的寫法，並且會提供對應的檔案 `kb_input2.h` 和 `kb_input2.c`。另外關於螢幕輸出，也會提供加強版 `console_draw2.h` 和 `console_draw2.c`，可以控制更多前景和背景顏色變化，做出更好看的效果。由於這一部分的內容對於學習 C 語言來說並不算是不可或缺的技術，所以我們只會在上課的時候大致講解一下用法，而不編入講義中。)

[練習 W10_04] 繼續練習二維陣列的使用。

輸入：用 `getchar()` 讀取一個 高 15 寬 30 的地圖，每個位置的值為 @ 或 ~。@ 代表土地 ~ 代表水。

輸出：將土地編號，相連的土地構成島嶼，每個島嶼要有不同的編號，最多會有十個島嶼。把編號過的地圖輸出。

編號要按照 0, 1, 2, ..., 9 順序，不能跳號。土地相連的定義，除了上下左右相鄰還包括斜向相鄰。

範例：

底下的程式碼可用來讀取資料

```
int map[15][30];
int c, j = 0, k = 0;
while (j < 15) {
    c = getchar();
    if (c=='@' || c=='~') {
        map[j][k] = c;
        k++;
        if (k == 30) {
            k = 0;
            j++;
        }
    }
}
```

[練習 W10_05] 還是關於二維陣列。

輸入：用 `getchar()` 讀一個 高 15 寬 20 的地圖，每個位置如果是 # 代表基地台，+ 則代表 sensor。

輸出：求出每個 sensor 到離它最近的基地台的距離。距離用所謂 Manhattan distance 來計算，也就是從 A 到 B 只能直走或橫走，不能斜向走，總共需要的步數即為 Manhattan distance。

第十章：Arrays and Pointers

W10_05 的輸入資料的讀取方式和 W10_04 類似，輸入和輸出的圖例如下：

+++++		4 3 2 1 1 2 2 3 4 5 6 7 6 5 4 3 2 1 2 3
+++##+++++		3 2 1 0 0 1 1 2 3 4 5 6 5 4 3 2 1 0 1 2
++++#+++++		4 3 2 1 0 1 0 1 2 3 4 5 4 3 4 3 2 1 2 3
++++#+++++		5 4 3 2 1 2 1 0 1 2 3 4 3 2 3 4 3 2 3 4
++++#+++++		6 5 4 3 2 3 2 1 0 1 2 3 2 1 2 3 4 3 4 5
++++#+++++		7 6 5 4 3 4 3 2 1 2 3 2 1 0 1 2 3 4 5 5
++++#+++++		6 5 4 3 4 5 4 3 2 3 3 2 1 0 1 2 3 4 5 4
++++#+++++		5 4 3 2 3 4 5 4 3 4 4 3 2 1 1 1 2 3 4 3
++++#+++++		4 3 2 1 2 3 4 5 4 5 4 3 2 1 0 0 1 2 3 2
+++#++++++		3 2 1 0 1 2 3 4 5 5 4 3 2 1 1 1 2 3 2 1
++++#+++++		4 3 2 1 2 3 4 5 5 4 3 2 1 0 0 0 1 2 1 0
++++#+++++		4 3 2 2 3 4 5 5 4 3 2 1 0 1 1 1 0 1 1 0
++++#+++++		3 2 1 2 3 4 5 4 3 2 1 1 1 2 2 2 1 2 2 1
+++#++++++		2 1 0 1 2 3 4 3 2 1 0 0 1 2 2 1 0 1 2 2
++++#+++++		3 2 1 2 3 4 5 4 3 2 1 1 2 3 3 2 1 2 3 3

指標與陣列 Pointers and Arrays

我們在上一個單元已經簡短地介紹過指標變數，我們學到了使用指標變數來記錄某個記憶體位址。指標和陣列有很密切的關連，我們等一下就會看到，陣列表示法其實只是使用指標的方式之一。陣列的名稱，同時也可以代表陣列的第一個元素的位址，底下的程式碼可以用來測試是否確實如此：

```
int a[10];
printf("%p %p %d\n", a, &a[0], a==&a[0]);
```

其中 `%p` 格式專門用來顯示指標變數所儲存的值（代表某個位址）。

[範例 E10_07.c]

```
#include<stdio.h>
#define SIZE 4
int main(void)
{
    char cdata[SIZE] = {'A', 'B', 'C', 'D'};
    char * ptc;
    int idata[SIZE] = {1, 2, 3, 4};
    int * pti;
    int k;

    ptc = cdata;
    pti = idata;

    printf("%20s %12s\n", "char", "int");
    for (k = 0; k < SIZE; k++)
        printf("pointers + %d: %10p %c %10p %d\n",
            k, ptc+k, *(ptc+k), pti+k, *(pti+k));

    return 0;
}
```

輸出：

	char	int
pointers + 0:	0022FF6C A	0022FF50 1
pointers + 1:	0022FF6D B	0022FF54 2
pointers + 2:	0022FF6E C	0022FF58 3
pointers + 3:	0022FF6F D	0022FF5C 4

從這個範例的輸出結果，我們發現如果把指標指向陣列的起始位址，例如 `ptc = cdata;` 則使用 `*(ptc+k)` 的效果和使用 `cdata[1]` 是一樣的，所以 `*(ptc+1)` 就是 `cdata[1]`。使用 `*` 符號的效果是做所謂 dereferencing 的動作，也就是把指標指到的位址裡面存放的值取出，所以 `*(ptc+1)` 就是把 `ptc+1` 這個位址裡存放的值（字元 'B'）取出。另外我們發現從 `ptc` 變成 `ptc+1` 和從 `pti` 變成 `pti+1` 的記憶體位址變化量不一樣，對 `char` 指標來說，`ptc+1` 會讓記憶體位址多 1 (byte)，但是對 `int` 指標來說 `pti+1` 則是讓記憶體位址移動了 4 (bytes)。

所以現在我們知道 `pti+3` 和 `&pti[3]` 是同樣的記憶體位址，而 `*(pti+3)` 和 `pti[3]` 都是取得同一個位址裡存放的值。我們用下一個範例來進一步測試各種指標和陣列通用的情形。

第十章：Arrays and Pointers

[範例 E10_08.c]

```

#include<stdio.h>
#define SIZE 4
int main(void)
{
    int idata[SIZE] = {1, 2, 3, 4};
    int *pti;
    int k;

    pti = idata; /* pti = &idata[0]; */

    for (k = 0; k < 4; k++) {
        printf("%d ", idata[k]);
        printf("%d ", *(pti + k));

        printf("%d ", *(idata + k));
        printf("%d ", *(&idata[0] + k));
        printf("%d ", pti[k]);

        printf("%d ", k[pti]);
        printf("%d\n", k[idata]);
    }

    return 0;
}

```

輸出：

1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	4	4	4	4

從範例的輸出結果可以看出上面這些寫法都是相通的。首先我們讓整數指標 **pti** 指向陣列 **idata** 的開頭，然後用各種方式取得陣列第 **k** 個元素的值，比較奇怪的是最後兩個寫法。由於陣列元素的寫法 **idata[k]** 是被定義成和 ***(idata + k)** 同義，而 **k[idata]** 也會被看成是 ***(k + idata)**，因此 **idata[k]** 和 **k[idata]** 的效果是一樣的。同理 **pti[k]** 和 **k[pti]** 也可以用相同道理來解釋。看了這麼多種寫法只是為了更了解陣列和指標的概念，實際使用的時候只要熟練一兩種固定的用法就夠了。

Functions, Arrays, and Pointers

假設我們想把陣列當作參數傳給某個 function，然後在 function 中計算陣列元素的總和，如果主程式裡的陣列叫做

```
int a[10] = {1, 1, 2, 3, 5, 8, 13, 21, 34, 55};
```

我們希望在主程式裡使用下面的方式呼叫 function 取得陣列總和，再把結果儲存在變數 **total** 裡面。

```
total = sum(a);
```

所以 **sum()** 這個 function 應該宣告成什麼樣子呢？由於我們知道陣列的名稱可以用來代表整個陣列的起始位址，也就是第一個元素的位址，而陣列 **a[]** 的每個元素都是 **int**，它的位址應該可以用「指向 **int** 的指標」來儲存或傳遞，所以 function 的 prototype 的合理宣告方式大概是

```
int sum(int *ap);
```

這樣 **sum()** 就可以藉由參數 **ap** (是個指標) 取得陣列的開頭位址，而由於陣列的元素是一個接著一個放置，所以找得到第一個就等於可以接著找到後面的元素。再來想想如何定義 function 的內容。由於我們想要計算陣列的元素的總和，所以 **sum()** 裡面應該會用到一個迴圈來把元素通通加起來，但是迴圈除了要知道從哪裡開始，還要知道什麼時候該結束，而我們只把陣列的開頭傳給 **sum()**，並沒有告訴 **sum()** 陣列有多大。因此，function 的 prototype 宣告應該更改一下，變成 **int sum(int *ap, int n);** 而 function 的內容就定義成

```

int sum(int * ap, int n)
{
    int i, total = 0;

    for (i = 0; i < n; i++)
        total += ap[i];      /* total = total + *(ap + i); */

    return total;
}

```

第十章：Arrays and Pointers

在參數宣告的地方，還有一種寫法是

```
int sum(int a[], int n);
```

意思和 `int sum(int *ap, int n);` 是一樣的，都是把一個用來儲存 `int` 資料的位址傳給 `sum()`。但是要特別注意，只有當宣告 function 的參數的時候，才可以用 `int a[]` 這種寫法來產生指標變數。(這種寫法的好處是可以清楚知道參數傳的位址是陣列的開頭位址。) 另外由於 function 的 prototype 可以省略變數名稱，所以底下四種宣告方式是一樣的

```
int sum(int * ap, int n);
int sum(int *, int);
int sum(int a[], int n);
int sum(int [], int);
```

不過如果是主體的 definition 就要把參數名稱寫出來，例如前面寫過

```
int sum(int * ap, int n)
{
    ...
}
```

底下是完整的範例和結果。

[範例 E10_09.c]

```
#include<stdio.h>
#define SIZE 10
int sum(int [], int);
int main(void)
{
    int arr[SIZE] = {1, 1, 2, 3, 5, 8, 13, 21, 34, 55};
    int ans;

    ans = sum(arr, SIZE);
    printf("The sum of the series is %d.\n", ans);
    printf("In main(): The size of arr is %d bytes.\n", sizeof arr);

    return 0;
}

int sum(int a[], int n)
{
    int i, total = 0;

    for (i = 0; i < n; i++)
        total += a[i];

    printf("In sum(): The size of a is %d bytes.\n", sizeof a);

    return total;
}
```

輸出：

```
In sum(): The size of a is 4 bytes.
The sum of the series is 143.
In main(): The size of arr is 40 bytes.
```

執行的結果有幾個地方需要注意。在 `sum()` 裡面看到的 `a` 的大小只有 4 bytes，這是因為對 `sum()` 來說，它只知道 `a` 是個指向整數的指標變數，對於 32 位元的系統來說，指標變數的大小是 4 bytes。在 `main()` 裡面 `arr` 被宣告成陣列，所以 `sizeof` 知道整個陣列的大小，所以得到總共 40 bytes 的大小，其中每個元素都是 `int` 所以是 4 bytes，總共有十個元素所以乘起來就是 40 bytes。

上面的範例裡為了要讓 `sum()` 知道陣列的開頭和陣列的長度，所以要傳兩個參數進去，但是這並不是讓 function 知道陣列的開頭和結尾的唯一方法。

下一個範例是另一種作法，用到的概念是傳入兩個指標變數當參數，其中一個儲存的是陣列的開頭位址，另一個用來儲存陣列的結尾位址。由於指標變數雖然儲存的是位址，但是畢竟也是一個變數，所以它的值可以改變，可以對它加減，也可以拿來做比較。所以只要把原本儲存了起始位址的指標變數的值慢慢增加，當指標變數所儲存的位址等於結尾位址的時候就知道該停止了。

第十章：Arrays and Pointers

[範例 E10_10.c]

```
#include<stdio.h>
#define SIZE 10
int sum_p(int *, int *);
int main(void)
{
    int arr[SIZE] = {1, 1, 2, 3, 5, 8, 13, 21, 34, 55};
    int ans;

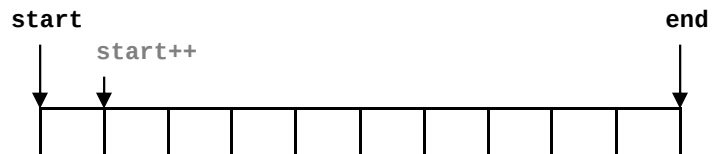
    ans = sum_p(arr, arr + SIZE);
    printf("The sum of the series is %d.\n", ans);

    return 0;
}

int sum_p(int *start, int *end)
{
    int total = 0;

    while (start < end) {
        total += *start;
        start++;
    }

    return total;
}
```



需要傳入的兩個參數都是指向整數的指標變數，在 `main()` 裡面呼叫 `sum_p()` 分別傳入陣列起始位址 `arr` 和陣列的結尾位置 `arr + SIZE` (最後一個元素的下一個整數的位址)。在 `sum_p()` 裡面靠 `start` 和 `end` 這兩個指向整數的指標變數來接收陣列的起始和結束位址，然後開始靠迴圈來把陣列的元素一個一個取出來。迴圈的進行條件是 `while (start < end)`，也就是比較兩個指標變數裡面儲存的位址的大小。我們會不斷移動 `start` 指的位址，如果 `(start < end)` 這個條件成立，表示 `start` 還沒到達陣列的結尾位址，所以在迴圈裡就可以用 `*start` 取出指標指的記憶體位址裡所儲存的整數值，然後累加到 `total` 裡，接著再做 `start++` 把指標移到下一個位址，對於整數指標來說就是把指標的值加 4，因為一個整數會佔用四個 bytes。在 `sum_p()` 的迴圈裡還有一個小地方可以稍微注意一下，如果把兩個 statements 合併變成

```
total += *start++;
```

這樣寫的意思也等於是

```
total = total + *(start++);
```

不過如果為了避免混淆，還是最規矩的寫法比較好。

雖然我們已經看過陣列和指標的互通性，例如不論 `a` 是陣列的名稱或是 `a` 是個指標變數，`a[i]` 和 `*(a+i)` 的意思都是一樣的。但是對於累加的運算 `a++` 來說，只有指標變數才可以這樣寫，陣列的名稱並不是變數，所以不可以做 `a++` 的動作。

我們再用下一個範例把指標的各種運算和使用方式回顧一次，包含指標如何取得某個記憶體位址、指標變數值的增減 (相當於改變指到的位址)、以及如何取出指標所指到的位址裡存放的值 (dereferencing a pointer)。

[範例 E10_11.c]

```
#include <stdio.h>
void show_int_array(int [], int);
int main(void)
{
    int arr[4] = {100, 200, 300, 400};
    int *ptr1, *ptr2;
    ptr1 = arr;
    ptr2 = &arr[3];
```

第十章：Arrays and Pointers

[範例 E10_11.c] 接上一頁

```

printf("&ptr1:%p, ptr1:%p, *ptr1:%d\n", &ptr1, ptr1, *ptr1);
printf("&ptr2:%p, ptr2:%p, *ptr2:%d\n\n", &ptr2, ptr2, *ptr2);

show_int_array(arr, 4);

ptr1++; *ptr1 = 201;
ptr2--; *ptr2 = 301;
printf("\nAfter ptr1++, ptr2--, ");
printf("*ptr1 = 201, *ptr2 = 301\n");
printf("&ptr1:%p, ptr1:%p, *ptr1:%d\n",
      &ptr1, ptr1, *ptr1);
printf("&ptr2:%p, ptr2:%p, *ptr2:%d\n\n",
      &ptr2, ptr2, *ptr2);
show_int_array(arr, 4);

return 0;
}
void show_int_array(int a[], int n)
{
    int *ip;
    for (ip = a; ip < (a+n); ip++) {
        printf("| %p ", ip);
    }
    printf(" |\n");
    for (ip = a; ip < (a+n); ip++) {
        printf("| %8d ", *ip);
    }
    printf(" |\n");
}

```

輸出：

```

&ptr1:0022FF5C, ptr1:0022FF60, *ptr1:100
&ptr2:0022FF58, ptr2:0022FF6C, *ptr2:400

| 0022FF60 | 0022FF64 | 0022FF68 | 0022FF6C |
|      100 |      200 |      300 |      400 |

After ptr1++, ptr2--, *ptr1 = 201, *ptr2 = 301
&ptr1:0022FF5C, ptr1:0022FF64, *ptr1:201
&ptr2:0022FF58, ptr2:0022FF68, *ptr2:301

| 0022FF60 | 0022FF64 | 0022FF68 | 0022FF6C |
|      100 |      201 |      301 |      400 |

```

有一個必須牢記在心的重點：**未做初始化的指標，絕對不能對牠做 dereferencing 的動作**。這一個重點很有可能一不小心會沒有注意到。例如底下的程式碼，先想想看會有什麼問題：

```

int *pt;
*pt = 5;

```

乍看之下似乎沒什麼問題，但是我們想要把 **pt** 所指到的位址裡面放入整數 **5**，那麼 **pt** 目前指到的位址在哪裡呢？答案是不確定。當我們宣告一個指標變數的時候，我們只會得到一個可以用來記錄記憶體位址的變數，至於牠所記錄的記憶體位址究竟在哪裡，以及那個位址是否可以被使用，都必須等到我們做了額外的動作之後才能確定。總之，一定要牢記並遵守指標初始化的原則，對於未初始化的指標，絕對不要對牠使用 ***** 符號來修改記憶體內容。

[**練習 W10_06] 分析底下的程式所做的事情，把程式的輸出結果寫出來。

```

#include <stdio.h>

int findmin(int a[], int left, int right);

int main(void)
{
    int array[] = { 3, 5, 2, 6, 7, 12, 10 };
    int min;

    /* find the smallest value of integers stored in array */
    min = findmin(array, 0, sizeof(array)/sizeof(int) -1);
    printf("The smallest element is %d.\n", min);

    return 0;
}

```

第十章：Arrays and Pointers

[**練習 W10_06] 接前一頁。

```
int findmin(int a[], int left, int right)
{
    int middle;
    int leftmin, rightmin;

    if (left == right)
        return a[left];

    middle = left + (right-left) / 2;
    leftmin = findmin(a, left, middle);
    rightmin = findmin(a, middle+1, right);

    printf("%d, %d\n", leftmin, rightmin);

    if (leftmin < rightmin)
        return leftmin;
    else
        return rightmin;
}
```

[**練習 W10_07] 分析程式碼，並且寫出執行後會得到什麼樣的輸出結果。

```
#include <stdio.h>
void perm(char a[], int n);
void swap(char *, char *);
int main(void)
{
    char a[] = "cat";
    perm(a, 3);
    return 0;
}
void perm(char a[], int n)
{
    int i;
    if (n == 1) {
        printf("%s\n", a);
    }
    else {
        for (i = 0; i < n; i++) {
            swap(&a[i], &a[n-1]);
            perm(a, n-1);
            swap(&a[i], &a[n-1]);
        }
    }
}
void swap(char *x, char *y)
{
    char tmp;

    tmp = *x;
    *x = *y;
    *y = tmp;
}
```

指標和二維陣列

關於二維陣列，我們可以再結合已經學到的指標概念，對它做更進一步的運用。假設我們宣告了一個二維陣列：

```
int z[4][2] = {{1,2}, {3,4}, {5,6}, {7,8}};
```

則 **z** 本身是一個陣列，總共有四個元素，而其中的每個元素本身也都是一個「包含了兩個元素的陣列」。由於 **z** 可以代表陣列第一個元素的位址，所以 **z** 的值和 **&z[0]** 是一樣的。而又由於 **z[0]** 本身也是個陣列（包含兩個整數），所以 **z[0]** 同時也代表著它的第一個元素的位址，也就是 **&z[0][0]**。對於二維陣列來說，**z** 所代表的位址，和 **z[0]** 代表的位址是一樣的，都是整個二維陣列最前面的那個整數元素的位址。

第十章：Arrays and Pointers

雖然 `z` 所代表的位址和 `z[0]` 代表的位址一樣，但是由於 `z` 和 `z[0]` 是不同型別的指標，所以對他們做位址加一的動作，不論是在意義上或是造成的效果，都不相同。如果計算 `z+1`，在這個例子裡得到的位址變化是增加八個 bytes，也就是兩個整數的大小。如果計算 `z[0]+1`，則位址變化則只會增加 4 bytes。因為意義上 `z+1` 相當於是 `&z[1]`，而 `z[0]+1` 則相當於 `&z[0][1]`。

接下來看看關於取出記憶體位址裡儲存的值的寫法。首先由於 `z[0]` 是 `z[0][0]` 這個元素的位址（`z[0]` 和 `&z[0][0]` 是一樣的），所以 `*(z[0])` 等於是 `*&z[0][0]` 也就等於是 `z[0][0]` 這個元素的值。而又由於 `z` 代表的位址等於 `&z[0]`，所以 `*z` 就等於是 `z[0]`，再套用前面的結果就得到 `*(z)` 等於是 `*(z[0])` 等於是 `*&z[0][0]`，所以也就是 `z[0][0]`。

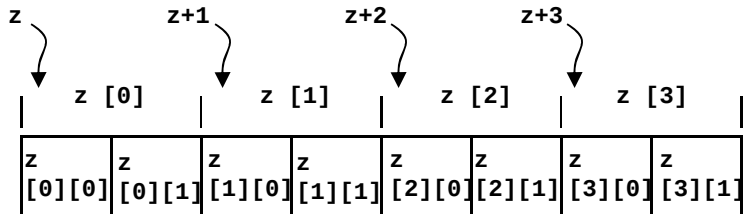
以上所有的動作可以用下面的範例來測試一次。

[範例 E10_12.c]

```
#include <stdio.h>
int main(void)
{
    int z[4][2] = {{1,2}, {3,4}, {5,6}, {7,8}};

    printf("    z: %p\n", z);
    printf("    &z[0]: %p\n", &z[0]);
    printf("    z[0]: %p\n", z[0]);
    printf("&z[0][0]: %p\n", &z[0][0]);
    printf("z[0][0]: %d\n", z[0][0]);
    printf("\n");
    printf("    z+1: %p\n", z+1);
    printf("    &z[1]: %p\n", &z[1]);
    printf("    z[0]+1: %p\n", z[0]+1);
    printf("&z[0][1]: %p\n", &z[0][1]);
    printf("\n");
    printf("    *z: %p\n", *z);
    printf("    z[0]: %p\n", z[0]);
    printf("    *z[0]: %d\n", *z[0]);
    printf("z[0][0]: %d\n", z[0][0]);
    printf("    **z: %d\n", **z);

    return 0;
}
```



輸出：

```
z: 0022FF50
&z[0]: 0022FF50
z[0]: 0022FF50
&z[0][0]: 0022FF50
z[0][0]: 1

z+1: 0022FF58
&z[1]: 0022FF58
z[0]+1: 0022FF54
&z[0][1]: 0022FF54

*z: 0022FF50
z[0]: 0022FF50
*z[0]: 1
z[0][0]: 1
**z: 1
```

接著就來看看如何宣告一個可以指向二維陣列的指標。用上一個程式為例，`z` 是一個二維陣列，也代表一個「包含兩個整數元素的陣列」的位址，所以我們可以宣告一個指向「包含兩個整數元素的陣列」的指標，用來儲存這個陣列的位址，宣告的方式是

```
int (*pz)[2];
```

這樣宣告的意思是說，`pz` 是一個指到整數陣列的指標，而牠所指到的陣列包含了兩個整數元素。要注意上面的括號一定要加，因為如果不加的話，代表的是另一種意義：

```
int *py[3];
```

這種宣告方式是表示 `py` 是一個「有三個元素的陣列」，其中每個元素都是「指向整數的指標」，或者是說，每個元素都記錄著某個記憶體位址。有了對應到二維陣列的指標之後，我們就可以把這個指標指向二維陣列，例如

```
pz = z;
```

然後就可以把 `pz` 這個指標當作二維陣列來使用，所以 `pz[2][1]` 就相當於是 `z[2][1]`。

[練習 W10_08] 根據變數的宣告來判斷下列動作是否合法。

```
pt = &a[0][0];
pt = a[0];
pa = a;
pa = b;
p2 = &pt;
*p2 = b[0];
p2 = b;
```

宣告：

```
int *pt;
int (*pa)[3];
int a[2][3];
int b[3][2];
int **p2;
```

第十章：Arrays and Pointers

如何傳二維陣列給 function

[範例 E10_13.c]

```

#include <stdio.h>
#define ROWS 3
#define COLS 4
int sum2d(int (*ap)[COLS], int rows);
/* int sum2d(int[][COLS], int); */
/* int sum2d(int a[][COLS], int rows); */
int main(void)
{
    int arr2[ROWS][COLS] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};

    printf("Sum = %d\n", sum2d(arr2, ROWS));

    return 0;
}
int sum2d(int a[][COLS], int rows)
{
    int r, c;
    int total = 0;

    for (r = 0; r < rows; r++)
        for (c = 0; c < COLS; c++)
            total += a[r][c];

    return total;
}

```

要把二維陣列傳給某個 function，必須要使用上面的範例裡的 function 宣告方式。由於一個二維陣列可以解讀成「由一維陣列組成的一維陣列」，但是傳遞參數時，我們能夠做的事情只是把位址 (某個大小為 4 bytes 的值) 傳給 functions，所以傳遞二維陣列其實應該是要傳遞位址給某個指標變數。那麼這個接收二維陣列的指標變數該宣告成什麼樣子呢？依照我們前面討論的指標型別，我們知道 `int (*ap)[COLS]` 這樣的指標，和 `int arr2[ROWS][COLS]` 二維陣列具有同樣的型別，都是一個包含 COLS 個元素的陣列的起始位址。所以我們把 function 宣告成 `int sum2d(int (*ap)[COLS], int rows)`；而呼叫的時候就把二維陣列的名稱傳進去就可以，例如 `sum2d(arr2, ROWS)`。要特別注意的地方是 [COLS] 這個東西一定要寫，因為這樣 compiler 才知道指標指到的是一個「包含了 COLS 個元素的整數陣列」。假如兩個指標 `pa` 和 `pb` 指到的陣列的大小不一樣，則這兩個指標的型別就不一樣，因為 `pa+1` 和 `pb+1` 的效果是不同的。

指標陣列

[範例 E10_14.c]

```

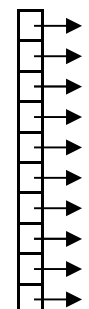
#include <stdio.h>
int main(void)
{
    int a[50], *aPtr[10];
    int i, j;

    for (i = 0; i < 50; i++) a[i] = i;
    for (i = 0; i < 10; i++) aPtr[i] = &a[i*5];

    for (i = 0; i < 10; i++) {
        for (j = 0; j < 5; j++) {
            printf("%3d ", aPtr[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

輸出：



0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24
25	26	27	28	29
30	31	32	33	34
35	36	37	38	39
40	41	42	43	44
45	46	47	48	49

這個簡短的範例主要是要示範如何用指標陣列 (每個陣列的元素都是指標) 來把一維陣列轉變成二維陣列來使用。我們先產生了一個指標陣列，然後再把其中的每個指標元素指向某個連續的位址。由於 `*(aPtr[0])` 的意義相當於 `aPtr[0][0]`，又或者 `*(aPtr[5]+2)` 的意義相當於 `aPtr[5][2]`，所以我們可以把 `aPtr` 當作二維陣列使用。

第十章：Arrays and Pointers

動態取得記憶體：使用 `malloc()` 和 `free()`

在 `stdlib.h` 裡面提供了兩個常用來管理記憶體的 functions，叫做 `malloc()` 和 `free()`，我們可以靠它們來取得和釋放記憶體。由於這樣的記憶體管理方式可以在程式正在執行的時候運作，所以稱作**動態記憶體管理**。底下的範例是一個簡單的用法，用 `malloc()` 產生一個動態的陣列，使用完畢要呼叫 `free()` 把記憶體釋放回去。

[範例 E10_15.c]

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    double *ptd;
    int array_size, i;
    printf("How many doubles do you want? ");
    scanf("%d", &array_size);

    ptd = (double *) malloc(array_size * sizeof (double));
    if (ptd == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }
    for (i = 0; i < array_size; i++) {
        ptd[i] = (double) rand() / RAND_MAX;
    }
    for (i = 0; i < array_size; i++) {
        if (*(ptd+i) > 0.5)
            printf("%d: %lf\n", i, ptd[i]);
    }
    free(ptd);
    return 0;
}
```

輸出：

```
How many doubles do you want? 10
1: 0.563585
3: 0.808741
4: 0.585009
7: 0.895962
8: 0.822840
9: 0.746605
```

傳給 `malloc()` 的參數是我們想要取得的記憶體大小，以 byte 為單位，所以如果我們想要一個「包含十個 `double` 元素的陣列」，我們要傳 `10 * sizeof(double)` 給 `malloc()`。由於 `malloc()` 傳回來的是指向 `void` 型別的指標，所以我們必須依照想要的型別，對指標做強制轉換：`ptd = (double*)malloc(array_size* sizeof(double));`，才能把符合型別規定的指標設定給 `ptd`。如果 `malloc()` 呼叫失敗，沒辦法取得記憶體，則會傳回 `NULL`，所以如果指標等於 `NULL`，程式就不能再進行下去，直接用 `exit(EXIT_FAILURE)`；來結束程式並傳回錯誤訊息給作業系統。同樣地，`exit()` 這個 function 和 `EXIT_FAILURE` 常數也都是 `stdlib.h` 提供的。取得了記憶體之後，我們就可以透過 `ptd` 這個指標來存取那塊記憶體，不論是用 `*(ptd + i)` 這種指標取值的方式，或是用陣列的方式 `ptd[i]` 都可以。最後當取得的記憶體不需要再使用，就用 `free(ptd)`；把它釋放。(附註：程式如果不斷取得記憶體，但沒有正確地用 `free()` 釋放不再用到的空間，一旦原本用來記錄位址的指標變數的值被改變，例如指到了別的地方，則原本那塊記憶體就沒有人能找得到，也就沒人能再使用它，這會造成所謂的 memory leak 的問題。)

[練習 W10_09] 解釋下面的程式碼，並描述會得到什麼樣的輸出結果。

```
#include <stdio.h>
int main(void)
{
    int *a, **b, i, j, rows, cols;
    scanf("%d %d", &rows, &cols);

    a = (int *) malloc(rows * cols * sizeof(int));
    for (i = 0; i < rows*cols; i++) a[i] = i;
    b = (int **) malloc(rows * sizeof(int *));
    for (i = 0; i < rows; i++) b[i] = &a[i*cols];
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            printf("%3d ", b[i][j]);
        }
        printf("\n");
    }
    free(b);
    free(a);
    return 0;
}
```

輸出：

10	5			
0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24
25	26	27	28	29
30	31	32	33	34
35	36	37	38	39
40	41	42	43	44
45	46	47	48	49

第十章：Arrays and Pointers

到這裡為止，關於 C 語言，我們已經學了不少，但是也有很多東西我們還沒教。不過不用太擔心，大家已經具備了該有的基礎知識，對於沒有教的技術，其實都已有能力能夠很快地學會。底下整理出其中比較重要的主題，希望大家有空可以去了解一下，進一步提升使用 C 語言的熟練度。

1. C Standard Library 提供許多有用的 functions，讓我們免去了凡事都自己來的麻煩，有些功能直接用別人寫好的現成 functions，可以替我們節省很多時間。大家可以去翻翻使用手冊 (課程網頁上有放 The GNU C Library Reference Manual 的 PDF 檔的連結)，大致瀏覽一遍建立一些粗略的印象。其實這些 functions 學也學不完，也沒必要全部都了解，只有當你需要使用到某些 functions 時，再去查它們的使用方法就夠了；要用的時候再去查它需要傳入什麼樣的參數，以及會傳回什麼值。底下列舉一些常用的 functions，大家可以去查詢它們的功能。

string.h: `strcat()`, `strcmp()`, `strcpy()`, `strtok()`, `memcpy()`, `memset()`, ...
stdio.h: `fopen()`, `fclose()`, `fflush()`, `fseek()`, `fsanf()`, `fprintf()`, `sprintf()`, `scanf()`, `fread()`, `fwrite()`, ...

如果對 C 語言的語法規則和 C Standard Library 有興趣，可以再去參考底下這本書

S.P. Harbison III and G.L. Steele Jr., *C: A Reference Manual, Fifth Edition*, Prentice Hall, 2002.

這本書雖然不適合拿來學習 C 語言，但是卻是很好的工具書。(有點像是字典，雖然不能直接拿來學習寫作文，但是寫作文有時候需要查字典)。如果已經有了相當的基礎，又想要對瑣碎的細節有所了解，可以去找來讀一讀。

2. Bitwise operators。C 語言提供了一些位元運算符號，讓我們對變數做較低階的處理。將來你們學了組合語言、邏輯設計、以及微算機系統，就會對這些低階運算很熟悉。

3. Storage classes 和 linkage。我們沒有提到 local 變數和 global 變數的差別，也沒有提到 **static** 或 **extern** 變數。

4. 使用 C **struct**、指標、以及動態記憶體管理等等功能，去實作出各種進階的資料結構。這一部份等到將來學到了資料結構 (Data Structures) 這門課，會有很詳細的解釋。要成為真正的程式高手，資料結構和演算法 (Algorithms) 是必定要學會的武器。程式語言本身只是個工具，你可以試圖去學會使用 C、C++、Java、Python、LISP、Perl、Ruby 等等各式各樣的語言，或是學會一些獨特的小技巧，或用這些語言來寫出一些應用程式，但是真正重要的內涵在於你使用的資料結構和演算法。研究某種特定語言，只會讓你成為那個語言的專家；只有當你精通了資料結構和演算法，才會讓你變成程式高手，才能創造出有效率的程式、解決實際的難題。希望未來大家學習這些科目的時候，會花很多心力去把它們學得很好。

上面列出的幾項重要主題，其中一部分的內容可以參考講義後面的附錄。

附錄

什麼是指標？

指標其實也是一個變數，只是比較特別的是它所儲存的值是(另一個變數的)記憶體位址。

和一般的變數比較一下，譬如：`char a;` 和 `int b;` 分別表示 `a` 所儲存的是一個字元而 `b` 所儲存的是一個整數。而 `char *ap;` 和 `int *bp;` 則表示 `ap` 是一個指到字元的指標而 `bp` 是一個指到整數的指標。

這裡出現了星號 `*` operator，就是靠這個符號讓 compiler 知道你需要用到指標變數 `ap` 或 `bp` 來儲存另外一個變數的記憶體位址。

對於一般的變數我們會用 `a = '9';` 或是 `b = 3;` 的敘述來指定 `a` 和 `b` 變數所儲存的值。同樣的我們也可以指定指標變數所儲存的值。但是我們不能像對一般變數一樣隨意指定 `'9'` 或是 `3` 這樣的常數值給指標，因為指標變數的值是記憶體位址，所以指標的值必須是真正存在而且有意義的位址才行。

下面是程式裡面常用的兩種設定指標的值的方式，一個是直接複製，把一個已經儲存了「某個記憶體位址」的指標的值設給另一個指標，譬如 `bp = cp;` (假設 `cp` 已經有值)。另一個是用 `&` 符號來取得一般變數的位址：`bp = &b;`；這樣的動作可以想成是把整數指標 `bp` 指到整數 `b`，其實就是把整數 `b` 的記憶體位址儲存到 `bp` 裡頭。

這裡有一個非常基本的前提就是程式中所定義的一般變數，在程式執行的過程中其實會被存放在記憶體的某個位置，所以程式執行過程中記憶體某個位置會被用來代表前面定義的字元變數 `a` 和整數 `b`，而 `a = '9';` 和 `b = 3;` 就會把 `'9'` 放進代表 `a` 的那個位置，而把 `3` 放進代表 `b` 的那個位置。

一個 `char` 型別的變數佔用的記憶體空間是 1 byte，而一個 `int` 變數通常是 4 bytes，那麼一個指標變數需要佔用多少 bytes？

不論是指到哪一種型別的指標 `char *ap;` 或 `int *bp;` 它們需要的記憶體空間在 32-bit 系統下都只要 4 bytes，因為指標變數所儲存的是另一個變數的記憶體位址，而 32-bit 系統的記憶體位址只需要 4 bytes 就能表示。(變數需要用到多少記憶體來儲存，可以用 `sizeof` 來測試。)

[例外] 唯一可以直接設定給指標的整數值是 `0`，但習慣上通常不寫 `0` 而是使用 `stdio.h` 中所定義的 `NULL` 常數來表示。由於 `0` 這個位址並不會真的被使用到，所以常會用 `NULL` 來判斷某個指標是否指到有用的位址，譬如：一開始先設定 `bp = NULL;` 經過一連串運算後，你的程式可能需要用 `if (bp != NULL) ...` 來判斷 `bp` 是否已經被指定了一個有意義的位址。

[練習 WA_01]

請用圖示方式來表示 `b` 和 `bp` 的關聯。假如 `b` 所儲存的值為 `3`，而 `b` 這個變數的記憶體位址是 `100000`。

如何取得指標所指到的記憶體位址中所儲存的值？如何修改指標所指到的記憶體位址中所儲存的值？

這時候就輪到星號 `*` 上場了。我們可以在指標變數的前面加上星號來將指標變成一般的變數，譬如 `*bp`，不過有一個非常重要的前提：加了 `*` 的指標必須已經指向某個記憶體位址，也就是必須已經被初始化(initialization)。所以下面這樣的寫法是錯的：

```
int *bp;
*bp = 3;
```

要像下面這樣才行

```
int *bp;
int b;
bp = &b;
*bp = 3;
```

因為 `*bp` 相當於把 `bp` 所儲存的記憶體位址轉換成那個位址所儲存的變數。如果 `bp` 是一個指向整數的指標，則 `*bp` 就是 `bp` 所指到的整數。所以 `bp` 當然必須真的有指到某個整數才行，這樣 `*bp` 才有意義。

如果沒有先對指標作初始化就直接想用 `*` 將指標轉成它所指到的變數，由於未初始化的指標根本還不知道指到記憶體的哪個位址，所以用了 `*` 的下場通常就是程式當掉。

[練習 WA_02]

產生一個 `float` 變數 `x` 和一個指向 `float` 的指標 `xp`，先直接把 `x` 的值設為 `3.1`，然後再透過 `xp` 讓 `x` 的值增加 `0.04`。

為什麼需要指標？

在 C 程式裡面，指標的用途實在太廣了。譬如，指標可以讓我們定出 linked lists 和 trees 等等各式各樣有用的資料結構。就目前所學到的範圍來說，我們可以用指標達到 function 呼叫時 call-by-reference 的效果。

為什麼需要 call-by-reference？

又要舉出已經看過的例子：**swap()**

如果要寫一個 function 可以把傳入的兩個整數變數的值交換，該怎麼寫

```
#include <stdio.h>
void swap(double, double);
int main(void)
{
    double x = 5.5, y = 7.7;
    printf("x=%lf, y=%lf\n", x, y);
    swap(x, y); /* x and y will not be swapped */
    printf("x=%lf, y=%lf\n", x, y);

    return 0;
}
/* useless swap */
void swap(double a, double b)
{
    double tmp;
    tmp = a;
    a = b;
    b = tmp;
}
```

上面的寫法當然沒用，因為 **a** 和 **b** 都只屬於 **swap()**，一但程式執行的流程離開了 **swap()**，**a** 和 **b** 就不見了。而且你的主程式呼叫 **swap()** 時只是把 **x** 和 **y** 的值，複製到 **a** 和 **b** 之中。你可以把主程式中的呼叫 **swap(x, y);** 改成 **swap(5, 7);**，在這個例子中這兩種的意思是一樣的，所以可以看出在這種寫法下，**swap()** 所做的事一點意義也沒有。

[練習 WA_03]

試著用 debugger 來 trace 主程式呼叫 **swap()** 的過程，看看局部變數 **a** 和 **b** 什麼時候會存在。

大家應該都已經非常熟悉 **swap()**，知道要改成下面的寫法才對

```
void swap(double *a, double *b)
{
    double tmp;
    tmp = *a;
    *a = *b;
    *b = tmp;
}
```

而呼叫的地方應該改成 **swap(&x, &y);**，因為我們必須把存放 **x** 和 **y** 兩個變數的記憶體位址 **&x** 和 **&y** 告訴 **swap()**，讓 **swap()** 到那兩個位址去找，參數傳進去後相當於做了 **a = &x;** 和 **b = &y;** 的動作。然後用 ***a** 和 ***b** 來設定那兩個記憶體位址中所儲存的值。

[練習 WA_04]

寫一個 function 傳入整數值 **n**，求出 **1+2+...+n** 和 **1*1+2*2+...+n*n** 並把答案分別儲存到主程式中的兩個變數 **m** 和 **s** 中。

指標與陣列的關連？

在 C 程式裏所有可以用陣列方式表示的寫法，都可以改用指標方式來達成。

先來回顧一下陣列。陣列相對於指標來說，算是比較容易理解和使用，因為陣列只是儲存在記憶體中的一串連續元素，譬如 `int a[10];` 就是連續十個整數，而 `a` 是這一串連續整數的代稱，代表的是第一個元素的位址（第一個元素的 index 是 0，C 語言中陣列的編號由 0 開始）。陣列的使用很簡單，`a[0]` 代表第一個元素，`a[9]` 代表第十個元素，所以我們可以把 `a[0]` 或 `a[9]` 都當作是一般的整數來使用，只不過比較特別的是它們在記憶體中的存放位置是連續的。

既然陣列的元素在記憶體中的存放位置是連續的，那麼如果我們知道第一個元素的位址，不就可以透過指標來找到其他的元素了嗎？想通了這個道理，你的程式就可以藉由指標，更靈活地存取陣列元素。

如何將一個指標指到陣列的第一個元素？方法就和取得一般變數的位址一樣（譬如 `bp = &b;`）：

```
int *pa;
pa = &a[0];
```

另一種寫法是 `pa = a;` 因為就像前面說的，`a` 這個名稱本身所代表的意義是這個陣列的起始位址。

將指標指到了陣列的開頭位址，再來就要看看如何存取其他元素。在這之前，先再回顧一下指標的位址運算原則。

`(pa+1)` 所代表的記憶體位址和 `pa` 差多少呢？

這和 `pa` 這個指標所指到的變數型別有關，由於 `int *pa;` 而整數需要 4 bytes 的記憶體來儲存，所以 `(pa+1)` 和 `pa` 差 4 bytes。

[練習 WA_05]

假如是 `char *pa;` 則 `(pa+1)` 和 `pa` 差多少？

由此衍伸出來，我們得到四種存取陣列元素的寫法，

```
int a[10], *pa;
pa = &a[0];
*pa = 1;
pa[1] = 2;
a[2] = 3;
*(a + 3) = 4;
```

其中第四種用法要稍微注意一下，陣列名稱 `a` 代表陣列開頭元素的位址，但是它並不是一個指標變數，程式 compile 之後它所代表的相對位址就會確定，譬如在 1000 這個位址，而 `&a[2]` 就變成 1008，而 `(a + 3)` 就變成 1012。所以 `a++` 這樣的動作對陣列來說是不行的；雖然陣列的名稱在大多數情況下用法和指標沒兩樣，但是它終究只是那一連串陣列元素的代稱（代表著開頭位址），而不是一個指標變數，所以不能去改變它的值。（實際上，compiler 看到 `a[2]` 這樣的寫法時，會把它翻譯成 `*(a+2*sizeof(int))`，然後再把 `a` 用位址取代。）

除此此外，在 function 的參數宣告中 `void f(int ary[])` 和 `void f(int *ary)` 陣列與指標這兩種不同寫法是完全相通的。（可能會讓人覺得比較奇怪的是對於 `int ary[]` 這種用法，在 `f()` 裡頭可以使用 `ary++`，因為對 function 來說，參數 `ary` 其實是個存放在堆疊記憶體中的變數，不過這已經有些離題了。）

同樣的，呼叫時傳遞位址也可以用兩種方式

```
f(a);
f(&a[0]);
```

甚至如果有需要，不一定要傳陣列的開頭位址，譬如

```
f(a + 2);
f(&a[2]);
```

[練習 WA_06]

寫個小程序測試一下，如果 `pa = &a[2];` 那麼 `pa[-1]` 或 `*(pa-2)` 這樣的用法可以用嗎？

[練習 WA_07]

寫一個 function `f()`，傳入兩個各自已經由小到大排序好的正整數陣列 `a` 和 `b`，以及一個比 `a` 和 `b` 合起來還長的空陣列 `c`，（假設 `a` 和 `b` 的最後一個元素都是 -1 代表陣列結束），`f()` 要將 `a` 和 `b` 合併並且從大到小排好儲存在 `c` 裡頭。

```
void f(int a[], int b[], int c[]);
```

字元陣列與字串

關於字元陣列其實只要把前面回顧的整數陣列的 `int` 改成 `char`。而一個字串則必須在結尾的地方多加入一個 `'\0'` 字元，表示字串到此結束。其餘的相關用法其實都只是指標和陣列的用法。

[練習 WA_08]

說明並圖示下面兩種寫法的差異：

```
char str1[] = "piece of cake";
```

```
char *str2 = "piece of cake"; /* 附註：ANSI C 要求 compiler 可以支援到長度 509 的字串 */
```

[練習 WA_09]

假設在 `main()` 裡面有下列三行，請問 `str1`、`str2`、`str3` 的內容會是什麼？

```
char str1[100];
```

```
char str2[100] = {'a'};
```

```
char str3[100] = "";
```

[練習 WA_10]

寫一個 function 傳入一個字串，傳回該字串的長度 (整數值)。

[練習 WA_11]

寫一個 function `f()`，傳入一個字串，判斷是否為 palindrome (像是 "level", "wasitacatisaw")。

指標陣列 Pointer Arrays (Pointers to Pointers)

陣列的元素除了可以是整數、浮點數、字元之外，當然也可以是指標。所以我們可以產生一個指標陣列，其中每個元素儲存了某個位址。

[練習 WA_12]

用圖示表現出下面的字串

```
char *ptrarray[] = {"piece", "of", "cake"};
```

二維陣列 Two-Dimensional Arrays

[練習 WA_13]

圖示下面兩種寫法的差別

```
char *str1[] = {"piece", "of", "cake"};
```

```
char str2[][8] = {"piece", "of", "cake"};
```

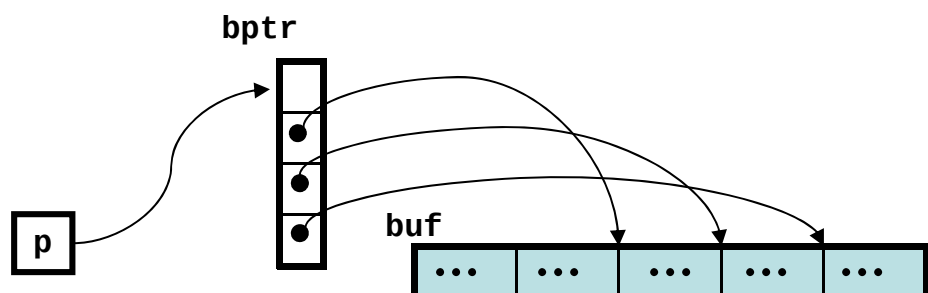
```
#include <stdio.h>
int main(void)
{
    char *str1[] = {"piece", "of", "cake"};
    char str2[][8] = {"piece", "of", "cake"};
    int i, j;
    for (i=0; i<3; i++) {
        for (j=0; j<8; j++) {
            printf("%c", str1[i][j]);
            printf("\n");
        }
    }
    for (i=0; i<3; i++) {
        for (j=0; j<8; j++) {
            printf("%c", str2[i][j]);
            printf("\n");
        }
    }
    return 0;
}
```

[練習 WA_14]

二維陣列使用起來很方便，譬如用來儲存矩陣或影像。但是在 C 程式中我們必須預先給定陣列大小，譬如 `int a[10][20]`。這個練習是要藉由指標陣列來模擬動態產生二維陣列的機制。

假設程式最前面已經定了兩個 global 陣列：`char buf[1000000]`；是一個字元陣列。`char *bptr[1000]`；是一個字元指標的陣列。

試著寫出一個 function 叫做 `char **mtx(int m, int n)`；傳入的參數 `m` 和 `n` 代表我們想要產生的二維陣列的大小，回傳值則是一個指向指標的指標。`mtx()` 要替我們設定 `bptr` 的指標內容（指到 `buf`），然後傳回一個指標，讓我們能用例如 `p = mtx(5, 10)`；的方式取得一個二維陣列，然後就可以在程式把 `p` (type 是 `char **`) 當作二維陣列來使用，譬如 `p[2][5] = 7`。在 `mtx()` 裡面應該會包含兩個 `static` 變數，用來記住已經被使用掉的 `buf` 和 `bptr` 有多少。如果 `mtx()` 發現沒有足夠的空間產生二維陣列，就把回傳值設為 `NULL`。



什麼是 C Functions ?

程式是一長串運算的組合，這些運算可以被區分拆解成一些主要步驟或是性質類似的任務，將這些步驟獨立出來就形成 functions。

使用 functions 有不少好處：

1. 程式變得比較清晰易懂，也方便除錯，可以單獨測試每個 function 是否做出正確的運算；
2. 可重複使用自己曾經寫過或別人已經寫好的 functions，這樣能節省許多程式開發的時間。

譬如你的程式裡面需要計算 $a[0]+a[1]+\dots+a[9]$ ， $a[10]+a[11]+\dots+a[19]$ ，和 $b[0]+b[1]+\dots+b[9]$

```
int i, sum1, sum2, sum3;
for ( i=0, sum1=0; i<10; i++ ) {
    sum1 += a[i];
}
for ( i=10, sum2=0; i<20; i++ ) {
    sum2 += a[i];
}
for ( i=0, sum3=0; i<10; i++ ) {
    sum3 += b[i];
}
```

你會發現兩個迴圈其實非常相似，所以可以寫一個 function 來處理這兩個迴圈要做的事情，然後靠傳入不同的參數來改變運算的內容：

```
int SUM(int array[], int length)
{
    int i, answer;
    for (i=0, answer=0; i<length; answer += array[i++]);
    return answer;
}
```

所以原來的程式碼就變成

```
int sum1, sum2, sum3;
sum1 = SUM(&a[0], 10);
sum2 = SUM(&a[10], 10);
sum3 = SUM(&b[0], 10);
```

每一個步驟做的事情就變得很清楚，而且只要你確定 **SUM()** 的程式是正確的，剩下的就只要注意該傳什麼參數，而不需要再擔心計算過程會出問題。

另外一種常見的情況是使用現成的 libraries 所包含 functions，譬如 C Standard Library 就包含一些常用數學運算或是字串處理所需的 functions，使用時只要在程式最前面用

```
#include <math.h>
#include <string.h>
```

把現成的 functions 宣告引入，就能在接下來的程式碼中使用 **sqrt(x)**，**strcmp(s1, s2)** 等等函式。

另外，自己寫的 functions（像是前面的 **SUM()**），同樣也要在程式最前面加入 function prototype 的宣告

```
int SUM(int a[], int len);
```

讓你的 compiler 先知道有這樣的 function 存在。

在撰寫一個 function 時，除了 function 本身要做的事情之外，還有兩個重點需要考慮，分別是要傳入的參數是什麼，以及要回傳的是什麼？以前面的 **SUM()** 為例，參數有兩個：**int array[]** 和 **int length** 回傳值的型別則是 **int**。

假如 function 沒有回傳值，就要宣告成 **void**，例如 **void f(int a);**，假如連參數也沒有，就變成

```
void f(void);
```

[練習 WB_01]

配合陣列和字串來複習一下。假設用 **char a[100];** 和 **char b[100];** 儲存長度小於一百位數的正整數字串，**char c[200]** 用來儲存計算結果。請下面寫出幾個 functions 做 **a** 和 **b** 的加減乘除然後把答案存到 **c**：

```
void addLong(char c[], char a[], char b[]);
void subtractLong(char c[], char a[], char b[]);
void multiplyLong(char c[], char a[], char b[]);
void divideLong(char c[], char a[], char b[]);
```

什麼是 Recursive Functions ?

自己呼叫自己的 function 就是 recursive function。譬如

```
void f(char *a)
{
    if (*a == '\0') return;
    f(a+1);
    putchar(*a);
}
```

[練習 WB_02]

修改上面的 function，把輸入字串 "abc" 輸出為 "abccba"。用圖示來說明程式執行的流程。

[練習 WB_03]

修改練習 WB_02，改成處理中文字串 (一個 big5 中文字要用兩個 characters)。

寫 recursive functions 最需要注意的是停止條件，不然有可能不停呼叫自己，最後把所有可用的記憶體資源都用光。(呼叫 function 時如果傳參數，這些參數需要用記憶體存放；另外，只要 function 還沒執行到 **return**，那個 function 中的 local variables 也都會持續佔用記憶體，所謂的 stack。所以每多一層 function call，就多用了一些記憶體。)

什麼時候需要 Recursive Functions ?

所有可以用 recursive 方式來寫的程式都可以用 iterative 方式來寫 (**while** 或 **for**)；反之亦然。既然如此，為什麼不全部用 iteration 就好，幹嘛用 recursion，不但比較耗記憶體，而且執行速度又不會比較快 (因為處理參數也需要額外的時間。)

原因是有些演算法用 recursive 方式來寫比較易懂，也比較簡潔，譬如一些搜尋和排序的演算法。除此之外，有些資料結構，譬如 trees，用 recursive 方式來定義及處理會比較方便。

[練習 WB_04]

重新寫出兩個 functions，用 recursive 方式計算字串長度和判斷字串是否為 palindrome。

在一個 Function 裡面定義的變數和 Function 外定義的變數有什麼差別？

依照變數的有效或可建範圍，我們可以把它們劃分成 local variables 和 global variables，定義在 function 裡面的變數只有那個 function 可以看得得到，而定義在整個程式最前面，在所有 functions 外面的變數，則是全部的 functions 都看得到。在命名方面，同一個 scope 裡同一個名字當然不能用兩次，譬如

```
int a;
double a;
```

但是在 function 裡的 local variable 可以和 global variable 同名，這種情況 global variable 在那個 function 裡面會被當作不存在。

[練習 WB_05]

下面的程式會顯示什麼？

```
#include <stdio.h>
int aa = 100;
void f(void)
{
    int aa = 3;
    printf("f: %d\n", aa++);
}
int main(void)
{
    printf("main: %d\n", aa++);
    f();
    printf("main: %d\n", aa++);
    f();
    return 0 ;
}
```

上面的例子其實也顯示了變數存在於記憶體的時間長短。一般的 local variables 只有當 function 被呼叫的時候才會自動出現在記憶體 (stack) 中，而當 function 結束後就不見。而 global variables 則是在整個程式執行過程都會一直存在。

如何讓 Function 裡面的變數一直存在？

在變數的型別前面加上 **static**。

```
void f(void)
{
    static int aa = 3;
    printf( "f: %d\n", aa++ );
}
```

你可以用 **static** 變數來記錄 function 被呼叫了幾次。除此之外，如果你想持續記錄某些資料，但是又不想像 global variables 一樣可以被全部的 functions 隨意修改，就可以用 **static variables**，只透過某個特定的 function 來修改。不過還是無法阻止使用指標方式去修改，譬如

```
#include <stdio.h>
int aa = 100;
int* f(void)
{
    static int aa = 3;

    printf("f: %d\n", aa++);
    return &aa;
}
int main(void)
{
    int *aptr;

    printf("main: %d\n", aa++);
    aptr = f();
    printf("main: %d\n", aa++);
    *aptr = 10;
    f();
    return 0 ;
}
```

[練習 WB_06]

試著解釋一下上面的程式碼的執行結果。如果把上面 **f()** 裡的 **static** 字拿掉會怎樣？

其他 Recursive Functions 範例：計算最大公因數

```
#include <stdio.h>

int gcd(int a, int b); /* gcd() 需要傳回答案 (整數) */

int main(void)
{
    int x, y;
    scanf("%d%d", &x, &y);
    printf("%d\n", gcd(x, y));
    return 0;
}

int gcd(int a, int b)
{
    int ans; /* 制式寫法 */
    if (b > 0) {
        printf("(%d, %d)\n", a, b);
        ans = gcd(b, a%b);
    } else {
        ans = a;
    }
    /* 如果 b == 0 則 a 就是答案 */
    return ans;
}
```

其他 Recursive Functions 範例：快速計算最大公因數

```
#include <stdio.h>

int bgcd(int a, int b); /* bgcd() 需要傳回答案 (整數) */

int main(void)
{
    int x, y;

    scanf("%d%d", &x, &y); /* 輸入兩個整數 */
    printf("%d\n", bgcd(x, y));

    return 0;
}

/*
    這個遞迴公式的原理不難
    主要是依照 a 和 b 的奇偶數關係
    如果其中一個是奇數另一個是偶數
    則 2 絕對不會是它們的公因數
    因此直接把 2 這個因數去掉
    新的問題變成計算
    bgcd(a/2, b) 或 bgcd(a, b/2)
    這樣問題就變小了
    例如 bgcd(24, 15)
    其實只要算 bgcd(12, 15) 就可以 因為不會有 2 這個公因數
*/
int bgcd(int a, int b)
{
    int ans; /* 制式寫法 */
    int tmp;

    if (a < b) { /* 如果 a 比 b 小就調換 */
        tmp = a; /* 確保 a 總是大於等於 b */
        a = b;
        b = tmp;
    }

    /* 兩種停止條件 */
    if (b == 1)
        return 1;

    if (b == 0)
        return a;

    if (a % 2 == 0) { /* 如果 a 是偶數 */

        if (b % 2 == 0) { /* 如果 b 是偶數 */
            ans = 2 * bgcd(a/2, b/2);
        } else { /* b 是奇數 */
            ans = bgcd(a/2, b);
        }

    } else { /* a 是奇數 */

        if (b % 2 == 0) { /* 如果 b 是偶數 */
            ans = bgcd(a, b/2);
        } else { /* b 是奇數 */
            ans = bgcd((a-b)/2, b);
        }

    }

    return ans; /* 制式寫法 */
}
```

其他 Recursive Functions 範例：遞迴方式計算平方和

```
#include <stdio.h>
int ssum(int a);

int main(void)
{
    int x;
    scanf("%d", &x);
    printf("%d\n", ssum(x));
    return 0;
}

/* 遞迴公式: ssum(a) = a*a + ssum(a-1) */
/* 同時還會顯示 a*a+(a-1)*(a-1)+...+1*1 */
int ssum(int a)
{
    int ans; /* 制式寫法 */

    if (a>1) {
        printf("%d*d+", a, a); /* 除了計算 ans 之外 還顯示一些計算過程 */
        ans = a*a + ssum(a-1);
    }
    else {
        printf("1*1");
        ans = 1;
    }
    return ans; /* 制式寫法 */
}
```

其他 Recursive Functions 範例：顯示整數的二進位表示法

```
#include <stdio.h>
void i2b(int n);
void i2b_wrong(int n);

int main(void)
{
    int x;
    scanf("%d", &x);
    i2b(x); /* 呼叫 i2b() 把 x 的二進位表示法顯示出來 */
    printf("\n");
    i2b_wrong(x); /* 錯誤的寫法 */
    printf("\n");
    return 0;
}

/* 關鍵在於 遞迴呼叫的時候
   所有在 遞迴式 之前的程式碼 都會以呼叫的順序來進行
   但是在 遞迴式 之後的程式碼 會以呼叫的相反順序進行 */
void i2b(int n)
{
    if (n > 0) { /* 如果 n > 0 就把 n 除以 2 的餘數顯示出來 */
        i2b(n/2); /* 然後把問題化簡成為 顯示 n/2 的二進位表示法 */
        printf("%d", n%2); /* 把最低位的 bit 顯示出來 */
    }
    /* 如果 n == 0 就不要再遞迴下去 */
    /* 什麼事都不用做了 */
}

void i2b_wrong(int n) /* 錯誤的寫法 */
{
    if (n > 0) {
        printf("%d", n%2);
        i2b_wrong(n/2);
    }
}
```

我們用一個簡短的範例程式示範如何開檔和讀檔。範例程式會從一個純文字檔案 5000_words.txt 裡讀取資料，請從課程網頁 (參考書籍與連結的補充資料)，下載 5000_words.txt，檔案裡面包含了五千個英文單字：

```
5000_words.txt 檔案片段
...
procedure n. A manner or method of acting.
proceed v. To renew motion or action, as after rest or interruption.
proclamation n. Any announcement made in a public manner.
procrastinate v. To put off till tomorrow or till a future time.
procrastination n. Delay.
...
```

程式一開始先宣告一個型別是 **FILE** 的指標 **fp**，然後用 **fopen()** 開啟要讀取的純文字檔，並把取得的檔案指標用變數 **fp** 記下來。有了這個指標，就可以用 **fscanf()** 或 **fgetc()** 循序讀取檔案內容。程式裡面還用了 **feof()** 來判斷是否已讀到檔案結尾，最後當檔案使用完畢則要用 **fclose()** 來關閉檔案。除了這個範例使用的幾個 functions 之外，還有幾個常用的與檔案有關的 functions，例如 **fprintf()**、**fread()**、**fwrite()**、**fseek()**，詳細的用法可以查詢使用手冊 (課程網頁上有 The GNU C Library Reference Manual 的 PDF 檔的連結)。

```
#include <stdio.h>
#include <stdlib.h> /* 為了用 srand() 和 rand() */
#include <time.h> /* 為了用 time() 產生亂數種子 */
#define LINE_LEN 40 /* 假設單字長度不超過 40 個字元 */
#define NUM_WORDS 5000 /* 總共有 5000 個單字 */
char dictionary[NUM_WORDS][LINE_LEN]; /* 儲存單字的陣列 每個 row 都是一個字串 儲存一個單字 */

int main(void)
{
    FILE *fp; /* 開檔案 需要一個指標指到開啟的檔案 */
    int nw;
    int i;

    /* fopen() 的用法就是傳入檔名字串 以及開檔方式 "r" 表示只要讀檔 */
    /* fopen() 會傳回指到檔案的指標 失敗的或那個指標會是 NULL 表示沒有指到任何地方 */
    if ( (fp = fopen("5000_words.txt", "r")) == NULL ) {
        printf("Cannot open file.\n");
        exit(1);
    }
    /*
    用 fscanf() 把所有的單字都讀進來
    fscanf() 和 scanf() 的用法相同
    只是要多傳一個參數 把檔案指標傳給 fscanf()
    才能夠知道是要從哪個檔案讀資料
    feof(fp) 可以判斷檔案 fp 是否已經讀到了盡頭
    如果已經到了檔案結尾 feof(fp) 會傳回 true
    fgetc(fp) 則只會讀取一個字元 我們只想讀取單字
    所以用 fgetc(fp) 把剩下的單字解釋略過 一直讀到換行 再繼續讀下個單字
    */
    nw = 0;
    while (!feof(fp) && nw < NUM_WORDS) {
        fscanf(fp, "%s", dictionary[nw]);
        while ( !feof(fp) && fgetc(fp) != '\n' ) ;
        nw++;
    }
    fclose(fp);
    /*
    隨機選 10 個單字出來
    dictionary[][] 的每一個 row 都是一個字串
    例如 dictionary[1002] 儲存的是 "convertible" 這個字串
    所以 dictionary[1002][0] 就是 'c' 而 dictionary[1002][1] 就是 'o'
    以此類推 dictionary[1002][10] 是 'e' 然後 dictionary[1002][11] 是 '\0'
    */
    srand(time(NULL));
    for (i=0; i<10; i++) {
        printf("%s\n", dictionary[rand()%NUM_WORDS]);
    }
    return 0;
}
```

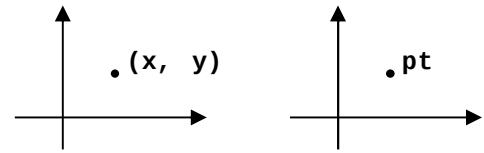
什麼是 C Structures ? 如何產生 `int`、`double`、字串、陣列等等基本資料型態之外，更複雜的資料儲存方式？

Structures 是多個相關的變數的集合，用一個共同的名稱來統稱。這樣的方式在使用上較方便，也會讓程式更簡潔易懂。譬如要描述平面上的點座標，可以用

```
int x, y;
```

若使用 structures 則可以自定一個叫做 `t_point` 的資料型態，寫成

```
struct t_point {
    int x;
    int y;
};
```



關鍵字 `struct` 後面接著的就是關於 `t_point` 這個自定的資料結構的宣告，`t_point` 是自己替這個資料結構取的名字，括號中間就是這個 structure 所包含的資料結構，括號後面要記得加上分號做結束。宣告過 structure 之後可以用它來定義變數

```
struct t_point pt;
```

所以 `pt` 會包含 `x` 和 `y` 兩個 members，要設定或更改 members 的內容，最直接的方式是使用 member operator `.` 來存取。

```
pt.x = 10;
pt.y = 20;
```

已經宣告過的 structure 可以再拿來宣告另一個 structure，例如

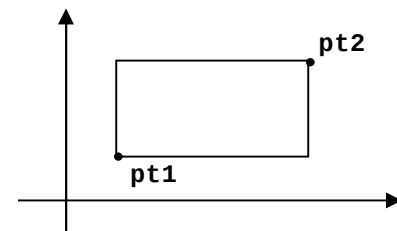
```
struct t_rect {
    struct t_point pt1;
    struct t_point pt2;
};
```

然後用它來產生變數

```
struct t_rect screen;
```

以及存取 members

```
printf("%d %d\n", screen.pt1.x, screen.pt1.y);
```



如何使用 C Structures 來做運算？

Structures 可以使用的運算元只有 `=`、`&`、`.`，其他的運算則必須自己寫 functions 來達到我們想要的功能，例如，想要比較兩個 structure 變數相不相等，不能直接用 `==` 或 `!=`，另外相加 `+` 和相減 `-` 也不能用。接下來就來看幾個自定 functions 的例子。

```
struct t_point makePoint(int, int);
struct t_point midPoint(struct t_point, struct t_point);

struct t_point makePoint(int x, int y)
{
    struct t_point pt;
    pt.x = x;
    pt.y = y;
    return pt;
}

struct t_point midPoint(struct t_point p1, struct t_point p2)
{
    p1.x = (p1.x + p2.x) / 2;
    p1.y = (p1.y + p2.y) / 2;
    return p1; /* 把 p1 回傳，會將 p1 的 members 的值一一複製 */
}
```

在主程式裡呼叫 `makePoint()` 來設定 `struct t_point` 變數

```
struct t_rect screen;
struct t_point mpt;
screen.pt1 = makePoint(1, 1);
screen.pt2 = makePoint(1024, 768);
mpt = midPoint(screen.pt1, screen.pt2);
```

[練習 WD_01]

寫一個 function `ptInRect()` 來判斷某個點 `p` 是否落在某個長方形 `r` 裡面，是則回傳 `1`，否則回傳 `0`。

```
int ptInRect(struct t_point p, struct t_rect r);
```

Structures 參數的傳遞方式

傳遞 structure 變數到 function，會用 call-by-value 方式，所以在 function 裡改變 structure 的 members 的值，並不會影響外部的 structure 變數的內容。

Structures 同樣可以用指標方式來達到 call-by-reference 的效果，通常用傳遞指標的方式會比傳遞整個 structure 來得有效率 (call-by-value 需要複製整個 structure 變數的內容)。

在討論指標之前，先來看一個額外的功能。C 提供 **typedef** 來讓我們宣告新的型別名稱，這樣接下來宣告變數會比較方便。譬如，在原來 `struct t_point` 的宣告之後，使用 **typedef** 宣告一個新名字來取代原來的寫法。

```
struct t_point {
    int x;
    int y;
};
typedef struct t_point Point;
```

原本產生變數必須使用

```
struct t_point pt;
```

現在只要用

```
Point pt;
```

所以現在宣告一個指到 `struct t_point` 的指標可以用

```
struct t_point *pp;
```

或

```
Point *pp;
```

把 `pp` 指到 `pt` 就和一般變數一樣，要使用 `&` 來取得 structure 的位址：

```
pp = &pt;
```

透過指標來存取 structure 的 members 有兩種寫法，一種是用標準的 `*` 先找到指標指的內容，再用 `.` 來取出 member：

```
(*pp).x = 10; /* 一定要加括號 */
```

另一種則是用 `->` 符號，相當於上面的縮寫

```
pp->x = 10;
```

[練習 WD_02]

宣告新的型別 `Rect` 來代表 `struct t_rect`，產生指到 `Rect screen` 的指標，透過指標修改 `screen` 範圍。

Structures 陣列

陣列的使用更能說明 structures 的必要性。以 `struct t_point` 為例，要產生 100 個平面上的點來記錄 100 組座標，如果不使用 structure 要寫成

```
int x[100], y[100];
```

使用 structure 變成

```
struct t_point pt[100]; /* 或使用前面 typedef 所宣告的新型別名稱 Point pt[100]; */
```

由於在這個例子中，平面點座標應該是一體的資料，包含 `x` 和 `y` 座標，所以使用 structure 的方式更合理。另外，計算 structure 陣列的大小，仍然可以使用 `sizeof`

```
sizeof(pt) / sizeof(Point)
```

和一般變數相同，structure 陣列參數的傳遞是用 call-by-reference 的方式，譬如 `f(pt)` 會傳陣列的起始位址。

[練習 WD_03]

寫出 function `randPoint()` 隨機在某個長方形 `r` 範圍內產生 `n` 個點。

```
void randPoint(Rect r, Point p[], int n);
```

產生的點儲存在 `p[]` 裡面。然後再寫出 function `meanPoint()` 計算 `p[]` 裡頭的 `n` 個點的 `mean`。

```
Point meanPoint(Point p[], int n);
```

Unions

除了 structures 之外，C 還提供另一種衍伸型別 unions，讓我們用同一塊區域來儲存不同類型的資料，例如

```
union u_tag {
    int ival;
    float fval;
    char *sval;
};
union u_tag u;
```

用這樣方式產生的變數 `u` 會有剛好足夠的空間來儲存 `int`、`float`、`char*` 三種型別中最大的型別資料。變數 `u` 每次只能儲存三種型別資料中的一種，我們必須自己記得目前 `u` 所儲存的資料是哪一種型別。

執行程式看看下面的輸出是什麼，就會對 `union` 的作用更了解。

```
printf("%d\n", sizeof(u)); /* 4 雖然三個成員各為 4 bytes 但必須共用同一個空間 */
u.ival = 10;
printf("%d\n", u.ival);    /* 10 */
u.fval = 20.0;             /* 後來設的值會把前面設的蓋掉 */
printf("%f\n", u.fval);    /* 20.000000 */
printf("%d\n", u.ival);    /* 1101004800 沒有意義 因為浮點數和整數的儲存方式不同 */
u.sval = "abcde";
printf("%d\n", u.ival);    /* 4202504 指標所指到的位址 */
printf("%s\n", u.sval);    /* abcde */
printf("%s\n", u.ival);    /* abcde 位址的儲存方式如果和整數的方式相同就能顯示出字串 */
```

Bit Manipulations

C 語言提供了一些 operators，讓我們能用更低階的方式存取和修改資料。這些 operators 包括

`&` (AND)、`|` (OR)、`^` (XOR)、`>>` (right shift)、`<<` (left shift)、`~` (one's compliment)

有時候我們需要儲存的資訊可能只有 `1` 和 `0` 兩種值，譬如記錄某種狀態 "有" 或 "無" (所謂的 flag)。當我們需要記錄大量這一類的資料時，若使用 `int` 來記錄會太浪費空間，這種情況就適合使用 bit 運算，只需用到原本的空間的 1/32。

```
#include <stdio.h>
void dispBits( unsigned val )
{
    int i;
    unsigned mask = 1 << 31; /* 1000...000 1 後面接 31 個 0 */
    for (i=0; i<32; i++) { /* (mask & val) != 0 如果是 true 會印出 1 */
        printf("%d", ((mask & val) != 0) ); /* (mask & val) != 0 如果是 false 會印出 0 */
        mask = mask >> 1; /* mask 的 1 向右移動 1 格 用來取下一個 bit */
    }
    printf("\n");
}
int main(void)
{
    unsigned n;
    n = 100;
    dispBits(n);
    dispBits(0x1f); /* 0x1f 是 31 的十六進位表示，換成二進位是 000...00011111 */
    dispBits(n & 0x1f);
    dispBits(n | 0x1f);
    dispBits(n ^ 0x1f);
    dispBits( ~n );
    return 0;
}
```

用圖示來解釋下面的運作原理

寫出 `unsigned rightRotate(x, n)` 傳回 `x` 向右 rotate `n` bits 之後的結果。

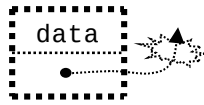
第一部份用 `(x & ~(~0 << n))` 取出最右邊 `n` 個 bits，然後向左 shift `(sizeof(x)*8 - n)` bits，
第二部份用 `(x >> n)` 把 `x` 向右 shift `n` bits，把兩個部份做 OR 組合起來就可以做出向右 rotate 的效果。

Self-Referential Structures

學過指標和 **struct** 之後，就可以靠結合這兩樣東西，變出很多花樣。我們會介紹一些基本的動態資料結構，包括 linked lists、stacks、queues、trees。這些資料結構搭配演算法，就會有很多實際用途。

先來看 Self-Referential Structures，如何讓 **struct** 中包含指向同一種 **struct** 的指標。標準寫法如下：

```
struct t_node {
    int data;
    struct t_node *nextPtr;
};
```



關鍵就是第二個 member，是一個能夠指向 **struct t_node** 的指標。我們用這樣的 **struct** 來產生兩個變數，**struct t_node n1, n2;**

還記得當你產生指標的時候，牠還沒被初始化，也就是還沒指到有意義的位址，所以還不能拿來使用。所謂有意義的位址，必須是已經產生的變數所在的位址，用 **&** 符號來取得，譬如 **&n2** 就是一個有意義的位址。所以，我們可以寫出下面的敘述：

```
n1.nextPtr = &n2;
```

再來 **n2.nextPtr** 也可幫牠指到一個合法的地方，譬如 **NULL**：

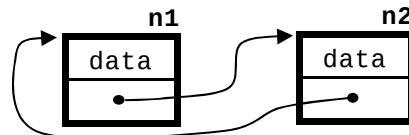
```
n2.nextPtr = NULL;
```

當然也可以把牠指向 **&n1** 位址

```
n2.nextPtr = &n1;
```

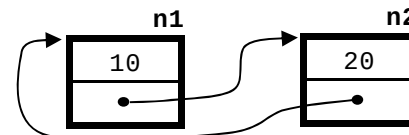
n1 和 **n2** 的 **data** 都還沒設定，我們現在可以用 **n1.nextPtr** 和 **n2.nextPtr** 互相幫對方設定。

```
(*n1.nextPtr).data = 20;
(*n2.nextPtr).data = 10;
```



上面的敘述還有另一種寫法：

```
n1.nextPtr->data = 20;
n2.nextPtr->data = 10;
```



[練習 WE_01]

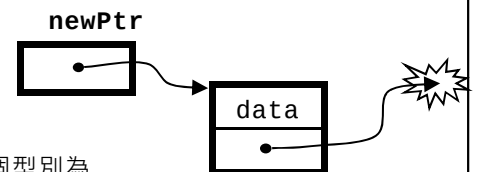
先產生三個 **struct t_node** 變數，分別叫做 **n1**、**n2**、**n3**

n1.nextPtr 指向了 **&n2** 位址，**n2.nextPtr** 指向 **&n3** 位址，**n3.nextPtr** 指向 **NULL** 位址，然後用一個新的指標 **struct t_node *p** 指向 **&n1** 位址。寫一個迴圈，只使用 **p** 來把 **n1**、**n2**、和 **n3** 的 **data** 都設成 **500**。

Dynamic Memory Allocation

前面的例子中，**n1**、**n2**、**n3** 都是在 compile 的時候就決定了，因為一開始我們就假設要用到三個變數。假如我們想讓程式在執行的時候，才依照外在狀況產生新的 **struct t_node**，就必須動態取得記憶體來儲存變數。在 **stdlib.h** 裡頭有兩個 functions: **malloc()** 和 **free()**，我們可以靠這兩個 functions 動態取得和釋放記憶體。用 **malloc()** 取得的記憶體大小是動態設定的，**malloc()** 會傳回取得的整塊記憶體的起始位址，你的程式就要靠這個起始位址找到你取得的整塊記憶體。程式開頭要加 **#include <stdlib.h>**。

```
struct t_node *newPtr;
newPtr = (struct t_node *)malloc(sizeof(struct t_node));
```



用 **sizeof(struct t_node)** 算出需要多少個 bytes。

然後用 **malloc()** 取得足夠的記憶體區塊的起始位址。**malloc()** 會傳回一個型別為

void* 的指標代表起始位址，**void*** 的意思是指向不特定的型別，所以你可以把這樣的指標的值設給「宣告成其他任意型別」的指標，譬如 **int*** 的指標。如果 **malloc()** 無法取得記憶體則會傳回 **NULL**。

(註：不管是指向哪一種型別的指標，儲存的值都是長度 4 bytes 的記憶體位址，所以這樣的設定很合理。真正的差異是當你要做類似 **p++** 這樣的指標增加的動作時，必須要知道 **p** 究竟是指向哪一種型別，才能知道每次 **++** 到底要移動多少個 bytes 的位址。你可以試試看對一個 **void*** 的指標做 **++** 的動作，compiler 可能會有會有類似 warning: wrong type argument to increment 的警告訊息。)

Dynamic Memory Allocation

用 `malloc()` 取得記憶體，使用完後要記得釋放，不然記憶體會越用越少。下面的敘述會把剛才前面取得的記憶體釋放

```
free(newPtr);
```

可見得，記住原先取得的記憶體的起始位址，是一件非常重要的事情。要是忘了那個起始位址，到時候就無法正確釋放記憶體。

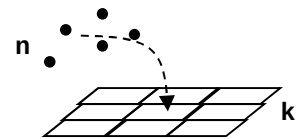
你可以試試看如果釋放一個錯誤的位址會有什麼後果，或是把已經 `free()` 的位址再 `free()` 一次看看。



(註：執行 `free(newPtr)` 會把 `newPtr` 指到的位址傳給 `free()`，然後 `free()` 會負責把對應位址原先佔用的那些記憶體釋放掉，但 `newPtr` 的值並不會變 (因為 call-by-value)，也就是說 `newPtr` 還是指到相同的記憶體位址，但是這塊記憶體的內容已經不再受到保護，所以不該再用 `newPtr` 去存取那塊記憶體，雖然不一定會立刻發生錯誤。更進一步說明，對大多數 `malloc()` 和 `free()` 的 implementation 來說，所謂取得和釋放其實只是改變記憶體的使用狀態。`malloc()` 和 `free()` 做的事情只是去記錄哪些記憶體區塊被佔用以及哪些可用。所以，執行完 `free(newPtr)` 之後，乍看之下可能會覺得沒發生什麼變化，因為原先那些記憶體內的資料可能都還存在，但是無論如何你的程式都不該再去存取已經被 `free()` 的位址的記憶體內容。總之，使用 `malloc()` 和 `free()` 要特別小心。另外順帶提一下，`malloc()` 和 `free()` 通常就是用接下來要介紹的 linked lists 來管理記憶體。)

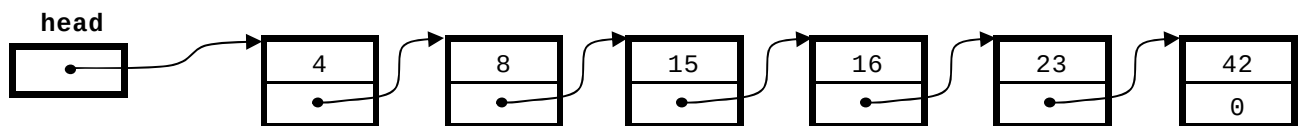
[練習 WE_02]

讓使用者輸入 `n` 和 `k` 兩個整數。宣告一個 `struct t_point` 包含 `int x` 和 `int y`。用 `malloc()` 產生一個型別為 `struct t_point`，長度為 `n` 的陣列，取名為 `P`。隨機設定每個 `P[i]` 的 `x` 和 `y` ($0 \sim k-1$ 的整數，總共有 $k \times k$ 個格子)，把落在每個格子內的點數秀出來。最後把 `P` 佔用的記憶體釋放掉。



Linked Lists

結合前面兩項功能，self-referential structures 和 dynamic memory allocation，就可以動態地做出 linked lists 這種資料結構。



接下來要討論的問題就是如何產生一個 linked list，如何在某個位置插入一個 node，如何拿掉某個 node。假設我們希望維持 linked list 所存的資料的順序，node 的 data 值要由小排到大。

需要寫出幾個 functions:

```
struct t_node* insert(struct t_node *np, int val);
struct t_node* delete(struct t_node *np, char val);
void dispList(struct t_node *np);
```

主程式從產生 `head` 開始

```
struct t_node *head;
head = NULL;
```

呼叫 `insert()` 把某個值加入 `head` 所指到的 linked list 的適當位置 (node 的 data 值要從小排到大)

```
head = insert(head, 4); /* 經過 insert() 之後，head 可能需要指到不同位址 */
head = insert(head, 8); /* 所以要傳回新的位址給 head */
head = insert(head, 13);
```

或是呼叫 `delete()` 把值從 `head` 所指到的 linked list 中刪除

```
head = delete(head, 13);
head = delete(head, 4);
head = delete(head, 11);
```

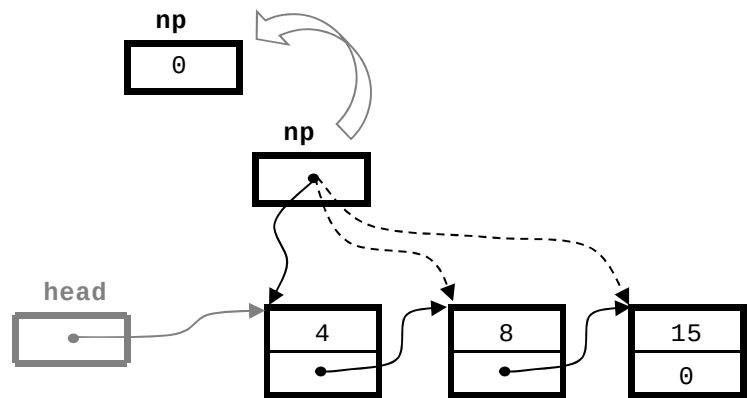
或是呼叫 `dispList()` 把 `head` 所指到的 linked list 的內容顯示出來

```
dispList(head);
```

Linked Lists

先來看怎麼寫 `void dispList(struct t_node *np);`

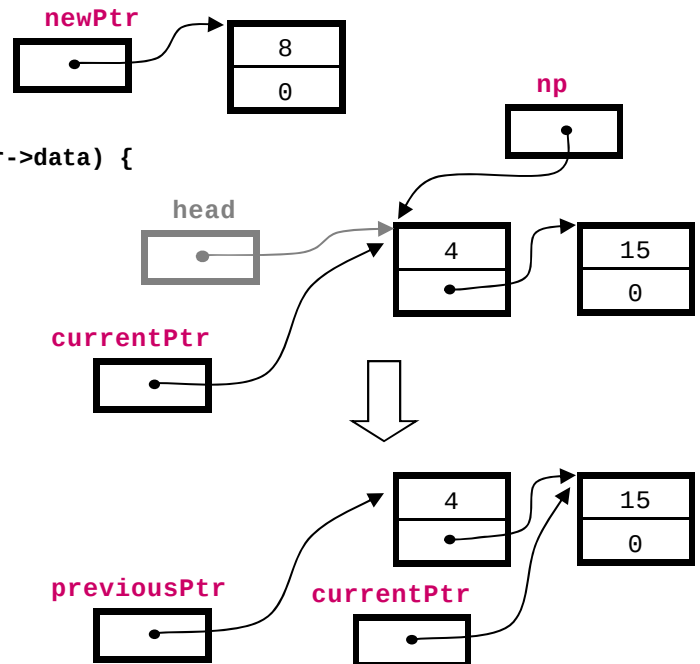
```
void dispList(struct t_node *np)
{
    if ( np == NULL ) {
        printf("List is empty.\n\n");
    }
    else {
        printf("The list is:\n");
        while (np != NULL) {
            printf("%d--> ", np->data);
            np = np->nextPtr;
        }
        printf("NULL\n\n");
    }
}
```



再來看 `struct t_node* insert(struct t_node *np, int val);`

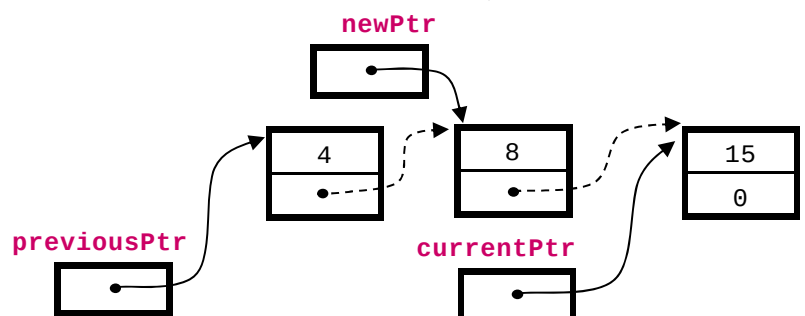
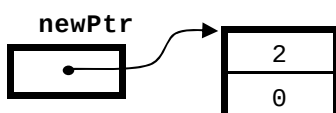
```
struct t_node* insert(struct t_node *np, int val)
{
    struct t_node *newPtr, *previousPtr, *currentPtr;

    newPtr = (struct t_node *)malloc(sizeof(struct t_node));
    if (newPtr != NULL) {
        newPtr->data = val;
        newPtr->nextPtr = NULL;
        previousPtr = NULL;
        currentPtr = np;
        while (currentPtr!=NULL && val>currentPtr->data) {
            previousPtr = currentPtr;
            currentPtr = currentPtr->nextPtr;
        }
        if (previousPtr == NULL) {
            newPtr->nextPtr = np;
            np = newPtr;
        }
        else {
            previousPtr->nextPtr = newPtr;
            newPtr->nextPtr = currentPtr;
        }
        return np;
    }
    else {
        printf("記憶體不夠啦.\n");
        return NULL;
    }
}
```



[練習 WE_03]

`head = insert(head, 2);` 會有哪些動作?

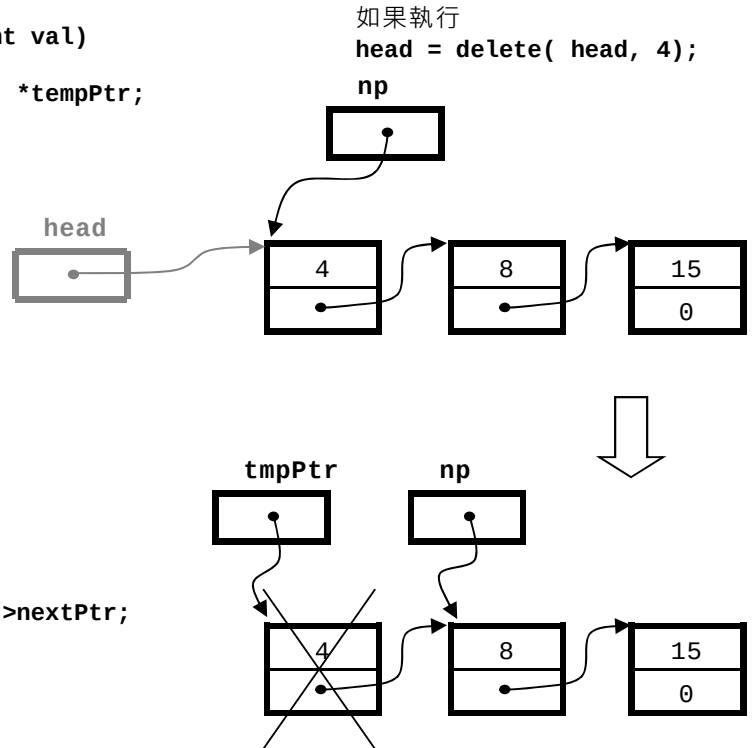


Linked Lists

最後看怎麼寫 `struct t_node* delete(struct t_node *np, int val);`

```
struct t_node* delete(struct t_node *np, int val)
{
    struct t_node *previousPtr, *currentPtr, *tempPtr;

    if (val == np->data) {
        tempPtr = np;
        np = np->nextPtr;
        free( tempPtr );
        return np;
    }
    else {
        previousPtr = np;
        currentPtr = np->nextPtr;
        while (currentPtr != NULL &&
               currentPtr->data != val) {
            previousPtr = currentPtr;
            currentPtr = currentPtr->nextPtr;
        }
        if (currentPtr != NULL) {
            tempPtr = currentPtr;
            previousPtr->nextPtr = currentPtr->nextPtr;
            free( tempPtr );
            return np;
        }
    }
    return NULL;
}
```



[練習 WE_04]

畫圖解釋上面的例子如果接著執行

`head = delete(head, 15);`

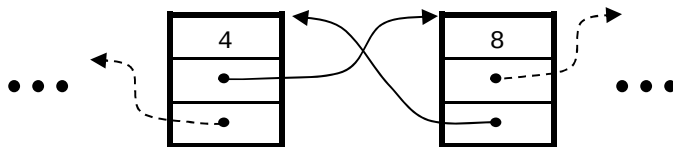
以及

`head = delete(head, 7);`

會做哪些動作。

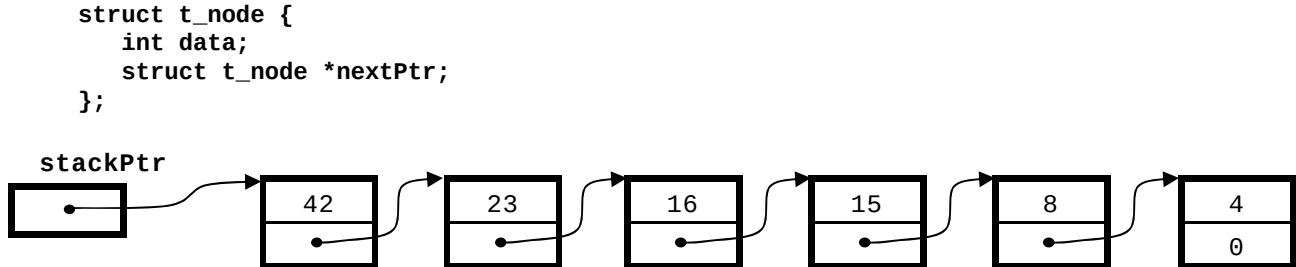
[練習 WE_05]

試著建立 doubly linked lists。



Stacks

看過 linked lists 的操作方式，再來就要介紹 stacks 這種資料結構。Stacks 是想像將一筆筆資料疊在一起，新加入的資料只能加在 stack 最上層，而且也只能從 stack 最上層移除資料，就是所謂的 last-in first-out (LIFO) 原則。可想而知，實作 stacks 的方式其實和 linked lists 一樣，還是用 **struct** 和指標的結合來做，只是對於 stacks 來說 **struct t_node** 的 **insert()** 和 **delete()** 只能發生在最前面的 node，一般習慣將這兩個動作稱作 push 和 pop。此外，和 linked lists 一樣，需要一個指標指到 stack 的最頂端位址，而 stack 最底層的 node 的 **nextPtr** 欄位也要指到 **NULL**。先來看用下面這種 **struct t_node** 製作出的 stack：



我們需要寫出能達到 push 和 pop 兩種效果的 functions。有了製作 linked lists 的經驗，這次我們就來看看如何用 call-by-reference 來寫這兩個 functions。

為了要將 stack 的內容印出來，我們可以直接拿 **displist()** 來修改寫出 **printStack()**。

```

void printStack(struct t_node *currentPtr)
{
    if (currentPtr == NULL) {
        printf("The stack is empty.\n\n");
    }
    else {
        printf("The stack is:\n");
        while (currentPtr != NULL) {
            printf("%d --> ", currentPtr->data);
            currentPtr = currentPtr->nextPtr;
        }
        printf("NULL\n\n");
    }
}

```

將新的資料加入 stack 需要 **void push(struct t_node * *topPtr, int val);**

```

void push(struct t_node * *topPtr, int val)
{
    struct t_node *newPtr;
    newPtr = (struct t_node *)malloc(sizeof(struct t_node));
    if (newPtr != NULL) {
        newPtr->data = val;
        newPtr->nextPtr = *topPtr;
        *topPtr = newPtr;
    } else {
        printf("%d not inserted. No memory available.\n", val);
    }
}

```

參數 **topPtr** 出現兩個星號是因為我們想要有 call-by-reference 的效果。在 **main()** 裡假設我們有一個指標指向 stack 的最上層位址：

```

struct t_node *stackPtr = NULL;

```

用 call-by-reference 方式呼叫，要用類似下面的方式，把 **stackPtr** 這個指標變數本身的位址傳給 **push()**：

```

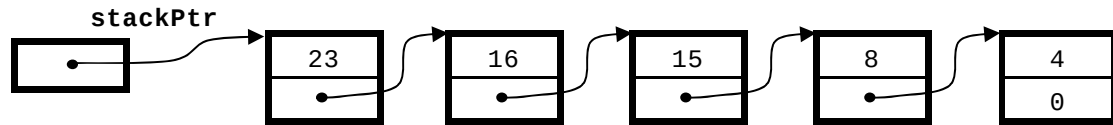
push(&stackPtr, 42);

```

由於 **stackPtr** 已經是 **struct t_node *** 這樣的型別，所以 **&stackPtr** 的型別就是 **struct t_node ****，出現兩個星號。如此一來，在 **push()** 裡面對 **topPtr** 所做的修改，才能夠真的影響到 **main()** 裡的 **stackPtr**。

push() 的示意圖

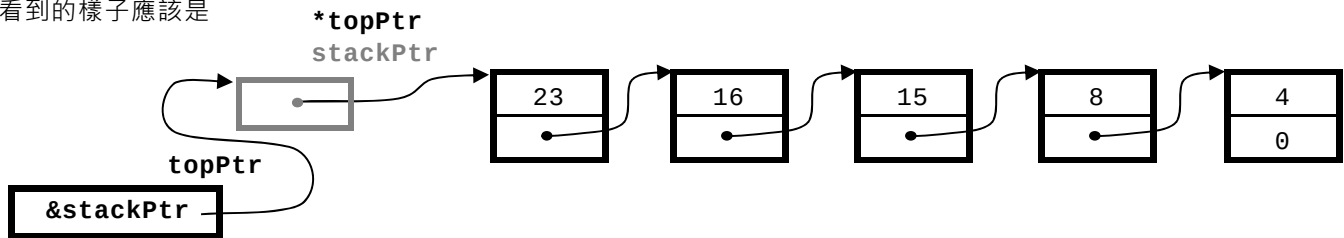
假設我們已經用了一連串 `push()` 產生了下面的 stack (這些數字是依照什麼順序 push ?):



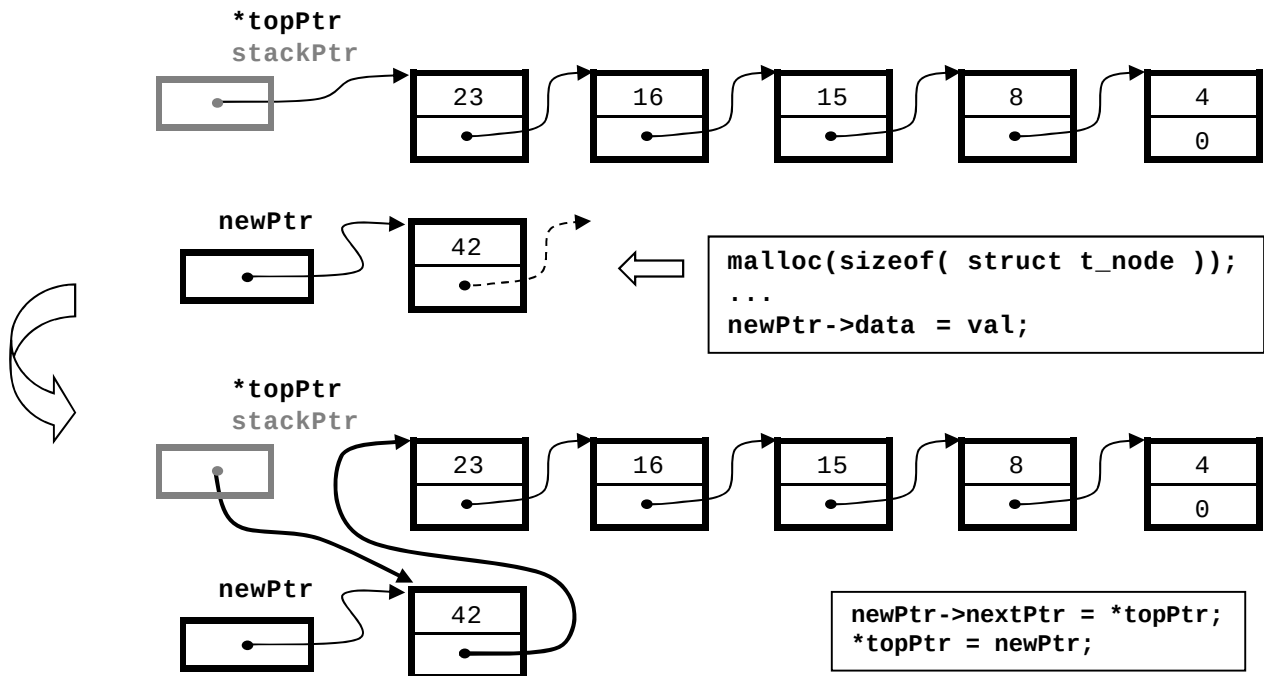
在這樣的情況下再呼叫 `push(&stackPtr, 42);` 進入

```
void push(struct t_node * *topPtr, int val)
{ ...
}
```

看到的樣子應該是



`topPtr` 這個指標所記錄的位址是 `&stackPtr`，所以 `*topPtr` 相當於 `stackPtr`，記錄的是由 `struct t_node` 組成的 stack 的起始位址。



Call-by-Reference 的觀念

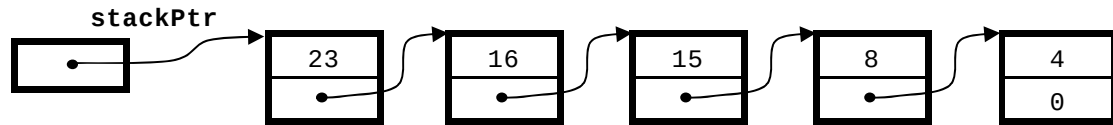
假設你有兩個 functions: `f1()` 和 `f2()`，在 `f1()` 裡面會呼叫 `f2()`，而 `f1()` 裡面有一個變數 `x`。如果想要在 `f2()` 裡面修改 `f1()` 的變數 `x`，就必須用 call-by-reference 的方式把 `x` 傳給 `f2()`，也就是把 `x` 的位址告訴 `f2()` 而讓 `f2()` 能直接對那個位址裡面儲存的資料做修改。

```
int f1()
{
    double x;
    ...
    f2(&x)
    ...
}

int f2(double *y)
{
    ...
    *y = 5.8;
    ...
}
```

pop()

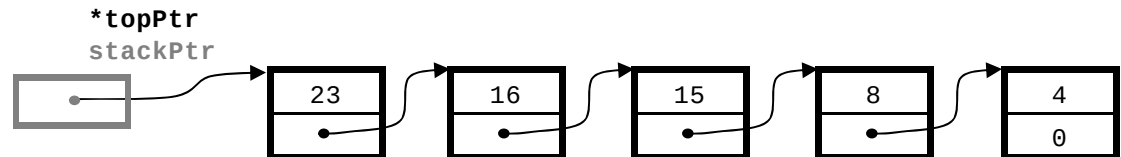
假設我們已經用了一連串 **push()** 產生了下面的 stack:



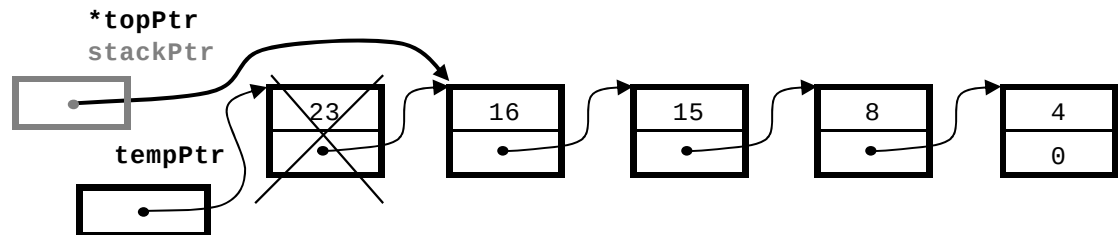
在這樣的情況下再呼叫 **pop(&stackPtr);** 進入

```
int pop(struct t_node * *topPtr)
{
    struct t_node *tempPtr;
    int popValue;
    if (*topPtr == NULL) return -1;
    tempPtr = *topPtr;
    popValue = (*topPtr)->data;
    *topPtr = (*topPtr)->nextPtr;
    free(tempPtr);
    return popValue;
}
```

在 **pop()** 裡看到的樣子應該是



執行 **pop()** 應該要傳回 **23** 這個整數，然後 **stackPtr** 要指到儲存 **16** 的 **struct t_node**，再把原本儲存 **23** 的 **struct t_node** 所佔用的空間釋放掉。



主程式的內容

```
int main()
{
    struct t_node *stackPtr = NULL;
    int i;
    push(&stackPtr, 4);
    push(&stackPtr, 8);
    push(&stackPtr, 15);
    push(&stackPtr, 16);
    push(&stackPtr, 23);
    printStack(stackPtr);
    i = pop(&stackPtr);
    printStack(stackPtr);
    printf("%d is popped.\n", i);

    return 0;
}
```

[練習 WE_06]

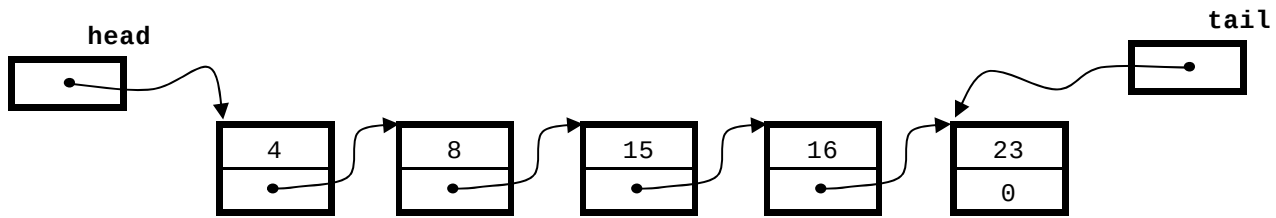
把 **printStack()** 改寫成 recursive 版本。

[練習 WE_07]

使用 stack 判斷一個字串的括號是否正確對應，例如 "(((1+2) + (3+4)) * 5 + (6+7) * 8) * 9"。

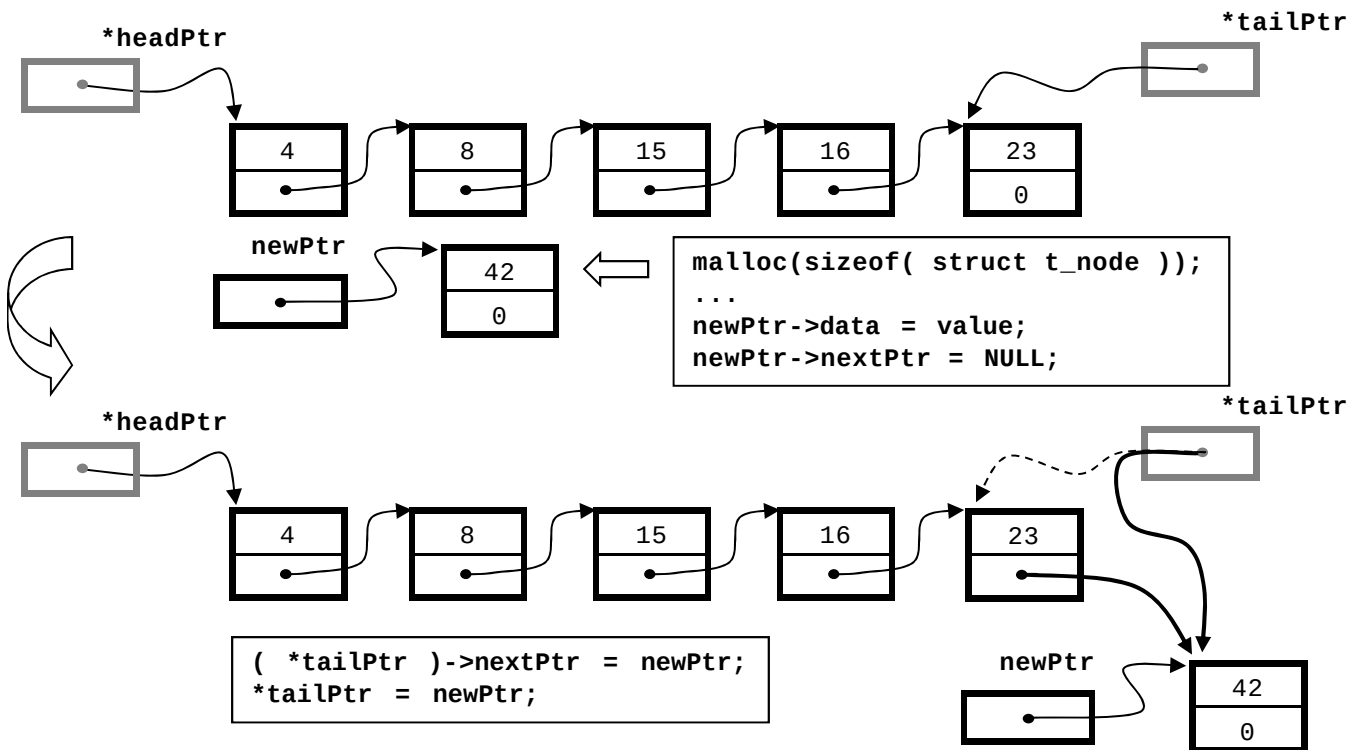
Queues

雖然 queues 和 stacks 都是用 linked lists 的形式來儲存資料，但是 queues 的操作原則是 first-in first-out (FIFO)，也就是平常的排隊原則，先排隊的先結帳，後來加入的要排在隊伍最後面。相對於 stacks 的 `push()` 和 `pop()` 這兩個動作，queues 需要 `enqueue()` 和 `dequeue()` 兩個 functions。至於 `printQueue()` 則和 `printStack()` 的方式完全一樣。除此之外，由於 stack 不論加入或移除都是從 stack 的最上端，但是 queue 則是從末端加入、前端移除，所以 queue 除了要有一個 `head` 指標指到 queue 的前端，還需要 `tail` 指標指到 queue 的末端。



在這樣的情況下再呼叫 `enqueue(&head, &tail, 42);`

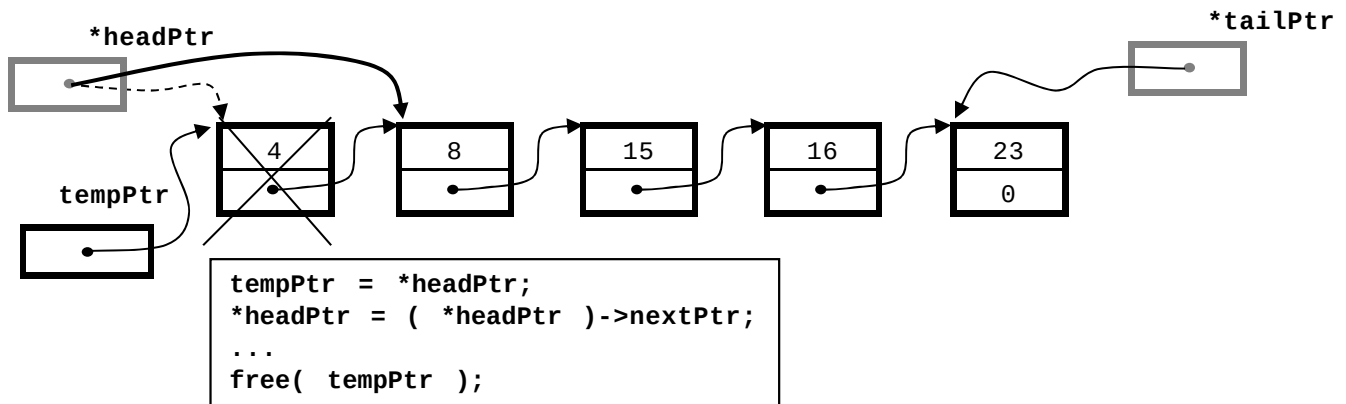
```
void enqueue(struct t_node * *headPtr, struct t_node * *tailPtr, int val)
{ ...
}
```



```
void enqueue(struct t_node * *headPtr, struct t_node * *tailPtr, char val)
{
    struct t_node *newPtr;
    newPtr = (struct t_node *)malloc(sizeof( struct t_node ));
    if (newPtr != NULL) {
        newPtr->data = val;
        newPtr->nextPtr = NULL;
        if (*headPtr == NULL) { /* 假如 queue 還是空的，則在第一個 node 被加入之後， */
            *headPtr = newPtr; /* headPtr 和 tailPtr 會暫時都指向第一個 node */
        } else {
            (*tailPtr)->nextPtr = newPtr;
        }
        *tailPtr = newPtr;
    } else {
        printf("%c not inserted. No memory available.\n", val);
    }
}
```

dequeue()

dequeue() 的動作很簡單：把 **head** 指標從第一個 node 移到第二個 node，然後把第一個 node 的空間釋放。同樣地，因為 **head** 指標一旦移動到第二個 node，第一個 node 的位址就沒有人記得，所以要先用 **tempPtr** 把第一個 node 的位址先記住，才能移動 **head** 指標。



```

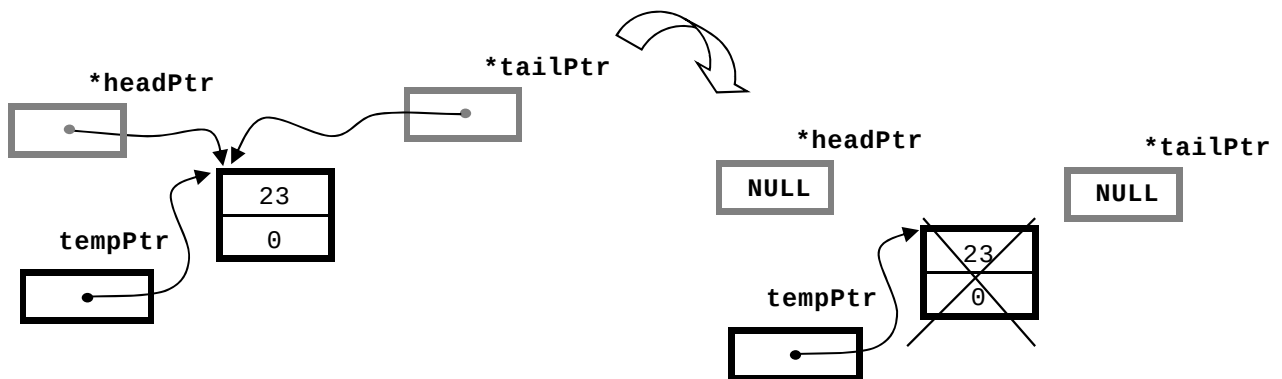
int dequeue(struct t_node * *headPtr, struct t_node * *tailPtr)
{
    int val;
    struct t_node *tempPtr;
    if (*headPtr == NULL) return -1;
    val = (*headPtr)->data;
    tempPtr = *headPtr;
    *headPtr = (*headPtr)->nextPtr;

    if (*headPtr == NULL) {
        *tailPtr = NULL;
    }
    free(tempPtr);
    return val;
}

```

當只剩下一個 node 的時候，**(*headPtr)->nextPtr** 指到 **NULL**，所以執行

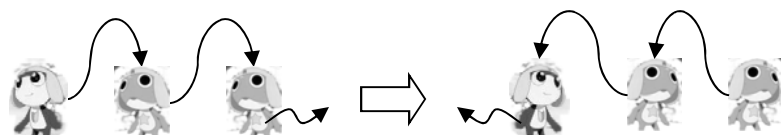
***headPtr = (*headPtr)->nextPtr;** 之後 ***headPtr** 指向 **NULL**，在這種情況下 ***tailPtr** 也應該指到 **NULL**，回到 empty queue 的狀態。

**[練習 WE_08]**

寫一個 function

```
void reverse(struct t_node * *headPtr, struct t_node * *tailPtr);
```

把 queue 反過來。



[練習 WE_08] (續)

```

void reverse(struct t_node ** headPtr, struct t_node ** tailPtr)
{
    struct t_node *currentPtr;
    struct t_node *previousPtr;
    struct t_node *tempPtr;

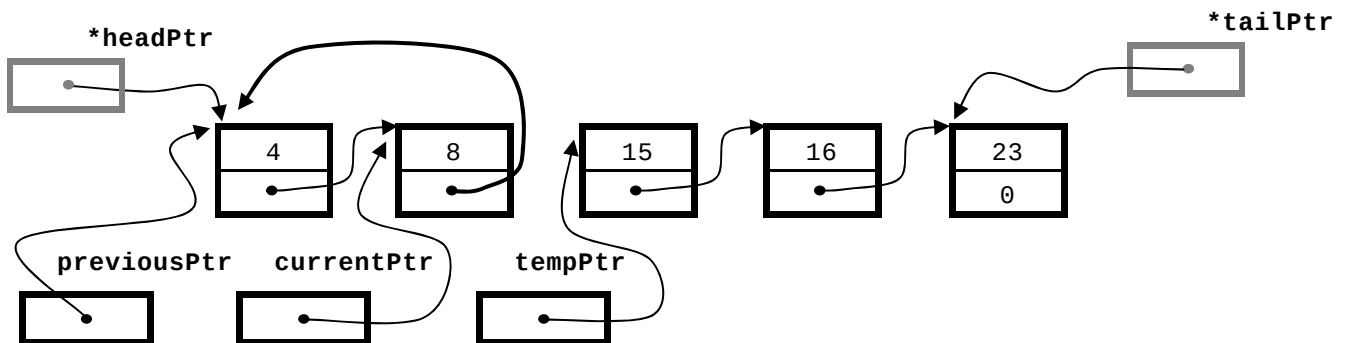
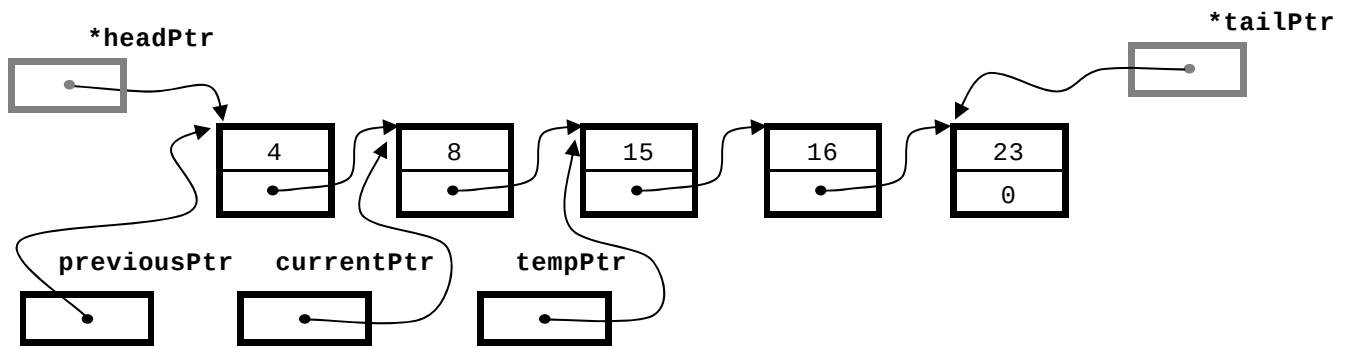
    if (*headPtr != *tailPtr) { /* 如果 queue 只剩一個 node, 或是 queue 是空的,
                                則 *headPtr 和 *tailPtr 會相等, 這時候並不需要做任何動作 */
        previousPtr = *headPtr;
        currentPtr = previousPtr->nextPtr; /* previousPtr、currentPtr 指到相鄰 nodes */

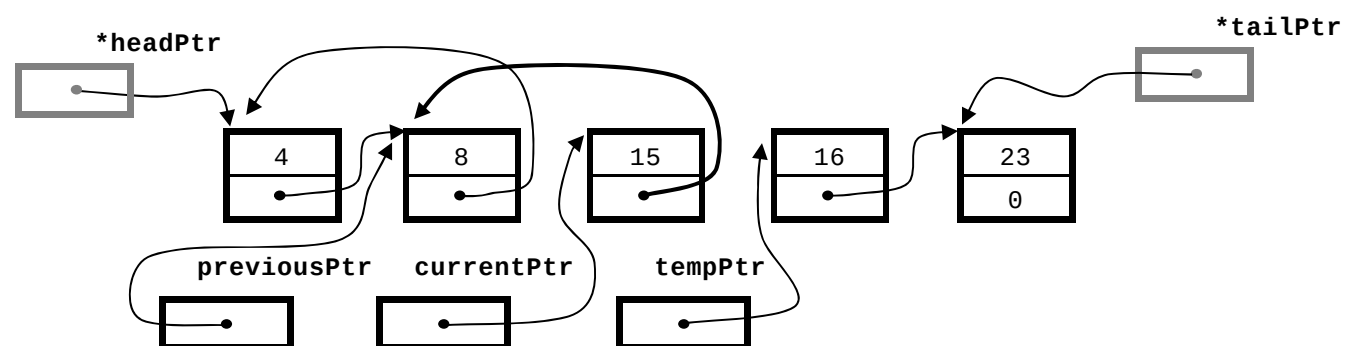
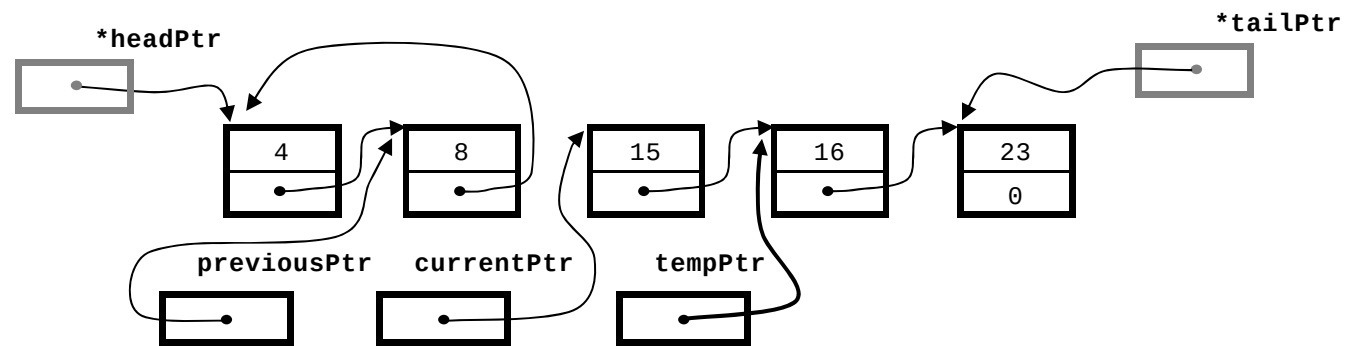
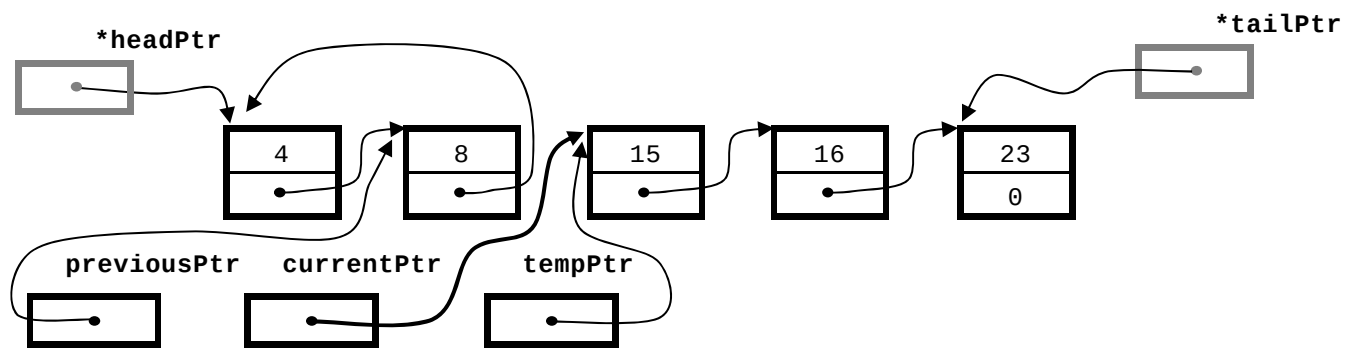
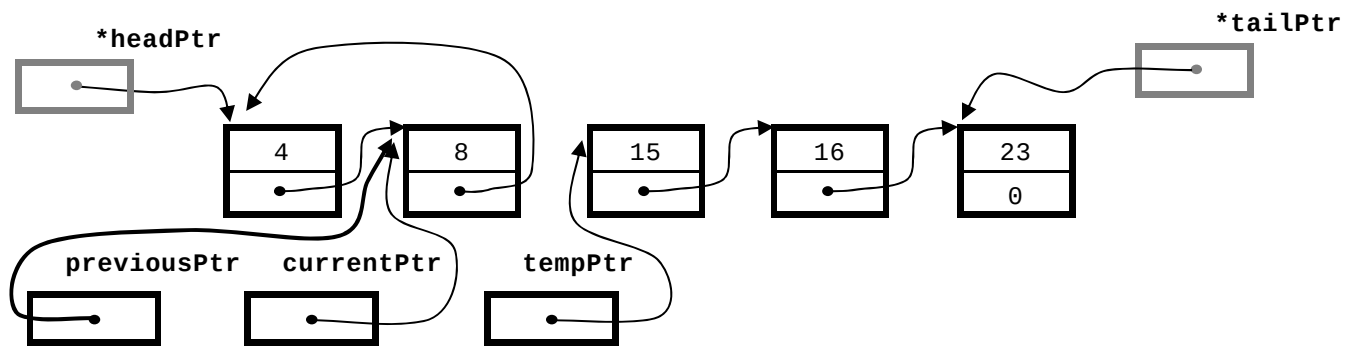
        while(currentPtr != *tailPtr) {
            tempPtr = currentPtr->nextPtr; /* 先記住下一個 node 的位址 */
            currentPtr->nextPtr = previousPtr; /* 記住後更改指標 讓牠往回指 */
            previousPtr = currentPtr; /* previousPtr 往下移動一個 node */
            currentPtr = tempPtr; /* currentPtr 也往下移到剛才記住的位址 */
        }
        (*tailPtr)->nextPtr = previousPtr; /* 迴圈結束後還要再收尾
                                           把原本指到 null 的指標改指到前一個 node */

        tempPtr = *tailPtr; /* swap 三步驟又來了 */
        *tailPtr = *headPtr;
        *headPtr = tempPtr;

        (*tailPtr)->nextPtr = NULL; /* 把新的 *tailPtr 的 nextPtr 指到 NULL */
    }
}

```

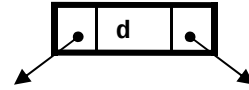




Trees

終於要介紹 trees 這種有趣的資料結構。大家以後學 data structures 和 algorithms 的時候會遇到各式各樣奇怪的樹，在這裡我們只簡單介紹 binary search trees (二元搜尋樹)，從名字可看出 binary search trees 其實是 binary trees 的一種特例。所謂 binary trees 是指每個 node 最多可以有兩個指標指向別的 nodes，我們必須稍微修改 `struct t_node` 的宣告：

```
struct t_treeNode {
    struct t_treeNode *leftPtr;
    int data;
    struct t_treeNode *rightPtr;
};
```



所以一個 binary tree 的每個節點都長得像 `struct t_treeNode` 這樣的結構，也就是除了有 `data` 欄位儲存資料之外，還包含了兩個指向 `struct t_treeNode` 的指標。

[回顧]

指標變數，用來記錄記憶體位址，在 32-bit 定址系統下，每個指標變數都用 4 bytes 來記錄某個記憶體位址。

既然都是 4 bytes，為什麼指標變數還要有型別，譬如 `struct t_treeNode *leftPtr;`？

這是因為我們對指標變數本身並不是很感興趣，比較在乎的是指標變數所記錄的那個記憶體位址裡面所存放的資料，但是指標變數記錄的只是資料的開頭位址，到底資料本身有多大，就要靠資料的型別來決定。

另外當我們對指標變數作加減的運算時，也必須靠資料的型別才能知道 `leftPtr+1` 這樣的動作到底會移動多少位址。

當我們使用 `leftPtr` 的時候，我們得到的是 `leftPtr` 這個變數裡面的值，也就是某個記憶體位址。

當我們使用 `*leftPtr` 的時候，我們得到的是 `leftPtr` 所記錄的那個記憶體位址的記憶體裡面所存放的資料。

當我們寫 `rightPtr = leftPtr;` 我們的意思是把 `leftPtr` 所記錄的「記憶體位址」複製給 `rightPtr`，所以 `rightPtr` 和 `leftPtr` 就會記錄了相同的位址。

當我們寫 `*rightPtr = *leftPtr;` 我們的意思是把 `leftPtr` 所記錄的「記憶體位址裡面的資料」複製到 `rightPtr` 所記錄的記憶體位址的記憶體裡面。

用圖書館的比喻，假設我想介紹你看一本書，我可以告訴你請到第幾樓的第幾個書櫃的第幾層，找出第幾本書來看。另一種可能是我自己去把那本書找出來，然後影印一本(違法，不過這只是比喻)，然後放到你的書櫃裡面。

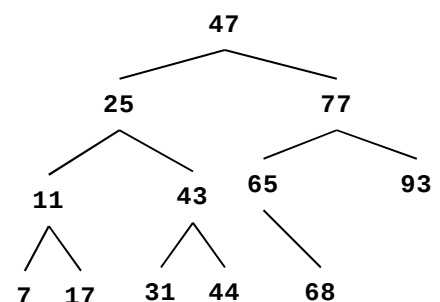
所以當我們說把某個指標指到另一個指標所指的位址，意思是把指標變數記錄的「記憶體位址」複製給另一個指標變數。如果能牢記這一點，再回頭看看 linked lists、stacks、queues 這些動態資料結構的加入和刪除的動作，應該會比較容易想通。

我們已經學過使用 `malloc()` 來取得記憶體，用 `free()` 來釋放記憶體。假設你用 `malloc(sizeof(int)*80);` 取得了一大塊記憶體，你要怎麼使用這塊記憶體呢？他並沒有一個像是 `a[]` 或 `x` 之類的名字，那要怎麼把資料存到這塊記憶體中，或是到底該怎麼知道這塊記憶體在哪裡。由此可知 `malloc()` 必須傳回這塊記憶體的起始位址，這樣你就可以用一個指標變數把這個位址記起來，然後就可以透過這個指標找到這塊記憶體，然後存取裡面的資料。釋放記憶體也是一樣，只要告訴 `free()` 你想要釋放哪個位址的記憶體，譬如 `free(leftPtr);`，它就會找到那個位址，然後依照當初 `malloc()` 取得記憶體時留下的記錄，把該位址之後正確大小的記憶體區塊釋放。

Binary Search Trees

在開始寫程式產生 binary search trees 之前，先來看一個實際例子，並且了解一下它的定義。

1. 最上面的那個 node 稱作 root；root 只會指別人，不會被別人指。
2. 最下面的那些 node 稱作 leaf nodes；leaf nodes 只會被指，不會指別人。
3. 相連的 nodes 之間的關係：指別人的稱作 parent，被別人指的稱作 child。
4. 每個 node 左右兩邊連著的通常被稱作 left subtree 左子樹和 right subtree 右子樹。
5. Binary search trees 的特性是對任意一個 node 來說牠的左子樹每個 node 的值都比牠小，而右子樹每個 node 的值都比牠大。

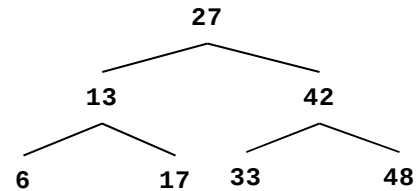


如何走訪一個 Binary Search Tree 的每一個 Node ?

preorder (中左右): 27 13 6 17 42 33 48

inorder (左中右): 6 13 17 27 33 42 48 (相當於做排序)

postorder (左右中): 6 17 13 33 48 42 27



[bst.c]

```

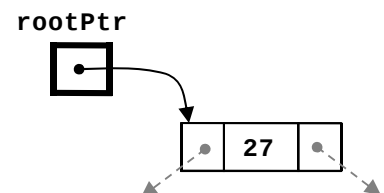
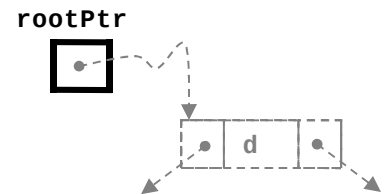
#include <stdio.h>
#include <stdlib.h>
struct t_treeNode {
    struct t_treeNode *leftPtr; /* 指向左子樹 (記錄左子樹的 root 的位址) */
    int data;
    struct t_treeNode *rightPtr; /* 指向右子樹 (記錄右子樹的 root 的位址) */
};
typedef struct t_treeNode TreeNode; /* 為了方便 替 struct t_treeNode 取個新名字 */
typedef TreeNode *   TreeNodePtr; /* 替 TreeNode* 也取一個新名字，可以避免使用兩個星號 */

void insertNode(TreeNodePtr *treePtr, int value); /* 參數使用 call-by-reference */
void inOrder(TreeNodePtr treePtr);
void preOrder(TreeNodePtr treePtr);
void postOrder(TreeNodePtr treePtr);

int main( void )
{
    TreeNodePtr rootPtr = NULL;

    insertNode(&rootPtr, 27);
    insertNode(&rootPtr, 42);
    insertNode(&rootPtr, 13);
    insertNode(&rootPtr, 6);
    insertNode(&rootPtr, 33);
    insertNode(&rootPtr, 48);
    insertNode(&rootPtr, 17);

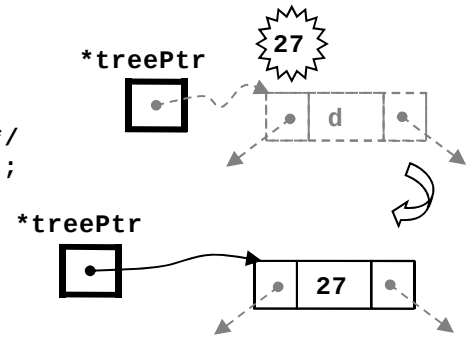
    printf("\n\nThe preOrder traversal is:\n");
    preOrder(rootPtr);
    printf("\n\nThe inOrder traversal is:\n");
    inOrder(rootPtr);
    printf("\n\nThe postOrder traversal is:\n");
    postOrder(rootPtr);
    return 0;
}
  
```



```

/* 用 recursive 方式加入新的 node */
void insertNode(TreeNodePtr *treePtr, int value)
{
    if (*treePtr == NULL) { /* *treePtr 記錄的位址是 NULL */
        *treePtr = (TreeNodePtr)malloc(sizeof( TreeNode ));
        if (*treePtr != NULL) {
            (*treePtr)->data = value;
            (*treePtr)->leftPtr = NULL;
            (*treePtr)->rightPtr = NULL;
        }
    }
    else {
        printf("%d not inserted. No memory available.\n", value);
    }
}
else {
    if (value < (*treePtr)->data) {
        insertNode(&(*treePtr)->leftPtr, value);
    }
    else if (value > (*treePtr)->data) {
        insertNode(&(*treePtr)->rightPtr, value);
    }
    else { /* 如果要放入的值重複 (已經在 tree 裡面) 就不再放入 */
        printf("重複");
    }
}
}
}

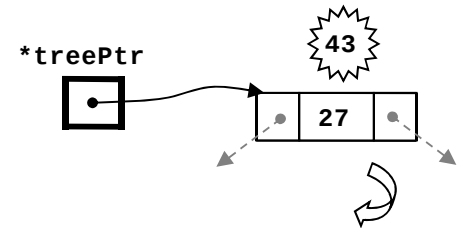
```



```

/* 用 recursive 方式走完一個 tree */
void inOrder(TreeNodePtr treePtr)
{
    if (treePtr != NULL) {
        inOrder(treePtr->leftPtr);
        printf("%3d", treePtr->data);
        inOrder(treePtr->rightPtr);
    }
}

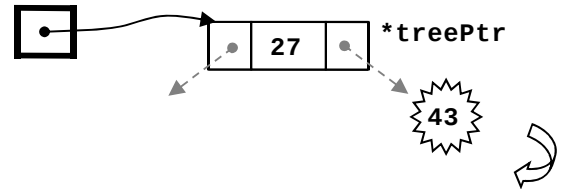
```



```

void preOrder(TreeNodePtr treePtr)
{
    if (treePtr != NULL) {
        printf("%3d", treePtr->data);
        preOrder(treePtr->leftPtr);
        preOrder(treePtr->rightPtr);
    }
}

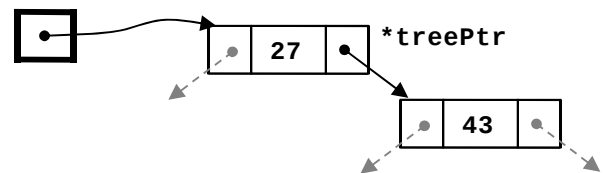
```



```

void postOrder(TreeNodePtr treePtr)
{
    if (treePtr != NULL) {
        postOrder(treePtr->leftPtr);
        postOrder(treePtr->rightPtr);
        printf("%3d", treePtr->data);
    }
}

```



[練習 WE_09]

前面介紹的 binary search tree 不允許重複的值出現，每個 node 存放的值都不一樣。如果要把它改成可以允許重複的值，可以在 `struct t_treeNode` 多加一個欄位來記錄重複的次數，這樣做會多佔用一些空間，你可以想想看有沒有其他作法。另外還有一個問題是如何寫 `deleteNode()`，可以傳入某個值，把存放這個值的 node 刪除，

```
int deleteNode(TreeNodePtr *treePtr, int value);
```

關鍵在於如何把斷掉的樹接起來。